AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 1: Introduction

Adam Prugel-Bennett

September 15, 2026

1 Keywords

- Course organisation, vectors, matrices, transposes, determinants, gradients, random variables, expectations

2 Main Points

2.1 About the Course

- The course is dual coded as AICE3001 (third year Artificial Intelligence) and AICE6002 (MSc Machine Learning), with slightly different assessment for the two
- Notes are on Moodle and at `https://ecs-vlc.github.io/aice3001/`
- **Why learn mathematics when LLMs can do so much?** Because you need to be able to trust your own judgement. LLMs are useful, but if you cannot check what they tell you, you are disempowered
- Learning machines are notoriously hard to understand. Mathematics lets us reason rigorously, build intuition we can rely on, describe the world precisely, and gives us a set of tools that turn out to be hugely useful
- Mathematics is a language that takes time to learn. Everyone can crack the code, but only by working with it—which is why the tutorials should be treated as compulsory
- Students start from very different places. If you are strong, become stronger by helping others; if you are starting from a weak base, judge yourself by your progress rather than by comparison
- **The golden rule of mathematics: understand every step.** When you don't understand a step, stop, break the problem down, and ask. Asking "stupid" questions is how everyone learns
- The course covers calculus, vector spaces, convexity, optimisation, probability, statistics and information theory: the foundations of most of machine learning

2.2 What We Assume You Know

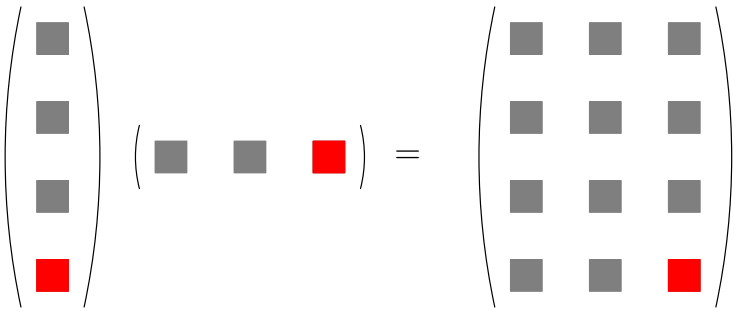
- A-level mathematics and some university mathematics: functions such as $f(x) = x^2 + 3$, simple algebra, and elementary calculus such as $f'(x) = 2\,x$
- The rest of this note summarises the assumed background. We will revisit most of it in more depth, but if it is unfamiliar you should revise it now, because other machine learning modules rely on it

2.3 Vectors

- Vectors in $\mathbb{R}^n$ are written in bold and are *column* vectors,

$$\boldsymbol{x} = \begin{pmatrix} 1.2 \\ 2.3 \\ 4.7 \end{pmatrix}$$

- The corresponding *row* vector is the transpose, $\boldsymbol{x}^\mathsf{T} = (1.2, 2.3, 4.7)$. Keeping track of rows and columns lets us write vector equations succinctly and unambiguously
- The *inner product* (dot product) of two vectors of the same length is $\boldsymbol{a}^\mathsf{T}\boldsymbol{b} = \sum_i a_i\, b_i$. It encodes the angle between them: $\boldsymbol{a}^\mathsf{T}\boldsymbol{b} = \|\boldsymbol{a}\|\ \|\boldsymbol{b}\| \cos(\theta)$
- The *outer product* $\boldsymbol{a}\,\boldsymbol{b}^\mathsf{T}$ is a matrix with elements $a_i\, b_j$, defined for vectors of any sizes



- The *Euclidean norm* (2-norm) measures size,

$$\|\boldsymbol{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{\boldsymbol{x}^\mathsf{T}\boldsymbol{x}}$$

  Other norms exist; this one is intimately tied to the inner product

2.4 Matrices

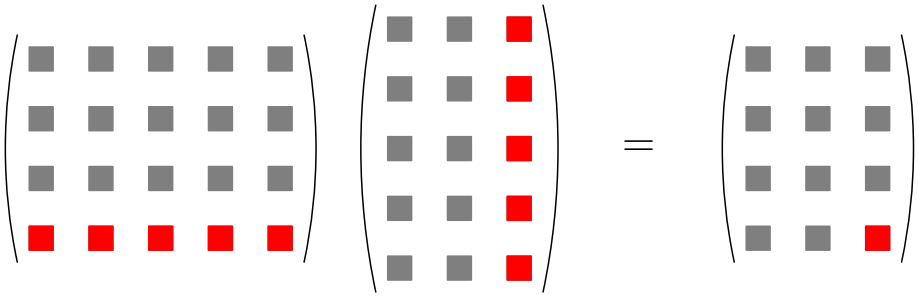
- An $m \times n$ matrix, $\mathbf{M} \in \mathbb{R}^{m\times n}$, has m rows and n columns, and maps vectors in $\mathbb{R}^n$ to vectors in $\mathbb{R}^m$:

$$\mathbf{M}\,\boldsymbol{x} = \begin{pmatrix} 1.1 & -2.1 & 6.2 \\ 9.3 & 3.1 & -4.4 \end{pmatrix} \begin{pmatrix} 1.2 \\ 2.3 \\ 4.7 \end{pmatrix} = \begin{pmatrix} 25.63 \\ -2.39 \end{pmatrix}$$

- Each output element is a linear sum of the inputs, $b_i = \sum_j M_{ij}\, a_j$, so for any real c

$$\mathbf{M}\,(c\,\boldsymbol{a}) = c\,(\mathbf{M}\,\boldsymbol{a})$$
$$\mathbf{M}\,(\boldsymbol{a} + \boldsymbol{b}) = \mathbf{M}\,\boldsymbol{a} + \mathbf{M}\,\boldsymbol{b}$$

- If $\mathbf{A} \in \mathbb{R}^{\ell\times m}$ and $\mathbf{B} \in \mathbb{R}^{m\times n}$ then $\mathbf{A}\,\mathbf{B}$ is the mapping "first apply $\mathbf{B}$, then $\mathbf{A}$", $\mathbf{A}\,\mathbf{B}\,\boldsymbol{x} = \mathbf{A}\,(\mathbf{B}\,\boldsymbol{x})$. It is an $\ell \times n$ matrix computed by matrix multiplication (rows times columns)



- Matrix multiplication is associative, $(\mathbf{A}\,\mathbf{B})\,\mathbf{C} = \mathbf{A}\,(\mathbf{B}\,\mathbf{C})$, but in general not commutative, $\mathbf{A}\,\mathbf{B} \neq \mathbf{B}\,\mathbf{A}$
- A column vector can be treated as an $n \times 1$ matrix, so matrix-vector multiplication is a special case of matrix multiplication

2.5 Transposes

- The transpose swaps rows and columns, $[\mathbf{M}^\mathsf{T}]_{ij} = [\mathbf{M}]_{ji}$, and $(\mathbf{M}^\mathsf{T})^\mathsf{T} = \mathbf{M}$
- A little book keeping shows $(\mathbf{A}\,\mathbf{B})^\mathsf{T} = \mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}$
- A matrix is *symmetric* if $\mathbf{M} = \mathbf{M}^\mathsf{T}$. A 1×1 matrix is always symmetric, which gives two identities we will use constantly:

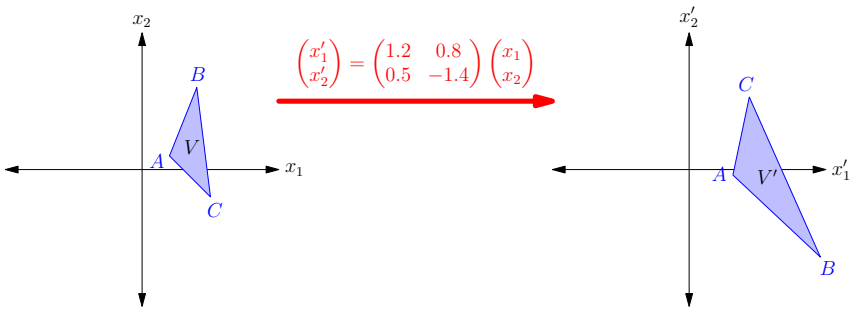
$$\boldsymbol{a}^\mathsf{T}\boldsymbol{b} = \boldsymbol{b}^\mathsf{T}\boldsymbol{a} \qquad\qquad \boldsymbol{a}^\mathsf{T}\mathbf{M}\,\boldsymbol{b} = \boldsymbol{b}^\mathsf{T}\mathbf{M}^\mathsf{T}\boldsymbol{a}$$

2.6 Determinants

- Treat each point of a region of volume V in $\mathbb{R}^n$ as a vector and map it by a square matrix $\mathbf{M}$. The image has volume V', and

$$\det(\mathbf{M}) = \pm\frac{V'}{V}$$

  where the sign records whether the mapping includes a reflection



- Because $\mathbf{A}\,\mathbf{B}$ means "apply $\mathbf{B}$ then $\mathbf{A}$", volume scalings multiply: $\det(\mathbf{A}\,\mathbf{B}) = \det(\mathbf{A})\,\det(\mathbf{B})$
- Determinants are only defined for square matrices; a non-square matrix maps between spaces of different dimension, where volumes cannot be compared
- A *singular* matrix squashes space onto a lower dimensional subspace, so its determinant is 0

2.7 Multivariate Functions and Gradients

- Machine learning uses functions $f : \mathbb{R}^n \to \mathbb{R}^m$ heavily, and very often scalar functions $f : \mathbb{R}^n \to \mathbb{R}$, such as a learning machine $\hat{f}(\boldsymbol{x}, \boldsymbol{w})$ or a loss function $L(\boldsymbol{w})$
- A function is *linear* if $f(c\,\boldsymbol{x}) = c\,f(\boldsymbol{x})$ and $f(\boldsymbol{x} + \boldsymbol{y}) = f(\boldsymbol{x}) + f(\boldsymbol{y})$
- The *gradient* of a scalar function $f : \mathbb{R}^n \to \mathbb{R}$ is the column vector of partial derivatives

$$\boldsymbol{\nabla} f(\boldsymbol{x}) = \begin{pmatrix} \partial f/\partial x_1 \\ \vdots \\ \partial f/\partial x_n \end{pmatrix}$$

- It is orthogonal to the contours of f and points in the direction in which f increases fastest

2.8 Probability

- A *random variable* assigns a number to the outcome of a random event: 0 for tails and 1 for heads is a discrete random variable; the temperature in Hyde Park at noon tomorrow is a continuous one
- For a discrete random variable the *probability mass function* $\mathbb{P}[X = x]$ lies between 0 and 1 and $\sum_{x \in \mathcal{X}} \mathbb{P}[X = x] = 1$
- For a continuous random variable the probability of any single value is zero, and we use the *probability density*

$$f_X(x) = \lim_{\delta x \to 0} \frac{\mathbb{P}[x < X \leq x + \delta x]}{\delta x}$$

- The *expectation* of a function $g(X)$ is its average value,

$$\mathbb{E}[g(X)] = \sum_{x \in \mathcal{X}} \mathbb{P}[X = x]\ g(x) \quad \text{or} \quad \mathbb{E}[g(X)] = \int f_X(x)\, g(x)\, \mathrm{d}x$$

- Expectation is linear, $\mathbb{E}[c\, g(X)] = c\, \mathbb{E}[g(X)]$ and $\mathbb{E}[g(X) + h(X)] = \mathbb{E}[g(X)] + \mathbb{E}[h(X)]$, and the expectation of a constant is itself, $\mathbb{E}[c] = c$

3 Exercises

These are warm-ups to check the assumed background.

3.1 Matrix multiplication does not commute

- Compute $\mathbf{A}\,\mathbf{B}$ and $\mathbf{B}\,\mathbf{A}$ for

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \qquad \mathbf{B} = \begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix}$$

3.2 Transpose of a product

- Prove $(\mathbf{A}\,\mathbf{B})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$ by writing out the elements

3.3 Determinants as areas

- Compute the determinant of $\mathbf{M} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ and say what it tells you about the image of the unit square

3.4 Expectations

- For a fair six-sided die X, compute $\mathbb{E}[X]$, $\mathbb{E}\left[X^2\right]$ and the variance $\mathbb{E}\left[X^2\right] - \mathbb{E}[X]^2$
- Use the definition of expectation to show $\mathbb{E}[a\, X + b] = a\, \mathbb{E}[X] + b$

4 Answers

4.1 Matrix multiplication does not commute

- $$\mathbf{A}\,\mathbf{B} = \begin{pmatrix} 7 & 2 \\ 3 & 1 \end{pmatrix} \qquad \mathbf{B}\,\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 7 \end{pmatrix}$$

  which are different

4.2 Transpose of a product

- $[(\mathbf{A}\,\mathbf{B})^\mathsf{T}]_{ij} = [\mathbf{A}\,\mathbf{B}]_{ji} = \sum_k A_{jk}\,B_{ki}$
- $[\mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}]_{ij} = \sum_k [\mathbf{B}^\mathsf{T}]_{ik}\,[\mathbf{A}^\mathsf{T}]_{kj} = \sum_k B_{ki}\,A_{jk}$
- These are the same sum, so the matrices are equal

4.3 Determinants as areas

- $\det(\mathbf{M}) = 2 \times 3 - 1 \times 1 = 5$
- The unit square is mapped to a parallelogram of area 5, with no reflection since the determinant is positive

4.4 Expectations

- $\mathbb{E}[X] = \frac{1}{6}(1 + 2 + \cdots + 6) = 3.5$
- $\mathbb{E}\left[X^2\right] = \frac{1}{6}(1 + 4 + 9 + 16 + 25 + 36) = 91/6$
- The variance is $91/6 - 49/4 = 35/12 \approx 2.92$
- For a discrete variable $\mathbb{E}[a\,X + b] = \sum_x \mathbb{P}[X = x]\,(a\,x{+}b) = a\sum_x \mathbb{P}[X = x]\;x{+}b\sum_x \mathbb{P}[X = x] = a\,\mathbb{E}[X] + b$, using $\sum_x \mathbb{P}[X = x] = 1$ (the continuous case is identical with integrals)

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 2: Differential Calculus

Adam Prugel-Bennett

September 22, 2026

1 Keywords

- Differentiation, linearity, product rule, chain rule, inverse functions, gradients, Jacobians, matrix derivatives, traces

2 Main Points

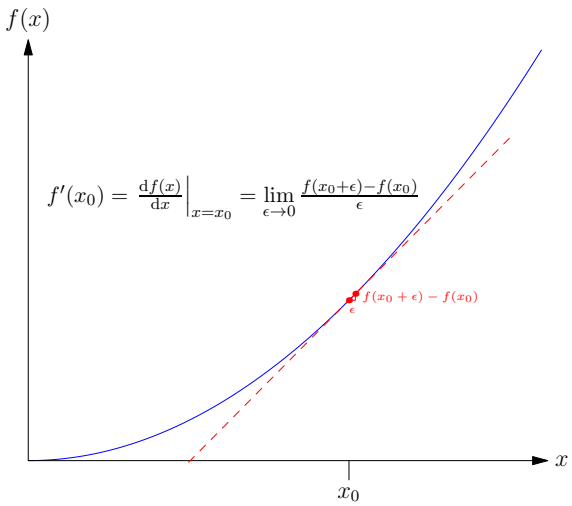
2.1 Why Calculus?

- Differentiation is the fundamental tool of optimisation, and optimisation is how learning machines are trained
- Integration is how we take expectations over continuous distributions
- You met the rules at A-level, but they are easy to forget, and we will use some quite sophisticated tricks. It pays to understand where the tricks come from

2.2 Differentiation from First Principles

- The derivative is the limit of the slope of a chord,

$$f'(x) = \frac{\mathrm{d}f(x)}{\mathrm{d}x} = \lim_{\epsilon\to 0} \frac{f(x+\epsilon) - f(x)}{\epsilon}$$



- An equivalent statement, used over and over again in this course, is

$$f(x+\epsilon) = f(x) + \epsilon\, f'(x) + O(\epsilon^2)$$

To find a derivative, expand $f(x+\epsilon)$ and read off the coefficient of ϵ

- **Polynomials.**
$$\frac{\mathrm{d}x^2}{\mathrm{d}x} = \lim_{\epsilon\to 0} \frac{(x^2 + 2\,\epsilon\,x + \epsilon^2) - x^2}{\epsilon} = \lim_{\epsilon\to 0}(2\,x + \epsilon) = 2\,x$$
More generally $(x+\epsilon)^n = x^n + n\,\epsilon\,x^{n-1} + O(\epsilon^2)$, so $\mathrm{d}x^n/\mathrm{d}x = n\,x^{n-1}$

- **Linearity.** Splitting the limit,
$$\frac{\mathrm{d}}{\mathrm{d}x}\left(a\,f(x) + b\,g(x)\right) = a\,f'(x) + b\,g'(x)$$
Differentiation is a linear operation

2.3 Product and Chain Rules

- **Product rule.** Multiplying out the expansions of $f(x+\epsilon)$ and $g(x+\epsilon)$,
$$f(x+\epsilon)\,g(x+\epsilon) = f(x)\,g(x) + \epsilon\left(f'(x)\,g(x) + f(x)\,g'(x)\right) + O(\epsilon^2)$$
so the derivative of $h(x) = f(x)\,g(x)$ is $h'(x) = f'(x)\,g(x) + f(x)\,g'(x)$

- **Chain rule.** For $h(x) = f(g(x))$,
$$h(x+\epsilon) = f\left(g(x) + \epsilon\,g'(x) + O(\epsilon^2)\right) = f(g(x)) + \epsilon\,g'(x)\,f'(g(x)) + O(\epsilon^2)$$
so $h'(x) = g'(x)\,f'(g(x))$

- Together the product and chain rules let us differentiate almost anything

- The chain rule is often easier to remember as
$$\frac{\mathrm{d}f(g(x))}{\mathrm{d}x} = \frac{\mathrm{d}f(g)}{\mathrm{d}g}\,\frac{\mathrm{d}g(x)}{\mathrm{d}x}$$
For example
$$\frac{\mathrm{d}\,\mathrm{e}^{\cos(x^2)}}{\mathrm{d}x} = \frac{\mathrm{d}\,\mathrm{e}^{\cos(x^2)}}{\mathrm{d}\cos(x^2)}\,\frac{\mathrm{d}\cos(x^2)}{\mathrm{d}x^2}\,\frac{\mathrm{d}x^2}{\mathrm{d}x} = \mathrm{e}^{\cos(x^2)}\left(-\sin(x^2)\right)\,2\,x = -2\,x\,\sin(x^2)\,\mathrm{e}^{\cos(x^2)}$$

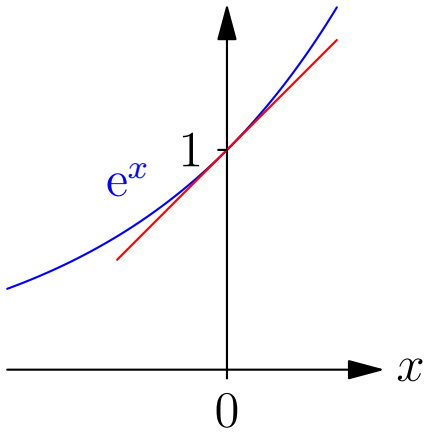
2.4 Inverse Functions

- If $g = f^{-1}$, so $g(f(x)) = x$, differentiating both sides with the chain rule gives $f'(x)\,g'(f(x)) = 1$

- Writing $y = f(x)$,
$$\frac{\mathrm{d}f^{-1}(y)}{\mathrm{d}y} = \frac{1}{f'\left(f^{-1}(y)\right)}$$

2.5 Exponentials and Logarithms

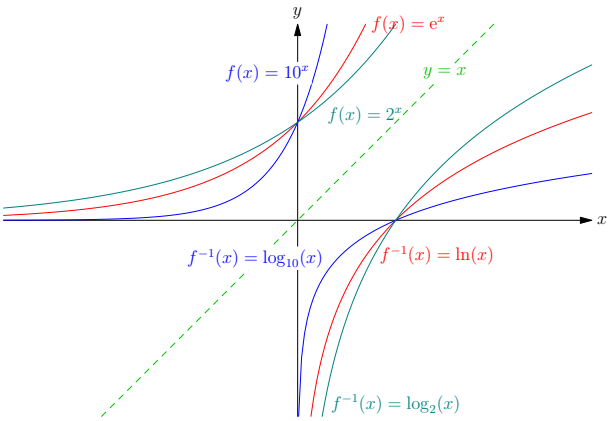
- Since $\mathrm{e}^{\epsilon} = 1 + \epsilon + O(\epsilon^2)$ and $\mathrm{e}^{x+\epsilon} = \mathrm{e}^x\,\mathrm{e}^{\epsilon}$,
$$\mathrm{e}^{x+\epsilon} = \mathrm{e}^x + \epsilon\,\mathrm{e}^x + O(\epsilon^2) \qquad \Longrightarrow \qquad \frac{\mathrm{d}\,\mathrm{e}^x}{\mathrm{d}x} = \mathrm{e}^x$$



- By the chain rule $\mathrm{d}\,e^{c\,x}/\mathrm{d}x = c\,e^{c\,x}$, and more generally $\mathrm{d}\,e^{g(x)}/\mathrm{d}x = g'(x)\,e^{g(x)}$
- For any base, $a^x = e^{x\ln(a)}$, so $\mathrm{d}\,a^x/\mathrm{d}x = \ln(a)\,a^x$
- The natural logarithm is the inverse of the exponential, $\ln(e^x) = x$ and $e^{\ln(y)} = y$. The inverse function rule with $f(x) = e^x$ gives

$$\frac{\mathrm{d}\ln(y)}{\mathrm{d}y} = \frac{1}{e^{\ln(y)}} = \frac{1}{y}$$

- The lecture animation shows e^x and $\ln(x)$ as reflections of one another in the line $y = x$: reflecting a curve swaps rise and run, which is why the slopes of inverse functions are reciprocals



2.6 Derivatives in High Dimensions

- A function $f : \mathbb{R}^n \to \mathbb{R}$ changes at different rates in different directions. The derivative in a unit direction $\boldsymbol{u}$ is the *directional derivative*

$$\partial_{\boldsymbol{u}} f(\boldsymbol{x}) = \lim_{\epsilon \to 0} \frac{f(\boldsymbol{x} + \epsilon\,\boldsymbol{u}) - f(\boldsymbol{x})}{\epsilon}$$

- Taking $\boldsymbol{u} = \boldsymbol{\delta}_i$, the unit vector along the ith axis, gives the partial derivative $\partial f(\boldsymbol{x})/\partial x_i$
- Expanding to first order,

$$f(\boldsymbol{x} + \epsilon\,\boldsymbol{u}) = f(\boldsymbol{x}) + \epsilon\,\boldsymbol{u}^{\mathsf{T}}\boldsymbol{\nabla} f(\boldsymbol{x}) + O(\epsilon^2)$$

where the *gradient* $\boldsymbol{\nabla} f(\boldsymbol{x})$ is the column vector of partial derivatives. This is the start of the high dimensional Taylor expansion

2.7 Computing Gradients

- **Component-wise.** Writing $\boldsymbol{w}^\mathsf{T}\mathbf{M}\,\boldsymbol{w} = \sum_{i,j} w_i\, M_{ij}\, w_j$ and differentiating with respect to w_k picks out the terms with $i = k$ or $j = k$:

$$\frac{\partial}{\partial w_k} \sum_{i,j} w_i\, M_{ij}\, w_j = \sum_j M_{kj}\, w_j + \sum_i w_i\, M_{ik}$$

  so $\boldsymbol{\nabla}\, \boldsymbol{w}^\mathsf{T}\mathbf{M}\,\boldsymbol{w} = \left(\mathbf{M} + \mathbf{M}^\mathsf{T}\right) \boldsymbol{w}$. Tedious, but when you are confused, go back to components

- **By expansion.** A slicker method is to expand $f(\boldsymbol{x} + \epsilon\,\boldsymbol{u})$. For $f(\boldsymbol{x}) = \boldsymbol{x}^\mathsf{T}\mathbf{M}\,\boldsymbol{x} + \boldsymbol{a}^\mathsf{T}\boldsymbol{x}$,

$$\begin{aligned} f(\boldsymbol{x} + \epsilon\,\boldsymbol{u}) &= f(\boldsymbol{x}) + \epsilon \left(\boldsymbol{u}^\mathsf{T}\mathbf{M}\,\boldsymbol{x} + \boldsymbol{x}^\mathsf{T}\mathbf{M}\,\boldsymbol{u} + \boldsymbol{a}^\mathsf{T}\boldsymbol{u}\right) + O(\epsilon^2) \\ &= f(\boldsymbol{x}) + \epsilon\, \boldsymbol{u}^\mathsf{T} \left(\mathbf{M}\,\boldsymbol{x} + \mathbf{M}^\mathsf{T}\boldsymbol{x} + \boldsymbol{a}\right) + O(\epsilon^2) \end{aligned}$$

  using $\boldsymbol{x}^\mathsf{T}\mathbf{M}\,\boldsymbol{u} = \boldsymbol{u}^\mathsf{T}\mathbf{M}^\mathsf{T}\boldsymbol{x}$ and $\boldsymbol{a}^\mathsf{T}\boldsymbol{u} = \boldsymbol{u}^\mathsf{T}\boldsymbol{a}$. Reading off the coefficient, $\boldsymbol{\nabla} f(\boldsymbol{x}) = \mathbf{M}\,\boldsymbol{x} + \mathbf{M}^\mathsf{T}\boldsymbol{x} + \boldsymbol{a}$

2.8 Jacobians and the Chain Rule in High Dimensions

- Consider a composition $h(\boldsymbol{x}) = f(\boldsymbol{g}(\boldsymbol{x}))$ with $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^m$ and $f : \mathbb{R}^m \to \mathbb{R}$. The vector valued $\boldsymbol{g}$ is just m scalar functions $g_1, \ldots, g_m$

- Perturbing each one,

$$\boldsymbol{g}(\boldsymbol{x} + \epsilon\,\boldsymbol{u}) = \boldsymbol{g}(\boldsymbol{x}) + \epsilon\,\mathbf{J}\,\boldsymbol{u} + O(\epsilon^2)$$

  where the *Jacobian* $\mathbf{J}$ is the $m \times n$ matrix with $J_{ij} = \partial g_i(\boldsymbol{x})/\partial x_j$

$$\mathbf{J} = \begin{pmatrix} \frac{\partial g_1(\boldsymbol{x})}{\partial x_1} & \frac{\partial g_1(\boldsymbol{x})}{\partial x_2} & \cdots & \frac{\partial g_1(\boldsymbol{x})}{\partial x_n} \\ \frac{\partial g_2(\boldsymbol{x})}{\partial x_1} & \frac{\partial g_2(\boldsymbol{x})}{\partial x_2} & \cdots & \frac{\partial g_2(\boldsymbol{x})}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial g_m(\boldsymbol{x})}{\partial x_1} & \frac{\partial g_m(\boldsymbol{x})}{\partial x_2} & \cdots & \frac{\partial g_m X(\boldsymbol{x})}{\partial x_n} \end{pmatrix}$$

- Then, to first order,

$$h(\boldsymbol{x} + \epsilon\,\boldsymbol{u}) = f\left(\boldsymbol{g}(\boldsymbol{x}) + \epsilon\,\mathbf{J}\,\boldsymbol{u}\right) = h(\boldsymbol{x}) + \epsilon\,\boldsymbol{u}^\mathsf{T}\mathbf{J}^\mathsf{T}\boldsymbol{\nabla}_{\boldsymbol{g}} f(\boldsymbol{g}(\boldsymbol{x}))$$

- Comparing with $h(\boldsymbol{x} + \epsilon\,\boldsymbol{u}) = h(\boldsymbol{x}) + \epsilon\,\boldsymbol{u}^\mathsf{T}\boldsymbol{\nabla}_{\boldsymbol{x}} h(\boldsymbol{x})$ gives the chain rule in high dimensions,

$$\boldsymbol{\nabla}_{\boldsymbol{x}} h(\boldsymbol{x}) = \mathbf{J}^\mathsf{T}\, \boldsymbol{\nabla}_{\boldsymbol{g}} f(\boldsymbol{g}(\boldsymbol{x})) \qquad \text{or} \qquad \frac{\partial h(\boldsymbol{x})}{\partial x_i} = \sum_{j=1}^{m} \frac{\partial f(\boldsymbol{g})}{\partial g_j}\, \frac{\partial g_j(\boldsymbol{x})}{\partial x_i}$$

- Backpropagation in a neural network is this rule applied layer by layer: the gradient is passed backwards by multiplying by transposed Jacobians

2.9 Differentiating with Respect to Matrices

- Losses often depend on a matrix of weights, e.g. $L(\mathbf{W}) = (\boldsymbol{a}^\mathsf{T}\mathbf{W}\,\boldsymbol{b} - c)^2$. A minimum is a point $\mathbf{W}^*$ where, for every matrix $\mathbf{U}$, $L(\mathbf{W}^* + \epsilon\,\mathbf{U}) - L(\mathbf{W}^*) = O(\epsilon^2)$

- Define $\partial L(\mathbf{W})/\partial \mathbf{W}$ as the matrix of partial derivatives, with elements $\partial L/\partial W_{ij}$. The same argument as for vectors gives

$$L(\mathbf{W} + \epsilon\,\mathbf{U}) = L(\mathbf{W}) + \epsilon\, \mathrm{tr}\left(\mathbf{U}^\mathsf{T} \frac{\partial L(\mathbf{W})}{\partial \mathbf{W}}\right) + O(\epsilon^2)$$

  where $\mathrm{tr}(\mathbf{U}^\mathsf{T}\mathbf{G}) = \sum_{ij} U_{ij}\, G_{ij}$ is the inner product between matrices

- **Example.** For $L(\mathbf{W}) = (\boldsymbol{a}^\mathsf{T}\mathbf{W}\,\boldsymbol{b} - c)^2$,

$$L(\mathbf{W} + \epsilon\,\mathbf{U}) = L(\mathbf{W}) + 2\,\epsilon \left(\boldsymbol{a}^\mathsf{T}\mathbf{W}\,\boldsymbol{b} - c\right) \boldsymbol{a}^\mathsf{T}\mathbf{U}\,\boldsymbol{b} + O(\epsilon^2)$$

  and $\boldsymbol{a}^\mathsf{T}\mathbf{U}\,\boldsymbol{b} = \sum_{ij} a_i\, U_{ij}\, b_j = \mathrm{tr}(\mathbf{U}^\mathsf{T}\boldsymbol{a}\,\boldsymbol{b}^\mathsf{T})$, so $\partial L/\partial \mathbf{W} = 2\left(\boldsymbol{a}^\mathsf{T}\mathbf{W}\,\boldsymbol{b} - c\right) \boldsymbol{a}\,\boldsymbol{b}^\mathsf{T}$

2.10 Traces

- The trace is the sum of the diagonal elements, $\operatorname{tr}\mathbf{A} = \sum_i A_{ii} = \operatorname{tr}\mathbf{A}^\mathsf{T}$
- It is linear, $\operatorname{tr}(c\,\mathbf{A}) = c\operatorname{tr}\mathbf{A}$ and $\operatorname{tr}(\mathbf{A}+\mathbf{B}) = \operatorname{tr}\mathbf{A} + \operatorname{tr}\mathbf{B}$
- $\operatorname{tr}(\mathbf{A}\,\mathbf{B}) = \sum_{ij} A_{ij}\,B_{ji} = \operatorname{tr}(\mathbf{B}\,\mathbf{A})$, so a trace is unchanged by cycling the matrices in a product:

$$\operatorname{tr}(\mathbf{A}\,\mathbf{B}\,\mathbf{C}\,\mathbf{D}) = \operatorname{tr}(\mathbf{D}\,\mathbf{A}\,\mathbf{B}\,\mathbf{C}) = \operatorname{tr}(\mathbf{C}\,\mathbf{D}\,\mathbf{A}\,\mathbf{B}) = \operatorname{tr}(\mathbf{B}\,\mathbf{C}\,\mathbf{D}\,\mathbf{A})$$

- **Quick matrix differentiation.** Since $\partial_\mathbf{U} f(\mathbf{X}) = \operatorname{tr}\left(\mathbf{U}^\mathsf{T}\,\partial f(\mathbf{X})/\partial\mathbf{X}\right)$, expand, then use the trace rules to move $\mathbf{U}^\mathsf{T}$ to the front. For example

$$\partial_\mathbf{U} \operatorname{tr}(\mathbf{A}\,\mathbf{X}\,\mathbf{B}) = \operatorname{tr}(\mathbf{A}\,\mathbf{U}\,\mathbf{B}) = \operatorname{tr}(\mathbf{B}^\mathsf{T}\mathbf{U}^\mathsf{T}\mathbf{A}^\mathsf{T}) = \operatorname{tr}(\mathbf{U}^\mathsf{T}\mathbf{A}^\mathsf{T}\mathbf{B}^\mathsf{T})$$

  so $\partial \operatorname{tr}(\mathbf{A}\,\mathbf{X}\,\mathbf{B})/\partial\mathbf{X} = \mathbf{A}^\mathsf{T}\mathbf{B}^\mathsf{T}$
- Many more useful results are collected in *The Matrix Cookbook*

3 Exercises

3.1 First principles

- Show from the definition that $\mathrm{d}(1/x)/\mathrm{d}x = -1/x^2$

3.2 Chain rule

- Differentiate x^x (hint: write it as an exponential)

3.3 Inverse functions

- Use the inverse function rule to find $\mathrm{d}\arcsin(y)/\mathrm{d}y$

3.4 The logistic function

- Show that $\sigma(x) = 1/(1+\mathrm{e}^{-x})$ satisfies $\sigma'(x) = \sigma(x)\,(1-\sigma(x))$

3.5 Least squares

- By expanding $f(\boldsymbol{w}+\epsilon\,\boldsymbol{u})$, find the gradient of $f(\boldsymbol{w}) = \|\boldsymbol{y} - \mathbf{X}\,\boldsymbol{w}\|^2$

3.6 A matrix derivative

- Find $\partial L/\partial\mathbf{W}$ for $L(\mathbf{W}) = \|\mathbf{W}\,\boldsymbol{x} - \boldsymbol{y}\|^2$

4 Answers

4.1 First principles

-

$$\frac{1}{\epsilon}\left(\frac{1}{x+\epsilon} - \frac{1}{x}\right) = \frac{1}{\epsilon}\,\frac{x-(x+\epsilon)}{x\,(x+\epsilon)} = -\frac{1}{x\,(x+\epsilon)} \to -\frac{1}{x^2}$$

4.2 Chain rule

- $x^x = \mathrm{e}^{x\ln(x)}$, so by the chain and product rules

$$\frac{\mathrm{d}x^x}{\mathrm{d}x} = (\ln(x)+1)\,\mathrm{e}^{x\ln(x)} = (\ln(x)+1)\,x^x$$

4.3 Inverse functions

- With $f = \sin$ and $f' = \cos$,

$$\frac{\mathrm{d}\arcsin(y)}{\mathrm{d}y} = \frac{1}{\cos(\arcsin(y))} = \frac{1}{\sqrt{1-y^2}}$$

using $\cos(\theta) = \sqrt{1-\sin^2(\theta)}$, which is non-negative on the range of $\arcsin$

4.4 The logistic function

- $$\sigma'(x) = \frac{\mathrm{e}^{-x}}{(1+\mathrm{e}^{-x})^2} = \frac{1}{1+\mathrm{e}^{-x}}\,\frac{\mathrm{e}^{-x}}{1+\mathrm{e}^{-x}} = \sigma(x)\,(1-\sigma(x))$$

since $1-\sigma(x) = \mathrm{e}^{-x}/(1+\mathrm{e}^{-x})$

4.5 Least squares

- $f(\boldsymbol{w}+\epsilon\,\boldsymbol{u}) = \|(\boldsymbol{y}-\mathbf{X}\,\boldsymbol{w}) - \epsilon\,\mathbf{X}\,\boldsymbol{u}\|^2 = f(\boldsymbol{w}) - 2\,\epsilon\,\boldsymbol{u}^\mathsf{T}\mathbf{X}^\mathsf{T}(\boldsymbol{y}-\mathbf{X}\,\boldsymbol{w}) + O(\epsilon^2)$
- So $\boldsymbol{\nabla} f(\boldsymbol{w}) = -2\,\mathbf{X}^\mathsf{T}(\boldsymbol{y}-\mathbf{X}\,\boldsymbol{w}) = 2\,\mathbf{X}^\mathsf{T}(\mathbf{X}\,\boldsymbol{w}-\boldsymbol{y})$

4.6 A matrix derivative

- $L(\mathbf{W}+\epsilon\,\mathbf{U}) = \|(\mathbf{W}\,\boldsymbol{x}-\boldsymbol{y}) + \epsilon\,\mathbf{U}\,\boldsymbol{x}\|^2 = L(\mathbf{W}) + 2\,\epsilon\,(\mathbf{W}\,\boldsymbol{x}-\boldsymbol{y})^\mathsf{T}\mathbf{U}\,\boldsymbol{x} + O(\epsilon^2)$
- $(\mathbf{W}\,\boldsymbol{x}-\boldsymbol{y})^\mathsf{T}\mathbf{U}\,\boldsymbol{x} = \mathrm{tr}\left(\mathbf{U}^\mathsf{T}(\mathbf{W}\,\boldsymbol{x}-\boldsymbol{y})\,\boldsymbol{x}^\mathsf{T}\right)$ by the same argument as the example above, so

$$\frac{\partial L}{\partial \mathbf{W}} = 2\,(\mathbf{W}\,\boldsymbol{x}-\boldsymbol{y})\,\boldsymbol{x}^\mathsf{T}$$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 3: Taylor's Expansion

Adam Prugel-Bennett

October 7, 2026

1 Keywords

- Taylor expansion, convergence, remainder, gradient, tangent plane, Hessian, minima, maxima, saddle points

2 Main Points

2.1 Polynomial Approximation

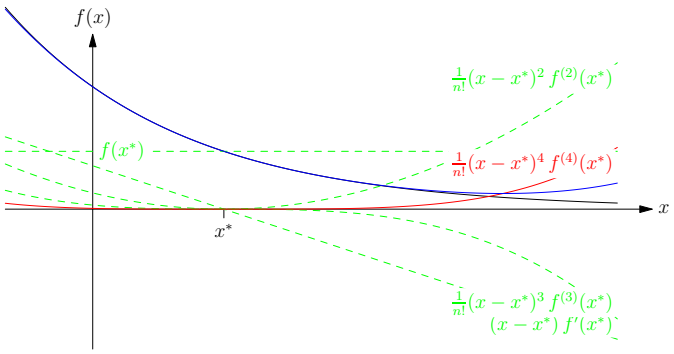
- Much of machine learning is about approximating functions, and every approximation scheme assumes something about how regular the function is
- A Taylor expansion approximates a smooth function near a point x^* by a polynomial

$$f(x) \approx a_0 + a_1(x - x^*) + a_2(x - x^*)^2 + \cdots + a_n(x - x^*)^n$$

- Differentiating both sides i times and setting $x = x^*$ picks out a single coefficient, so if we demand that the two sides agree in value and in their first n derivatives at x^* then $a_i = f^{(i)}(x^*)/i!$
- This gives the Taylor expansion

$$f(x) \approx f(x^*) + f'(x^*)\,(x - x^*) + \frac{1}{2}f''(x^*)\,(x - x^*)^2 + \cdots + \frac{1}{n!}f^{(n)}(x^*)\,(x - x^*)^n$$

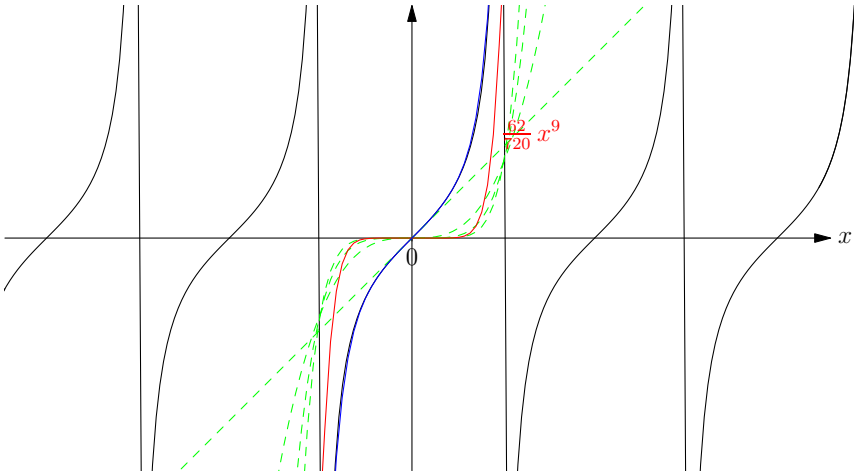
- The figure shows successive Taylor approximations to e^x around $x^* = 0$; each extra term extends the region where the approximation is good



2.2 Convergence

- Adding terms improves the approximation close to x^*, provided the higher derivatives exist
- Polynomials only diverge as $x \to \pm\infty$, so no polynomial can follow a function that blows up at a finite point, such as $1/x$, $1/(x-5)^2$ or $\tan(x)$

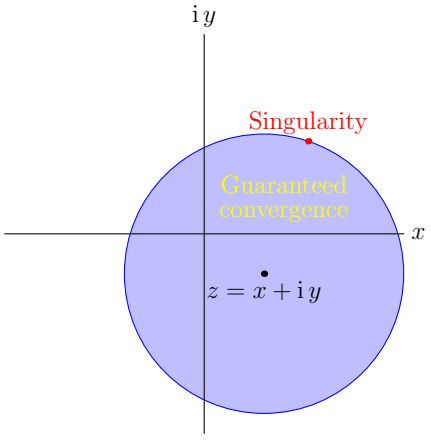
- For such functions the Taylor series only converges in a limited region. The lecture animation builds up the expansion of $\tan(x)$ term by term; the final frame is below. However many terms we add, the series fails beyond the singularities at $x = \pm\pi/2$



- To understand where a Taylor series converges we need to extend the function into the complex plane, $z = x + \mathrm{i}\,y$
- A function is *analytic* where its derivative

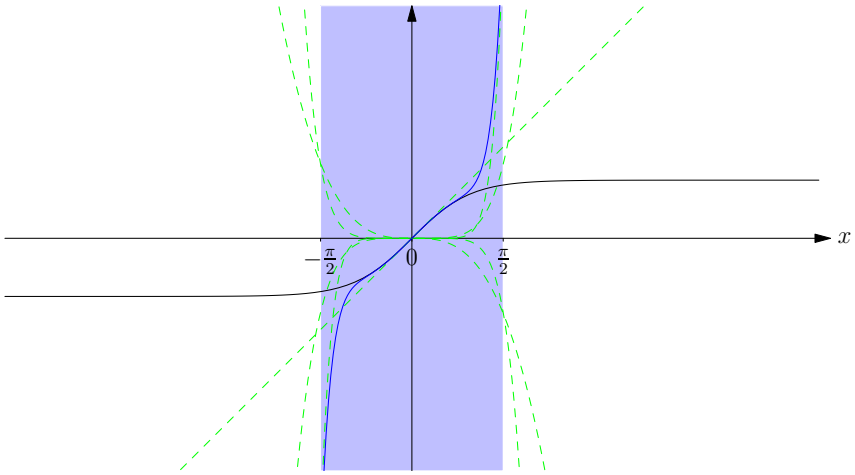
$$\frac{\mathrm{d}f(z)}{\mathrm{d}z} = \lim_{\epsilon \to 0} \frac{f(z+\epsilon) - f(z)}{\epsilon}$$

  is the same whichever direction the complex number ϵ approaches zero from
  - polynomials, exponentials, trigonometric functions, logarithms, ratios of polynomials and most "special functions" are analytic except at isolated singular points
- **Key result:** the Taylor series of an analytic function about z^* converges inside the largest circle centred on z^* that contains no singularity. The radius of that circle is the *radius of convergence*



- Singularities off the real axis matter even when the function looks perfectly well behaved for real x
  - $f(z) = 1/(z+\mathrm{i})^2$ is smooth for every real x, but it is singular at $z = -\mathrm{i}$, so its Taylor series about 0 only converges for $|x| < 1$
- **The hyperbolic tangent.** $\tanh(x)$ has no singularities on the real line. But $\tan(\mathrm{i}\,x) = \mathrm{i}\tanh(x)$, and $\tan$ is singular at $\pi/2 + k\pi$, so $\tanh$ is singular at $z = \pm\mathrm{i}\,\pi/2, \pm 3\mathrm{i}\,\pi/2, \ldots$ on the imaginary axis

- The expansion of $\tanh(x)$ about 0 therefore converges only for $|x| < \pi/2$, even though $\tanh$ itself is bounded everywhere



2.3 The Remainder

- A Taylor expansion comes with an exact error term. Stopping after n terms,

$$f(x) = \sum_{i=0}^{n-1} \frac{(x-x^*)^i}{i!} f^{(i)}(x^*) + \frac{(x-x^*)^n}{n!} f^{(n)}(\theta_x)$$

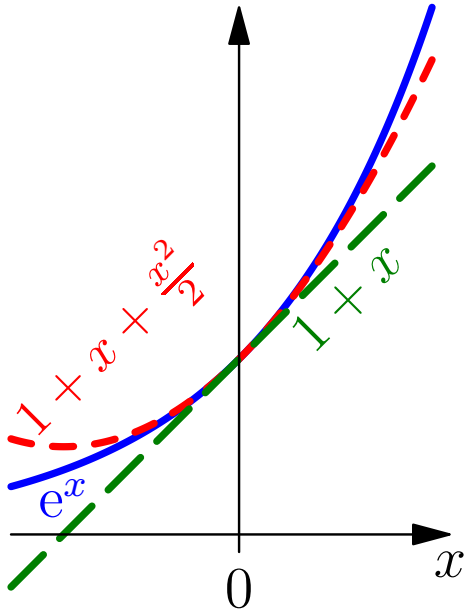
  where θ_x is some number between x^* and x

- This last term, the *remainder*, is very useful for proving inequalities, because we often know the sign of $f^{(n)}$ even when we don't know θ_x

- Expanding e^x about 0,

$$\mathrm{e}^x = 1 + x + \frac{x^2}{2}\,\mathrm{e}^{\theta_x}$$

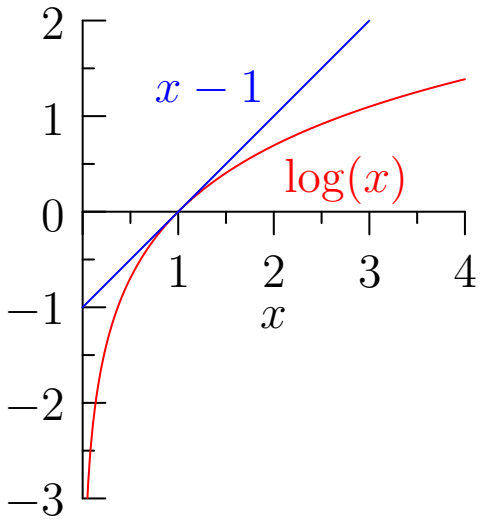
  and since $\mathrm{e}^{\theta_x} > 0$ we get the much-used bound $\mathrm{e}^x \geq 1 + x$ for all x



- Expanding $\log(x)$ about $x = 1$,

$$\log(x) = (x-1) - \frac{(x-1)^2}{2\,\theta^2}$$

so $\log(x) \leq x - 1$, or equivalently $\log(1 + x) \leq x$



- The proof of the remainder formula comes from integrating $f^{(n)}$ repeatedly; it was skipped in the lecture

2.4 High Dimensional Taylor Expansion

- For $f : \mathbb{R}^n \to \mathbb{R}$ we approximate by a multivariate polynomial and fix the coefficients by matching derivatives at $\boldsymbol{x}^*$ in the same way. To second order,

$$f(\boldsymbol{x}) \approx f(\boldsymbol{x}^*) + (\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T} \boldsymbol{\nabla} f(\boldsymbol{x}^*) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T} \mathbf{H} (\boldsymbol{x} - \boldsymbol{x}^*)$$

- $\mathbf{H}$ is the *Hessian*, the matrix of second derivatives

$$H_{ij} = \frac{\partial^2 f(\boldsymbol{x}^*)}{\partial x_i \, \partial x_j}$$

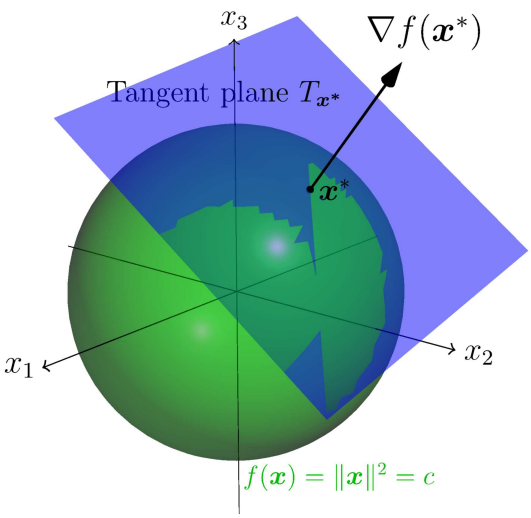
- For functions with continuous second derivatives the order of differentiation does not matter, so the Hessian is symmetric, $\mathbf{H} = \mathbf{H}^\mathsf{T}$
- Do not confuse the Hessian (second derivatives of a scalar function) with the Jacobian (first derivatives of a vector-valued function)

2.5 The Tangent Plane and Steepest Ascent

- For a smooth function the set $f(\boldsymbol{x}) = c$ is an $(n-1)$-dimensional surface, the high dimensional analogue of a contour line
- The *tangent plane* at $\boldsymbol{x}^*$ is the plane that touches this surface at $\boldsymbol{x}^*$. Moving within it changes f only at second order, so it is the set of points with

$$(\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T} \boldsymbol{\nabla} f(\boldsymbol{x}^*) = 0$$

that is, the tangent plane is orthogonal to the gradient



- Moving a small distance ϵ in a unit direction $\boldsymbol{u}$,

$$f(\boldsymbol{x}^* + \epsilon\,\boldsymbol{u}) = f(\boldsymbol{x}^*) + \epsilon\,\boldsymbol{u}^\mathsf{T}\boldsymbol{\nabla} f(\boldsymbol{x}^*) + O(\epsilon^2)$$

- By the Cauchy–Schwarz inequality $\boldsymbol{u}^\mathsf{T}\boldsymbol{\nabla} f \le \|\boldsymbol{\nabla} f\|$, with equality when $\boldsymbol{u}$ points along $\boldsymbol{\nabla} f$. So
  - $\boldsymbol{\nabla} f(\boldsymbol{x}^*)$ is the direction of *steepest ascent*
  - $-\boldsymbol{\nabla} f(\boldsymbol{x}^*)$ is the direction of *steepest descent*
- This is only a local statement: it holds while the $O(\epsilon^2)$ term is negligible

2.6 The Hessian at Stationary Points

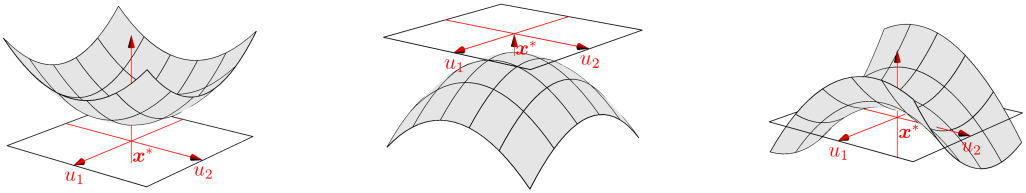
- At a *stationary point*, $\boldsymbol{\nabla} f(\boldsymbol{x}^*) = \mathbf{0}$, the first order term vanishes and

$$f(\boldsymbol{x}^* + \epsilon\,\boldsymbol{u}) = f(\boldsymbol{x}^*) + \frac{1}{2}\epsilon^2\,\boldsymbol{u}^\mathsf{T}\mathbf{H}\,\boldsymbol{u} + O(\epsilon^3)$$

  so the sign of $\boldsymbol{u}^\mathsf{T}\mathbf{H}\,\boldsymbol{u}$ decides whether f goes up or down
- $\mathbf{H}$ is real and symmetric, so it has real eigenvalues λ_i and orthonormal eigenvectors $\boldsymbol{v}_i$ ($\mathbf{H}\,\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i$, $\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = \delta_{ij}$)
- Writing any direction as $\boldsymbol{u} = \sum_i c_i\,\boldsymbol{v}_i$,

$$\boldsymbol{u}^\mathsf{T}\mathbf{H}\,\boldsymbol{u} = \left(\sum_i c_i\,\boldsymbol{v}_i^\mathsf{T}\right)\left(\sum_j c_j\,\lambda_j\,\boldsymbol{v}_j\right) = \sum_{i,j} \lambda_j\,c_i\,c_j\,\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = \sum_i \lambda_i\,c_i^2$$

- Hence
  - all $\lambda_i > 0$: $\mathbf{H}$ is *positive definite*, $\mathbf{H} \succ 0$, and $\boldsymbol{x}^*$ is a local minimum
  - all $\lambda_i < 0$: $\mathbf{H}$ is *negative definite*, $\mathbf{H} \prec 0$, and $\boldsymbol{x}^*$ is a local maximum
  - eigenvalues of both signs: $\boldsymbol{x}^*$ is a saddle point
  - if some $\lambda_i = 0$ the second order test is inconclusive and higher order terms decide



2.7 Finding Optima

- To minimise a loss $L(\boldsymbol{w})$ we look for $\boldsymbol{\nabla} L(\boldsymbol{w}^*) = \mathbf{0}$ with a positive definite Hessian
- Except for very simple losses there are many local minima, and $\boldsymbol{\nabla} L = \mathbf{0}$ cannot be solved analytically
- Instead we iterate, using gradient descent

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - r\, \boldsymbol{\nabla} L(\boldsymbol{w}^{(t)})$$

  which moves in the locally steepest downhill direction. Steps shrink as the gradient shrinks near a minimum. Choosing r well is subtle; we return to this in the optimisation lecture

3 Exercises

3.1 Radius of convergence

- Write down the Taylor expansion of $\log(1+x)$ about $x = 0$ up to the x^4 term. Without doing any further calculation, for what values of x must the series converge?

3.2 A bound from the remainder

- Use the remainder form of Taylor's expansion to show that $\cos(x) \geq 1 - x^2/2$ for all real x

3.3 Classifying a stationary point

- For $f(x, y) = x^2 + 3\,x\,y + y^2$ find the gradient and the Hessian. Show that the origin is the only stationary point and classify it

3.4 Quadratic functions

- For $f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\mathbf{A}\,\boldsymbol{x} - \boldsymbol{b}^{\mathsf{T}}\boldsymbol{x}$ with $\mathbf{A}$ symmetric, expand $f(\boldsymbol{x} + \epsilon\,\boldsymbol{u})$ exactly. Hence find the gradient and Hessian, and explain why the second order Taylor expansion of f is exact

4 Answers

4.1 Radius of convergence

- The derivatives are $f^{(k)}(x) = (-1)^{k-1}(k-1)!/(1+x)^k$, so

$$\log(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots$$

- $\log(1+z)$ is singular at $z = -1$, a distance 1 from the expansion point, so the series converges for $|x| < 1$

4.2 A bound from the remainder

- With $f(x) = \cos(x)$ we have $f(0) = 1$, $f'(0) = 0$ and $f''(\theta) = -\cos(\theta)$. Stopping after two terms,

$$\cos(x) = 1 + 0 \cdot x - \frac{x^2}{2}\cos(\theta_x)$$

- Since $\cos(\theta_x) \leq 1$ and $x^2 \geq 0$, it follows that $\cos(x) \geq 1 - x^2/2$

4.3 Classifying a stationary point

- $\nabla f = (2x + 3y,\ 3x + 2y)^\mathsf{T}$. Setting this to zero gives a linear system with determinant $2 \times 2 - 3 \times 3 = -5 \neq 0$, so the only solution is $x = y = 0$
- The Hessian is constant,

$$\mathbf{H} = \begin{pmatrix} 2 & 3 \\ 3 & 2 \end{pmatrix}$$

  with eigenvalues 2 ± 3, i.e. 5 and -1, and eigenvectors $(1, 1)/\sqrt{2}$ and $(1, -1)/\sqrt{2}$
- The eigenvalues have opposite signs, so the origin is a saddle point: f increases along $(1, 1)$ and decreases along $(1, -1)$

4.4 Quadratic functions

- Expanding,

$$\begin{aligned} f(\boldsymbol{x} + \epsilon\, \boldsymbol{u}) &= \frac{1}{2}(\boldsymbol{x} + \epsilon\, \boldsymbol{u})^\mathsf{T} \mathbf{A}\, (\boldsymbol{x} + \epsilon\, \boldsymbol{u}) - \boldsymbol{b}^\mathsf{T}(\boldsymbol{x} + \epsilon\, \boldsymbol{u}) \\ &= f(\boldsymbol{x}) + \epsilon\, \boldsymbol{u}^\mathsf{T}\, (\mathbf{A}\, \boldsymbol{x} - \boldsymbol{b}) + \frac{1}{2}\epsilon^2\, \boldsymbol{u}^\mathsf{T} \mathbf{A}\, \boldsymbol{u} \end{aligned}$$

  using $\boldsymbol{x}^\mathsf{T} \mathbf{A}\, \boldsymbol{u} = \boldsymbol{u}^\mathsf{T} \mathbf{A}^\mathsf{T} \boldsymbol{x} = \boldsymbol{u}^\mathsf{T} \mathbf{A}\, \boldsymbol{x}$ for symmetric $\mathbf{A}$
- Comparing with the Taylor expansion, $\nabla f = \mathbf{A}\, \boldsymbol{x} - \boldsymbol{b}$ and $\mathbf{H} = \mathbf{A}$
- There are no terms beyond ϵ^2, so the second order expansion is exact. This is why quadratics are such a useful model of a loss function near its minimum

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 4: Vector Spaces

Adam Prugel-Bennett

October 7, 2026

1 Keywords

- Fields, vector spaces, metrics, Lipschitz functions, norms, spectral norm, regularisation

2 Main Points

2.1 Scalars (Fields)

- A vector space is built over a set of *scalars*, formally a *field*: numbers we can add, subtract, multiply and divide (except by 0) with the usual rules
- The rationals, the reals and the complex numbers are fields. The integers are not, since 2 has no integer inverse
- We will almost always use the reals

2.2 Vector Spaces

- A set of objects $\mathcal{V}$, with addition and multiplication by a scalar, is a *vector space* if
  1. $a\,\boldsymbol{v} \in \mathcal{V}$ and $\boldsymbol{v} + \boldsymbol{w} \in \mathcal{V}$ for all $\boldsymbol{v}, \boldsymbol{w} \in \mathcal{V}$ (closure)
  2. $\boldsymbol{v} + \boldsymbol{w} = \boldsymbol{w} + \boldsymbol{v}$ (commutativity of addition)
  3. $(\boldsymbol{u} + \boldsymbol{v}) + \boldsymbol{w} = \boldsymbol{u} + (\boldsymbol{v} + \boldsymbol{w})$ (associativity of addition)
  4. $\boldsymbol{v} + \mathbf{0} = \boldsymbol{v}$ (additive identity $\mathbf{0}$)
  5. $\boldsymbol{v} + (-\boldsymbol{v}) = \mathbf{0}$ (additive inverse $-\boldsymbol{v}$)
  6. $1\,\boldsymbol{v} = \boldsymbol{v}$ (multiplicative identity)
  7. $a\,(b\,\boldsymbol{v}) = (a\,b)\,\boldsymbol{v}$ (associativity of scalar multiplication)
  8. $a\,(\boldsymbol{v} + \boldsymbol{w}) = a\,\boldsymbol{v} + a\,\boldsymbol{w}$ and $(a + b)\,\boldsymbol{v} = a\,\boldsymbol{v} + b\,\boldsymbol{v}$ (distributivity)
- Most of these hold automatically if addition and scalar multiplication behave like ordinary arithmetic. The one to watch is closure: adding two members, or scaling one, must not take you outside the set
- Column vectors in $\mathbb{R}^n$, matrices of a fixed shape, infinite sequences and functions on an interval all form vector spaces. Anything satisfying the axioms is a "vector"
- Bounded vectors (say $\|\boldsymbol{v}\| \leq 1$) do *not* form a vector space: $2\,\boldsymbol{v}$ may fall outside the set

2.3 Distances (Metrics)

- A *distance* or *metric* satisfies 4 conditions
  1. $d(\boldsymbol{x}, \boldsymbol{y}) \geq 0$ (non-negativity)
  2. $d(\boldsymbol{x}, \boldsymbol{y}) = 0$ if and only if $\boldsymbol{x} = \boldsymbol{y}$ (identity of indiscernibles)
  3. $d(\boldsymbol{x}, \boldsymbol{y}) = d(\boldsymbol{y}, \boldsymbol{x})$ (symmetry)
  4. $d(\boldsymbol{x}, \boldsymbol{y}) \leq d(\boldsymbol{x}, \boldsymbol{z}) + d(\boldsymbol{z}, \boldsymbol{y})$ (triangular inequality)
- You can define different distances for the same set of objects (e.g. Euclidean and Manhattan distance)
- Often we use *pseudo-metrics* that break one or other of the conditions

2.4 Mappings and Lipschitz Functions

- A function $f : \mathcal{V} \to \mathcal{W}$ maps one vector space to another (possibly the same one). Mappings compose: for matrices, $\mathbf{A}\,\mathbf{B}\,\boldsymbol{x} = \mathbf{A}\,(\mathbf{B}\,\boldsymbol{x})$ means "apply $\mathbf{B}$, then $\mathbf{A}$"
- A useful measure of how much a mapping can amplify changes is the *Lipschitz condition*: f is K-Lipschitz if, for all x and y,

  $$d(f(x), f(y)) \leq K\, d(x, y)$$

  for some fixed $K < \infty$
  - Lipschitz functions are continuous (they have no jumps), but not every continuous function is Lipschitz: $\sqrt{x}$ on $[0, 1]$ is continuous but infinitely steep at 0
  - For a differentiable $f : \mathbb{R} \to \mathbb{R}$, the smallest K is the largest value of $|f'(x)|$
- If $K < 1$ the mapping is *contractive*. The *Banach fixed-point theorem* says a contractive mapping (of $\mathbb{R}^n$, or any complete metric space, into itself) has exactly one fixed point $x^* = f(x^*)$, and iterating $x_{t+1} = f(x_t)$ from any start converges to it. Each step shrinks the distance to x^* by at least a factor K: $d(x_{t+1}, x^*) = d(f(x_t), f(x^*)) \leq K\, d(x_t, x^*)$. This is a standard way to prove that an iterative algorithm converges

2.5 Norms

- Norms provide a measure of the size of a vector. They satisfy three conditions
  1. $\|\boldsymbol{v}\| > 0$ if $\boldsymbol{v} \neq \mathbf{0}$ and $\|\mathbf{0}\| = 0$ (non-negativity)
  2. $\|a\,\boldsymbol{v}\| = |a|\,\|\boldsymbol{v}\|$ ("linearity")
  3. $\|\boldsymbol{u} + \boldsymbol{v}\| \leq \|\boldsymbol{u}\| + \|\boldsymbol{v}\|$ (triangular inequality)
- If not all of these conditions hold we have a *pseudo-norm*
- **Vector norms**
  - Common examples
    1. Euclidean or 2-norm: $\|\boldsymbol{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$
    2. p-norm ($p \geq 1$): $\|\boldsymbol{v}\|_p = (\sum_{i=1}^n |v_i|^p)^{1/p}$
    3. 1-norm: $\|\boldsymbol{v}\|_1 = \sum_{i=1}^n |v_i|$
    4. ∞-norm or max-norm: $\|\boldsymbol{v}\|_\infty = \max_i |v_i|$
  - The 1-, 2- and ∞-norms are all p-norms
  - The "0-norm", the number of non-zero components, is only a pseudo-norm, as $\|2\,\boldsymbol{v}\|_0 = \|\boldsymbol{v}\|_0 \neq 2\,\|\boldsymbol{v}\|_0$

- **Norms for regularisation.** When many weight vectors $\boldsymbol{w}$ fit the training data equally well we usually prefer a "small" one, so we minimise $\text{loss}(\boldsymbol{w}) + \eta \, \|\boldsymbol{w}\|$. This is equivalent to minimising the loss subject to $\|\boldsymbol{w}\| \leq c$: confining $\boldsymbol{w}$ to a *ball* of the chosen norm. Each norm means something different by small
  - $\|\boldsymbol{w}\|_2$ small: no component is large; the weight is spread out
  - $\|\boldsymbol{w}\|_1$ small: a few large components are tolerated if the rest are tiny. The 1-norm ball is a diamond with corners on the axes, and the optimum often lands on a corner where most components are exactly zero. This is why L_1 regularisation (the *lasso*) picks out a few features
  - $\|\boldsymbol{w}\|_\infty$ small: only the largest component is controlled
  - Choosing a norm is choosing what you mean by "small"
- **Matrix norms**
  - Examples
    1. The Frobenius norm is $\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} |A_{ij}|^2}$
    2. The *spectral norm* is the largest factor by which $\mathbf{A}$ can stretch a vector
    $$\|\mathbf{A}\|_2 = \max_{\boldsymbol{v} \neq \mathbf{0}} \frac{\|\mathbf{A}\,\boldsymbol{v}\|_2}{\|\boldsymbol{v}\|_2}$$
    For a symmetric matrix this is its largest $|\lambda|$. For a general matrix it is the largest singular value (see the SVD lecture), which can be bigger than any $|\lambda|$
    3. There are many others: the 1-norm, max-norm, nuclear norm, ...
  - The determinant is not a norm: it can be negative and does not scale correctly
  - Many (but not all) matrix norms satisfy
  $$\|\mathbf{A}\,\mathbf{B}\| \leq \|\mathbf{A}\| \times \|\mathbf{B}\|$$
  which lets us bound the norm of a product quickly
  - A matrix norm $\|\cdot\|_a$ and a vector norm $\|\cdot\|_b$ are *compatible* if
  $$\|\mathbf{M}\boldsymbol{v}\|_b \leq \|\mathbf{M}\|_a \times \|\boldsymbol{v}\|_b$$
  The spectral and Euclidean norms are compatible; this is just the definition of the spectral norm. The Frobenius and Euclidean norms are compatible too, since $\|\mathbf{M}\|_2 \leq \|\mathbf{M}\|_F$
  - The sensitivity of a mapping is measured by its *condition number* $\|\mathbf{M}\| \, \left\|\mathbf{M}^{-1}\right\|$: the ratio of its largest to smallest stretch (see the eigensystems notes)

2.6 The Lipschitz Constant of a Network

- A deep network applies layers in turn. Ignoring biases, a linear layer is $\boldsymbol{x}_n = \mathbf{L}_n \, \boldsymbol{x}_{n-1}$, interleaved with non-linearities such as ReLUs and pooling
- Lipschitz constants multiply under composition: if g is K_g-Lipschitz and f is K_f-Lipschitz then $\|f(g(\boldsymbol{x})) - f(g(\boldsymbol{y}))\| \leq K_f \, \|g(\boldsymbol{x}) - g(\boldsymbol{y})\| \leq K_f \, K_g \, \|\boldsymbol{x} - \boldsymbol{y}\|$
- By compatibility, a linear layer has $K = \|\mathbf{L}_n\|_2$. ReLUs and max-pooling never stretch distances (they can shrink them), so $K = 1$. Hence
$$K_{\text{net}} \leq \prod_n \|\mathbf{L}_n\|_2$$
This bound can be very loose (see Exercise 5)

- **Why care?** $\|f(\boldsymbol{x}+\boldsymbol{\delta})-f(\boldsymbol{x})\| \leq K\,\|\boldsymbol{\delta}\|$ bounds how much the output can move when the input is nudged
  - *Adversarial examples* are tiny $\boldsymbol{\delta}$ that change the classification; a network with small K is harder to fool
  - $K \gg 1$ lets signals (and gradients) explode; $K \ll 1$ lets them vanish
  - Wasserstein GANs need a critic with $K \leq 1$. One way to enforce this is *spectral normalisation*: divide each $\mathbf{L}_n$ by $\|\mathbf{L}_n\|_2$
  - Some bounds on how well a network generalises depend on K
  - But make K too small and the network is too smooth to fit the data

2.7 Function Norms

- The most common function norms, for f defined on a region $\mathcal{R}$, are
  1. The L_2-norm
  $$\|f\|_{L_2} = \sqrt{\int_{\boldsymbol{x}\in\mathcal{R}} f^2(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}}$$
  2. The L_1-norm
  $$\|f\|_{L_1} = \int_{\boldsymbol{x}\in\mathcal{R}} |f(\boldsymbol{x})|\,\mathrm{d}\boldsymbol{x}$$
  3. The ∞ or max-norm
  $$\|f\|_{\infty} = \max_{\boldsymbol{x}\in\mathcal{R}} |f(\boldsymbol{x})|$$
- Function norms also define vector spaces: the L_2 space is the set of functions with $\|f\|_{L_2} < \infty$, and similarly for L_1
- In these spaces two functions that differ only on a set of zero size (e.g. at a few isolated points) count as the same function, so that $\|f\| = 0$ only for the zero function

2.8 Norms and Metrics

- Every norm induces a distance $d(\boldsymbol{x},\boldsymbol{y}) = \|\boldsymbol{x}-\boldsymbol{y}\|$, so every normed space is also a metric space (Exercise 6). For example the Frobenius distance between matrices is $d_F(\mathbf{A},\mathbf{B}) = \|\mathbf{A}-\mathbf{B}\|_F$
- Next lecture adds an *inner product*, which gives us angles as well as lengths

3 Exercises

1. Which of these are vector spaces (with the usual addition and scalar multiplication)? Give a reason
   (a) Vectors in $\mathbb{R}^2$ with $x_1 \geq 0$
   (b) Polynomials of degree at most 3
   (c) Vectors in $\mathbb{R}^2$ with $x_1 + x_2 = 0$
   (d) Vectors in $\mathbb{R}^2$ with $x_1 + x_2 = 1$
2. For $\boldsymbol{v} = (3,-4,0,1)^{\mathsf{T}}$ compute $\|\boldsymbol{v}\|_1$, $\|\boldsymbol{v}\|_2$, $\|\boldsymbol{v}\|_\infty$ and $\|\boldsymbol{v}\|_0$. Check that $\|\boldsymbol{v}\|_\infty \leq \|\boldsymbol{v}\|_2 \leq \|\boldsymbol{v}\|_1$

3. Find a Lipschitz constant K, or show none exists, for
   (a) $f(x) = 3x + 1$ on $\mathbb{R}$
   (b) $f(x) = \sin(x)$ on $\mathbb{R}$
   (c) $f(x) = x^2$ on $\mathbb{R}$, and on $[-1, 1]$
   (d) $\mathrm{ReLU}(x) = \max(0, x)$ on $\mathbb{R}$
4. Show that $f(x) = x/2 + 1$ is contractive, find its fixed point, and iterate $x_{t+1} = f(x_t)$ from $x_0 = 0$. How fast does the error shrink?
5. Let $\mathbf{L}_1 = \begin{pmatrix} 3 & 0 \\ 0 & 0.5 \end{pmatrix}$ and $\mathbf{L}_2 = \begin{pmatrix} 0.5 & 0 \\ 0 & 3 \end{pmatrix}$
   (a) Find $\|\mathbf{L}_1\|_2$ and $\|\mathbf{L}_1\|_F$
   (b) Compare $\|\mathbf{L}_2 \mathbf{L}_1\|_2$ with the bound $\|\mathbf{L}_2\|_2 \, \|\mathbf{L}_1\|_2$. What does this say about the bound on K_{net}?
6. Show that $d(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|$ satisfies the four conditions for a metric

4 Experiments

4.1 Norms

- In python, compare the spectral norm with the largest stretch of many random unit vectors, and with the largest eigenvalue

```
import numpy as np

v = np.array([3, -4, 0, 1.0])
print([np.linalg.norm(v, p) for p in (1, 2, np.inf)])   # 1-, 2- and max-norms

rng = np.random.default_rng(0)
A = rng.normal(size=(3, 3))                  # not symmetric
u = rng.normal(size=(3, 100000))
u /= np.linalg.norm(u, axis=0)               # 100000 random unit vectors
print(np.linalg.norm(A @ u, axis=0).max())   # largest stretch found
print(np.linalg.norm(A, 2))                  # the spectral norm
print(np.abs(np.linalg.eigvals(A)).max())    # largest |eigenvalue|: smaller
print(np.linalg.norm(A, 'fro'))              # Frobenius norm: larger
```

- The largest stretch found is (just below) the spectral norm, about 1.82. The largest $|\lambda|$ is only about 1.54: for a non-symmetric matrix the eigenvalues do not tell you how much it can stretch a vector

5 Answers

1. Vector spaces?
   (a) No: $(1, 0)^\mathsf{T}$ is in the set but $-1 \times (1, 0)^\mathsf{T}$ is not (closure fails)
   (b) Yes: sums and multiples of polynomials of degree at most 3 are polynomials of degree at most 3, and the zero polynomial is in the set
   (c) Yes: if $x_1 + x_2 = 0$ and $y_1 + y_2 = 0$ then $(x_1 + y_1) + (x_2 + y_2) = 0$ and $a\,x_1 + a\,x_2 = 0$. It is a line through the origin
   (d) No: it does not contain $\mathbf{0}$, and adding two members gives $x_1 + x_2 = 2$

2. $\|\boldsymbol{v}\|_1 = 3+4+0+1 = 8$, $\|\boldsymbol{v}\|_2 = \sqrt{9+16+0+1} = \sqrt{26} \approx 5.10$, $\|\boldsymbol{v}\|_\infty = 4$ and $\|\boldsymbol{v}\|_0 = 3$. Indeed $4 \leq 5.10 \leq 8$

3. Lipschitz constants

   (a) $|f(x) - f(y)| = 3\,|x-y|$, so $K = 3$

   (b) $|f'(x)| = |\cos(x)| \leq 1$, so $K = 1$ (by the mean value theorem $|\sin x - \sin y| = |\cos \xi|\,|x-y|$ for some ξ)

   (c) $|x^2 - y^2| = |x+y|\,|x-y|$ and $|x+y|$ is unbounded on $\mathbb{R}$, so no K works. On $[-1,1]$, $|x+y| \leq 2$ so $K = 2$

   (d) $K = 1$: if $x, y \geq 0$ the difference is unchanged; if both are negative it becomes 0; if $x \geq 0 > y$ then $|\mathrm{ReLU}(x) - \mathrm{ReLU}(y)| = x < x - y$. ReLU never stretches but can shrink distances

4. $|f(x) - f(y)| = \frac{1}{2}|x-y|$, so $K = \frac{1}{2} < 1$. The fixed point solves $x = x/2 + 1$, i.e. $x^* = 2$. From $x_0 = 0$: $1, 1.5, 1.75, 1.875, \ldots$. The error $x^* - x_t = 2^{1-t}$ halves at every step, exactly the factor K

5. Spectral norms of layers

   (a) A diagonal matrix stretches the i-th axis by $|L_{ii}|$, so $\|\mathbf{L}_1\|_2 = 3$, while $\|\mathbf{L}_1\|_F = \sqrt{9 + 0.25} \approx 3.04$

   (b) $\mathbf{L}_2\,\mathbf{L}_1 = 1.5\,\mathbf{I}$, so $\|\mathbf{L}_2\mathbf{L}_1\|_2 = 1.5$, but the bound gives $3 \times 3 = 9$. Each layer stretches a different direction, and the second undoes the first. The product bound on K_{net} is safe but can be very pessimistic

6. Non-negativity and $d = 0 \Leftrightarrow \boldsymbol{x} = \boldsymbol{y}$ follow from norm condition 1. Symmetry: $\|\boldsymbol{x} - \boldsymbol{y}\| = \|(-1)(\boldsymbol{y} - \boldsymbol{x})\| = |-1|\,\|\boldsymbol{y} - \boldsymbol{x}\|$. Triangular inequality: $\|\boldsymbol{x} - \boldsymbol{y}\| = \|(\boldsymbol{x} - \boldsymbol{z}) + (\boldsymbol{z} - \boldsymbol{y})\| \leq \|\boldsymbol{x} - \boldsymbol{z}\| + \|\boldsymbol{z} - \boldsymbol{y}\|$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 5: Inner Product Spaces

Adam Prugel-Bennett

September 29, 2026

1 Keywords

- Inner-product, operators

2 Main Points

- For some vector spaces we can define an *inner product* between pairs of vectors
- Inner products are scalars associated with two elements in a vector space
- They are generally denoted by $\langle \boldsymbol{x}, \boldsymbol{y} \rangle$
- To be an inner-product requires satisfying 5 axioms
  1. $\langle \boldsymbol{x}, \boldsymbol{x} \rangle \geq 0$ for all $\boldsymbol{x} \in \mathcal{V}$
  2. $\langle \boldsymbol{x}, \boldsymbol{x} \rangle = 0$ if and only if $\boldsymbol{x} = \boldsymbol{0}$
  3. $\langle \alpha\, \boldsymbol{x}, \boldsymbol{y} \rangle = \alpha\, \langle \boldsymbol{x}, \boldsymbol{y} \rangle$
  4. $\langle \boldsymbol{x}, \boldsymbol{y} + \boldsymbol{z} \rangle = \langle \boldsymbol{x}, \boldsymbol{y} \rangle + \langle \boldsymbol{x}, \boldsymbol{z} \rangle$
  5. $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \langle \boldsymbol{y}, \boldsymbol{x} \rangle$
- The inner-product induces a norm: $\|x\| = \sqrt{\langle \boldsymbol{x}, \boldsymbol{x} \rangle}$
- For normal vectors (i.e. $\boldsymbol{x} \in \mathbb{R}^n$) the standard inner product is the dot-product

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \boldsymbol{x}^\mathsf{T} \boldsymbol{y} = \sum_{i=1}^{n} x_i\, y_i$$

- We can define an inner product between functions

$$\langle f, g \rangle = \int_{x \in \mathcal{I}} f(x)\, g(x)\, \mathrm{d}x$$

- For matrices we can define the Frobenius inner-product

$$\langle \mathbf{A}, \mathbf{B} \rangle_F = \operatorname{Tr} \mathbf{A}^\mathsf{T} \mathbf{B} = \sum_{i,j} A_{ij}\, B_{ij}$$

  which defines the Frobenius norm

$$\|\mathbf{A}\|_F = \sqrt{\langle \mathbf{A}, \mathbf{A} \rangle_F} = \sqrt{\sum_{i,j} A_{ij}^2}$$

  The Frobenius norm is compatible with the Euclidean norm ($\|\mathbf{A}\boldsymbol{x}\| \leq \|\mathbf{A}\|_F\, \|\boldsymbol{x}\|$), but it is not *induced* by any vector norm, unlike the spectral norm.

- A really important inequality is the *Cauchy-Schwarz* inequality

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle^2 \leq \langle \boldsymbol{x}, \boldsymbol{x} \rangle \ \langle \boldsymbol{y}, \boldsymbol{y} \rangle = \|\boldsymbol{x}\|^2 \ \|\boldsymbol{y}\|^2$$

- Inner products allow us to define the notion of similarity

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \|\boldsymbol{x}\| \, \|\boldsymbol{y}\| \, \cos(\theta)$$
$$\langle f(x), g(x) \rangle = \|f(x)\| \, \|g(x)\| \, \cos(\theta)$$

- $\cos(\theta)$ can be seen as a measure of the correlation between vectors (or functions)
- Because of Cauchy-Schwarz $\langle \boldsymbol{x}, \boldsymbol{y} \rangle / (\|\boldsymbol{x}\| \, \|\boldsymbol{y}\|)$ lies between -1 and 1 (so that we can represent this quantity by the cosine of an angle)

2.1 Coordinates or Basis Vectors

- Any set of vectors that span the entire vector space can be considered a set of basis vectors or coordinates
- If our bases are linearly independent then we have a set of non-degenerate basis functions where each vector is unique
- The most convenient set of basis vectors are those where the bases are normalised and orthogonal $\langle \boldsymbol{b}_i, \boldsymbol{b}_j \rangle = \delta_{ij}$
- **Basis Functions**
  - For a function space we can consider a set of basis functions
  - A familiar set of functions define on the interval $[0, 2\,\pi]$ are the Fourier functions

$$\{1, \cos(\theta), \sin(\theta), \cos(2\,\theta), \sin(2\,\theta), \cdots\}$$

  - This set is orthogonal: $\langle b_i(\theta), b_j(\theta) \rangle = 0$ whenever $i \neq j$. It is not normalised — for example $\langle 1, 1 \rangle = 2\pi$ while $\langle \cos\theta, \cos\theta \rangle = \pi$ — so divide each by its own norm to get an *orthonormal* set
  - There are many orthogonal polynomials that have similar properties
  - Given an *orthonormal* set of functions $\{b_i(\boldsymbol{x})\}$ we can decompose a function $f(\boldsymbol{x})$ as an (infinite) vector $\boldsymbol{f}$ with components $f_i = \langle f(\boldsymbol{x}), b_i(\boldsymbol{x}) \rangle$. If the set is orthogonal but not normalised the components are $f_i = \langle f, b_i \rangle / \langle b_i, b_i \rangle$
  - This allows us to represent any function as a point in an infinite-dimensional space

2.2 Projection Operators

- Given a vector $\boldsymbol{a}$ we can ask how much of it lies along another vector $\boldsymbol{b}$

$$\mathrm{proj}_{\boldsymbol{b}}(\boldsymbol{a}) = \frac{\langle \boldsymbol{a}, \boldsymbol{b} \rangle}{\langle \boldsymbol{b}, \boldsymbol{b} \rangle} \, \boldsymbol{b}$$

- Read this as two steps: the scalar $\langle \boldsymbol{a}, \boldsymbol{b} \rangle \, / \, \langle \boldsymbol{b}, \boldsymbol{b} \rangle$ says how many copies of $\boldsymbol{b}$ are needed, and multiplying by $\boldsymbol{b}$ puts the answer back in the direction of $\boldsymbol{b}$
- If $\boldsymbol{b}$ is normalised ($\|\boldsymbol{b}\| = 1$) this is just $\langle \boldsymbol{a}, \boldsymbol{b} \rangle \ \boldsymbol{b}$
- Using $\langle \boldsymbol{a}, \boldsymbol{b} \rangle = \|\boldsymbol{a}\| \, \|\boldsymbol{b}\| \cos(\theta)$ the length of the projection is $\|\boldsymbol{a}\| \cos(\theta)$, which is the picture you would draw

- The defining property of a projection is that doing it twice changes nothing — it is *idempotent*

$$\mathrm{proj}_{\boldsymbol{b}}\left(\mathrm{proj}_{\boldsymbol{b}}(\boldsymbol{a})\right) = \mathrm{proj}_{\boldsymbol{b}}(\boldsymbol{a})$$

  which follows because $\mathrm{proj}_{\boldsymbol{b}}(\boldsymbol{a})$ already points along $\boldsymbol{b}$, so projecting it again leaves it alone

- The **remainder** is what matters for what follows: $\boldsymbol{a} - \mathrm{proj}_{\boldsymbol{b}}(\boldsymbol{a})$ is orthogonal to $\boldsymbol{b}$, since

$$\left\langle \boldsymbol{a} - \frac{\langle \boldsymbol{a}, \boldsymbol{b}\rangle}{\langle \boldsymbol{b}, \boldsymbol{b}\rangle}\boldsymbol{b}, \boldsymbol{b} \right\rangle = \langle \boldsymbol{a}, \boldsymbol{b}\rangle - \frac{\langle \boldsymbol{a}, \boldsymbol{b}\rangle}{\langle \boldsymbol{b}, \boldsymbol{b}\rangle}\langle \boldsymbol{b}, \boldsymbol{b}\rangle = 0$$

- For vectors in $\mathbb{R}^n$ the projection is a matrix, $\mathbf{P} = \boldsymbol{b}\,\boldsymbol{b}^{\mathsf{T}}/(\boldsymbol{b}^{\mathsf{T}}\boldsymbol{b})$, and idempotency reads $\mathbf{P}^2 = \mathbf{P}$

- To project onto a whole subspace spanned by an orthonormal set $\{\boldsymbol{e}_1, \ldots, \boldsymbol{e}_k\}$ just add the projections, $\mathbf{P} = \sum_{i=1}^{k} \boldsymbol{e}_i \boldsymbol{e}_i^{\mathsf{T}}$. This is the operator that appears again in PCA

2.3 Gram-Schmidt Orthogonalisation

- Projections let us turn any set of linearly independent vectors $(\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n)$ into an orthonormal basis

- Take each vector in turn and subtract off everything it shares with the directions already found; normalise what is left

$$\begin{aligned}
\boldsymbol{u}_1 &= \boldsymbol{a}_1, & \boldsymbol{e}_1 &= \frac{\boldsymbol{u}_1}{\|\boldsymbol{u}_1\|} \\
\boldsymbol{u}_2 &= \boldsymbol{a}_2 - \mathrm{proj}_{\boldsymbol{u}_1}(\boldsymbol{a}_2), & \boldsymbol{e}_2 &= \frac{\boldsymbol{u}_2}{\|\boldsymbol{u}_2\|} \\
\boldsymbol{u}_3 &= \boldsymbol{a}_3 - \mathrm{proj}_{\boldsymbol{u}_1}(\boldsymbol{a}_3) - \mathrm{proj}_{\boldsymbol{u}_2}(\boldsymbol{a}_3), & \boldsymbol{e}_3 &= \frac{\boldsymbol{u}_3}{\|\boldsymbol{u}_3\|}
\end{aligned}$$

  and in general $\boldsymbol{u}_k = \boldsymbol{a}_k - \sum_{i<k} \mathrm{proj}_{\boldsymbol{u}_i}(\boldsymbol{a}_k)$

- **The projections are onto the $\boldsymbol{u}_i$, not the $\boldsymbol{a}_i$.** This is the easy mistake. Subtracting $\mathrm{proj}_{\boldsymbol{a}_2}(\boldsymbol{a}_3)$ instead of $\mathrm{proj}_{\boldsymbol{u}_2}(\boldsymbol{a}_3)$ does *not* leave something orthogonal to both, because $\boldsymbol{a}_2$ is not itself orthogonal to $\boldsymbol{a}_1$. Only $\boldsymbol{u}_2 = \boldsymbol{a}_2$ when $k = 2$, which is why the second step looks the same either way

- Each $\boldsymbol{u}_k$ is orthogonal to everything before it by construction: it is a remainder, and we showed above that remainders are orthogonal to what was subtracted

- If the $\boldsymbol{a}_i$ are *not* linearly independent some $\boldsymbol{u}_k$ comes out as $\mathbf{0}$, and the process tells you the set was degenerate

- The same algorithm works in any inner-product space. Applied to $1, x, x^2, \ldots$ with $\langle f, g\rangle = \int f g \,\mathrm{d}x$ it generates the orthogonal polynomials

- In $\mathbb{R}^n$ it is numerically poor: rounding errors accumulate and the later vectors drift away from orthogonality. Numerical libraries compute the same factorisation with Householder reflections instead. Gram-Schmidt is how you *understand* the result, not how you compute it

2.4 Operators

- Operators transform elements of a vector space

- Consider the transformation or operator $\mathcal{T} : \mathcal{V} \to \mathcal{V}'$

- This says that $\mathcal{T}$ maps some object $\boldsymbol{x} \in \mathcal{V}$ to an object $\boldsymbol{y} = \mathcal{T}[\boldsymbol{x}]$ in a new vector space $\mathcal{V}'$
- **Linear Operators**
  - Linear operators satisfy the two conditions
    1. $\mathcal{T}[a\,\boldsymbol{x}] = a\,\mathcal{T}[\boldsymbol{x}]$
    2. $\mathcal{T}[\boldsymbol{x} + \boldsymbol{y}] = \mathcal{T}[\boldsymbol{x}] + \mathcal{T}[\boldsymbol{y}]$
  - Linear operators are really important because we can understand them
  - For normal vectors the most general linear operation is

    $$\mathcal{T}[\boldsymbol{x}] = \mathbf{M}\,\boldsymbol{x}$$

    where $\mathbf{M}$ is a matrix
  - For functions the most general linear operator is a kernel function

    $$g(\boldsymbol{x}) = \mathcal{T}[f(\boldsymbol{x})] = \int K(\boldsymbol{x}, \boldsymbol{y})\, f(\boldsymbol{y})\, \mathrm{d}\boldsymbol{y}$$

    - Kernels appear in SVMs, SVRs, kernel-PCA, Gaussian Processes
- Often we are interested in operators that map vectors in a vector space to new vectors in the same vector space
  - $\mathcal{T} : \mathcal{V} \to \mathcal{V}$
  - The most general linear mapping for normal vectors that does this is a square matrix

2.5 Matrices and Mappings

- Matrices, $\mathbf{M}$ are linear maps from one point $\boldsymbol{x}$ to another $\boldsymbol{y} = \mathbf{M}\boldsymbol{x}$
- The product of a matrix $\mathbf{C} = \mathbf{AB}$ corresponds to applying the mapping $\mathbf{B}$ followed by $\mathbf{A}$
- For most matrices $\mathbf{AB} \neq \mathbf{BA}$ (we say matrix multiplication does not commute)
- There are pairs of matrices that do commute, but they need to share a special structure for this to happen
- Matrix multiplication is associative. That is, $(\mathbf{A}\,\mathbf{B})\mathbf{C} = \mathbf{A}(\mathbf{B}\,\mathbf{C})$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 6: Understand Mappings

Adam Prugel-Bennett

October 1, 2026

1 Keywords

- Least Squares, Under-constrained Systems, Ill-conditioning

2 Main Points

2.1 Inverse Problems

- Much of machine learning can be viewed as solving an inverse problem
- We collect data about the world by performing a series of measurements
- Our task is to infer properties of the world from the data
- In linear regression we have m observations $(\boldsymbol{x}_k, y_k)$ and want p unknown weights $\boldsymbol{w}$ such that $y_k = \boldsymbol{x}_k^\mathsf{T}\boldsymbol{w}$
- Stacking the feature vectors $\boldsymbol{x}_k^\mathsf{T}$ as the rows of the *design matrix* $\mathbf{X}$ (an $m \times p$ matrix) we can write this as $\boldsymbol{y} = \mathbf{X}\boldsymbol{w}$
- If $m = p$ and $\mathbf{X}$ is invertible then $\boldsymbol{w} = \mathbf{X}^{-1}\boldsymbol{y}$, but this is rarely the situation we are in

2.2 Least Squares (Over-Constrained Problems)

- We can have contradictory data that our model cannot explain
- This may arise because
  - We have errors in the data
  - Our data contains insufficient information
  - Our model is too simple
- If we have more training data than free parameters ($m > p$) this is likely to occur
- We typically solve this by introducing a loss function we minimise. The classic example is the squared loss

$$L(\boldsymbol{w}) = \sum_{k=1}^{m} \left(\boldsymbol{x}_k^\mathsf{T}\boldsymbol{w} - y_k\right)^2 = \|\mathbf{X}\,\boldsymbol{w} - \boldsymbol{y}\|^2$$

  This was introduced by Gauss to predict the orbit of the asteroid Ceres
- Setting the gradient to zero (see Exercise 1) gives

$$\boldsymbol{w} = \left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} = \mathbf{X}^{+}\boldsymbol{y}$$

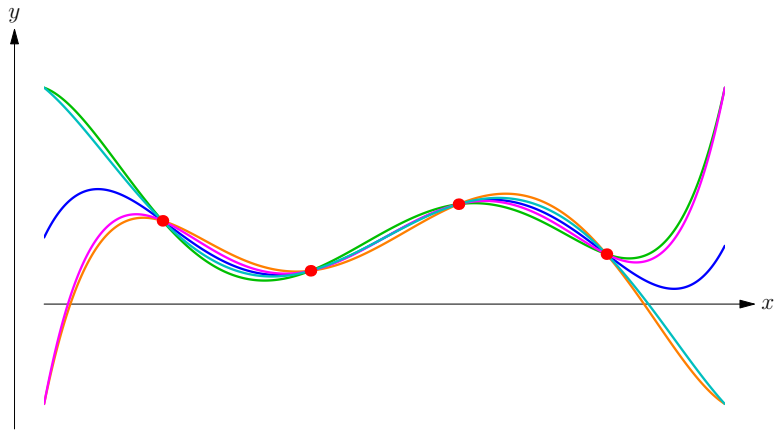
  where $\mathbf{X}^{+} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}$ is the pseudo-inverse
- This requires $\mathbf{X}^\mathsf{T}\mathbf{X}$ to be invertible, which is the case when the columns of $\mathbf{X}$ are linearly independent

2.3 Under-Constrained Problems

- We can also be in a situation when many models (learning machines) explain the data
- This will typically happen when we have more free parameters than data ($m < p$), or when we are unlucky and the training examples don't span the full space
- In linear regression $\mathbf{X}^\mathsf{T}\mathbf{X}$ is then singular (see Exercise 3), so the pseudo-inverse formula fails. The problem is said to be *ill-posed*
- As an example, consider fitting a polynomial of degree 5, $y = w_0 + w_1\,x + \cdots + w_5\,x^5$ (6 parameters), to 4 data points (x_i, y_i). If $p_0(x)$ is the cubic through the four points then

$$p(x) = p_0(x) + (a + b\,x)\,(x - x_1)(x - x_2)(x - x_3)(x - x_4)$$

  passes through all four points for *every* choice of a and b (Exercise 4). We have 6 unknowns and 4 equations, leaving 2 free directions. The figure shows five members of this family. They agree near the data but make wildly different predictions away from it



- Here we have to choose a particular model. To do this requires (implicitly or explicitly) making additional assumptions
- Ockham's razor suggests choosing the "simplest" solution, although what counts as simplest is a choice we make
- For linear regression, adding an L_2 regulariser $\eta\ \|\boldsymbol{w}\|^2$ to the loss makes the solution unique. We will see exactly how this works when we meet the singular value decomposition
- For high-dimensional inputs we can be over-constrained in some directions and under-constrained in others

2.4 Ill-Conditioning

- Even when we are not under-constrained our inverse can be very sensitive to the data
- That is, small errors can be strongly magnified
- A matrix that is close to singular is ill-conditioned: some of its eigenvalues are tiny compared with the largest, so it squashes space towards a lower-dimensional subspace. Inverting it stretches those directions back out, magnifying any noise in them
- A worked example: take two data points with two nearly identical features

$$\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1.001 \end{pmatrix}, \qquad \boldsymbol{y} = \begin{pmatrix} 2 \\ 2.001 + \delta \end{pmatrix}$$

Solving $\mathbf{X}\,\boldsymbol{w} = \boldsymbol{y}$, subtracting the first equation from the second gives $0.001\,w_2 = 0.001 + \delta$, so

$$\boldsymbol{w} = \begin{pmatrix} 1 - 1000\,\delta \\ 1 + 1000\,\delta \end{pmatrix}$$

- A change δ in the data produces a change $1000\,\delta$ in the weights. With $\delta = 0.001$ (a 0.05% change in y_2) the weights jump from $(1, 1)$ to $(0, 2)$ and the prediction for a new input $\boldsymbol{x} = (1, 0)^\mathsf{T}$ goes from 1 to 0
- The weights always move along $(-1, 1)^\mathsf{T}$. This is (very nearly) the eigenvector of $\mathbf{X}$ with the small eigenvalue (≈ 0.0005, against ≈ 2 for the other one)
- The data hardly constrain the weights in this direction: the two features are so alike that the data cannot tell us how to share the weight between them

- Common causes of ill-conditioning in machine learning
  - Roughly as many data points as parameters
  - Nearly collinear features (e.g. floor area and number of rooms)
  - Polynomial features $x, x^2, \ldots, x^9$ for $x \in [0, 1]$, which all look alike (see the experiments)
  - Features on very different scales (easily fixed by standardising each feature to zero mean and unit variance)
- Ill-conditioning leads to high variance in the bias-variance sense and hence poor generalisation
- Regularisers are one way of improving the conditioning

2.5 Least Squares in Practice

- The formula $\boldsymbol{w} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$ is for understanding, not for computing
- Use a least squares solver such as `np.linalg.lstsq(X, y)`, which works with $\mathbf{X}$ directly
- Forming $\mathbf{X}^\mathsf{T}\mathbf{X}$ squares the ratio of the largest to smallest eigenvalues (the *condition number*, defined properly in the next lecture). If that ratio is 10^8 for $\mathbf{X}$ it becomes 10^{16} for $\mathbf{X}^\mathsf{T}\mathbf{X}$, beyond the roughly 16 significant figures of a double-precision number
- When $m < p$, `lstsq` returns the solution with the smallest $\|\boldsymbol{w}\|$, which is one version of the "simplest" solution. We will see why when we meet the singular value decomposition

2.6 Three Regimes

Regime	Exact fit?	Unique?	Method
$m > p$ (over-constrained)	usually not	yes (usually)	minimise $\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$
$m = p$	yes	yes (usually)	$\boldsymbol{w} = \mathbf{X}^{-1}\boldsymbol{y}$
$m < p$ (under-constrained)	yes, many	no	choose the "simplest"

- "Usually" means provided the columns of $\mathbf{X}$ are linearly independent
- Real problems mix these: over-constrained in some directions and (nearly) under-constrained in others. The nearly under-constrained directions are what make a problem ill-conditioned

3 Exercises

1. Derive the formula $\boldsymbol{w} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$ for the weight vector in linear regression by minimising $\|\mathbf{X}\,\boldsymbol{w} - \boldsymbol{y}\|^2$
2. By writing $\boldsymbol{w}^\mathsf{T}\mathbf{M}\,\boldsymbol{w} = \sum_{i,j} w_i\, M_{ij}\, w_j$ and differentiating with respect to w_k, show that $\boldsymbol{\nabla}\, \boldsymbol{w}^\mathsf{T}\mathbf{M}\,\boldsymbol{w} = (\mathbf{M} + \mathbf{M}^\mathsf{T})\,\boldsymbol{w}$
3. Show that if $\mathbf{X}$ is an $m \times p$ matrix with $m < p$ then $\mathbf{X}^\mathsf{T}\mathbf{X}$ is singular. (Hint: find a non-zero vector $\boldsymbol{v}$ with $\mathbf{X}\,\boldsymbol{v} = \mathbf{0}$)
4. For the polynomial example in the notes
   (a) Show that $p(x)$ passes through all four data points for any a and b, and that it is a polynomial of degree at most 5
   (b) (Harder) Show that *every* polynomial of degree at most 5 through the four points has this form
5. For the worked example with $\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1.001 \end{pmatrix}$
   (a) Verify the solution $\boldsymbol{w} = (1 - 1000\,\delta, 1 + 1000\,\delta)^\mathsf{T}$
   (b) Find the eigenvalues of $\mathbf{X}$ to 4 significant figures and their ratio
   (c) What happens to the weights if instead the two features were identical, $\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$?

4 Experiments

4.1 Ill-Conditioning and Under-Constrained Systems

- Using python (with numpy), try the following and check you understand the output

```
import numpy as np

# 1. The worked example: a tiny change in y, a big change in w
X = np.array([[1, 1], [1, 1.001]])
for y2 in [2.001, 2.002, 2.0005]:
    print(y2, np.linalg.solve(X, [2, y2]))
print(np.linalg.eigh(X))   # eigenvalues and eigenvectors (X is symmetric)

# 2. Polynomial features are ill-conditioned
x = np.linspace(0, 1, 50)
for degree in [1, 3, 5, 9]:
    Xp = np.vander(x, degree + 1, increasing=True)  # columns 1, x, x^2, ...
    print(degree, np.linalg.cond(Xp), np.linalg.cond(Xp.T @ Xp))

# 3. An under-constrained system: 3 equations, 6 unknowns
rng = np.random.default_rng(1)
X = rng.normal(size=(3, 6))
y = rng.normal(size=3)
w, *_ = np.linalg.lstsq(X, y)
print(np.allclose(X @ w, y))                    # fits the data exactly
print(np.linalg.eigvalsh(X.T @ X))              # three (numerically) zero eigenvalues
v = np.linalg.svd(X)[2][-1]                     # a direction with X v = 0
print(np.allclose(X @ (w + 5 * v), y))          # ...so this also fits exactly
print(np.linalg.norm(w), np.linalg.norm(w + 5 * v))  # but a larger norm
```

- In part 2, `np.linalg.cond` computes the condition number (covered next lecture). Notice that the condition number of $\mathbf{X}^\mathsf{T}\mathbf{X}$ is the square of that of $\mathbf{X}$, reaching about 10^{13} for degree 9
- In part 3, try adding other multiples of `v`. Why can the norm never be smaller than that of `w`?

5 Answers

1. Using $\|\boldsymbol{a}\|^2 = \boldsymbol{a}^\mathsf{T}\boldsymbol{a}$ and $\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y} = \boldsymbol{y}^\mathsf{T}\mathbf{X}\,\boldsymbol{w}$ (a number equals its own transpose)

$$\|\mathbf{X}\,\boldsymbol{w} - \boldsymbol{y}\|^2 = \boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\,\boldsymbol{w} - 2\,\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y} + \boldsymbol{y}^\mathsf{T}\boldsymbol{y}$$

   As $\mathbf{X}^\mathsf{T}\mathbf{X}$ is symmetric, Exercise 2 gives the gradient $2\,(\mathbf{X}^\mathsf{T}\mathbf{X}\,\boldsymbol{w} - \mathbf{X}^\mathsf{T}\boldsymbol{y})$. Setting this to zero gives $\mathbf{X}^\mathsf{T}\mathbf{X}\,\boldsymbol{w} = \mathbf{X}^\mathsf{T}\boldsymbol{y}$ and, if $\mathbf{X}^\mathsf{T}\mathbf{X}$ is invertible, the result

2. $$\frac{\partial}{\partial\, w_k}\sum_{i,j} w_i\, M_{ij}\, w_j = \sum_j M_{kj}\, w_j + \sum_i w_i\, M_{ik} = (\mathbf{M}\,\boldsymbol{w})_k + (\mathbf{M}^\mathsf{T}\boldsymbol{w})_k$$

3. The p columns of $\mathbf{X}$ are vectors in $\mathbb{R}^m$ with $m < p$, so they are linearly dependent: some non-zero $\boldsymbol{v}$ has $\mathbf{X}\,\boldsymbol{v} = \mathbf{0}$. Then $\mathbf{X}^\mathsf{T}\mathbf{X}\,\boldsymbol{v} = \mathbf{0}$, so $\boldsymbol{v}$ is an eigenvector of $\mathbf{X}^\mathsf{T}\mathbf{X}$ with eigenvalue 0 and $\mathbf{X}^\mathsf{T}\mathbf{X}$ has no inverse. Any multiple of $\boldsymbol{v}$ can be added to a solution without changing the fit

4. Polynomial example
   (a) At each data point one factor $(x - x_i)$ vanishes, so $p(x_i) = p_0(x_i) = y_i$. The degree is at most $1 + 4 = 5$
   (b) If $r(x)$ is another such polynomial then $r(x) - p_0(x)$ has degree at most 5 and is zero at $x_1, \ldots, x_4$, so it is divisible by $(x - x_1)\cdots(x - x_4)$. The quotient has degree at most 1, i.e. it is $a + b\,x$ for some a and b

5. Worked example
   (a) Substitute back: $(1{-}1000\,\delta){+}(1{+}1000\,\delta) = 2$ and $(1{-}1000\,\delta){+}1.001\,(1{+}1000\,\delta) = 2.001{+}\delta$
   (b) The eigenvalues satisfy $\lambda^2 - 2.001\,\lambda + 0.001 = 0$, giving $\lambda \approx 2.001$ and $\lambda \approx 0.0004999$, a ratio of about 4000
   (c) $\mathbf{X}$ is singular (eigenvalues 2 and 0). If $y_1 = y_2$ there are infinitely many solutions, $w_1 + w_2 = y_1$; otherwise there is no exact solution. The ill-conditioned problem has become under-constrained in the $(-1, 1)^\mathsf{T}$ direction

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 7: Eigensystems

Adam Prugel-Bennett

October 7, 2026

1 Keywords

- Eigenvectors, Orthogonal Matrices, Eigenvector Decomposition, Condition Number, Rank, Low Rank Approximation, Non-Normal Matrices

2 Main Points

2.1 Eigen-Systems

- We can understand ill-conditioning for linear regression by the eigen-decomposition of $\mathbf{M} = \mathbf{X}^\mathsf{T}\mathbf{X}$
- This should be revision
- You know that an *eigenvector*, $\boldsymbol{v}$, satisfies $\mathbf{M}\,\boldsymbol{v} = \lambda\,\boldsymbol{v}$
- For a symmetric matrix there are n real orthogonal eigenvectors
- You can prove they are orthogonal
- **Orthogonal Matrices**
  - Putting the n eigenvectors into a matrix $\mathbf{V}$ with columns $\boldsymbol{v}_i$ we obtain an orthogonal matrix
  - The defining property of an orthogonal matrix is $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{I}$
  - They correspond to rotations (with a possible reflection): they preserve lengths and angles, since $(\mathbf{V}\boldsymbol{x})^\mathsf{T}(\mathbf{V}\boldsymbol{y}) = \boldsymbol{x}^\mathsf{T}\mathbf{V}^\mathsf{T}\mathbf{V}\,\boldsymbol{y} = \boldsymbol{x}^\mathsf{T}\boldsymbol{y}$
  - $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{I}$ follows directly from the eigenvectors being orthonormal. $\mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{I}$ then follows because $\mathbf{V}$ is invertible: if $\mathbf{V}\boldsymbol{x} = \mathbf{0}$ then $\boldsymbol{x} = \mathbf{V}^\mathsf{T}\mathbf{V}\boldsymbol{x} = \mathbf{0}$
- **Matrix Decomposition**
  - We can decompose a symmetric matrix, $\mathbf{M}$ as

$$\mathbf{M} = \mathbf{V}\,\mathbf{\Lambda}\,\mathbf{V}^\mathsf{T}$$

  - Where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues of $\mathbf{M}$ (i.e. $\Lambda_{ii} = \lambda_i$)
  - $\mathbf{V}$ is the orthogonal matrix made up of the eigenvectors of $\mathbf{M}$
  - We can interpret the mapping of a symmetric matrix $\mathbf{M}$ as equivalent to
    1. a rotation defined by $\mathbf{V}^\mathsf{T}$
    2. scaling of the ith component by λ_i and
    3. a rotation backwards given by $\mathbf{V}$

- Equivalently if we work in a coordinate system defined by the eigenvectors of $\mathbf{M}$ (this forms an orthonormal basis set) then we just rescale in the directions $\boldsymbol{v}_i$ by λ_i
  - A symmetric matrix just squashes or expands in different orthogonal directions (this is what the eigensystem captures)

- **Inverse Matrices**
  - The inverse of a symmetric matrix $\mathbf{M}$ is given by

$$\mathbf{M}^{-1} = \mathbf{V}\,\mathbf{\Lambda}^{-1}\,\mathbf{V}^{\mathsf{T}}$$

  - Where $\mathbf{\Lambda}^{-1}$ is a diagonal matrix with elements $\Lambda_{ii}^{-1} = 1/\lambda_i$
  - This is only defined if all the eigenvalues of $\mathbf{M}$ are non-zero ($\mathbf{M}$ is said to be full rank)
  - If λ_i is very small then $1/\lambda_i$ is large and in taking the inverse $\mathbf{M}^{-1}\boldsymbol{x}$ any component of $\boldsymbol{x}$ in the direction $\boldsymbol{v}_i$ will get magnified by $1/\lambda_i$
  - For linear regression we invert $\mathbf{M} = \mathbf{X}^{\mathsf{T}}\mathbf{X}$
    - in directions where the training examples don't vary much the associated eigenvalue will be small and the inverse inherently unstable

2.2 Condition Number

- Suppose we solve $\mathbf{M}\,\boldsymbol{x} = \boldsymbol{b}$ for a symmetric, invertible $\mathbf{M}$, but $\boldsymbol{b}$ carries an error $\delta\boldsymbol{b}$. The solution then carries an error $\delta\boldsymbol{x} = \mathbf{M}^{-1}\delta\boldsymbol{b}$
- In the eigenvector basis $\mathbf{M}^{-1}$ just multiplies the component along $\boldsymbol{v}_i$ by $1/\lambda_i$, so

$$\|\delta\boldsymbol{x}\| \le \frac{\|\delta\boldsymbol{b}\|}{|\lambda_{\min}|} \qquad \text{and} \qquad \|\boldsymbol{b}\| = \|\mathbf{M}\,\boldsymbol{x}\| \le |\lambda_{\max}|\;\|\boldsymbol{x}\|$$

  Rearranging the second as $1/\|\boldsymbol{x}\| \le |\lambda_{\max}|/\|\boldsymbol{b}\|$ and multiplying it by the first gives

$$\frac{\|\delta\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \le \kappa\,\frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|}, \qquad \kappa = \frac{|\lambda_{\max}|}{|\lambda_{\min}|}$$

- κ is the *condition number*. A relative error in the data can be amplified by up to a factor κ in the solution
- For a symmetric matrix $|\lambda_{\max}|$ is the spectral norm $\|\mathbf{M}\|_2$ from the vector spaces lecture, and $|\lambda_{\min}|^{-1} = \left\|\mathbf{M}^{-1}\right\|_2$, so $\kappa = \|\mathbf{M}\|_2\,\left\|\mathbf{M}^{-1}\right\|_2$
- Example: in the mappings lecture $\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1.001 \end{pmatrix}$ has eigenvalues ≈ 2.0005 and 0.0005, so $\kappa \approx 4000$. Changing y_2 by 0.001 is a relative change of 3.5×10^{-4} in $\boldsymbol{y}$, but changed $\boldsymbol{w}$ from $(1, 1)$ to $(0, 2)$, a relative change of 1: an amplification of about 2800, within the bound

2.3 Rank and Null Space

- Writing out $\mathbf{M} = \mathbf{V}\,\mathbf{\Lambda}\,\mathbf{V}^{\mathsf{T}}$ column by column gives the useful form

$$\mathbf{M} = \sum_{i=1}^{n} \lambda_i\,\boldsymbol{v}_i\,\boldsymbol{v}_i^{\mathsf{T}}$$

  a sum of n rank-one pieces, one per eigenvector
- The *rank* of a symmetric matrix is its number of non-zero eigenvalues. (For a general square matrix the rank is the number of linearly independent columns; the two agree for symmetric matrices but not in general)

- The eigenvectors with zero eigenvalue span the *null space*: any $\boldsymbol{x} = \sum_k a_k \boldsymbol{v}_k$ built from them has $\mathbf{M}\boldsymbol{x} = \mathbf{0}$
- A matrix is *rank deficient* if it has a zero eigenvalue. Equivalently its columns are linearly dependent, or $\det(\mathbf{M}) = \prod_i \lambda_i = 0$. It then has no inverse: if $\boldsymbol{x}$ solves $\mathbf{M}\boldsymbol{x} = \boldsymbol{b}$ then so does $\boldsymbol{x}$ plus anything in the null space

2.4 Pseudo-Inverse of a Symmetric Matrix

- If $\mathbf{M}$ has k non-zero eigenvalues $\lambda_1, \ldots, \lambda_k$ we define

$$\mathbf{M}^{+} = \mathbf{V}\boldsymbol{\Lambda}^{+}\mathbf{V}^{\mathsf{T}} = \sum_{i=1}^{k} \frac{1}{\lambda_i}\, \boldsymbol{v}_i\, \boldsymbol{v}_i^{\mathsf{T}}, \qquad \boldsymbol{\Lambda}^{+} = \operatorname{diag}\Big(\tfrac{1}{\lambda_1}, \ldots, \tfrac{1}{\lambda_k}, 0, \ldots, 0\Big)$$

  i.e. invert the non-zero eigenvalues and leave the zero ones at zero
- If $\mathbf{M}\boldsymbol{x} = \boldsymbol{b}$ has solutions, $\boldsymbol{x} = \mathbf{M}^{+}\boldsymbol{b}$ is the one with no component in the null space. Since every other solution adds a null-space vector, which is orthogonal to it, this is the solution with the smallest $\|\boldsymbol{x}\|$
- If $\boldsymbol{b}$ has a component in the null space there is no exact solution, and $\mathbf{M}^{+}\boldsymbol{b}$ minimises $\|\mathbf{M}\boldsymbol{x} - \boldsymbol{b}\|$ instead
- This is the symmetric case of the Moore–Penrose pseudo-inverse described in the SVD notes

2.5 Low Rank Approximation

- Keeping only the k largest (in magnitude) eigenvalues gives the rank-k approximation $\mathbf{M}_k = \sum_{i=1}^{k} \lambda_i\, \boldsymbol{v}_i\, \boldsymbol{v}_i^{\mathsf{T}}$
  - It needs only k eigenvalues and k eigenvectors, $k(n+1)$ numbers rather than n^2
  - The error, $\mathbf{M} - \mathbf{M}_k$, is made of the discarded terms, so it is small when the discarded eigenvalues are small
- For an ill-conditioned matrix we can set the tiny eigenvalues to zero and use the pseudo-inverse. The directions that would have amplified noise by $1/\lambda_i$ are simply ignored
  - This trades a little bias (we lose whatever true signal lay in those directions) for a large reduction in variance
  - The experiment below shows it in action. Ridge regression, in the SVD lecture, is a smoother version of the same idea: it damps small eigenvalues rather than discarding them

2.6 Non-Normal Matrices

- Symmetric matrices belong to a larger class, the *normal* matrices, which satisfy $\mathbf{M}\mathbf{M}^{\dagger} = \mathbf{M}^{\dagger}\mathbf{M}$, where $\mathbf{M}^{\dagger} = (\mathbf{M}^{\mathsf{T}})^{*}$ is the conjugate transpose (for real matrices just $\mathbf{M}^{\mathsf{T}}$)
  - Normal matrices are exactly the matrices with a full set of orthonormal (possibly complex) eigenvectors
  - Hermitian matrices ($\mathbf{M} = \mathbf{M}^{\dagger}$) are normal; they matter in quantum theory, much less in machine learning
  - A real matrix can be normal without being symmetric: the rotation $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ is normal, with eigenvalues $\pm\mathrm{i}$

- Most non-symmetric real matrices are not normal, and their eigenvectors behave less nicely

- **Complex eigenvalues.** The eigenvalues are the roots of the *characteristic equation* $\det(\mathbf{M} - \lambda \mathbf{I}) = 0$. $(\mathbf{M} - \lambda \mathbf{I})\,\boldsymbol{v} = \mathbf{0}$ can only have a non-zero solution if $\mathbf{M} - \lambda \mathbf{I}$ is singular. This is a polynomial of order n in λ, so there are n roots (counting repeats), and in general they are complex
  - If $\mathbf{M}$ is real and λ is complex then $\boldsymbol{v}$ is complex too. Taking the complex conjugate of $\mathbf{M}\,\boldsymbol{v} = \lambda\,\boldsymbol{v}$ gives $\mathbf{M}\,\boldsymbol{v}^* = \lambda^*\,\boldsymbol{v}^*$, so complex eigenvalues and eigenvectors come in conjugate pairs

- **Defective matrices.** A non-symmetric matrix need not have n linearly independent eigenvectors; if it does not it is called *defective*
  - For example $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ has the repeated eigenvalue 1 but only one eigenvector, $(1, 0)^\mathsf{T}$
  - A randomly generated matrix is almost never defective, but defective matrices do turn up in structured problems

- **Left and right eigenvectors.** As well as the usual (right) eigenvectors $\mathbf{M}\,\boldsymbol{v}_i = \lambda_i\,\boldsymbol{v}_i$ there are left eigenvectors $\boldsymbol{u}_i^\mathsf{T}\mathbf{M} = \lambda_i\,\boldsymbol{u}_i^\mathsf{T}$, i.e. eigenvectors of $\mathbf{M}^\mathsf{T}$. As $\det(\mathbf{M}^\mathsf{T} - \lambda \mathbf{I}) = \det(\mathbf{M} - \lambda \mathbf{I})$ they have the same eigenvalues
  - For a symmetric matrix the left eigenvectors are just the transposes of the right ones
  - In general they differ, and the right eigenvectors are not orthogonal to one another. Instead the two sets are *biorthogonal*: computing $\boldsymbol{u}_i^\mathsf{T}\mathbf{M}\,\boldsymbol{v}_j$ in two ways (exactly as in the proof for symmetric matrices) gives $(\lambda_i - \lambda_j)\,\boldsymbol{u}_i^\mathsf{T}\boldsymbol{v}_j = 0$, so $\boldsymbol{u}_i^\mathsf{T}\boldsymbol{v}_j = 0$ whenever $\lambda_i \neq \lambda_j$

- **Eigen-decomposition.** If $\mathbf{M}$ is not defective, put the right eigenvectors in the columns of $\mathbf{V}$. Then $\mathbf{M}\,\mathbf{V} = \mathbf{V}\,\mathbf{\Lambda}$ and, as the columns are independent, $\mathbf{V}$ is invertible, so

$$\mathbf{M} = \mathbf{V}\,\mathbf{\Lambda}\,\mathbf{V}^{-1}$$

  - $\mathbf{V}$ is no longer orthogonal, so $\mathbf{V}^{-1} \neq \mathbf{V}^\mathsf{T}$
  - Multiplying $\mathbf{M} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ on the left by $\mathbf{V}^{-1}$ gives $\mathbf{V}^{-1}\mathbf{M} = \mathbf{\Lambda}\,\mathbf{V}^{-1}$: the rows of $\mathbf{V}^{-1}$ are the left eigenvectors, scaled so that $\boldsymbol{u}_i^\mathsf{T}\boldsymbol{v}_i = 1$
  - We still get $\mathbf{M}^k = \mathbf{V}\,\mathbf{\Lambda}^k\,\mathbf{V}^{-1}$, but the geometric picture (rotate, stretch along orthogonal axes, rotate back) is lost. This is one reason we lean on the singular value decomposition, which works for every matrix

3 Exercises

1. Let $\mathbf{M}$ be a symmetric matrix with eigenvectors $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ and distinct eigenvalues $\lambda_1 \neq \lambda_2$. By computing $\boldsymbol{v}_2^\mathsf{T}\mathbf{M}\,\boldsymbol{v}_1$ in two different ways, show that $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ are orthogonal
2. Show that the eigenvalues of $\mathbf{X}^\mathsf{T}\mathbf{X}$ are never negative. (Hint: consider $\|\mathbf{X}\,\boldsymbol{v}\|^2$)
3. Using $\mathbf{M} = \mathbf{V}\,\mathbf{\Lambda}\,\mathbf{V}^\mathsf{T}$
   (a) Show that $\mathbf{M}^{-1} = \mathbf{V}\,\mathbf{\Lambda}^{-1}\,\mathbf{V}^\mathsf{T}$ by multiplying the two together
   (b) Show that $\mathbf{M}^k = \mathbf{V}\,\mathbf{\Lambda}^k\,\mathbf{V}^\mathsf{T}$
4. Find the eigenvalues and eigenvectors of $\mathbf{M} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ and check that $\mathbf{M} = \mathbf{V}\,\mathbf{\Lambda}\,\mathbf{V}^\mathsf{T}$
5. Non-normal matrices

(a) For $\mathbf{M} = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix}$ find the eigenvalues, the right eigenvectors $\boldsymbol{v}_1, \boldsymbol{v}_2$ and the left eigenvectors $\boldsymbol{u}_1, \boldsymbol{u}_2$. Check that $\boldsymbol{v}_1^\mathsf{T}\boldsymbol{v}_2 \neq 0$ but $\boldsymbol{u}_1^\mathsf{T}\boldsymbol{v}_2 = \boldsymbol{u}_2^\mathsf{T}\boldsymbol{v}_1 = 0$

(b) Verify that $\mathbf{M} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{-1}$ and that the rows of $\mathbf{V}^{-1}$ are the left eigenvectors

(c) Show that $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ has only one independent eigenvector

(d) Show that $\mathbf{R} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ is normal but not symmetric, and that $(1, -\mathrm{i})^\mathsf{T}$ is an eigenvector with eigenvalue i. What is the other eigenvector?

6. Show that $\mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T} = \sum_i \lambda_i \boldsymbol{v}_i \boldsymbol{v}_i^\mathsf{T}$, and use the sum form to check that $\mathbf{M}\boldsymbol{v}_j = \lambda_j \boldsymbol{v}_j$

7. For $\mathbf{M} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$

   (a) Find the eigenvalues, eigenvectors, rank and null space

   (b) Find $\mathbf{M}^+$

   (c) Solve $\mathbf{M}\boldsymbol{x} = (2, 2)^\mathsf{T}$. Write down all the solutions and show $\mathbf{M}^+(2, 2)^\mathsf{T}$ is the shortest

   (d) What does $\mathbf{M}^+$ give for $\boldsymbol{b} = (1, 3)^\mathsf{T}$, which has no exact solution?

8. Work through the derivation of the condition-number bound $\|\delta\boldsymbol{x}\| / \|\boldsymbol{x}\| \leq \kappa \, \|\delta\boldsymbol{b}\| / \|\boldsymbol{b}\|$. Find a choice of $\boldsymbol{b}$ and $\delta\boldsymbol{b}$ for which it is an equality

4 Experiments

4.1 Eigensystems

- In either Matlab/Octave or python generate random matrices and check the matrix identities

```
X = randn(5,4)  % generate a mock design matrix with 5 inputs of length 4
M = X'*X        % compute a symmetric matrix
[V,L] = eig(M)  % compute eigenvectors and eigenvalues
V*L*V'          % should be identical to M
V*V'            % should be the identity matrix (up to rounding precision)
V'*V            % should be the identity matrix (up to rounding precision)
x = randn(4,1)  % generate a random column matrix of length 4
y = randn(4,1)  % generate another random column matrix of length 4
xp = V*x        % apply V to x
yp = V*y        % apply V to y
norm(x)         % compute Euclidean norm of x
norm(xp)        % should be the same as the Euclidean norm of x
x'*y            % compute inner product of x and y
xp'*yp          % inner product of xp and yp (should be the same as above)

Z = rand(4,5)   % a design matrix with more unknowns than examples
W = Z'*Z        % a covariance-like matrix (but without subtracting the mean)
eig(W)          % compute eigenvalues (one should be 0 up to machine precision)
```

4.2 Non-Normal Matrices

- In python, look at the eigensystem of a random non-symmetric matrix

```
import numpy as np

rng = np.random.default_rng(2)
M = rng.normal(size=(4, 4))       # not symmetric
lam, V = np.linalg.eig(M)         # columns of V are right eigenvectors
print(lam)                        # complex eigenvalues come in conjugate pairs
print(np.allclose(M, V @ np.diag(lam) @ np.linalg.inv(V)))   # M = V L V^-1
U = np.linalg.inv(V)              # rows should be left eigenvectors
print(np.round(U @ V, 10))        # u_i^T v_j: the identity (biorthogonal)
print(np.round(V.conj().T @ V, 3))  # but the v_i are not orthogonal
```

4.3 Low Rank Pseudo-Inverse

- Regression with two almost identical features: compare inverting every eigenvalue of $\mathbf{X}^\mathsf{T}\mathbf{X}$ with dropping the tiny one

```
import numpy as np

rng = np.random.default_rng(0)
m = 50
X = rng.normal(size=(m, 3))
X[:, 2] = X[:, 1] + 1e-3 * rng.normal(size=m)   # feature 3 is almost feature 2
w_true = np.array([1.0, 2.0, 0.0])
C = X.T @ X                                      # symmetric 3x3
lam, V = np.linalg.eigh(C)                       # eigenvalues in ascending order
print(lam, lam[-1] / lam[0])                     # tiny eigenvalue; condition no.
for trial in range(3):                           # same X, fresh noise each time
    y = X @ w_true + 0.1 * rng.normal(size=m)
    w_full = V @ np.diag(1 / lam) @ V.T @ X.T @ y  # invert all eigenvalues
    lam_plus = np.where(lam > 1e-2 * lam[-1], 1 / lam, 0)  # drop the tiny one
    w_trunc = V @ np.diag(lam_plus) @ V.T @ X.T @ y
    print(np.round(w_full, 2), np.round(w_trunc, 2))
```

- The condition number is about 3×10^6. With every eigenvalue inverted the last two weights jump around from one noise sample to the next (e.g. $(6.27, -4.26)$ then $(-1.38, 3.36)$), though their sum stays near 2. Dropping the tiny eigenvalue gives about $(1, 1)$ every time
- Neither recovers the true $(2, 0)$: the data cannot tell the two features apart. The truncated answer splits the weight equally, the minimum-norm choice, and predicts just as well

5 Answers

1. On one hand $\boldsymbol{v}_2^\mathsf{T}(\mathbf{M}\,\boldsymbol{v}_1) = \lambda_1\,\boldsymbol{v}_2^\mathsf{T}\boldsymbol{v}_1$. On the other, as $\mathbf{M}^\mathsf{T} = \mathbf{M}$, $\boldsymbol{v}_2^\mathsf{T}\mathbf{M}\,\boldsymbol{v}_1 = (\mathbf{M}\,\boldsymbol{v}_2)^\mathsf{T}\boldsymbol{v}_1 = \lambda_2\,\boldsymbol{v}_2^\mathsf{T}\boldsymbol{v}_1$. Subtracting, $(\lambda_1 - \lambda_2)\,\boldsymbol{v}_2^\mathsf{T}\boldsymbol{v}_1 = 0$ and since $\lambda_1 \neq \lambda_2$ we must have $\boldsymbol{v}_2^\mathsf{T}\boldsymbol{v}_1 = 0$
2. If $\mathbf{X}^\mathsf{T}\mathbf{X}\,\boldsymbol{v} = \lambda\,\boldsymbol{v}$ then $\lambda\,\|\boldsymbol{v}\|^2 = \boldsymbol{v}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\,\boldsymbol{v} = \|\mathbf{X}\,\boldsymbol{v}\|^2 \geq 0$, so $\lambda \geq 0$. It is zero exactly when $\mathbf{X}\,\boldsymbol{v} = \mathbf{0}$: a direction the data say nothing about
3. Matrix powers
   (a) $\mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}\mathbf{V}\boldsymbol{\Lambda}^{-1}\mathbf{V}^\mathsf{T} = \mathbf{V}\boldsymbol{\Lambda}\boldsymbol{\Lambda}^{-1}\mathbf{V}^\mathsf{T} = \mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{I}$, using $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{I}$ twice
   (b) In $\mathbf{M}^k = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}\mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}\cdots\mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}$ every inner $\mathbf{V}^\mathsf{T}\mathbf{V}$ is $\mathbf{I}$, leaving $\mathbf{V}\boldsymbol{\Lambda}^k\mathbf{V}^\mathsf{T}$

4. $\det(\mathbf{M}-\lambda\mathbf{I}) = (2-\lambda)^2-1 = 0$ gives $\lambda = 3$ with $\boldsymbol{v} = (1,1)^\mathsf{T}/\sqrt{2}$ and $\lambda = 1$ with $\boldsymbol{v} = (1,-1)^\mathsf{T}/\sqrt{2}$. Then

$$\frac{1}{2}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}\begin{pmatrix}3 & 0\\ 0 & 1\end{pmatrix}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix} = \frac{1}{2}\begin{pmatrix}4 & 2\\ 2 & 4\end{pmatrix} = \mathbf{M}$$

5. Non-normal matrices

   (a) $\mathbf{M}$ is upper triangular, so $\det(\mathbf{M}-\lambda\mathbf{I}) = (1-\lambda)(3-\lambda)$ and the eigenvalues are 1 and 3. Right eigenvectors: $\boldsymbol{v}_1 = (1,0)^\mathsf{T}$ and $\boldsymbol{v}_2 = (1,1)^\mathsf{T}$. Left eigenvectors (eigenvectors of $\mathbf{M}^\mathsf{T}$): $\boldsymbol{u}_1 = (1,-1)^\mathsf{T}$ and $\boldsymbol{u}_2 = (0,1)^\mathsf{T}$. Then $\boldsymbol{v}_1^\mathsf{T}\boldsymbol{v}_2 = 1 \neq 0$, while $\boldsymbol{u}_1^\mathsf{T}\boldsymbol{v}_2 = 1-1 = 0$ and $\boldsymbol{u}_2^\mathsf{T}\boldsymbol{v}_1 = 0$. (With these choices $\boldsymbol{u}_1^\mathsf{T}\boldsymbol{v}_1 = \boldsymbol{u}_2^\mathsf{T}\boldsymbol{v}_2 = 1$ already.)

   (b) $\mathbf{V} = \begin{pmatrix}1 & 1\\ 0 & 1\end{pmatrix}$, $\mathbf{V}^{-1} = \begin{pmatrix}1 & -1\\ 0 & 1\end{pmatrix}$, whose rows are $\boldsymbol{u}_1^\mathsf{T}$ and $\boldsymbol{u}_2^\mathsf{T}$, and

$$\mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{-1} = \begin{pmatrix}1 & 1\\ 0 & 1\end{pmatrix}\begin{pmatrix}1 & 0\\ 0 & 3\end{pmatrix}\begin{pmatrix}1 & -1\\ 0 & 1\end{pmatrix} = \begin{pmatrix}1 & 3\\ 0 & 3\end{pmatrix}\begin{pmatrix}1 & -1\\ 0 & 1\end{pmatrix} = \begin{pmatrix}1 & 2\\ 0 & 3\end{pmatrix} = \mathbf{M}$$

   (c) The only eigenvalue is 1 (twice). An eigenvector must satisfy $\begin{pmatrix}0 & 1\\ 0 & 0\end{pmatrix}\boldsymbol{v} = \mathbf{0}$, i.e. $v_2 = 0$, so every eigenvector is a multiple of $(1,0)^\mathsf{T}$

   (d) $\mathbf{R}\,\mathbf{R}^\mathsf{T} = \mathbf{R}^\mathsf{T}\mathbf{R} = \mathbf{I}$ (it is a rotation by $90°$), but $\mathbf{R}^\mathsf{T} \neq \mathbf{R}$. $\mathbf{R}\,(1,-\mathrm{i})^\mathsf{T} = (\mathrm{i},1)^\mathsf{T} = \mathrm{i}\,(1,-\mathrm{i})^\mathsf{T}$. The other eigenvector is the complex conjugate, $(1,\mathrm{i})^\mathsf{T}$, with eigenvalue $-\mathrm{i}$. As $\mathbf{R}$ is normal these two are orthogonal, using the complex inner product (conjugate the first vector): $(1,\mathrm{i})^\dagger(1,-\mathrm{i})^\mathsf{T} = 1\cdot 1 + (-\mathrm{i})(-\mathrm{i}) = 1-1 = 0$

6. $\mathbf{V}\boldsymbol{\Lambda}$ has columns $\lambda_i\boldsymbol{v}_i$, and a matrix with columns $\boldsymbol{a}_i$ times one with rows $\boldsymbol{v}_i^\mathsf{T}$ is $\sum_i \boldsymbol{a}_i\boldsymbol{v}_i^\mathsf{T}$. So $\mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T} = \sum_i \lambda_i\,\boldsymbol{v}_i\boldsymbol{v}_i^\mathsf{T}$. Then $\mathbf{M}\,\boldsymbol{v}_j = \sum_i \lambda_i\,\boldsymbol{v}_i\,(\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j) = \lambda_j\,\boldsymbol{v}_j$, since $\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = 0$ unless $i = j$

7. Rank-deficient example

   (a) $\lambda = 2$ with $\boldsymbol{v}_1 = (1,1)^\mathsf{T}/\sqrt{2}$, and $\lambda = 0$ with $\boldsymbol{v}_2 = (1,-1)^\mathsf{T}/\sqrt{2}$. The rank is 1 and the null space is all multiples of $(1,-1)^\mathsf{T}$

   (b) $\mathbf{M}^+ = \frac{1}{2}\boldsymbol{v}_1\boldsymbol{v}_1^\mathsf{T} = \frac{1}{4}\begin{pmatrix}1 & 1\\ 1 & 1\end{pmatrix}$

   (c) The solutions are $x_1 + x_2 = 2$, i.e. $(1,1)^\mathsf{T} + t\,(1,-1)^\mathsf{T}$ for any t, with $\|\boldsymbol{x}\|^2 = 2 + 2t^2$. This is smallest at $t = 0$, and $\mathbf{M}^+(2,2)^\mathsf{T} = \frac{1}{4}(4,4)^\mathsf{T} = (1,1)^\mathsf{T}$

   (d) $\mathbf{M}^+(1,3)^\mathsf{T} = \frac{1}{4}(4,4)^\mathsf{T} = (1,1)^\mathsf{T}$, giving $\mathbf{M}\,\boldsymbol{x} = (2,2)^\mathsf{T}$. That is the closest point to $(1,3)^\mathsf{T}$ that $\mathbf{M}$ can reach: the projection of $(1,3)^\mathsf{T}$ onto $(1,1)^\mathsf{T}$

8. Equality needs both inequalities to be tight: $\boldsymbol{x}$ along the eigenvector of $\lambda_{\max}$, so that $\|\mathbf{M}\boldsymbol{x}\| = |\lambda_{\max}|\,\|\boldsymbol{x}\|$, and $\delta\boldsymbol{b}$ along the eigenvector of $\lambda_{\min}$. So take $\boldsymbol{b} = \boldsymbol{v}_{\max}$ and $\delta\boldsymbol{b} = \epsilon\,\boldsymbol{v}_{\min}$: the signal lies in the direction $\mathbf{M}$ stretches most and the noise in the one it stretches least

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 8: Principal Component Analysis (PCA)

Adam Prugel-Bennett

October 7, 2026

1 Keywords

- Covariance matrices, positive semi-definite, dimensionality reduction, PCA, reconstruction error, duality

2 Main Points

2.1 Why Reduce Dimensions?

- Data often varies significantly in only a few directions, even when it has many features. You cannot usually see this by looking at the numbers
- Projecting onto the few directions of greatest variation gives fewer, more informative features for a learning machine, and discards directions that are mostly noise

2.2 Notation

- We have m data points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_m$, each with p features, and sample mean $\boldsymbol{\mu} = \frac{1}{m}\sum_k \boldsymbol{x}_k$
- In this lecture we collect the *centred* data as the columns of a $p \times m$ matrix

$$\mathsf{X} = \frac{1}{\sqrt{m-1}}\left(\boldsymbol{x}_1 - \boldsymbol{\mu}, \boldsymbol{x}_2 - \boldsymbol{\mu}, \cdots, \boldsymbol{x}_m - \boldsymbol{\mu}\right)$$

  This is the transpose of the design matrix used in linear regression (one row per data point), and it includes the factor $1/\sqrt{m-1}$

2.3 The Direction of Maximum Variation

- The spread of the data along a unit vector $\boldsymbol{v}$ ($\|\boldsymbol{v}\| = 1$) is the variance of the projections $\boldsymbol{v}^\mathsf{T}(\boldsymbol{x}_k - \boldsymbol{\mu})$

$$\sigma^2(\boldsymbol{v}) = \frac{1}{m-1}\sum_{k=1}^{m}\left(\boldsymbol{v}^\mathsf{T}(\boldsymbol{x}_k - \boldsymbol{\mu})\right)^2 = \boldsymbol{v}^\mathsf{T}\mathsf{C}\,\boldsymbol{v}$$

  where, writing $\left(\boldsymbol{v}^\mathsf{T}\boldsymbol{a}\right)^2 = \boldsymbol{v}^\mathsf{T}\boldsymbol{a}\,\boldsymbol{a}^\mathsf{T}\boldsymbol{v}$ and taking $\boldsymbol{v}$ outside the sum, C is the *covariance matrix*

$$\mathsf{C} = \frac{1}{m-1}\sum_{k=1}^{m}(\boldsymbol{x}_k - \boldsymbol{\mu})\,(\boldsymbol{x}_k - \boldsymbol{\mu})^\mathsf{T} = \mathsf{X}\,\mathsf{X}^\mathsf{T}$$

- To maximise $\boldsymbol{v}^\mathsf{T}\mathsf{C}\,\boldsymbol{v}$ subject to $\boldsymbol{v}^\mathsf{T}\boldsymbol{v} = 1$ we use a Lagrange multiplier. Setting the gradient of $\mathcal{L} = \boldsymbol{v}^\mathsf{T}\mathsf{C}\,\boldsymbol{v} - \lambda\,(\boldsymbol{v}^\mathsf{T}\boldsymbol{v} - 1)$ to zero gives

$$2\,\mathsf{C}\,\boldsymbol{v} - 2\,\lambda\,\boldsymbol{v} = \boldsymbol{0} \quad \Rightarrow \quad \mathsf{C}\,\boldsymbol{v} = \lambda\,\boldsymbol{v}$$

  (we used $\boldsymbol{\nabla}\,\boldsymbol{v}^\mathsf{T}\mathsf{C}\,\boldsymbol{v} = 2\,\mathsf{C}\,\boldsymbol{v}$ for symmetric C, from the mappings lecture)

- So the stationary directions are the eigenvectors of $\mathbf{C}$, and the variance along eigenvector $\boldsymbol{v}_i$ is $\boldsymbol{v}_i^\mathsf{T}\mathbf{C}\,\boldsymbol{v}_i = \lambda_i\,\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_i = \lambda_i$. The direction of greatest variation is the eigenvector with the largest eigenvalue
- The next best direction *orthogonal to the first* is the eigenvector with the second largest eigenvalue, and so on

2.4 Properties of the Covariance Matrix

- $C_{ij} = \frac{1}{m-1}\sum_k (x_{ik}-\mu_i)(x_{jk}-\mu_j)$ measures how features i and j vary together, the sample version of $\mathrm{Cov}(X,Y) = \mathbb{E}[(X-\mathbb{E}[X])(Y-\mathbb{E}[Y])]$. The diagonal holds the variances of the individual features
- $\mathbf{C}$ is built from *outer products* $\boldsymbol{a}\,\boldsymbol{a}^\mathsf{T}$ (a $p\times p$ matrix), compared with the inner product $\boldsymbol{a}^\mathsf{T}\boldsymbol{a}$ (a number)
- $\mathbf{C}$ is symmetric, so its eigenvectors can be chosen orthonormal (eigensystems lecture)
- $\mathbf{C}$ is *positive semi-definite*: for any $\boldsymbol{v}$,

  $$\boldsymbol{v}^\mathsf{T}\mathbf{C}\,\boldsymbol{v} = \boldsymbol{v}^\mathsf{T}\mathbf{X}\,\mathbf{X}^\mathsf{T}\boldsymbol{v} = \left\|\mathbf{X}^\mathsf{T}\boldsymbol{v}\right\|^2 \geq 0$$

  Applying this to an eigenvector shows every eigenvalue is $\lambda_i = \boldsymbol{v}_i^\mathsf{T}\mathbf{C}\,\boldsymbol{v}_i / \left\|\boldsymbol{v}_i\right\|^2 \geq 0$ (they are variances, after all)
- The total variance, summed over all features, is $\mathrm{Tr}\,\mathbf{C} = \sum_i C_{ii} = \sum_i \lambda_i$. The fraction $\sum_{i\leq k}\lambda_i / \sum_i \lambda_i$ is the "variance explained" by the first k components
- **Zero eigenvalues.** $\lambda_i = 0$ exactly when there is no variation at all along $\boldsymbol{v}_i$
  - The m centred points sum to $\mathbf{0}$, so they span at most an $(m-1)$-dimensional subspace. If $m \leq p$, $\mathbf{C}$ must have zero eigenvalues
  - With $m \geq p+1$ points that do not all lie in one hyperplane, every eigenvalue is positive: $\mathbf{C}$ is *positive definite*, full rank and invertible
  - Typically, with $m > p$, $\mathbf{C}$ is positive definite unless the features are linearly related (e.g. one is the sum of others)
- **The ellipsoid picture.** The surface $\boldsymbol{x}^\mathsf{T}\mathbf{C}^{-1}\boldsymbol{x} = 1$ is an ellipsoid whose principal axes point along the eigenvectors. Along $\boldsymbol{v}_i$, putting $\boldsymbol{x} = r\,\boldsymbol{v}_i$ gives $r^2/\lambda_i = 1$, so the radius is $\sqrt{\lambda_i}$: the standard deviation of the data in that direction

2.5 Principal Component Analysis

- The recipe
  1. Compute the mean $\boldsymbol{\mu}$ and the covariance matrix $\mathbf{C}$
  2. Find its eigenvalues and eigenvectors
  3. Keep the k eigenvectors with the largest eigenvalues, the *principal components*
  4. Project each input into the space they span
- The projection uses the $k\times p$ matrix whose rows are the principal components

  $$\mathbf{P} = \begin{pmatrix} \boldsymbol{v}_1^\mathsf{T} \\ \vdots \\ \boldsymbol{v}_k^\mathsf{T} \end{pmatrix}, \qquad \boldsymbol{z} = \mathbf{P}\,(\boldsymbol{x}-\boldsymbol{\mu})$$

  The k numbers $z_j = \boldsymbol{v}_j^\mathsf{T}(\boldsymbol{x}-\boldsymbol{\mu})$ are the new features

- **Reconstruction.** We can map back to an approximation of the input

$$\hat{\boldsymbol{x}} = \boldsymbol{\mu} + \sum_{j=1}^{k} z_j \, \boldsymbol{v}_j$$

- **Reconstruction error.** All p eigenvectors form an orthonormal basis, so $\boldsymbol{x} - \boldsymbol{\mu} = \sum_{j=1}^{p} z_j \, \boldsymbol{v}_j$ exactly, and the error is the part we dropped, $\boldsymbol{x} - \hat{\boldsymbol{x}} = \sum_{j>k} z_j \, \boldsymbol{v}_j$. By orthonormality $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2 = \sum_{j>k} z_j^2$, and averaging over the data (with the same $1/(m-1)$) each z_j^2 averages to the variance along $\boldsymbol{v}_j$, which is λ_j. So

$$\frac{1}{m-1} \sum_{k'=1}^{m} \|\boldsymbol{x}_{k'} - \hat{\boldsymbol{x}}_{k'}\|^2 = \sum_{j>k} \lambda_j$$

  the sum of the eigenvalues we ignored. No other k-dimensional subspace does better, which is why PCA keeps the largest eigenvalues

- **Choosing k.** Plot the eigenvalues in decreasing order (as for the handwritten digits in the lecture) and keep enough components to explain most of the variance. There is no single right answer

- Images of handwritten digits: the leading eigenvectors look like blurred digits, and a few tens of them reconstruct a recognisable digit from its hundreds of pixels

2.6 Duality

- For images p is huge: a 256×256 image has $p = 65\,536$ pixels, and $\mathbf{C}$ would have over 4×10^9 entries. But m images span at most an $(m-1)$-dimensional subspace, usually with $m \ll p$

- Work instead with the $m \times m$ *dual matrix* $\mathbf{D} = \mathbf{X}^\mathsf{T}\mathbf{X}$, whose entries $D_{kl} = \frac{1}{m-1}(\boldsymbol{x}_k - \boldsymbol{\mu})^\mathsf{T}(\boldsymbol{x}_l - \boldsymbol{\mu})$ are inner products between data points. Building it costs $O(p\,m^2)$

- If $\mathbf{D}\,\boldsymbol{v} = \lambda\,\boldsymbol{v}$, multiply on the left by $\mathbf{X}$:

$$\mathbf{X}\,\mathbf{X}^\mathsf{T}\mathbf{X}\,\boldsymbol{v} = \lambda\,\mathbf{X}\,\boldsymbol{v} \quad \Rightarrow \quad \mathbf{C}\,\boldsymbol{u} = \lambda\,\boldsymbol{u}, \qquad \boldsymbol{u} = \mathbf{X}\,\boldsymbol{v}$$

  so every eigenvector of $\mathbf{D}$ with $\lambda \neq 0$ gives one of $\mathbf{C}$ with the same eigenvalue. (If $\lambda = 0$ then $\mathbf{X}\boldsymbol{v} = \mathbf{0}$ and there is nothing to map)

- $\mathbf{C}$ and $\mathbf{D}$ therefore have the same *non-zero* eigenvalues. The bigger matrix just has extra zeros

- **Normalising.** $\|\mathbf{X}\,\boldsymbol{v}\|^2 = \boldsymbol{v}^\mathsf{T}\mathbf{D}\,\boldsymbol{v} = \lambda$ for a unit $\boldsymbol{v}$, so the unit eigenvector of $\mathbf{C}$ is $\boldsymbol{u} = \mathbf{X}\,\boldsymbol{v}/\sqrt{\lambda}$

- We have worked in the *dual space*, spanned by the examples. The same trick reappears in SVMs and kernel methods, and the pair $\mathbf{X}\mathbf{X}^\mathsf{T}$, $\mathbf{X}^\mathsf{T}\mathbf{X}$ is the heart of the SVD (next lecture)

3 Exercises

1. Show that $\frac{1}{m-1}\sum_k \left(\boldsymbol{v}^\mathsf{T}(\boldsymbol{x}_k - \boldsymbol{\mu})\right)^2 = \boldsymbol{v}^\mathsf{T}\mathbf{C}\,\boldsymbol{v}$
2. Two data points $\boldsymbol{y}^1 = (2, 4, 5)^\mathsf{T}$ and $\boldsymbol{y}^2 = (8, 6, 1)^\mathsf{T}$ (the example in the lecture)
   (a) Find $\boldsymbol{\mu}$ and $\mathbf{X}$ (with $m = 2$ the factor $1/\sqrt{m-1}$ is 1)
   (b) Find $\mathbf{D} = \mathbf{X}^\mathsf{T}\mathbf{X}$ and its eigenvalues and eigenvectors
   (c) Use the dual to find the principal component of $\mathbf{C}$, and explain why $\mathbf{C}$ has two zero eigenvalues

3. Show that $\operatorname{Tr}\mathbf{C}$ equals the sum of the variances of the individual features, and also the sum of the eigenvalues

4. Data in two dimensions have covariance matrix $\mathbf{C} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$

   (a) Find the principal components and the variance along each

   (b) What fraction of the variance does the first component explain?

   (c) What are the radii and directions of the axes of the ellipse $\boldsymbol{x}^{\mathsf{T}}\mathbf{C}^{-1}\boldsymbol{x} = 1$?

5. Show that if $\boldsymbol{v}$ is a unit eigenvector of $\mathbf{D}$ with eigenvalue $\lambda > 0$ then $\mathbf{X}\,\boldsymbol{v}/\sqrt{\lambda}$ is a unit eigenvector of $\mathbf{C}$

4 Experiments

4.1 PCA by hand

- Generate data in 10 dimensions that really lies close to a plane, and check the claims in these notes

```
import numpy as np

rng = np.random.default_rng(0)
m, p = 200, 10
basis = rng.normal(size=(p, 2))  # data really lives near a plane
Y = (basis @ rng.normal(size=(2, m))).T + 0.1 * rng.normal(size=(m, p))
mu = Y.mean(axis=0)
X = (Y - mu).T / np.sqrt(m - 1)  # p x m, columns are centred points
C = X @ X.T  # covariance matrix (p x p)
lam, V = np.linalg.eigh(C)
lam, V = lam[::-1], V[:, ::-1]  # largest eigenvalue first
print(np.round(lam, 3))                          # two big, eight tiny
print(np.allclose(C, np.cov(Y.T)))  # agrees with numpy's covariance

for k in (1, 2, 3):
    P = V[:, :k].T  # k x p projection matrix
    Z = P @ (Y - mu).T                           # k x m new features
    Yhat = mu + (V[:, :k] @ Z).T                 # reconstruction
    err = np.sum((Y - Yhat) ** 2) / (m - 1)
    # the error should equal the sum of the discarded eigenvalues
    print(k, round(err, 4), round(lam[k:].sum(), 4))

# duality: with more features than examples use the m x m matrix
Xs = X[:, :5]  # pretend we have only 5 examples
print(np.round(np.linalg.eigvalsh(Xs.T @ Xs)[::-1], 4))  # 5 eigenvalues of D
print(np.round(np.linalg.eigvalsh(Xs @ Xs.T)[::-1], 4))  # same, plus zeros
```

- Two eigenvalues (about 12 and 2.7) stand out; the other eight are around 0.01, the noise. Keeping $k = 2$ components cuts the reconstruction error from about 2.8 to 0.08, and in every case the error equals the sum of the discarded eigenvalues

- The 5×5 dual matrix has the same non-zero eigenvalues as the 10×10 matrix, which just has five extra zeros

5 Answers

1. $\left(\boldsymbol{v}^{\mathsf{T}}\boldsymbol{a}\right)^2 = (\boldsymbol{v}^{\mathsf{T}}\boldsymbol{a})(\boldsymbol{a}^{\mathsf{T}}\boldsymbol{v}) = \boldsymbol{v}^{\mathsf{T}}\boldsymbol{a}\,\boldsymbol{a}^{\mathsf{T}}\boldsymbol{v}$, since $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{a}$ is a number and equals its transpose. With $\boldsymbol{a} = \boldsymbol{x}_k - \boldsymbol{\mu}$, summing over k and taking $\boldsymbol{v}$ outside the sum gives $\boldsymbol{v}^{\mathsf{T}}\mathbf{C}\,\boldsymbol{v}$

2. The lecture example

   (a) $\boldsymbol{\mu} = (5, 5, 3)^{\mathsf{T}}$ and $\mathbf{X} = \begin{pmatrix} -3 & 3 \\ -1 & 1 \\ 2 & -2 \end{pmatrix}$

   (b) $\mathbf{D} = \begin{pmatrix} 14 & -14 \\ -14 & 14 \end{pmatrix}$, with $\lambda = 28$, $\boldsymbol{v} = (-1, 1)^{\mathsf{T}}/\sqrt{2}$ and $\lambda = 0$, $\boldsymbol{v} = (1, 1)^{\mathsf{T}}/\sqrt{2}$

   (c) $\mathbf{X}\,\boldsymbol{v} = (6, 2, -4)^{\mathsf{T}}/\sqrt{2}$ and dividing by $\sqrt{28}$ gives $\boldsymbol{u} = (3, 1, -2)^{\mathsf{T}}/\sqrt{14}$, with variance 28. The two centred points are $\pm(3, 1, -2)^{\mathsf{T}}$: they lie on a line through the origin, so there is no variation in the two directions perpendicular to it

3. $\operatorname{Tr}\mathbf{C} = \sum_i C_{ii}$ and C_{ii} is the variance of feature i. Using $\mathbf{C} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{\mathsf{T}}$ and the cyclic property of the trace, $\operatorname{Tr}\mathbf{C} = \operatorname{Tr}\boldsymbol{\Lambda}\mathbf{V}^{\mathsf{T}}\mathbf{V} = \operatorname{Tr}\boldsymbol{\Lambda} = \sum_i \lambda_i$

4. The matrix from the eigensystems notes

   (a) $\lambda = 3$ along $(1, 1)^{\mathsf{T}}/\sqrt{2}$ and $\lambda = 1$ along $(1, -1)^{\mathsf{T}}/\sqrt{2}$. The variances are 3 and 1

   (b) $3/(3 + 1) = 75\%$

   (c) Radii $\sqrt{3}$ along $(1, 1)^{\mathsf{T}}$ and 1 along $(1, -1)^{\mathsf{T}}$

5. $\mathbf{C}\,(\mathbf{X}\boldsymbol{v}) = \mathbf{X}\,(\mathbf{X}^{\mathsf{T}}\mathbf{X}\,\boldsymbol{v}) = \lambda\,\mathbf{X}\boldsymbol{v}$, so it is an eigenvector, and $\|\mathbf{X}\boldsymbol{v}\|^2 = \boldsymbol{v}^{\mathsf{T}}\mathbf{X}^{\mathsf{T}}\mathbf{X}\,\boldsymbol{v} = \lambda\,\boldsymbol{v}^{\mathsf{T}}\boldsymbol{v} = \lambda$, so dividing by $\sqrt{\lambda}$ makes it a unit vector

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 9: Singular Value Decomposition (SVD)

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Singular Value Decomposition, SVD, spectral norm, condition number, general linear maps, ridge regression

2 Main Points

2.1 Singular Value Decomposition

- Any $m \times p$ matrix, $\mathbf{X}$ can be decomposed as $\mathbf{X} = \mathbf{U}\,\mathbf{S}\,\mathbf{V}^\mathsf{T}$ (for a design matrix, m is the number of data points and p the number of features)
  - $\mathbf{U}$ is an $m \times m$ orthogonal matrix
  - $\mathbf{S}$ is an $m \times p$ matrix with zeros everywhere except the diagonal where $S_{ii} = s_i \geq 0$
  - $\mathbf{V}$ is a $p \times p$ orthogonal matrix
- The values s_i are known as the *singular values* of $\mathbf{X}$
- The columns $\boldsymbol{u}_i$ of $\mathbf{U}$ and $\boldsymbol{v}_i$ of $\mathbf{V}$ are the left and right *singular vectors*. Column by column the SVD says $\mathbf{X}\,\boldsymbol{v}_i = s_i\,\boldsymbol{u}_i$ and $\mathbf{X}^\mathsf{T}\boldsymbol{u}_i = s_i\,\boldsymbol{v}_i$
- If $\mathbf{X}$ is symmetric and positive semi-definite (e.g. a covariance matrix) then $\mathbf{U} = \mathbf{V}$ and $\mathbf{S} = \mathbf{\Lambda}$: the SVD is the eigen-decomposition. For a general symmetric matrix $s_i = |\lambda_i|$, and the columns of $\mathbf{U}$ and $\mathbf{V}$ belonging to a negative eigenvalue differ in sign
- **Where it comes from.** The SVD can be built from the eigenvectors of a symmetric matrix; the derivation is in the next section
- **Economical SVD**
  - If $m > p$ some algorithms won't bother outputting the last $m - p$ columns of $\mathbf{U}$
  - If $p > m$ some algorithms won't bother outputting the last $p - m$ columns of $\mathbf{V}$
  - In this case it will output a square $\mathbf{S}$ matrix
  - In numpy, `U, s, Vt = np.linalg.svd(X)` gives the full form and `np.linalg.svd(X, full_matrices=False)` the economical one. Note that numpy returns the singular values as a vector `s` and returns $\mathbf{V}^\mathsf{T}$, not $\mathbf{V}$
- **Finding the SVD.** In principle we can take whichever of $\mathbf{X}\,\mathbf{X}^\mathsf{T}$ and $\mathbf{X}^\mathsf{T}\mathbf{X}$ is smaller, find its eigenvalues s_i^2 and eigenvectors, and recover the other set of singular vectors from $\mathbf{U} = \mathbf{X}\,\mathbf{V}\,\mathbf{S}^{-1}$ (for the non-zero s_i). Good libraries do not do this: forming $\mathbf{X}^\mathsf{T}\mathbf{X}$ squares the condition number (see the mappings notes), so they use a more stable algorithm on $\mathbf{X}$ itself

2.2 Where the SVD Comes From

- Take $m \le p$ (otherwise apply the argument to $\mathbf{X}^\mathsf{T}$) and build the $(m+p)\times(m+p)$ matrix

$$\mathbf{B} = \begin{pmatrix} \mathbf{0} & \mathbf{X} \\ \mathbf{X}^\mathsf{T} & \mathbf{0} \end{pmatrix}$$

  It is symmetric, so it has real eigenvalues and orthonormal eigenvectors. This is the whole trick: everything we know about symmetric matrices transfers to an arbitrary $\mathbf{X}$

- **Eigenvectors come in pairs.** Write an eigenvector as $(\boldsymbol{u}, \boldsymbol{v})$, with $\boldsymbol{u} \in \mathbb{R}^m$ and $\boldsymbol{v} \in \mathbb{R}^p$. Then $\mathbf{B}\,(\boldsymbol{u}, \boldsymbol{v}) = s\,(\boldsymbol{u}, \boldsymbol{v})$ says exactly

$$\mathbf{X}\,\boldsymbol{v} = s\,\boldsymbol{u}, \qquad \mathbf{X}^\mathsf{T}\boldsymbol{u} = s\,\boldsymbol{v}$$

  Changing the sign of $\boldsymbol{v}$ changes the sign of both right-hand sides, so $(\boldsymbol{u}, -\boldsymbol{v})$ is an eigenvector with eigenvalue $-s$. The eigenvalues therefore come as m pairs $\pm s_i$ (with $s_i \ge 0$) together with $p - m$ zeros, whose eigenvectors have the form $(\mathbf{0}, \boldsymbol{v}_k)$ with $\mathbf{X}\,\boldsymbol{v}_k = \mathbf{0}$

- Applying $\mathbf{X}^\mathsf{T}$ to the first equation gives $\mathbf{X}^\mathsf{T}\mathbf{X}\,\boldsymbol{v} = s\,\mathbf{X}^\mathsf{T}\boldsymbol{u} = s^2\,\boldsymbol{v}$, and similarly $\mathbf{X}\,\mathbf{X}^\mathsf{T}\boldsymbol{u} = s^2\,\boldsymbol{u}$: the s_i^2 are the eigenvalues of $\mathbf{X}^\mathsf{T}\mathbf{X}$ and $\mathbf{X}\,\mathbf{X}^\mathsf{T}$

- **Normalisation: why $\|\boldsymbol{u}\| = \|\boldsymbol{v}\|$.** Normalise the eigenvector so that $\|\boldsymbol{u}\|^2 + \|\boldsymbol{v}\|^2 = 1$. If its eigenvalue s is not zero, the two halves have the same length. Take the inner product of $\mathbf{X}\,\boldsymbol{v} = s\,\boldsymbol{u}$ with $\boldsymbol{u}$, and of $\mathbf{X}^\mathsf{T}\boldsymbol{u} = s\,\boldsymbol{v}$ with $\boldsymbol{v}$:

$$s\,\|\boldsymbol{u}\|^2 = \boldsymbol{u}^\mathsf{T}\mathbf{X}\,\boldsymbol{v}, \qquad s\,\|\boldsymbol{v}\|^2 = \boldsymbol{v}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{u}$$

  The right-hand sides are equal, since $\boldsymbol{v}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{u} = (\boldsymbol{u}^\mathsf{T}\mathbf{X}\,\boldsymbol{v})^\mathsf{T}$ and a number equals its own transpose. So $s\,\|\boldsymbol{u}\|^2 = s\,\|\boldsymbol{v}\|^2$, and dividing by $s \ne 0$,

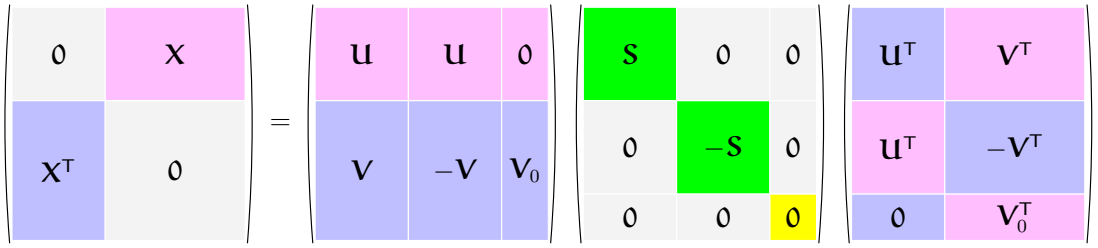
$$\|\boldsymbol{u}\|^2 = \|\boldsymbol{v}\|^2 = \tfrac{1}{2}$$

  - In words: $\mathbf{X}$ takes $\boldsymbol{v}$ to $s\,\boldsymbol{u}$ and $\mathbf{X}^\mathsf{T}$ takes $\boldsymbol{u}$ back to $s\,\boldsymbol{v}$. Both directions scale by the same factor, so neither half can be longer than the other
  - Another route: $(\boldsymbol{u}, \boldsymbol{v})$ and $(\boldsymbol{u}, -\boldsymbol{v})$ have eigenvalues s and $-s$, which differ when $s \ne 0$, so they are orthogonal: $0 = \|\boldsymbol{u}\|^2 - \|\boldsymbol{v}\|^2$
  - It fails when $s = 0$. The equations become $\mathbf{X}\,\boldsymbol{v} = \mathbf{0}$ and $\mathbf{X}^\mathsf{T}\boldsymbol{u} = \mathbf{0}$, which no longer link $\boldsymbol{u}$ to $\boldsymbol{v}$. For example the eigenvectors $(\mathbf{0}, \boldsymbol{v}_k)$ have $\|\boldsymbol{u}\| = 0$ and $\|\boldsymbol{v}\| = 1$
  - Similarly, $(\boldsymbol{u}_i, \boldsymbol{v}_i)$ is orthogonal to both $(\boldsymbol{u}_j, \pm\boldsymbol{v}_j)$ for $j \ne i$, giving $\boldsymbol{u}_i^\mathsf{T}\boldsymbol{u}_j = \boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = 0$. (For simplicity we take every $s_i > 0$)

- **Stacking the eigenvectors.** Put the eigenvectors into the orthogonal matrix $\mathbf{W}$ and the eigenvalues into the diagonal matrix $\mathbf{D}$:

$$\mathbf{W} = \begin{pmatrix} \mathbf{U} & \mathbf{U} & \mathbf{0} \\ \mathbf{V} & -\mathbf{V} & \mathbf{V}_0 \end{pmatrix}, \qquad \mathbf{D} = \begin{pmatrix} \mathbf{S} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\mathbf{S} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{pmatrix}$$

  where $\mathbf{U}$ $(m \times m)$ and $\mathbf{V}$ $(p \times m)$ have columns $\boldsymbol{u}_i$ and $\boldsymbol{v}_i$, $\mathbf{V}_0$ $(p \times (p-m))$ has columns $\boldsymbol{v}_k$, and $\mathbf{S} = \operatorname{diag}(s_1, \ldots, s_m)$. The eigen-decomposition of $\mathbf{B}$ is $\mathbf{B} = \mathbf{W}\,\mathbf{D}\,\mathbf{W}^\mathsf{T}$



- **Multiplying out.** First multiply $\mathbf{W}$ by the diagonal matrix $\mathbf{D}$, which scales each block column. Note that the $-\mathbf{V}$ block meets $-\mathbf{S}$, giving $+\mathbf{VS}$, and $\mathbf{V}_0$ meets $\mathbf{0}$

$$\begin{pmatrix} \mathbf{0} & \mathbf{X} \\ \mathbf{X}^\mathsf{T} & \mathbf{0} \end{pmatrix} = \begin{pmatrix} \mathbf{US} & -\mathbf{US} & \mathbf{0} \\ \mathbf{VS} & \mathbf{VS} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \mathbf{U}^\mathsf{T} & \mathbf{V}^\mathsf{T} \\ \mathbf{U}^\mathsf{T} & -\mathbf{V}^\mathsf{T} \\ \mathbf{0} & \mathbf{V}_0^\mathsf{T} \end{pmatrix}$$

- Then multiply by $\mathbf{W}^\mathsf{T}$. The top-left block is $\mathbf{USU}^\mathsf{T} - \mathbf{USU}^\mathsf{T} = \mathbf{0}$, the top-right is $\mathbf{USV}^\mathsf{T} + \mathbf{USV}^\mathsf{T} = 2\,\mathbf{USV}^\mathsf{T}$, and similarly for the bottom row

$$\begin{pmatrix} \mathbf{0} & \mathbf{X} \\ \mathbf{X}^\mathsf{T} & \mathbf{0} \end{pmatrix} = \begin{pmatrix} \mathbf{0} & 2\mathbf{USV}^\mathsf{T} \\ 2\mathbf{VSU}^\mathsf{T} & \mathbf{0} \end{pmatrix}$$

- Comparing the two sides block by block, $\mathbf{X} = 2\,\mathbf{U}\,\mathbf{S}\,\mathbf{V}^\mathsf{T}$. The columns of $\mathbf{U}$ and $\mathbf{V}$ are orthogonal but have length $1/\sqrt{2}$, so define $\tilde{\mathbf{U}} = \sqrt{2}\,\mathbf{U}$ and $\tilde{\mathbf{V}} = \sqrt{2}\,\mathbf{V}$, whose columns are orthonormal. Then

$$\mathbf{X} = \tilde{\mathbf{U}}\,\mathbf{S}\,\tilde{\mathbf{V}}^\mathsf{T}$$

  which is the (economical) SVD. Adding the columns of $\mathbf{V}_0$ to $\tilde{\mathbf{V}}$ completes it to a $p \times p$ orthogonal matrix and gives the full SVD

2.3 Spectrum and the Spectral Norm

- The singular values, ordered $s_1 \geq s_2 \geq \cdots \geq 0$, are the (singular value) *spectrum* of $\mathbf{X}$
- The largest, $s_{\max}$, is the most that $\mathbf{X}$ can stretch a vector. It is the *spectral norm* from the vector spaces lecture:

$$\|\mathbf{X}\|_2 = \max_{\boldsymbol{x} \neq \mathbf{0}} \frac{\|\mathbf{X}\,\boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2} = s_{\max}$$

- **Proof.** Write $\boldsymbol{x} = \sum_i c_i\,\boldsymbol{v}_i$ in the orthonormal basis of right singular vectors, so $\|\boldsymbol{x}\|^2 = \sum_i c_i^2$. Then $\mathbf{X}\,\boldsymbol{x} = \sum_i c_i\,s_i\,\boldsymbol{u}_i$, and as the $\boldsymbol{u}_i$ are orthonormal too

$$\|\mathbf{X}\,\boldsymbol{x}\|^2 = \sum_i s_i^2\,c_i^2 \leq s_{\max}^2 \sum_i c_i^2 = s_{\max}^2\,\|\boldsymbol{x}\|^2$$

  with equality when $\boldsymbol{x} = \boldsymbol{v}_{\max}$, which is mapped to $\mathbf{X}\,\boldsymbol{v}_{\max} = s_{\max}\,\boldsymbol{u}_{\max}$

- Consequences
  - For a symmetric matrix $\|\mathbf{X}\|_2 = \max_i |\lambda_i|$, as used in the eigensystems notes
  - For any square matrix $\|\mathbf{X}\|_2 \geq \max_i |\lambda_i|$ (if $\mathbf{X}\boldsymbol{v} = \lambda\boldsymbol{v}$ then $\boldsymbol{v}$ is stretched by $|\lambda|$, which cannot exceed $s_{\max}$). The two can differ a lot for non-symmetric matrices (Exercise 3)
  - $\|\mathbf{X}\|_F^2 = \sum_{ij} X_{ij}^2 = \sum_i s_i^2 \geq s_{\max}^2$, so $\|\mathbf{X}\|_2 \leq \|\mathbf{X}\|_F$ (Exercise 2)
  - For a square invertible matrix the smallest stretch is $s_{\min}$ and $\left\|\mathbf{X}^{-1}\right\|_2 = 1/s_{\min}$, so the condition number is $\kappa = \|\mathbf{X}\|_2 \left\|\mathbf{X}^{-1}\right\|_2 = s_{\max}/s_{\min}$. This generalises the eigenvalue ratio for symmetric matrices
  - Spectral normalisation of a network layer (vector spaces notes) divides $\mathbf{L}$ by $s_{\max}$ so that $\|\mathbf{L}\|_2 = 1$

2.4 General Linear Mapping

- Recall that matrices are the most general linear operators
- Since any matrix $\mathbf{X}$ can be written as $\mathbf{U}\,\mathbf{S}\,\mathbf{V}^\mathsf{T}$ we can interpret any linear mapping as doing three operations
  - A rotation (with possibly a reflection) defined by $\mathbf{V}^\mathsf{T}$
  - A rescaling of each coordinate by s_i
  - A rotation (with possibly a reflection) defined by $\mathbf{U}$
- When $\mathbf{X}$ is not square, $\mathbf{S}$ also either squashes some directions to zero or embeds the result in a space with extra dimensions
- The *rank* of any matrix is its number of non-zero singular values (the number of linearly independent rows or columns)
- For a square matrix, using $|\mathbf{AB}| = |\mathbf{A}|\,|\mathbf{B}|$ and $|\mathbf{U}| = \pm 1$ for an orthogonal matrix, $|\det \mathbf{X}| = \prod_i s_i$: the singular values multiply up to the change of volume
- **Duality**
  - Using $\mathbf{X} = \mathbf{U}\,\mathbf{S}\,\mathbf{V}^\mathsf{T}$ then
    - $\mathbf{C} = \mathbf{X}\,\mathbf{X}^\mathsf{T} = \mathbf{U}\mathbf{S}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T}$
    - $\mathbf{D} = \mathbf{X}^\mathsf{T}\mathbf{X} = \mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{S}\mathbf{V}^\mathsf{T}$
  - $\mathbf{S}\mathbf{S}^\mathsf{T}$ and $\mathbf{S}^\mathsf{T}\mathbf{S}$ are diagonal matrices whose non-zero diagonal elements are s_i^2

2.5 Ridge Regression

- Ridge regression is linear regression with an L_2 regulariser
- Adding a regulariser $\eta\,\|\boldsymbol{w}\|^2$ to the squared loss, the weights, $\boldsymbol{w}^*$, that minimise the loss function are given by $\boldsymbol{w}^* = (\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\,\mathbf{I})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$
- Using $\mathbf{X} = \mathbf{U}\,\mathbf{S}\,\mathbf{V}^\mathsf{T}$ then

$$\boldsymbol{w}^* = \mathbf{V}\,\hat{\mathbf{S}}^+\mathbf{U}^\mathsf{T}\boldsymbol{y}$$

  where $\hat{\mathbf{S}}^+$ is a regularised pseudo-inverse of $\mathbf{S}$ given by

$$\hat{\mathbf{S}}^+ = (\mathbf{S}^\mathsf{T}\mathbf{S} + \eta\,\mathbf{I})^{-1}\mathbf{S}^\mathsf{T}$$

  - If $\eta = 0$ (and all $s_i > 0$) this is equal to the pseudo-inverse of $\mathbf{S}$
- $\hat{\mathbf{S}}^+$ is a $p \times m$ matrix which is zero everywhere except on the diagonal, where $\hat{S}^+_{ii} = \frac{s_i}{s_i^2 + \eta}$
  - Note if $s_i = 0$ linear regression has an infinity of solutions and the pseudo-inverse of $\mathbf{X}$, as we derived it, does not exist (setting $\eta = 0$ we get $S^+_{ii} = 1/s_i$ which is not defined when $s_i = 0$). See the Moore–Penrose pseudo-inverse below
  - In the regularised case $\hat{S}^+_{ii} = 0$ (we have selected one of the solutions that minimise the squared error)
  - If $s_i \ll \eta$ then without the regularisation term the inverse is very ill-conditioned while with the regularisation term $\hat{S}^+_{ii}$ will be small
  - If $s_i \gg \eta$ then $\hat{S}^+_{ii} \approx \frac{1}{s_i} = S^+_{ii}$
- Adding an L_2 regulariser means that the optimum weights, $\boldsymbol{w}^*$, will be less sensitive to the training data reducing the variance in the bias-variance dilemma

2.6 The Moore–Penrose Pseudo-Inverse (going further)

- The pseudo-inverse we derived, $\mathbf{X}^{+} = (\mathbf{X}^{\mathsf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathsf{T}}$, only exists when $\mathbf{X}^{\mathsf{T}}\mathbf{X}$ is invertible. That needs every one of the p singular values to be non-zero, which is impossible when $m < p$ (the under-constrained case)
- Using the SVD it can be written $\mathbf{X}^{+} = \mathbf{V}\,\mathbf{S}^{+}\mathbf{U}^{\mathsf{T}}$, where $\mathbf{S}^{+}$ is the $p \times m$ diagonal matrix with $S^{+}_{ii} = 1/s_i$
- The *Moore–Penrose pseudo-inverse* makes one change: wherever $s_i = 0$ it sets $S^{+}_{ii} = 0$ instead of $1/s_i$

  $$S^{+}_{ii} = \begin{cases} 1/s_i & s_i \neq 0 \\ 0 & s_i = 0 \end{cases}$$

  This always exists. When $\mathbf{X}^{\mathsf{T}}\mathbf{X}$ is invertible it is the same as our pseudo-inverse, and when $\mathbf{X}$ is square and invertible it is $\mathbf{X}^{-1}$
- $\boldsymbol{w} = \mathbf{X}^{+}\boldsymbol{y}$ minimises $\|\mathbf{X}\,\boldsymbol{w} - \boldsymbol{y}\|^2$ and, of all the weight vectors that do so, it is the one with the smallest $\|\boldsymbol{w}\|$
- To see why, rotate to the coordinates given by the SVD: let $\boldsymbol{z} = \mathbf{V}^{\mathsf{T}}\boldsymbol{w}$ and $\boldsymbol{c} = \mathbf{U}^{\mathsf{T}}\boldsymbol{y}$. Orthogonal matrices do not change lengths, so $\|\boldsymbol{w}\| = \|\boldsymbol{z}\|$ and

  $$\|\mathbf{X}\,\boldsymbol{w} - \boldsymbol{y}\|^2 = \left\|\mathbf{U}^{\mathsf{T}}(\mathbf{U}\,\mathbf{S}\,\mathbf{V}^{\mathsf{T}}\boldsymbol{w} - \boldsymbol{y})\right\|^2 = \|\mathbf{S}\,\boldsymbol{z} - \boldsymbol{c}\|^2 = \sum_{i:\, s_i \neq 0} (s_i\, z_i - c_i)^2 + \sum_{i:\, s_i = 0} c_i^2$$

  (in the second sum i runs over the rows of $\mathbf{S}$ that have no non-zero singular value)
  - The loss is minimised by $z_i = c_i / s_i$ whenever $s_i \neq 0$
  - The other components of $\boldsymbol{z}$ do not appear in the loss at all: these are the directions $\boldsymbol{v}_i$ with $\mathbf{X}\,\boldsymbol{v}_i = \mathbf{0}$, where the data give no information
  - Every choice of them fits equally well. Setting them to zero gives the smallest $\|\boldsymbol{z}\| = \|\boldsymbol{w}\|$, and that is exactly what $S^{+}_{ii} = 0$ does
- It is also the limit of ridge regression as the regularisation vanishes: as $\eta \to 0$, $\hat{S}^{+}_{ii} = s_i/(s_i^2 + \eta)$ tends to $1/s_i$ if $s_i \neq 0$ and is 0 if $s_i = 0$
- This is what `np.linalg.pinv` computes, and `np.linalg.lstsq` returns the same minimum-norm solution. In practice "zero" means below a small tolerance, so tiny singular values caused by rounding errors are discarded rather than inverted

3 Exercises

3.1 Ridge regression

- Ridge regression is just linear regression with an L_2 regulariser
  1. Derive the optimal weights in ridge regression
  2. Show that using $\mathbf{X} = \mathbf{U}\,\mathbf{S}\,\mathbf{V}^{\mathsf{T}}$ then $\boldsymbol{w}^{*} = \mathbf{V}\,(\mathbf{S}^{\mathsf{T}}\mathbf{S} + \eta\,\mathbf{I})^{-1}\mathbf{S}^{\mathsf{T}}\,\mathbf{U}^{\mathsf{T}}\boldsymbol{y}$
- See answers

3.2 Spectral norm

1. Show that $\|\mathbf{X}\,\boldsymbol{x}\|_2 \le s_{\max}\,\|\boldsymbol{x}\|_2$, with equality for $\boldsymbol{x} = \boldsymbol{v}_{\max}$
2. Show that $\|\mathbf{X}\|_F^2 = \operatorname{Tr}\mathbf{X}^\mathsf{T}\mathbf{X} = \sum_i s_i^2$, and hence that $\|\mathbf{X}\|_2 \le \|\mathbf{X}\|_F$
3. For $\mathbf{J} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$
   (a) Show that both eigenvalues of $\mathbf{J}$ are 1
   (b) Find the singular values of $\mathbf{J}$ from the eigenvalues of $\mathbf{J}^\mathsf{T}\mathbf{J}$, and hence $\|\mathbf{J}\|_2$ and the condition number
   (c) Repeat for the symmetric matrix $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$, whose eigenvalues are 3 and -1

4 Experiments

4.1 SVD

- Check the claims in these notes on a random 3×4 matrix

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(3, 4))
U, s, Vt = np.linalg.svd(X)       # s is a vector, Vt is V transposed
S = np.zeros_like(X)
np.fill_diagonal(S, s)            # the 3 x 4 matrix S
V = Vt.T
print(np.allclose(U @ S @ Vt, X))                        # X = U S V^T
print(np.allclose(U @ U.T, np.eye(3)), np.allclose(V.T @ V, np.eye(4)))
print(np.linalg.eigvalsh(X @ X.T)[::-1], s**2)           # eigenvalues are s_i^2
print(np.linalg.eigvalsh(X.T @ X)[::-1])                 # the same, plus a 0

# spectral norm: the largest stretch of any unit vector
print(np.linalg.norm(X, 2), s[0])                        # both s_max
print(np.linalg.norm(X @ V[:, 0]), np.linalg.norm(X @ V[:, 1]))  # s_1, s_2
x = rng.normal(size=4)
print(np.linalg.norm(X @ x) <= s[0] * np.linalg.norm(x))   # never above s_max
print(np.linalg.norm(X, 'fro')**2, np.sum(s**2))           # Frobenius^2

# ridge regression through the SVD
eta = 0.1
lhs = np.linalg.inv(X.T @ X + eta * np.eye(4))
rhs = V @ np.linalg.inv(S.T @ S + eta * np.eye(4)) @ V.T
print(np.allclose(lhs, rhs))
print(np.linalg.cond(X.T @ X))    # X^T X is singular (rank 3): enormous
```

- Every check prints `True`. The unit vector $\boldsymbol{v}_1$ is stretched by exactly s_1 and $\boldsymbol{v}_2$ by s_2
- $\mathbf{X}^\mathsf{T}\mathbf{X}$ is 4×4 but only has rank 3, so it has no inverse. Rounding errors mean numpy may still return one, with a condition number around 10^{16}: a warning sign never to ignore. Adding $\eta\,\mathbf{I}$ makes it safely invertible

5 Answers

5.1 Ridge regression

1. It is straightforward to show
$$\boldsymbol{w}^* = (\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\,\mathbf{I})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$$

2. The only hard part is to show that
$$(\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\,\mathbf{I})^{-1} = \mathbf{V}\,(\mathbf{S}^\mathsf{T}\mathbf{S} + \eta\,\mathbf{I})^{-1}\mathbf{V}^\mathsf{T}$$
   - It is easy to show that $\mathbf{X}^\mathsf{T}\mathbf{X} = \mathbf{V}\,\mathbf{S}^\mathsf{T}\mathbf{S}\,\mathbf{V}^\mathsf{T}$
   - But we also have $\mathbf{I} = \mathbf{V}\,\mathbf{V}^\mathsf{T}$ as $\mathbf{V}$ is an orthogonal matrix
   - Thus $\mathbf{M} = \mathbf{X}^\mathsf{T}\mathbf{X} + \eta\,\mathbf{I} = \mathbf{V}\,(\mathbf{S}^\mathsf{T}\mathbf{S} + \eta\mathbf{I})\mathbf{V}^\mathsf{T} = \mathbf{V}\,\mathbf{W}\,\mathbf{V}^\mathsf{T}$ where $\mathbf{W} = \mathbf{S}^\mathsf{T}\mathbf{S} + \eta\mathbf{I}$
   - But $(\mathbf{A}\,\mathbf{B}\,\mathbf{C})^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}$ (which we can verify by multiplying $\mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}$ on either the left or right by $\mathbf{A}\,\mathbf{B}\,\mathbf{C}$)
   - Thus $\mathbf{M}^{-1} = (\mathbf{V}\,\mathbf{W}\,\mathbf{V}^\mathsf{T})^{-1} = (\mathbf{V}^\mathsf{T})^{-1}\,\mathbf{W}^{-1}\mathbf{V}^{-1} = \mathbf{V}\,\mathbf{W}^{-1}\mathbf{V}^\mathsf{T}$ where we use $\mathbf{V}^{-1} = \mathbf{V}^\mathsf{T}$ as $\mathbf{V}$ is an orthogonal matrix

5.2 Spectral norm

1. See the proof in the main points: expand $\boldsymbol{x}$ in the right singular vectors, so $\|\mathbf{X}\boldsymbol{x}\|^2 = \sum_i s_i^2 c_i^2 \leq s_{\max}^2 \sum_i c_i^2$. For $\boldsymbol{x} = \boldsymbol{v}_{\max}$ only one c_i is non-zero and the inequality is an equality

2. $\sum_{ij} X_{ij}^2 = \sum_j (\mathbf{X}^\mathsf{T}\mathbf{X})_{jj} = \operatorname{Tr}\mathbf{X}^\mathsf{T}\mathbf{X}$. With $\mathbf{X}^\mathsf{T}\mathbf{X} = \mathbf{V}\,\mathbf{S}^\mathsf{T}\mathbf{S}\,\mathbf{V}^\mathsf{T}$ and the cyclic property of the trace this is $\operatorname{Tr}\mathbf{S}^\mathsf{T}\mathbf{S} = \sum_i s_i^2$. Every term is non-negative, so the sum is at least $s_{\max}^2$

3. Two 2×2 matrices

   (a) $\mathbf{J}$ is triangular, so its eigenvalues are its diagonal entries, 1 and 1

   (b) $\mathbf{J}^\mathsf{T}\mathbf{J} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$ has $\lambda^2 - 3\lambda + 1 = 0$, so $\lambda = (3 \pm \sqrt{5})/2$ and the singular values are $s = (\sqrt{5} \pm 1)/2$, i.e. 1.618 and 0.618. So $\|\mathbf{J}\|_2 = (1 + \sqrt{5})/2 \approx 1.618$ (the golden ratio) although every eigenvalue is 1, and $\kappa = 1.618/0.618 \approx 2.62$. The eigenvalues of a non-symmetric matrix do not tell you how much it stretches

   (c) Here $s_i = |\lambda_i| = 3$ and 1, so the spectral norm is 3 and $\kappa = 3$. For a symmetric matrix the eigenvalues do tell the whole story

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 10: Optimisation

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Gradient descent, Newton's method, linear, super-linear and quadratic convergence, condition number, BFGS, Levenberg–Marquardt

2 Main Points

2.1 Machine Learning as Optimisation

- Once you've designed your learning machine and chosen your loss function the rest is optimisation: find the weights $\boldsymbol{w}$ that minimise the loss $L(\boldsymbol{w})$
- This lecture covers the classical methods, which work well for up to thousands of parameters (L-BFGS: millions) when we can afford gradients computed from all the data. The next lecture covers the very large problems of deep learning

2.2 Gradient Descent

- At a minimum (or a maximum, or a saddle point) $\boldsymbol{\nabla} L = \mathbf{0}$. For most models we cannot solve this, so we search iteratively
- The gradient points in the direction in which the loss increases fastest, so we step the other way
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - r\,\boldsymbol{\nabla} L(\boldsymbol{w}^{(t)})$$
- The learning rate (step size) r is the difficulty: too small and we crawl, too large and we diverge

2.3 Newton's Method

- Near a minimum almost every loss is quadratic (higher order terms in the Taylor expansion are negligible). In one dimension, if $f(x) = a + \frac{b}{2}(x - x^*)^2$ then $f'(x) = b\,(x - x^*)$ and $f''(x) = b$, so
$$x^* = x - \frac{f'(x)}{f''(x)}$$
- In many dimensions the second derivative becomes the Hessian, $H_{ij} = \partial^2 L/\partial w_i\,\partial w_j$, and
$$\boldsymbol{w}^* = \boldsymbol{w} - \mathbf{H}^{-1}\boldsymbol{\nabla} L(\boldsymbol{w})$$
- For a loss that is not exactly quadratic we iterate this. Close to a minimum it converges *quadratically*: an error ϵ becomes one of order ϵ^2, then ϵ^4, and so on, roughly doubling the number of correct digits each step (Exercise 4)

- Its drawbacks: the Hessian has N^2 entries and is costly to compute and invert, and far from a minimum Newton's method can head in the wrong direction (towards a maximum or saddle, or anywhere)

2.4 Step Size and Different Length Scales

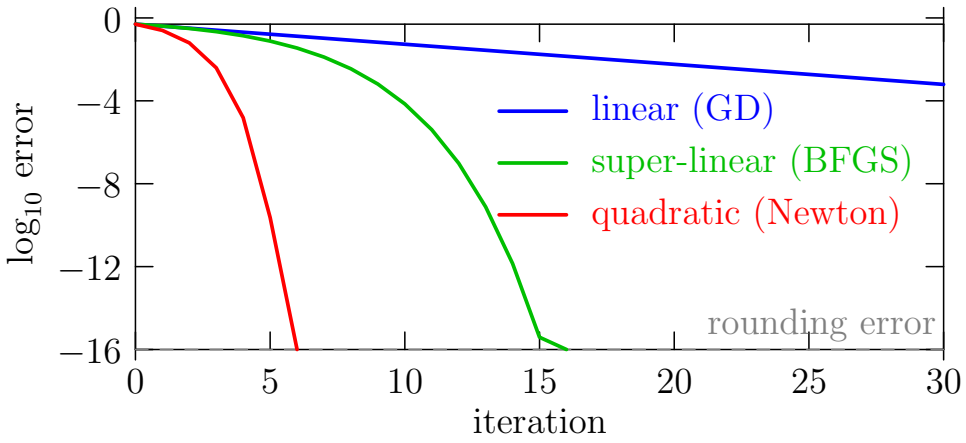
- Near a minimum $\boldsymbol{w}^*$ take the quadratic approximation $L(\boldsymbol{w}) \approx L(\boldsymbol{w}^*) + \frac{1}{2}(\boldsymbol{w}-\boldsymbol{w}^*)^\mathsf{T}\mathbf{H}(\boldsymbol{w}-\boldsymbol{w}^*)$. A gradient step gives
$$\boldsymbol{w}^{(t+1)} - \boldsymbol{w}^* = (\mathbf{I} - r\,\mathbf{H})\,(\boldsymbol{w}^{(t)} - \boldsymbol{w}^*)$$

- Write $\mathbf{H} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}$ and $u_i = \boldsymbol{v}_i^\mathsf{T}(\boldsymbol{w} - \boldsymbol{w}^*)$, the error along the i-th eigenvector. Then each component evolves on its own:
$$u_i^{(t+1)} = (1 - r\,\lambda_i)\,u_i^{(t)}$$

- Every component must shrink, $|1 - r\lambda_i| < 1$, so we need $r < 2/\lambda_{\max}$. The steepest direction limits the step size

- With r that small, the flattest direction shrinks by $1 - r\lambda_{\min} > 1 - 2\lambda_{\min}/\lambda_{\max}$ per step. The best fixed step, $r = 2/(\lambda_{\max} + \lambda_{\min})$, gives the factor
$$\frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$
where κ is the condition number of the Hessian (eigensystems notes). Badly conditioned problems ($\kappa \gg 1$) are slow

- **Rates of convergence.** If the error shrinks by a constant factor each step, convergence is *linear* (a straight line on a log plot). If the factor itself tends to zero it is *super-linear*, and if the error is squared each step it is *quadratic*



Gradient descent is linear, quasi-Newton methods such as BFGS are super-linear, and Newton's method is quadratic

2.5 Line Minimisation and Zig-Zags

- Rather than a fixed step we can minimise along the gradient direction (a line search) at every step

- At the line minimum the new gradient is orthogonal to the direction we came from, so in a narrow valley successive steps are at right angles: the path zig-zags across the valley rather than going along it

2.6 Quasi-Newton Methods

- These use curvature information without computing the full Hessian
- **Conjugate gradient** chooses search directions that do not undo the progress made along earlier ones. With exact line searches it reaches the minimum of an N-dimensional quadratic in N steps
- **BFGS** (Broyden–Fletcher–Goldfarb–Shanno) builds an approximation $\mathbf{B} \approx \mathbf{H}^{-1}$ from the gradients it has already seen, requiring at each step that

  $$\mathbf{B}\left(\boldsymbol{\nabla} L(\boldsymbol{w}^{(t+1)}) - \boldsymbol{\nabla} L(\boldsymbol{w}^{(t)})\right) = \boldsymbol{w}^{(t+1)} - \boldsymbol{w}^{(t)}$$

  (for a quadratic, $\boldsymbol{\nabla} L$ changes by exactly $\mathbf{H}$ times the step). It steps along $-\mathbf{B}\,\boldsymbol{\nabla} L$ with a line search and converges super-linearly
  - $\mathbf{B}$ is $N \times N$. *L-BFGS* keeps only the last ten or so steps and gradient changes, so its memory is $O(N)$
- **Levenberg–Marquardt** is for least-squares losses $L(\boldsymbol{w}) = \|\boldsymbol{\epsilon}(\boldsymbol{w})\|^2$ with residuals $\epsilon_i = f(\boldsymbol{x}_i|\boldsymbol{w}) - y_i$
  - Linearise the residuals about the current weights, $\boldsymbol{\epsilon}(\boldsymbol{w}) \approx \boldsymbol{\epsilon}(\boldsymbol{w}^{(k)}) + \mathbf{J}(\boldsymbol{w} - \boldsymbol{w}^{(k)})$, where $\mathbf{J}$ is the Jacobian $J_{ij} = \partial\epsilon_i/\partial w_j$
  - Minimising the linearised loss gives the Gauss–Newton step $\boldsymbol{w}^{(k+1)} = \boldsymbol{w}^{(k)} - (\mathbf{J}^\mathsf{T}\mathbf{J})^{-1}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)})$: Newton's method with $\mathbf{J}^\mathsf{T}\mathbf{J}$ standing in for the Hessian
  - Adding a penalty $\nu \left\|\boldsymbol{w} - \boldsymbol{w}^{(k)}\right\|^2$ on the step size gives the Levenberg–Marquardt step (Exercise 5)

    $$\boldsymbol{w}^{(k+1)} = \boldsymbol{w}^{(k)} - (\mathbf{J}^\mathsf{T}\mathbf{J} + \nu\,\mathbf{I})^{-1}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)})$$

  - Large ν gives a short step along the negative gradient; small ν gives Gauss–Newton. ν is reduced when the linear model predicts the drop in loss well and increased when it does not (a *trust region*)
- Non-gradient methods (Nelder–Mead, Powell's method, evolutionary strategies) also exist, but are not competitive in high dimensions

2.7 Where These Methods Are Used

- Logistic regression: scikit-learn's default solver is L-BFGS, and Newton's method is the standard fit for generalised linear models
- Gaussian processes: the kernel hyper-parameters are usually fitted with L-BFGS
- Curve fitting (non-linear least squares): Levenberg–Marquardt, e.g. SciPy's `curve_fit`
- Support vector machines: training is a quadratic programme (later lectures)
- With millions to billions of parameters, and gradients estimated from mini-batches, we need the methods of the next lecture

3 Exercises

1. **Divergence.** Take $L(w) = \frac{1}{2}w^2$ (one dimension) and update $w^{(t+1)} = w^{(t)} - r\,L'(w^{(t)})$
   (a) Compute the optimum step size
   (b) What is $r_{\max}$ such that we no longer converge if $r \geq r_{\max}$?

(c) If $r > r_{\max}$ calculate how $w^{(t)}$ grows with time

2. **Quadratic optima.** Consider the 2-d loss $L(\boldsymbol{w}) = w_1^2/2 + w_2^2 - w_1\, w_2$

   (a) Compute the gradient

   (b) Compute the Hessian

   (c) Compute the eigenvalues of the Hessian

   (d) Plot the contour lines of $L(\boldsymbol{w})$

3. **Condition number.** For the loss in Exercise 2

   (a) Find the condition number κ and the largest stable step size

   (b) With the best fixed step $r = 2/(\lambda_{\max} + \lambda_{\min})$, how many gradient steps reduce the error by a factor 10^6?

   (c) How many steps does Newton's method need?

4. **Newton's method converges quadratically.** Let $\epsilon = x - x^*$ be the error before a Newton step and ϵ' the error after. By Taylor expanding $f'(x)$ and $f''(x)$ about x^*, show that

$$\epsilon' \approx \frac{f'''(x^*)}{2\, f''(x^*)}\, \epsilon^2$$

5. **Levenberg–Marquardt.** Show that minimising $\|\boldsymbol{\epsilon} + \mathbf{J}\,\boldsymbol{d}\|^2 + \nu\, \|\boldsymbol{d}\|^2$ over the step $\boldsymbol{d}$ gives $\boldsymbol{d} = -(\mathbf{J}^\mathsf{T}\mathbf{J} + \nu\, \mathbf{I})^{-1}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}$. What happens when ν is very large?

4 Experiments

4.1 Comparing optimisers on the Rosenbrock function

- The Rosenbrock function $L = 100\,(w_2 - w_1^2)^2 + (1 - w_1)^2$ (the example in the lecture, with residuals $\epsilon_1 = 10\,(w_2 - w_1^2)$ and $\epsilon_2 = 1 - w_1$) has a long, curved, narrow valley

```
import numpy as np
from scipy.optimize import minimize, least_squares, rosen, rosen_der, rosen_hess

x0 = np.array([-1.2, 1.0])        # the classic start; the minimum is at (1, 1)

x, r = x0.copy(), 1e-3            # gradient descent: a larger r diverges
for t in range(1, 200001):
    x = x - r * rosen_der(x)
    if np.linalg.norm(x - 1) < 1e-6: break
print("gradient descent:", t, "steps")

for method in ["BFGS", "L-BFGS-B", "trust-exact"]:   # trust-exact: Newton
    hess = rosen_hess if method == "trust-exact" else None
    res = minimize(rosen, x0, method=method, jac=rosen_der, hess=hess, tol=1e-12)
    print(f"{method}: {res.nit} iterations")

# Levenberg-Marquardt on the residuals from the lecture
eps = lambda x: np.array([10 * (x[1] - x[0]**2), 1 - x[0]])
res = least_squares(eps, x0, method="lm", xtol=1e-12)
print("Levenberg-Marquardt:", res.nfev, "function evaluations")
```

- Gradient descent needs about 34,000 steps; BFGS, L-BFGS, Newton (with a trust region) and Levenberg–Marquardt need 20–40. Try increasing `r` to see gradient descent diverge

5 Answers

1. The update is $w^{(t+1)} = (1 - r)\, w^{(t)}$

   (a) The optimum step size is $r = 1$, which jumps straight to the minimum at $w = 0$

   (b) $r_{\max} = 2$, where $|1 - r| = 1$

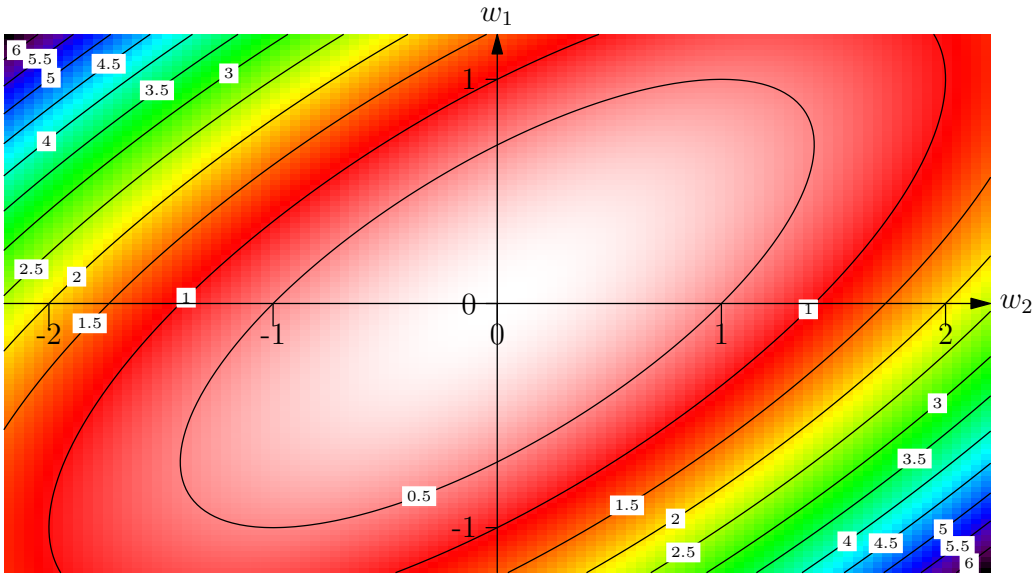
   (c) $w^{(t)} = (1 - r)^t\, w^{(0)}$: its size grows as $(r - 1)^t$, with alternating sign

2. Quadratic optima

   (a)
   $$\nabla L(\boldsymbol{w}) = \begin{pmatrix} w_1 - w_2 \\ 2\, w_2 - w_1 \end{pmatrix}$$

   (b)
   $$\mathbf{H} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix}$$

   (c) Let $T = \operatorname{tr} \mathbf{H} = 3$ and $D = \det(\mathbf{H}) = 1$ then $\lambda = \frac{1}{2}\left(T \pm \sqrt{T^2 - 4\,D}\right) = (3 \pm \sqrt{5})/2 = \{0.382, 2.618\}$

   (d)
   

3. Condition number

   (a) $\kappa = 2.618/0.382 \approx 6.85$, and stability needs $r < 2/2.618 \approx 0.76$

   (b) $r = 2/3$ gives the factor $(\kappa - 1)/(\kappa + 1) \approx 0.745$ per step, so we need n with $0.745^n = 10^{-6}$: $n = \ln 10^{-6} / \ln 0.745 \approx 47$ steps

   (c) One: the loss is exactly quadratic, so a single Newton step lands on the minimum

4. Expanding about x^*, where $f'(x^*) = 0$: $f'(x) \approx f''\epsilon + \frac{1}{2} f'''\epsilon^2$ and $f''(x) \approx f'' + f'''\epsilon$ (derivatives at x^*). Then

   $$\epsilon' = \epsilon - \frac{f'(x)}{f''(x)} \approx \epsilon - \epsilon\, \frac{1 + \frac{f'''}{2f''}\epsilon}{1 + \frac{f'''}{f''}\epsilon} \approx \epsilon - \epsilon \left(1 - \frac{f'''}{2f''}\epsilon\right) = \frac{f'''}{2f''}\,\epsilon^2$$

   using $1/(1 + a) \approx 1 - a$ for small a

5. The gradient with respect to $\boldsymbol{d}$ is $2\mathbf{J}^\mathsf{T}(\boldsymbol{\epsilon} + \mathbf{J}\boldsymbol{d}) + 2\nu\, \boldsymbol{d}$. Setting it to zero gives $(\mathbf{J}^\mathsf{T}\mathbf{J} + \nu\, \mathbf{I})\, \boldsymbol{d} = -\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}$. For large ν, $\boldsymbol{d} \approx -\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}/\nu = -\nabla L/(2\nu)$: a short gradient-descent step

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 11: Stochastic Gradient Descent

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- SGD, momentum, Nesterov, RMSProp, Adam, AdamW, learning-rate schedules, loss landscapes

2 Main Points

2.1 Optimisation at Scale

- Deep networks have millions to billions of parameters. A Hessian with 10^9 parameters would have 10^{18} entries, so Newton and quasi-Newton methods are out; we use gradients only
- **Mini-batches.** Rather than the gradient of the loss over the whole dataset $\mathcal{D}$ we use a random mini-batch $\mathcal{B} \subset \mathcal{D}$, $|\mathcal{B}| \ll |\mathcal{D}|$:

$$\boldsymbol{\nabla} L_{\mathcal{B}}(\boldsymbol{w}) = \frac{1}{|\mathcal{B}|} \sum_{(\boldsymbol{x},y)\in\mathcal{B}} \boldsymbol{\nabla} L(\boldsymbol{x}, y|\boldsymbol{w})$$

  Averaging (rather than summing) makes this an unbiased estimate of the full gradient $\boldsymbol{\nabla} L_{\mathcal{D}}$. Following it is *stochastic gradient descent* (SGD)

  - Each step is much cheaper, so we can take many small steps, and over many steps the random errors average out
- **Automatic differentiation** computes gradients of almost any program, which is what made very large models practical
- **Why the slower asymptotic convergence doesn't matter much**
  - We spend most of the time far from a quadratic minimum, where the asymptotic rates do not apply
  - With ReLUs the loss is not twice differentiable, so the Taylor argument behind Newton's method breaks down
  - We really want low generalisation error; the training loss is only a surrogate, so its exact minimum is not that important

2.2 One Step Size, Many Curvatures

- From the last lecture: along the i-th eigenvector of the Hessian a gradient step multiplies the error by $1 - r\,\lambda_i$, so we need $r < 2/\lambda_{\max}$, and the flattest directions then shrink by at best $1 - 2/\kappa$ per step, with κ the condition number
- Too large an r diverges, quickly giving NaNs. Everything in this lecture is a way round this conflict

2.3 Momentum

- In a narrow valley gradient descent steps consistently along the flat direction but bounces across the steep one. Momentum remembers earlier steps

$$\boldsymbol{v}^{(t+1)} = \gamma\, \boldsymbol{v}^{(t)} - r\, \boldsymbol{\nabla} L_{\mathcal{B}}(\boldsymbol{w}^{(t)})$$
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \boldsymbol{v}^{(t+1)}$$

  with $0 < \gamma < 1$ (typically 0.9): how much velocity is kept, so $1 - \gamma$ acts as friction

- Where the gradient keeps the same sign the steps add up, to an effective step $r/(1-\gamma)$ (ten times larger for $\gamma = 0.9$, Exercise 3). Where it alternates they cancel

- **Nesterov momentum** takes the gradient where momentum is about to carry us,

$$\boldsymbol{v}^{(t+1)} = \gamma\, \boldsymbol{v}^{(t)} - r\, \boldsymbol{\nabla} L\big(\boldsymbol{w}^{(t)} + \gamma\, \boldsymbol{v}^{(t)}\big)$$

  - With well-tuned momentum (heavy-ball or Nesterov) the error factor per step on a quadratic improves from about $1 - 2/\kappa$ to about $1 - 2/\sqrt{\kappa}$: the number of steps scales as $\sqrt{\kappa}$ rather than κ. For $\kappa = 10^4$ that is roughly 50 steps per factor e instead of 5000
  - Convergence is still *linear*: momentum fixes the condition number, not the order of convergence. Super-linear convergence needs curvature information (BFGS, last lecture)
  - On general convex problems Nesterov's method reaches error $O(1/t^2)$ after t steps, against $O(1/t)$ for gradient descent; no method that only uses gradients can do better. The optional module COMP6260, Optimisation for Machine Learning, covers this properly

2.4 Adaptive Methods

- Different parameters need different step sizes. Adaptive methods rescale the step for each parameter separately, which roughly makes the valleys more spherical

- **RMSProp** keeps a running average of the squared gradient for each parameter and divides by its square root

$$S_i^{(t+1)} = (1-\gamma)\, S_i^{(t)} + \gamma \left(\frac{\partial L_{\mathcal{B}}}{\partial w_i}\right)^2$$
$$w_i^{(t+1)} = w_i^{(t)} - \frac{r}{\sqrt{S_i^{(t+1)}} + \epsilon} \frac{\partial L_{\mathcal{B}}}{\partial w_i}$$

  Parameters with large gradients take small steps and those with small gradients large ones; each moves about r per step, whatever the scale of the loss

- **Adam** (adaptive moment estimation) is RMSProp plus momentum. It keeps running averages of the gradient and of its square,

$$M_i^{(t+1)} = (1-\beta)\, M_i^{(t)} + \beta\, \frac{\partial L_{\mathcal{B}}}{\partial w_i}, \qquad S_i^{(t+1)} = (1-\gamma)\, S_i^{(t)} + \gamma \left(\frac{\partial L_{\mathcal{B}}}{\partial w_i}\right)^2$$

  - Both start at zero, so early on they are biased towards zero. After $t+1$ updates a constant input x gives only $(1-(1-\beta)^{t+1})\, x$ (Exercise 1), so Adam corrects them:

$$\hat{M}_i^{(t+1)} = \frac{M_i^{(t+1)}}{1-(1-\beta)^{t+1}} \qquad\qquad \hat{S}_i^{(t+1)} = \frac{S_i^{(t+1)}}{1-(1-\gamma)^{t+1}}$$

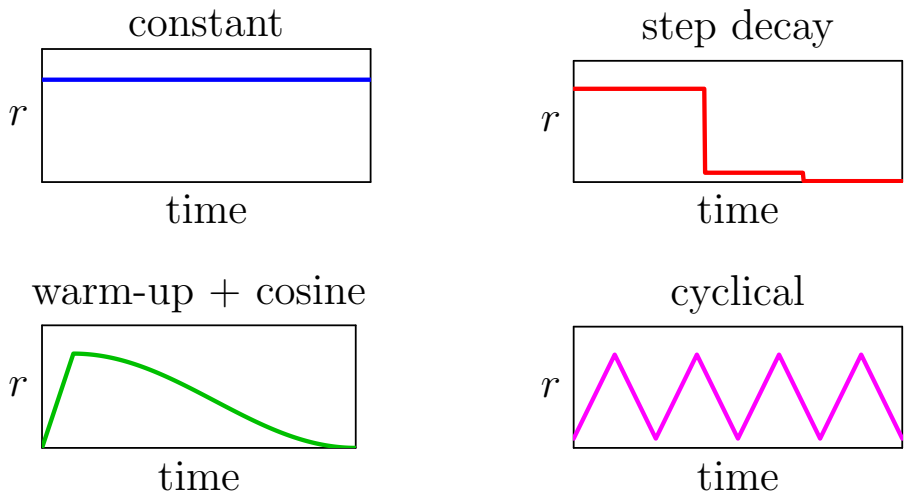
- The update is
$$w_i^{(t+1)} = w_i^{(t)} - \frac{r}{\sqrt{\hat{S}_i^{(t+1)}} + \epsilon} \hat{M}_i^{(t+1)}$$
- In this notation the usual defaults are $\beta = 0.1$ and $\gamma = 0.001$ (the literature writes $\beta_1 = 1 - \beta = 0.9$ and $\beta_2 = 1 - \gamma = 0.999$)
- Adam and its variants are very robust: a network that will not train with SGD often trains with Adam

- **AdamW.** Weight decay pulls the weights towards zero. Adding $\frac{\eta}{2} \|\boldsymbol{w}\|^2$ to the loss adds $\eta\, w_i$ to each gradient, which Adam then divides by $\sqrt{\hat{S}_i}$, so weights with large gradients are hardly decayed at all. AdamW applies the decay to the weights directly, $w_i \leftarrow w_i - r\,\eta\, w_i$, outside the adaptive rescaling (Exercise 4). It is the default for training transformers

- **Covariance.** Standard vector operations (addition, scalar and matrix multiplication) give results that do not depend on the coordinate system: rotate the coordinates and you get the rotated answer. Element-wise operations do not have this property
  - Adaptive methods rescale each coordinate separately, so they are not invariant to rotations
  - If two weights interact, the curvature can be high along $\boldsymbol{e}_i + \boldsymbol{e}_j$ and low along $\boldsymbol{e}_i - \boldsymbol{e}_j$. Rescaling w_i and w_j separately cannot equalise this

2.5 Learning-Rate Schedules

- In practice the learning rate is changed during training



  - Large r early: the noisy steps cross small valleys and find the big ones
  - Small r late: settle into a valley. Dividing r by ten when progress stalls often gives a sudden improvement (step decay)
  - Cosine decay lowers r smoothly; cyclical schedules repeatedly raise and lower it
  - A short *warm-up* at the start stops the first steps diverging before Adam's running averages have settled; common when training transformers

2.6 Loss Landscapes

- We minimise over an enormous space. Unless the loss is convex there will be many local minima, and for non-convex losses no efficient algorithm is guaranteed to find the global minimum. In such large spaces we may never reach even a local minimum

- Mini-batch noise means the weights keep moving; a smaller step averages over more batches and follows the true gradient more closely
- **Symmetries**
  - Permuting the neurons of a layer (or the feature maps of a CNN), together with their weights, gives the same network: $n!$ equivalent solutions for n hidden units
  - For odd activations such as $\tanh$, flipping the sign of a unit's input and output weights also changes nothing
  - There are continuous symmetries too: in a linear network $\mathbf{W}_2\mathbf{W}_1 = (\mathbf{W}_2\mathbf{U})(\mathbf{U}^{-1}\mathbf{W}_1)$ for any invertible $\mathbf{U}$
  - So there are huge sets of weights with the same loss. Between two equivalent minima there must be a saddle point
  - Skip connections (ResNets, transformers) break the permutation symmetry, which seems to make optimisation easier

3 Exercises

1. **Removing the start-up bias.** Consider a running average

$$a^{(t+1)} = (1-\gamma)\, a^{(t)} + \gamma\, x^{(t)}$$

   with constant input $x^{(t)} = x$ and $a^{(0)} = 0$

   (a) Write $a^{(t)}$ as a sum

   (b) Using the sum of a geometric series,

$$\sum_{i=0}^{t-1} z^i = 1 + z + \cdots + z^{t-1} = \frac{1-z^t}{1-z}$$

   write $a^{(t)}$ in closed form

   (c) Find the correction that makes the corrected average equal x for all t

2. **Gradient descent in a quadratic minimum.** Minimise

$$f(\boldsymbol{x}) = \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T}\mathbf{Q}(\boldsymbol{x} - \boldsymbol{x}^*)$$

   using $\boldsymbol{x}(t+1) = \boldsymbol{x}(t) - r\,\boldsymbol{\nabla} f(\boldsymbol{x})$

   (a) Write down the update equation

   (b) Subtract $\boldsymbol{x}^*$ from both sides and write down an update equation for $\boldsymbol{x}(t) - \boldsymbol{x}^*$

   (c) Write a closed form solution for $\boldsymbol{x}(t) - \boldsymbol{x}^*$

   (d) Using the eigen-decomposition $\mathbf{Q} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}$ rewrite the closed form solution

   (e) Defining $\boldsymbol{u}(t) = \mathbf{V}^\mathsf{T}(\boldsymbol{x}(t) - \boldsymbol{x}^*)$ write the update equation for the components $u_i(t)$ and explain what happens when $\lambda_i > 2/r$

3. **Momentum adds up.** Suppose the gradient is the same, $\boldsymbol{g}$, at every step. Starting from $\boldsymbol{v}^{(0)} = \mathbf{0}$, find $\boldsymbol{v}^{(t)}$ and its limit as $t \to \infty$

4. **Why AdamW?** With the loss $L(\boldsymbol{w}) + \frac{\eta}{2}\|\boldsymbol{w}\|^2$

   (a) Show that plain gradient descent on this loss is the same as gradient descent on L followed by $w_i \leftarrow w_i - r\,\eta\, w_i$

   (b) Explain why this is no longer true for Adam

4 Experiments

4.1 Gradient descent, momentum and Nesterov

- On a quadratic $f(\boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^{\mathsf{T}}\mathbf{H}\boldsymbol{w}$ with a random positive definite Hessian, compare the methods with the same step size

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(5, 4))
H = X.T @ X                          # Hessian of f(w) = w^T H w / 2
lam = np.linalg.eigvalsh(H)
print("eigenvalues:", np.round(lam, 2), " kappa =", round(lam[-1] / lam[0]))

f = lambda w: w @ H @ w / 2
grad = lambda w: H @ w
w0 = rng.normal(size=4)
r = 1 / lam[-1]                      # safely below 2 / lambda_max

def run(gamma, nesterov=False, steps=200):
    w, v = w0.copy(), np.zeros(4)
    for t in range(steps):
        g = grad(w + gamma * v) if nesterov else grad(w)
        v = gamma * v - r * g
        w = w + v
    return f(w)

print("gradient descent:", run(0.0))
print("momentum       :", run(0.9))
print("Nesterov       :", run(0.9, nesterov=True))
print("r = 2.1/lambda_max diverges:", end=" ")
r = 2.1 / lam[-1]; print(run(0.0))
```

- Here $\kappa \approx 39$. After 200 steps gradient descent reaches $f \approx 5 \times 10^{-6}$, momentum 10^{-10} and Nesterov 10^{-14}; just over the stability limit gradient descent blows up. Try other values of `gamma` and `r`

5 Solutions

1. Removing the start-up bias

   (a) Writing $a^{(t)}$ as a sum

   $$a^{(1)} = (1-\gamma)\,a^{(0)} + \gamma\,x = \gamma\,x$$
   $$a^{(2)} = (1-\gamma)\,a^{(1)} + \gamma\,x = (1-\gamma)\,\gamma\,x + \gamma\,x$$
   $$a^{(t)} = \gamma\,x \sum_{i=0}^{t-1} (1-\gamma)^i$$

   (b) To prove the identity, multiply the series by $1-z$: $(1-z)(1+z+\cdots+z^{t-1}) = 1-z^t$. Then

   $$a^{(t)} = \gamma\,x\,\frac{1-(1-\gamma)^t}{1-(1-\gamma)} = x\left(1-(1-\gamma)^t\right)$$

   which only approaches x as $t \to \infty$

(c) Divide by $1-(1-\gamma)^t$:

$$\bar{a}^{(t)} = \frac{a^{(t)}}{1-(1-\gamma)^t} = x$$

Adam's $\hat{M}^{(t+1)}$ and $\hat{S}^{(t+1)}$ are this correction after $t+1$ updates

2. Gradient descent in a quadratic minimum

   (a) $\boldsymbol{x}(t+1) = \boldsymbol{x}(t) - r\,\mathbf{Q}(\boldsymbol{x}(t) - \boldsymbol{x}^*)$

   (b) $\boldsymbol{x}(t+1) - \boldsymbol{x}^* = (\mathbf{I} - r\,\mathbf{Q})(\boldsymbol{x}(t) - \boldsymbol{x}^*)$

   (c) $\boldsymbol{x}(t) - \boldsymbol{x}^* = (\mathbf{I} - r\,\mathbf{Q})^t(\boldsymbol{x}(0) - \boldsymbol{x}^*)$

   (d) $\boldsymbol{x}(t) - \boldsymbol{x}^* = \mathbf{V}(\mathbf{I} - r\,\mathbf{\Lambda})^t\mathbf{V}^\mathsf{T}(\boldsymbol{x}(0) - \boldsymbol{x}^*)$, because $\mathbf{I} - r\,\mathbf{Q} = \mathbf{V}(\mathbf{I} - r\,\mathbf{\Lambda})\mathbf{V}^\mathsf{T}$ and in its t-th power every inner $\mathbf{V}^\mathsf{T}\mathbf{V}$ is $\mathbf{I}$

   (e) $u_i(t) = (1 - r\,\lambda_i)^t\,u_i(0)$. If $\lambda_i > 2/r$ then $|1 - r\lambda_i| > 1$ and $u_i(t)$ grows exponentially: any component along $\boldsymbol{v}_i$ diverges if the curvature in that direction exceeds $2/r$

3. $\boldsymbol{v}^{(1)} = -r\boldsymbol{g}$, $\boldsymbol{v}^{(2)} = -r\boldsymbol{g}(1+\gamma)$, and in general $\boldsymbol{v}^{(t)} = -r\,\boldsymbol{g}\,(1-\gamma^t)/(1-\gamma)$, which tends to $-r\,\boldsymbol{g}/(1-\gamma)$: an effective step size $r/(1-\gamma)$

4. Why AdamW

   (a) The gradient of the penalty is $\eta\,\boldsymbol{w}$, so the step is $\boldsymbol{w} - r\,\boldsymbol{\nabla}L - r\,\eta\,\boldsymbol{w}$: a gradient step on L plus the decay $w_i \leftarrow w_i - r\,\eta\,w_i$

   (b) Adam divides each component of the gradient, including $\eta\,w_i$, by $\sqrt{\hat{S}_i}$ (and averages it into the momentum). Weights with large gradients get large $\hat{S}_i$ and so very little decay. AdamW decays every weight by the same fraction $r\,\eta$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 12: Constrained Optimisation

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Lagrangians, Inequalities, KKT, Linear Programming,

2 Main Points

2.1 Equality Constraints

- If we want to minimise $f(\boldsymbol{x})$ subject to the constraint $g(\boldsymbol{x}) = 0$ this is equivalent to solving the problem

$$\min_{\boldsymbol{x}} \max_{\alpha} \mathcal{L}(\boldsymbol{x}, \alpha)$$

  where $\mathcal{L}(\boldsymbol{x}, \alpha)$ is a *Lagrangian* given by

$$\mathcal{L}(\boldsymbol{x}, \alpha) = f(\boldsymbol{x}) - \alpha\, g(\boldsymbol{x})$$

- α is a Lagrange multiplier that is determined by the joint optimisation problem
- Note that we seek a saddle-point
  - We minimise with respect to $\boldsymbol{x}$ and maximise with respect to α
  - We can't escape this
    - if we multiply α by -1 we just changing the directions of the α axis but still have a saddle-point
    - if we multiply both terms by -1 then we would end up minimising with respect to α, but maximising with respect to $\boldsymbol{x}$
- The solution to our problem must satisfy

$$\boldsymbol{\nabla}\mathcal{L}(\boldsymbol{x}, \alpha) = \boldsymbol{\nabla} f(\boldsymbol{x}) - \alpha\, \boldsymbol{\nabla} g(\boldsymbol{x}) = 0, \qquad \frac{\partial \mathcal{L}(\boldsymbol{x}, \alpha)}{\partial \alpha} = g(\boldsymbol{x}) = 0$$

  - The second equation ensures that we sit on the constraint
  - The first equation says that the gradient of $f(\boldsymbol{x})$ must be perpendicular to the constraint
  - This is necessary for the solution to be a (local) minimum (i.e. we can not get an improvement by moving along the constraint)
  - There can be multiple solutions: these equations at a satisfied for any local minima on the constraint
- If we have multiple equality constraints we just use multiple Lagrange multipliers

2.2 Inequality constraints

- If we are minimising $f(\boldsymbol{x})$ subject to an inequality constraint $g(\boldsymbol{x}) \geq 0$ then one of two things can happen

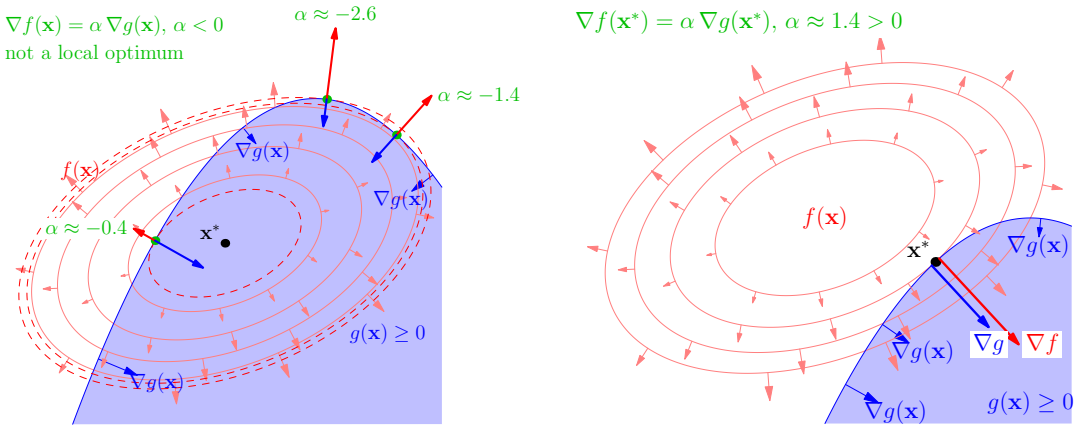
  1. Either we have a (local) minimum of $f(\boldsymbol{x})$ that satisfies the constraint or
  2. We have a local minimum on the constraint

- We can therefore solve this problem by again using a Lagrangian

$$\mathcal{L}(\boldsymbol{x}, \alpha) = f(\boldsymbol{x}) - \alpha\, g(\boldsymbol{x})$$

  with the additional constraint $\alpha \geq 0$

  - If the minimum lies in a region that satisfies the constraint then we just set $\alpha = 0$ and minimise $f(\boldsymbol{x})$
  - If the solution lies on the constraint we again have $\boldsymbol{\nabla} f(\boldsymbol{x}) = \alpha\, \boldsymbol{\nabla} g(\boldsymbol{x})$, but now $\alpha > 0$ which means that not only is there no improving direction along the constraint, but also the negative-gradient of $f(\boldsymbol{x})$ points in the direction where $g(\boldsymbol{x})$ becomes smaller, i.e. in the region that violates the constraint
    - note if $\alpha < 0$ we could find a better solution moving away from the constraint into the feasible region
    - In the figures below the unconstrained minimum $\boldsymbol{x}^*$ is feasible on the left. Three boundary points (green) still have $\boldsymbol{\nabla} f = \alpha\, \boldsymbol{\nabla} g$, but with $\alpha < 0$, so none is a local optimum. On the right the constraint excludes the unconstrained minimum, and at the constrained minimum $\alpha \approx 1.4 > 0$



  - *Karush-Kuhn-Tucker (KKT) Conditions*
    - For inequality constraints we require $\alpha \geq 0$ and $\alpha\, g(\boldsymbol{x}) = 0$ (*complementary slackness*), so either
      1. $\alpha = 0$: the constraint plays no part, and the minimum is an unconstrained one in the region $g(\boldsymbol{x}) \geq 0$ (it can happen to lie exactly on the boundary), or
      2. $\alpha > 0$ and the solution lies on $g(\boldsymbol{x}) = 0$

- If we have multiple inequality constraints we just introduce a Lagrange multiplier for each constraint with $\alpha \geq 0$

2.3 Duality

- Our problem of solving $f(\boldsymbol{x})$ subject to some constraints is known as the *primal problem*
- If our problem is sufficiently simple we can sometimes find a solution $\boldsymbol{x}^*(\boldsymbol{\alpha})$ that satisfies

$$\boldsymbol{\nabla} \mathcal{L}(\boldsymbol{x}^*(\boldsymbol{\alpha}), \boldsymbol{\alpha}) = 0$$

- This leaves us with the *dual problem*

$$\max_{\boldsymbol{\alpha}} \; \mathcal{L}(\boldsymbol{x}^*(\boldsymbol{\alpha}), \boldsymbol{\alpha})$$

possibly with constraints on $\boldsymbol{\alpha}$ (e.g. $\alpha_i \geq 0$) that arise from KKT conditions

- **Linear Programming**
  - In linear programming we are trying to find a value of $\boldsymbol{x}$ that minimises a linear objective function $\boldsymbol{c}^\mathsf{T}\boldsymbol{x}$ subject to linear constraints $\mathbf{M}\,\boldsymbol{x} = \boldsymbol{b}$ (and/or $\mathbf{M}\boldsymbol{x} \geq \boldsymbol{b}$)
  - We can write this as a Lagrange problem

$$\mathcal{L} = \boldsymbol{c}^\mathsf{T}\boldsymbol{x} - \boldsymbol{\alpha}^\mathsf{T}\left(\mathbf{M}\boldsymbol{x} - \boldsymbol{b}\right)$$

    (subject to constraints $\boldsymbol{\alpha} \geq 0$ if we have inequality constraints in the primal problem)
  - We can rearrange the Lagrangian as

$$\mathcal{L} = \boldsymbol{\alpha}^\mathsf{T}\boldsymbol{b} + \left(\boldsymbol{c}^\mathsf{T} - \boldsymbol{\alpha}^\mathsf{T}\mathbf{M}\right)\boldsymbol{x}$$

  - Using the identity $\boldsymbol{a}^\mathsf{T}\boldsymbol{b} = \boldsymbol{b}^\mathsf{T}\boldsymbol{a}$ we can rewrite this as

$$\mathcal{L} = \boldsymbol{b}^\mathsf{T}\boldsymbol{\alpha} - \boldsymbol{x}^\mathsf{T}\left(\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} - \boldsymbol{c}\right)$$

  - But we can now treat $\boldsymbol{x}$ as a Lagrange multiplier so we get the *dual problem*

$$\max_{\boldsymbol{\alpha}} \; \boldsymbol{b}^\mathsf{T}\boldsymbol{\alpha}$$

    subject to the constraint

$$\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} = \boldsymbol{c}$$

  - If the original constraints were inequalities, $\mathbf{M}\boldsymbol{x} \geq \boldsymbol{b}$, then $\boldsymbol{\alpha} \geq \mathbf{0}$; if also $\boldsymbol{x} \geq \mathbf{0}$ (as in the potato and rice example) the dual constraint becomes $\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} \leq \boldsymbol{c}$
  - The dimensionality of the dual problem can sometimes be much lower than that of the primal problem making it easier to solve

- **Quadratic Program**
  - In a quadratic program we have to minimise a quadratic loss $\boldsymbol{x}^\mathsf{T}\mathbf{Q}\,\boldsymbol{x}$ subject to linear constraints $\mathbf{M}\,\boldsymbol{x} = \mathbf{b}$ (or $\mathbf{M}\,\boldsymbol{x} \geq \mathbf{b}$)
  - For there to be a unique minimum $\mathbf{Q}$ must be positive definite (which is sometimes written $\mathbf{Q} \succ 0$)
  - We can write a Lagrangian

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\alpha}) = \boldsymbol{x}^\mathsf{T}\mathbf{Q}\,\boldsymbol{x} - \boldsymbol{\alpha}^\mathsf{T}\left(\mathbf{M}\,\boldsymbol{x} - \boldsymbol{b}\right)$$

  - The solution is given by $\max_{\boldsymbol{\alpha}} \min_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x}, \boldsymbol{\alpha})$
  - If the constraints are inequality constraints then $\alpha_i \geq 0$
  - The minimum with respect to $\boldsymbol{x}$ is given by

$$\boldsymbol{\nabla}_{\boldsymbol{x}}\mathcal{L}(\boldsymbol{x}, \boldsymbol{\alpha}) = 2\,\mathbf{Q}\,\boldsymbol{x} + \mathbf{M}^\mathsf{T}\boldsymbol{\alpha} = 0$$

  - So that $\boldsymbol{x}^* = \frac{1}{2}\mathbf{Q}^{-1}\mathbf{M}^\mathsf{T}$
  - Substituting this back into the Lagrangian we get the dual problem

$$\max_{\boldsymbol{\alpha}} -\frac{1}{4}\boldsymbol{\alpha}^\mathsf{T}\mathbf{M}\mathbf{Q}^{-1}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} + \boldsymbol{\alpha}^\mathsf{T}\boldsymbol{b}$$

    with $\alpha_i \geq 0$ if we started with inequality constraints
    - note in the derivation that we end up with two terms proportional to $\boldsymbol{\alpha}^\mathsf{T}\mathbf{M}\mathbf{Q}^{-1}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha}$ one partially cancelling the other

3 Exercises

3.1 Quadratic with a linear constraint

- Consider minimising $f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\mathsf{T}\mathbf{Q}\boldsymbol{x}$ subject to the constraint $\boldsymbol{a}^\mathsf{T}\boldsymbol{x} = b$
  1. Write a Lagrangian for this problem
  2. Find the minimum of the Lagrangian with respect to $\boldsymbol{x}$
  3. Write down and solve the dual problem
  4. Hence write down a solution to the primal problem
- See answers, but also experiments

3.2 Saddle Point

- Strangely (for me at least) the optimum of a constrained optimisation problem is given by the saddle-point of the Lagrangian
- Consider the problems of minimising $x^2/2$ subject to the constraint $x = 1$
  1. Write down the Lagrangian
  2. Calculate the Hessian matrix (matrix of second derivatives)
  3. Compute the eigenvalues of the Hessian (show that they have different signs everywhere so there are no maxima or minima)
- See answers

4 Experiments

4.1 Quadratic with a linear constraint

- Let X be a 10×5 random matrix with elements drawn from $\mathcal{N}(0, 1)$
- Let $\mathbf{Q} = \mathbf{X}^\mathsf{T}\mathbf{X}$
  - Check that this is positive definite
- Let $f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\mathsf{T}\mathbf{Q}\boldsymbol{x}$
- Let $\boldsymbol{a}$ be a random vector with 5 elements drawn from $\mathcal{N}(0, 1)$
- We want to minimise $f(\boldsymbol{x})$ subject to the constraint $\boldsymbol{a}^\mathsf{T}\boldsymbol{x} = 1$
- Work out the Lagrangian, $L(\boldsymbol{x}, \alpha)$ for this system
- Write an iterative gradient solver that
  1. Makes steps $\boldsymbol{x} \leftarrow \boldsymbol{x} - r\boldsymbol{\nabla}L(\boldsymbol{x}, \alpha)$
  2. Makes steps $\alpha \leftarrow \alpha + r\frac{\partial L(\boldsymbol{x},\alpha)}{\partial \alpha}$
- Note you will have to tune the learning step r
- Compare the solution you find by running your algorithm until convergence with the exact result (see exercise and/or answer)

```
function [x,alpha] = optimise(Q,a,r)
  x = rand(5,1);                                  % initialise x
  alpha = 0;                                        % initialise alpha
  for t=1:1000
    x = x - r*(Q*x-alpha*a);                    % x = x - r  \grad L
    alpha = alpha + r*(-(a'*x-1));          % alpha = alpha + r dL/d\alpha
  endfor
endfunction

[x,alpha] = optimise(Q,a,0.1)

alphaTheory = 1/(a'*Q*a)
xTheory = inv(Q)*a*alphaTheory
```

5 Answers

5.1 Quadratic with a linear constraint

1. The Lagrangian is given by

$$\mathcal{L}(\boldsymbol{x}, \alpha) = \frac{1}{2}\,\boldsymbol{x}^{\mathsf{T}}\mathbf{Q}\boldsymbol{x} - \alpha\,(\boldsymbol{a}^{\mathsf{T}}\boldsymbol{x} - b)$$

2. Minimising with respect to $\boldsymbol{x}$ we get

$$\boldsymbol{\nabla}\mathcal{L}(\boldsymbol{x}, \alpha) = \mathbf{Q}\,\boldsymbol{x} + \alpha\,\boldsymbol{a} = 0$$

   or $\boldsymbol{x} = \alpha\,\mathbf{Q}^{-1}\boldsymbol{a}$

3. Thus the dual problem is

$$\max_{\alpha} -\frac{1}{2}\alpha^2\,\boldsymbol{a}^{\mathsf{T}}\,\mathbf{Q}^{-1}\boldsymbol{a} + \alpha\,b$$

   - The solution to the dual problem is

$$\alpha = \frac{b}{\boldsymbol{a}^{\mathsf{T}}\,\mathbf{Q}^{-1}\boldsymbol{a}}$$

4. Thus the solution to the primal problem is

$$\boldsymbol{x} = \frac{b\,\mathbf{Q}^{-1}\boldsymbol{a}}{\boldsymbol{a}^{\mathsf{T}}\,\mathbf{Q}^{-1}\boldsymbol{a}}$$

   - Note that in most quadratic programming problems we are dealing with many inequality constraints so solving the dual problem in closed form isn't necessarily easy

5.2 Saddle Point

- Just do it

1. The Lagrangian is given bye

$$\mathcal{L} = \frac{x^2}{2} - \alpha\,(x - 1)$$

2. The Hessian is given by

$$\mathbf{H} = \begin{pmatrix} 1 & -1 \\ -1 & 0 \end{pmatrix}$$

3. The traces $T = 1$ and the determinant $D = -1$ So that

$$\lambda = \frac{T \pm \sqrt{T^2 - 4\,D}}{2} = \frac{1 \pm 5}{2} = \{1.618, -0.618\}$$

If you prefer you can compute the eigenvalues numerically

- Note that whatever we do the determinant will be negative leading to a negative eigenvalue (the determinant is equal to the product of eigenvalues). This would be true if we were maximising $-x^2/2$. You can change the constraints or the objective function, but you will still get eigenvalues of different signs.

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 13: Convexity

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Convex sets, convex functions, Jensen's inequality

2 Main Points

2.1 Convex Sets

- We are familiar geometrically with convex regions
- To define convexity we need to define an intermediate point $\boldsymbol{z} = a\,\boldsymbol{x} + (1-a)\,\boldsymbol{y}$
  - we require $a \in [0,1]$ (i.e. a is in the interval between 0 and 1) for $\boldsymbol{z}$ to be between $\boldsymbol{x}$ and $\boldsymbol{y}$
  - to define convexity we only need to have addition and scalar multiplication
- We can then define convexity in a very general way: a set $\mathcal{S}$ is convex if for every pair of points $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{S}$ and for every possible $a \in [0,1]$ then $\boldsymbol{z} = a\,\boldsymbol{x} + (1-a)\,\boldsymbol{y} \in \mathcal{S}$
- We can apply convexity to sets of complicated objects
- For example the set of positive semi-definite matrices forms a convex set
  - This follows from the fact the sum of two positive semi-definite matrices is also positive semi-definite and if we multiply a positive semi-definite matrix by a non-negative number then the matrix is still positive semi-definite

2.2 Convex Functions

- We can define a function, $f(\boldsymbol{x})$, to be a *convex function* if for all pairs of points in the domain of the function and all $a \in [0,1]$

$$f(a\,\boldsymbol{x} + (1-a)\,\boldsymbol{y}) \leq a\,f(\boldsymbol{x}) + (1-a)\,f(\boldsymbol{y})$$

- This means that the function sits on or below the straight chord joining the function values at any two points of its domain
- The epigraph of a function is the area that lies on or above the function
  - The epigraph of a convex function forms a convex region
  - If the epigraph of a function forms a convex region then the function is convex
- We can define *convex-down* or *concave* functions by reversing the inequality

$$f(a\,\boldsymbol{x} + (1-a)\,\boldsymbol{y}) \geq a\,f(\boldsymbol{x}) + (1-a)\,f(\boldsymbol{y})$$

- for clarity I will sometimes refer to "convex" functions as *convex-up* functions
- convex-down functions have similar (but opposite) properties to convex up functions

- A function where for every pair of distinct points $\boldsymbol{x} \neq \boldsymbol{y}$ and for any a such that $0 < a < 1$ (i.e. a lies strictly between 0 and 1)

$$f(a\,\boldsymbol{x} + (1-a)\,\boldsymbol{y}) < a\,f(\boldsymbol{x}) + (1-a)\,f(\boldsymbol{y})$$

  is said to be *strictly convex*

- Linear functions $f(\boldsymbol{x}) = \boldsymbol{m}^{\mathsf{T}}\boldsymbol{x} + c$ are both convex-up and convex-down functions
  - For a function to be a strictly convex function it cannot have a linear section

- Convex functions lie on or above their tangent plane (where they are differentiable)

$$f(\boldsymbol{x}) \geq f(\boldsymbol{x}_0) + (\boldsymbol{x} - \boldsymbol{x}_0)^{\mathsf{T}}\boldsymbol{\nabla} f(\boldsymbol{x}_0)$$

  - The tangent plane at $\boldsymbol{x}_0$ is the linear function that matches both the value $f(\boldsymbol{x}_0)$ and the gradient $\boldsymbol{\nabla} f(\boldsymbol{x}_0)$ there. (The gradient is orthogonal to the *contour* through $\boldsymbol{x}_0$, not to the tangent plane)
  - Where f is not differentiable (e.g. $|x|$ at $x = 0$) there is no unique tangent, but a convex function still lies above some line through each point (a *supporting* line, whose slope is called a *subgradient*). This is enough for everything below, including Jensen's inequality

- Why the second derivative is non-negative: the tangent lines at x and $x + \epsilon$ (with $\epsilon > 0$) give

$$\begin{aligned} f(x+\epsilon) &\geq f(x) + \epsilon\,f'(x) \\ f(x) &\geq f(x+\epsilon) - \epsilon\,f'(x+\epsilon) \end{aligned}$$

  Adding these, $0 \geq \epsilon\left(f'(x) - f'(x+\epsilon)\right)$, so $f'(x+\epsilon) \geq f'(x)$: the slope never decreases, and hence $f''(x) \geq 0$

- A necessary and sufficient condition for a twice-differentiable function to be convex is that its second derivative is non-negative or for multi-dimensional functions the Hessian is positive semi-definite
  - If the second derivative is positive (i.e. always greater than 0) or the Hessian is positive definite then the function is strictly convex
  - This is sufficient but not necessary: x^4 is strictly convex although its second derivative vanishes at $x = 0$

- Examples
  - Convex-up Functions
    - $f(x) = x^2$ is strictly convex since $f''(x) = 2 > 0$
    - $f(x) = x^{-2}$ (for $x > 0$) is strictly convex since $f''(x) = 6\,x^{-4} > 0$
    - $f(x) = x^4$ is convex since $f''(x) = 12\,x^2 \geq 0$ (in fact strictly convex, see above)
    - $f(x) = \mathrm{e}^{c\,x}$ is strictly convex for $c \neq 0$ as $f''(x) = c^2\,\mathrm{e}^{c\,x} > 0$
    - $f(\boldsymbol{x}) = \|\boldsymbol{x}\|^2$ is strictly convex since $\mathbf{H} = 2\,\mathbf{I} \succ 0$
    - $f(x) = |x|$ is convex since for $a \in [0,1]$ we have $|a\,x + (1-a)\,y| \leq a\,|x| + (1-a)\,|y|$ (the triangular inequality), with equality when $x\,y \geq 0$, so it is not strictly convex
    - $f(\boldsymbol{x}) = \|\boldsymbol{x}\|_1 = \sum_i |x_i|$ is convex, as a sum of convex functions

- $f(x, y) = x^2/y$ (for $y > 0$) is convex: its Hessian

$$\mathbf{H} = \frac{2}{y^3}\begin{pmatrix} y^2 & -x\,y \\ -x\,y & x^2 \end{pmatrix}$$

  has eigenvalues 0 and $2(x^2 + y^2)/y^3$, both non-negative

- Convex-down Functions
  1. $f(x) = -x^2$ is strictly convex-down since $f''(x) = -2 < 0$
  2. $f(x) = \sqrt{x}$ (for $x > 0$) is strictly convex-down since $f''(x) = -x^{-3/2}/4 < 0$
  3. $f(x) = \log(x)$ is strictly convex-down since $f''(x) = -1/x^2 < 0$

- A function $f(\boldsymbol{x})$ that is constrained to a convex domain ($\boldsymbol{x} \in \mathcal{S}$, where $\mathcal{S}$ is a convex set) is convex in that domain if for all pairs $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{S}$ and all $a \in [0, 1]$ we have

$$f(a\,\boldsymbol{x} + (1 - a)\,\boldsymbol{y}) \leq a\,f(\boldsymbol{x}) + (1 - a)\,f(\boldsymbol{y})$$

  - This is just a more precise definition of a convex function
  - Note that by limiting the domain of a function some non-convex functions may be convex over that domain
    - e.g. $\cos(x)$ is convex in the interval $[\pi/2, 3\pi/2]$, where $-\cos(x) \geq 0$, and $\sin(x)$ is convex-down in $[0, \pi]$
  - A convex function constrained to lie in a convex set will still be convex
  - Any combination of linear constraints will form a convex set
  - Non-linear constraints may or may not: with $\mathbf{M} \succeq 0$, $\boldsymbol{x}^\mathsf{T}\mathbf{M}\boldsymbol{x} \leq 1$ is a convex set (an ellipsoid) but $\boldsymbol{x}^\mathsf{T}\mathbf{M}\boldsymbol{x} \geq 1$ is not
  - Therefore convex functions restricted to satisfy linear constraints will be convex

- The minima of a convex function (if it has any) form a convex set
  - There can be no local minima that are not global: on the line from such a point to a lower point the function lies on or below the chord, so there would be lower points arbitrarily close by
  - A strictly convex function has at most one minimum: if $\boldsymbol{x} \neq \boldsymbol{y}$ were both minima, strict convexity would make $f((\boldsymbol{x} + \boldsymbol{y})/2)$ smaller than $f(\boldsymbol{x})$. It need not have one at all (e^x has none)

- A sum of convex functions with non-negative weights is convex
  - Adding a strictly convex function to a convex one gives a strictly convex function (add the two defining inequalities; one is strict)

- **Linear regression**
  - The loss function of linear regression is convex

$$L(\boldsymbol{w}) = \|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$$

    - The Hessian is $2\,\mathbf{X}^\mathsf{T}\mathbf{X}$ which is positive semi-definite which is a sufficient condition for $L(\boldsymbol{w})$ to be convex
  - Both the L_2 regulariser and the L_1 regulariser are convex
  - The L_2 regulariser is strictly convex so adding it (ridge regression, with $\eta > 0$) gives a unique solution. So does the elastic net (L_1 plus L_2), while the L_1 regulariser alone (lasso) gives a convex loss whose minimum need not be unique
  - Many machine learning algorithms are chosen because they involve minimising a convex function leading to a unique minimum

2.3 Jensen's Inequality

- For any convex-up function, if $\boldsymbol{x}$ is a random variable then

$$\mathbb{E}[f(\boldsymbol{x})] \geq f(\mathbb{E}[\boldsymbol{x}])$$

  - $\mathbb{E}[\cdots]$ denotes the expectation

- For any convex-down function

$$\mathbb{E}[f(\boldsymbol{x})] \leq f(\mathbb{E}[\boldsymbol{x}])$$

- These are known as *Jensen's Inequality*

- **Proof**

  - We can prove this (for differentiable f; a supporting plane does the same job otherwise) starting from the fact that $f(\boldsymbol{x})$ lies above the tangent plane at any point

$$f(\boldsymbol{x}) \geq f(\boldsymbol{x}^*) + (\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T} \boldsymbol{\nabla} f(\boldsymbol{x}^*)$$

  - This has to be true at the point $\boldsymbol{x}^* = \mathbb{E}[\boldsymbol{x}]$

$$f(\boldsymbol{x}) \geq f(\mathbb{E}[\boldsymbol{x}]) + (\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])^\mathsf{T} \boldsymbol{\nabla} f(\mathbb{E}[\boldsymbol{x}])$$

  - Taking expectations of both sides of the equation

$$\begin{aligned} \mathbb{E}[f(\boldsymbol{x})] &\geq f(\mathbb{E}[\boldsymbol{x}]) + (\mathbb{E}[\boldsymbol{x}] - \mathbb{E}[\boldsymbol{x}])^\mathsf{T} \boldsymbol{\nabla} f(\mathbb{E}[\boldsymbol{x}]) \\ &= f(\mathbb{E}[\boldsymbol{x}]) \qquad\qquad \square \end{aligned}$$

- Using Jensen's Inequality

  - Consider the strictly convex function $f(x) = x^2$ by Jensen's inequality

$$\mathbb{E}\left[x^2\right] \geq \mathbb{E}[x]^2$$

    - or $\mathbb{E}\left[x^2\right] - \mathbb{E}[x]^2 \geq 0$
      - the left-hand side is the variance so we see variances are non-negative
      - because $f(x) = x^2$ is strictly convex we only get equality where $\boldsymbol{x}$ doesn't vary at all

  - Consider the Kullback-Leibler (KL) divergence defined for discrete probability distributions defined over the same range as

$$\mathrm{KL}(f \| g) = -\sum_i f_i \log\left(\frac{g_i}{f_i}\right)$$

    - This is often used to measure how different distribution are from each other
    - Note if $g_i = f_i$ then $\mathrm{KL}(f \| g) = 0$ since $\log(1) = 0$
    - Now we can use Jensen's inequality to show that $\mathrm{KL}(f \| g) \geq 0$

$$\begin{aligned} \mathrm{KL}(f \| g) &= -\sum_i f_i \log\left(\frac{g_i}{f_i}\right) = -\mathbb{E}_f\left[\log\left(\frac{g_i}{f_i}\right)\right] \\ &\geq -\log\left(\mathbb{E}_f\left[\frac{g_i}{f_i}\right]\right) \\ &= -\log\left(\sum_i f_i \frac{g_i}{f_i}\right) = -\log\left(\sum_i g_i\right) = -\log(1) = 0 \end{aligned}$$

        * Here we are assuming we have random variable that take values $X_i = g_i/f_i$ that occur with probability f_i
        * The KL-divergence is therefore equal to $\mathbb{E}[-\log(X_i)]$
        * Since $-\log(x)$ is convex up we have by Jensen's inequality that the KL-divergence is greater than or equal to $-\log(\mathbb{E}[X_i]) = -\log(\sum_i f_i\, X_i)$
        * But $X_i = g_i/f_i$ so the KL-divergence is greater than or equal to $-\log(\sum_i g_i)$
        * But g_i is a probability so $\sum_i g_i = 1$ giving us our result
        * If some $f_i = 0$ we take $0\log(0) = 0$, and X_i is only defined where $f_i > 0$, so every sum runs over those i. Then $\sum_{i:f_i>0} g_i \leq 1$, so $-\log$ of it is still ≥ 0 and the result holds
    - This is known as the Gibbs' inequality after the mathematical physicist, J. Willard Gibbs, (founder of modern statistical mechanics) who first proved it
    - We often use KL-divergences when we want to choose the parameters of one probability distribution so that it approximates a second probability distribution

3 Exercises

3.1 Positive quadrant

- Prove that the set of vectors with non-negative elements form a convex set

3.2 Inverse of Convex Functions

1. Use the chain rule to compute the second derivative of $f(g(x))$
2. If $g(x) = f^{-1}(x)$ show that the second derivative of $f(g(x))$ vanishes
3. Use these results to derive an identity for the second derivative of $f^{-1}(x)$
4. Derive a condition for $f^{-1}(x)$ to be a convex-down function given that $f(x)$ is convex-up
5. Use this to show
   (a) $\sqrt{x}$ is a convex-down function
   (b) $\log(x)$ is a convex-down function

3.3 Cumulant Generating Function

- Here is something a bit harder (which you don't need to know)
- The cumulant generating function of a probability distribution $p(x)$ is defined as

$$G(\lambda) = \log\Big(\mathbb{E}\left[\mathrm{e}^{\lambda\, x}\right]\Big)$$

    - the expectation is over the random variable x drawn from $p(x)$

- It is called the cumulant generating function because if we take the nth derivative and set λ to zero we obtain the nth cumulant (i.e. $\kappa_n = G^{(n)}(0)$)
- The first cumulant is the mean, the second the variance while the third and fourth are proportional to the skewness and excess kurtosis
- Cumulant generating functions appear a lot when you work with probabilities, but go beyond this course
- Nevertheless let's show they are convex

1. Find the second derivative
2. Show that if $p(x)$ is a probability distribution then $q(x) = p(x)\,\mathrm{e}^{\lambda\,x}/\mathbb{E}\left[\mathrm{e}^{\lambda\,x}\right]$ is also a probability distribution
3. Hence show that the cumulant generating function is convex

- See answers

4 Answers

4.1 Positive quadrant

- Let $\mathcal{P}$ be the set of vectors with non-negative elements
- If $\boldsymbol{x} \in \mathcal{P}$ then if $c \geq 0$ we have $\boldsymbol{v} = c\,\boldsymbol{x} \in \mathcal{P}$ since each element of $\boldsymbol{v}$ will be non-negative (i.e. $v_i = c\,x_i \geq 0$)
- Also for any two vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{P}$ clearly $\boldsymbol{w} = \boldsymbol{x} + \boldsymbol{y} \in \mathcal{P}$ since $w_i = x_i + y_i$
- Thus for any two vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{P}$ and any $a \in [0, 1]$ the vector $\boldsymbol{z} = a\,\boldsymbol{x} + (1-a)\,\boldsymbol{y}$ will be in $\mathcal{P}$

4.2 Inverse of Convex Functions

1. Taking derivatives

$$\frac{\mathrm{d}^2 f(g(x))}{\mathrm{d}x^2} = \frac{\mathrm{d}f'(g(x))\,g'(x)}{\mathrm{d}x} = f''(g(x))\,(g'(x))^2 + f'(g(x))\,g''(x)$$

2. If $g(x) = f^{-1}(x)$ then $f(g(x)) = x$ and the second derivative vanishes
3. Using 1. and 2. we find (writing $f^{-1}(x)$ as $g(x)$)

$$g''(x) = -\frac{f''(g(x))\,(g'(x))^2}{f'(g(x))}$$

4. If $f(x)$ is convex then $f''(y) \geq 0$ for any y (including $y = f^{-1}(x)$) also $(g'(x))^2 \geq 0$ so for the inverse of $f(x)$ to be convex down we require $f'(f^{-1}(x)) > 0$
5. Use this to show
   (a) Let $f(x) = x^2$ for $x \geq 0$, so that $f''(x) = 2 > 0$ and $f'(y) = 2\,y$ which is non-negative if $y \geq 0$, but $f^{-1}(x) = \sqrt{x} > 0$ so $f'(f^{-1}(x)) \geq 0$ and consequently $\sqrt{x}$ is convex-down
   (b) Let $f(x) = \exp(x)$, so that $f''(x) = \exp(x) > 0$. But $f'(y) = \exp(y) > 0$ for all y so $f'(f^{-1}(x)) > 0$ which is sufficient to show $f^{-1}(x) = \log(x)$ is a convex-down function

4.3 Cumulant Generating Function

1. If $G(\lambda) = \log\bigl(\mathbb{E}\left[\mathrm{e}^{\lambda\,x}\right]\bigr)$ then

$$G'(\lambda) = \frac{\mathbb{E}\left[x\,\mathrm{e}^{\lambda\,x}\right]}{\mathbb{E}\left[\mathrm{e}^{\lambda\,x}\right]}$$

and

$$G''(\lambda) = \frac{\mathbb{E}\left[x^2\,\mathrm{e}^{\lambda\,x}\right]}{\mathbb{E}\left[\mathrm{e}^{\lambda\,x}\right]} - \frac{\mathbb{E}\left[x\,\mathrm{e}^{\lambda\,x}\right]^2}{\mathbb{E}\left[\mathrm{e}^{\lambda\,x}\right]^2}$$

2. - Now if $p(x)$ is a probability distribution it will be non-negative for all x and sum or integrate to 1

- But then $q(x) = p(x)\,\mathrm{e}^{\lambda x}/\mathbb{E}\left[\mathrm{e}^{\lambda x}\right]$ will be non-negative as $\mathrm{e}^{\lambda x} > 0$ and $\mathbb{E}\left[\mathrm{e}^{\lambda x}\right] > 0$ (the expectation of positive quantities will be positive)
- But
$$\int q(x)\,\mathrm{d}x = \frac{1}{\mathbb{E}\left[\mathrm{e}^{\lambda x}\right]} \int p(x)\,\mathrm{e}^{\lambda x}\,\mathrm{d}x = \frac{\mathbb{E}\left[\mathrm{e}^{\lambda x}\right]}{\mathbb{E}\left[\mathrm{e}^{\lambda x}\right]} = 1$$
- So $q(x)$ is non-negative and normalised so is a well defined probability distribution

3. Using the result of 1. and 2
$$G''(\lambda) = \frac{\mathbb{E}_p\left[x^2\,\mathrm{e}^{\lambda x}\right]}{\mathbb{E}_p\left[\mathrm{e}^{\lambda x}\right]} - \frac{\mathbb{E}_p\left[x\,\mathrm{e}^{\lambda x}\right]^2}{\mathbb{E}_p\left[\mathrm{e}^{\lambda x}\right]^2} = \mathbb{E}_q\left[x^2\right] - \mathbb{E}_q[x]^2 \geq 0$$
- since variances are non-negative

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 14: Support Vector Machines

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Support Vector Machines, maximum margins

2 Main Points

2.1 Overview

- Support vector machines are one of the most successful machine learning techniques when dealing with small data sets
  - They perform binary classification (you need many SVMs to do multi-class classification)
  - There are support vector machines for regression SVR (but I'm not sure their performance dominates other techniques)
- They are basically a perceptron that regularises itself by choosing the maximum margin separating plane
- Of course perceptrons only work with linearly separable data
  - SVMs overcome this limitations in two ways
    1. It often projects the data into a high dimensional extended feature space making it more likely that the data is linearly separable
       - In the extended feature space the features will usually be non-linear functions of the original features, so we can separate data in the extended feature space that isn't necessarily linearly separable in the original feature space
    2. It tolerates some errors using slack variables
       - We need to tune a hyper-parameter (usually called C) to decide how much slack we want to tolerate
- SVMs can work with a huge number of features by solving the problem in the dual space
  - The dual problem depends on the number of constraints
  - There is one constraint for each pattern which states that the pattern should be on or outside a margin
  - The dual problem only depends on the inner (dot) product between vectors in the (extended) feature space
  - As positive semi-definite kernels can be represented as inner products between vectors in an extended feature space (built from the eigenfunctions of the kernel) we often never need to explicitly compute the vectors in the extended feature space

2.2 Maximum Margins

- We consider data $\mathcal{D} = \{(\boldsymbol{x}_k, y_k) | k = 1, 2, \ldots, m\}$ with $\boldsymbol{x}_k \in \mathbb{R}^p$ and $y_k \in \{-1, 1\}$
- We define a dividing hyper-plane by its orthogonal vector $\boldsymbol{w}$ and a threshold b saying how far from the origin it is
- The gap between the hyper-plane and a data point $(\boldsymbol{x}_k, y_k)$ is given by

$$d_k = y_k \left(\frac{\boldsymbol{w}^\mathsf{T} \boldsymbol{x}_k}{\|\boldsymbol{w}\|} - b \right)$$

  - Note that y_k ensures the positive class is on one side of the dividing plane and the negative class on the other

- We want to constrain these gaps to be greater than or equal to a margin Δ

$$y_k \left(\frac{\boldsymbol{w}^\mathsf{T} \boldsymbol{x}_k}{\|\boldsymbol{w}\|} - b \right) \geq \Delta$$

- We can divide through by the margin size Δ giving

$$y_k \left(\hat{\boldsymbol{w}}^\mathsf{T} \boldsymbol{x}_k - \hat{b} \right) \geq 1$$

  - where $\hat{\boldsymbol{w}} = \boldsymbol{w} / (\Delta \, \|\boldsymbol{w}\|)$ and $\hat{b} = b / \Delta$

- We note that

$$\frac{1}{2} \|\hat{\boldsymbol{w}}\|^2 = \frac{1}{2 \, \Delta^2}$$

  - thus if we minimise $\|\hat{\boldsymbol{w}}\|^2 / 2$ subject to the constraint this is equivalent to maximising Δ
  - the factor of a half is just for convenience
  - we minimise $\|\hat{\boldsymbol{w}}\|^2$ rather than $\|\hat{\boldsymbol{w}}\|$ again for convenience
  - we will just drop the hat notation (i.e. we talk about $\boldsymbol{w}$ and b rather than $\hat{\boldsymbol{w}}$ and $\hat{b}$): after all what's in a name? (More seriously it has the same optimum whatever we call it)

- **Soft Margins**

  - So far we have constrained all the data points to lie outside the margin (this is known as *hard margins*)
  - We can instead allow some data points to lie within the margin (or even be misclassified)
    - This is known as a *soft margin* SVM
  - We do this by introducing slack variables s_k so that our constraints become

$$y_k \left(\boldsymbol{w}^\mathsf{T} \boldsymbol{x}_k - b \right) \geq 1 - s_k$$

    (I've dropped the hats for simplicity)
  - We punish the slack variables by adding $C \sum_{k=1}^{m} s_k$ to the objective function
  - We also constrain the slack variables so that $s_k \geq 0$ (so we don't get a reward if $s_k < 0$)

- Optimisation problem

- Our optimisation problem becomes

$$\min_{\boldsymbol{w},b,\boldsymbol{s}} \quad \frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{k=1}^{m} s_k$$

subject to

$$y_k\left(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_k - b\right) \geq 1 - s_k \quad \text{and} \quad s_k \geq 0$$

for $k = 1,\, 2,\, \ldots,\, m$

- We can write this as a Lagrange problem

$$\min_{\boldsymbol{w},b,\boldsymbol{s}} \max_{\boldsymbol{\alpha},\boldsymbol{\beta}} \quad \mathcal{L}(\boldsymbol{w}, b, \boldsymbol{s}, \boldsymbol{\alpha}, \boldsymbol{\beta})$$

where

$$\mathcal{L}(\boldsymbol{w}, b, \boldsymbol{s}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{k=1}^{m} s_k - \sum_{k=1}^{m}\alpha_k\left(y_k\left(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_k - b\right) - 1 + s_k\right) - \sum_{k=1}^{m}\beta_k\, s_k$$

  - α_k and β_k are Lagrange multipliers
    - α_k ensures the margin condition
    - β_k ensures the non-negativity of the slack variables
  - As they enforce inequality constraints we require they are non-negative

2.3 Extended Feature Space

- To help linear separability we can map our inputs to a high-dimensional feature space

$$\boldsymbol{x} \to \boldsymbol{\phi}(\boldsymbol{x})$$

- Note this is a function of the original feature vector meaning that if I change $\boldsymbol{x}$ I will get a different vector $\boldsymbol{\phi}(\boldsymbol{x})$
- Nevertheless $\boldsymbol{\phi}(\boldsymbol{x})$ is just a p' dimensional vector
- We call the space of $\boldsymbol{\phi}(\boldsymbol{x})$ the *extended feature space*
- We can define this extended feature mapping explicitly (e.g. we might decide that $\phi_1(\boldsymbol{x}) = x_1^2$, $\phi_2(\boldsymbol{x}) = x_1\, x_2$, etc.)
- More often this feature mapping is defined implicitly through a kernel function (more of that later)
- We usually choose the dimensionality, p', of the extended feature space to be much larger than that of the original feature space
- Usually we would be scared of this because it can lead to over-fitting
- However, because of choosing the maximal-margin hyper-plane this regularises the problem so strongly that we get good generalisation despite working in an enormous dimensional space
- The Lagrange problem in this extended feature space is

$$\mathcal{L}(\boldsymbol{w}, b, \boldsymbol{s}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{k=1}^{m} s_k - \sum_{k=1}^{m}\alpha_k\left(y_k\left(\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x}_k) - b\right) - 1 + s_k\right) - \sum_{k=1}^{m}\beta_k\, s_k$$

  - Note that the weight vector, $\boldsymbol{w}$, is now in the extended weight space

2.4 Dual Problem

- To obtain the dual problem we compute the minimum of the Lagrangian with respect to $\boldsymbol{w}$, b and $\boldsymbol{s}$
  - Weight minimisation
    $$\nabla_{\boldsymbol{w}} \mathcal{L} = \boldsymbol{w} - \sum_{k=1}^{m} \alpha_k \, y_k \, \boldsymbol{\phi}(\boldsymbol{x}_k) = 0$$
    - So that $\boldsymbol{w} = \sum_{k=1}^{m} \alpha_k \, y_k \, \boldsymbol{\phi}(\boldsymbol{x}_k)$
  - Threshold minimisation
    $$\frac{\partial \mathcal{L}}{\partial b} = \sum_{k=1}^{m} \alpha_k \, y_k = 0$$
    - This gives us a global constraint on the Lagrange multipliers α_k
  - Slack variable minimisation
    $$\frac{\partial \mathcal{L}}{\partial s_k} = C - \alpha_k - \beta_k = 0$$
    - Thus $\alpha_k = C - \beta_k$
    - When substituting back into the Lagrangian all the terms proportional to s_k cancel
    - But $\beta_k \geq 0$ so this implies $\alpha_k \leq C$ (recall that $\alpha_k \geq 0$ from the KKT condition)
- Substituting the optimal values of $\boldsymbol{w}$, b and $\boldsymbol{s}$ into the Lagrangian we obtain the dual problem

$$\max_{\boldsymbol{\alpha}} \; -\frac{1}{2} \sum_{k,l=1}^{m} \alpha_k \, \alpha_l \, y_k \, y_l \, \boldsymbol{\phi}^{\mathsf{T}}(\boldsymbol{x}_k) \, \boldsymbol{\phi}(\boldsymbol{x}_l) + \sum_{k=1}^{m} \alpha_k$$

subject to the conditions

$$\sum_{k=1}^{m} \alpha_k \, y_k = 0, \qquad \forall_k \; 0 \leq \alpha_k \leq C$$

  - Note that a large number of terms vanished (you really need to go through this carefully to see that this happens)
  - Amazingly adding slack variables is equivalent to preventing α_k becoming larger than C
  - the dual problem is another quadratic programming problem but involves the m Lagrange multipliers α_k rather than the p' weights
  - its objective is concave (its Hessian, with elements $-y_k \, y_l \, \boldsymbol{\phi}^{\mathsf{T}}(\boldsymbol{x}_k) \, \boldsymbol{\phi}(\boldsymbol{x}_l)$, is negative semi-definite), so every local maximum is global. The optimal $\boldsymbol{\alpha}$ need not be unique, but $\boldsymbol{w}$ is
  - importantly the dual problem depends only on $\boldsymbol{\phi}(\boldsymbol{x}_k)$ through the inner product $\boldsymbol{\phi}^{\mathsf{T}}(\boldsymbol{x}_k) \, \boldsymbol{\phi}(\boldsymbol{x}_l)$

2.5 Working in the extended feature space

- If we work in the original feature space then the dual problem depends on the inner (dot) product $\boldsymbol{x}_k^{\mathsf{T}} \boldsymbol{x}_l$
  - this is known as a linear SVM
  - quite often a linear SVM gives the best performance
  - sometimes it will pay to solve the primal problem rather than dual problem
  - however we can work with very high dimensional feature vectors

  - this is useful when we work with text where our input vectors are naturally high dimensional
  - In this case we would want to solve the dual problem

- We can explicitly define our extended feature vectors
  - this has the disadvantage that they could be expensive to compute, but for some applications there may be smart ways of doing this

- **Kernel Trick**
  - There is however a really nice trick
  - Any positive semi-definite kernel, $K(\boldsymbol{x}, \boldsymbol{y})$, can be decomposed as

$$K(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{\phi}^{\mathsf{T}}(\boldsymbol{x})\, \boldsymbol{\phi}(\boldsymbol{y})$$

    (we will talk much more about this in the next lecture)
  - obviously the functions $\phi_i(\boldsymbol{x})$ that form the elements of the vector $\boldsymbol{\phi}(\boldsymbol{x})$ depend on the kernel we choose
  - However, as the dual problem only depends on the inner product $\boldsymbol{\phi}^{\mathsf{T}}(\boldsymbol{x})\, \boldsymbol{\phi}(\boldsymbol{y})$ if we choose these so that $\boldsymbol{\phi}^{\mathsf{T}}(\boldsymbol{x})\, \boldsymbol{\phi}(\boldsymbol{y}) = K(\boldsymbol{x}, \boldsymbol{y})$ then we never have to explicitly compute $\boldsymbol{\phi}(\boldsymbol{x})$
  - Our dual problem in this case is

$$\max_{\boldsymbol{\alpha}} \; -\frac{1}{2} \sum_{k,l=1}^{m} \alpha_k\, \alpha_l\, y_k\, y_l\, K(\boldsymbol{x}_k, \boldsymbol{x}_l) + \sum_{k=1}^{m} \alpha_k$$

  - In using an SVM we have to determine which side of the dividing plane a new data point, $\boldsymbol{x}$, would lie
    - Our prediction would be

$$\operatorname{sgn}\bigl(\boldsymbol{w}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}) - b\bigr)$$

      * the function $\operatorname{sgn}(\cdot)$ outputs ± 1 depending on the sign of the argument
    - But our optimal weight vector is given by $\boldsymbol{w} = \sum_{k=1}^{m} \alpha_k\, y_k\, \boldsymbol{\phi}(\boldsymbol{x}_k)$
    - Thus our prediction will be

$$\operatorname{sgn}\left(\sum_{k=1}^{m} \alpha_k\, y_k\, \boldsymbol{\phi}^{\mathsf{T}}(\boldsymbol{x}_k)\, \boldsymbol{\phi}(\boldsymbol{x}) - b \right) = \operatorname{sgn}\left(\sum_{k=1}^{m} \alpha_k\, y_k\, K(\boldsymbol{x}_k, \boldsymbol{x}) - b \right)$$

    - Thus we don't need to compute the weight or the extended feature vectors

2.6 Practical Consideration

- To get SVMs to work in practice requires a lot of tuning
- Firstly you need to normalise the inputs
- You might need to **balance** your data set
  - SVMs perform poorly on problems where the training set has many more examples of one class than the other
  - You can balance the data set by ignoring examples from the large class or
  - Increasing the size of minority class by duplicating examples
  - Sometimes you can use data augmentation on the minority class

- This is only a problem where the data set is strongly unbalanced

- You need to choose C (this can vary by orders of magnitudes)
  - Typically found by exhaustive search start at 2^{-5} and doubling until you reach 2^{15}
  - You decide on what to use by testing on a validation set

- You also have to choose the right kernel
  - This could be no kernel (linear SVM)
  - A polynomial kernel (you have to try different degrees)
  - A radial basis kernel
  - Sometimes you use a specially designed kernel for the problem
  - Kernels also come with hyper-parameters (often called γ) which you also have to tune

- It is a lot of work but for many problems (particularly with small training set) it often leads to state-of-the-art performance

- **Multi-Class Classification**
  - Natively SVMs only perform binary classification
  - If you have multiple classes you need multiple SVMs
  - People use two strategies
    - *One-versus-all*: for each class train a separate SVM using all other classes as negative examples (but see note on balancing)
    - *All-pairs*: Train an SVM for every pair of classes (with c classes that is $c(c-1)/2$ SVMs, which gets expensive when there are many classes)
  - For one-versus-all you pick the class whose SVM is most confident; for all-pairs each SVM votes and the class with the most votes wins

3 Experiments

3.1 SVM in Practice

- SVMs are hard to code (they need a quadratic programming solver, which is complicated)
- However, there are lots of good implementations out there so have a go
- Stolen from the web...

```
from sklearn import datasets, svm, metrics
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

cancer = datasets.load_breast_cancer()
print("Features:", cancer.feature_names)    # 30 features
print("Labels:", cancer.target_names)       # 'malignant', 'benign'
print("Data shape:", cancer.data.shape)

# 70% training and 30% test
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, test_size=0.3, random_state=109)
```

```
# scale each feature to zero mean and unit variance, then a linear SVM
clf = make_pipeline(StandardScaler(), svm.SVC(kernel='linear'))
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

# accuracy: fraction of test points classified correctly
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
# precision: of the points predicted positive, the fraction that are
print("Precision:", metrics.precision_score(y_test, y_pred))
# recall: of the truly positive points, the fraction predicted positive
print("Recall:", metrics.recall_score(y_test, y_pred))
```

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 15: Kernel Trick

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- The Kernel Trick, SVMs, Regression

2 Main Points

2.1 Recap: the Kernel Trick

- SVM kernels are symmetric functions of two variables that can be factorised as an inner-product

$$K(\boldsymbol{x},\boldsymbol{y}) = \boldsymbol{\phi}(\boldsymbol{x})^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{y}) = \sum_{i=1}^{p'} \phi_i(\boldsymbol{x})\,\phi_i(\boldsymbol{y})$$

  - $\boldsymbol{\phi}(\boldsymbol{x})$ are vectors whose elements, $\phi_i(\boldsymbol{x})$, are real-valued functions of the features $\boldsymbol{x}$ (each feature vector $\boldsymbol{x}$ gives a different vector $\boldsymbol{\phi}(\boldsymbol{x})$)
  - p' is the dimensionality of the extended feature space which might be infinite
  - An immediate consequence of this is that the kernel is positive semi-definite
    - This follows because for any function $f(\boldsymbol{x})$ the quadratic form is non-negative

$$\begin{aligned}\iint f(\boldsymbol{x})\,K(\boldsymbol{x},\boldsymbol{y})\,f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\,\mathrm{d}\boldsymbol{y} &= \iint f(\boldsymbol{x})\,\boldsymbol{\phi}(\boldsymbol{x})^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{y})\,f(\boldsymbol{y})\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}\boldsymbol{y}\\ &= \sum_{i=1}^{p'}\left(\int f(\boldsymbol{x})\,\phi_i(\boldsymbol{x})\mathrm{d}\boldsymbol{x}\right)^2 \geq 0\end{aligned}$$

- The SVM dual problem only depends on the inner products $K(\boldsymbol{x}_k,\boldsymbol{x}_l)$, so we never compute $\boldsymbol{\phi}(\boldsymbol{x})$ (the *kernel trick*). The rest of this lecture asks which functions K have this form, and how to build them

2.2 Mercer's Theorem

- Kernel functions play the same role for functions as matrices do for normal vectors
  - that is they describe general linear transformations
  - for a function $f(\boldsymbol{x})$ the argument $\boldsymbol{x}$ can be seen as an index just like i is the index of element v_i of a vector $\boldsymbol{v}$
  - we will consider only symmetric kernels (that is, where $K(\boldsymbol{x},\boldsymbol{y}) = K(\boldsymbol{y},\boldsymbol{x})$
  - these play a similar role as symmetric matrices
- Eigensystems for Kernels

- $\psi(\boldsymbol{y})$ is said to be an eigenfunction of a kernel functions if

$$\int K(\boldsymbol{x}, \boldsymbol{y})\, \psi(\boldsymbol{y})\, \mathrm{d}\, \boldsymbol{y} = \lambda\, \psi(\boldsymbol{x})$$

- In an analogy to the eigen-decomposition of a symmetric matrix we can define the eigen-decomposition of a symmetric kernel function

$$K(\boldsymbol{x}, \boldsymbol{y}) = \sum_i \lambda_i\, \psi_i(\boldsymbol{x})\, \psi_i(\boldsymbol{y})$$

- This is known as **Mercer's Theorem**
- We proved the decomposition for matrices
  - the difficult part in the proof is that you need the eigenvectors to span the vector space
  - this is intuitively obvious if there are n orthogonal eigenvectors in an n-dimensional space
  - it is harder in functions spaces and you need to define the vector space you are modelling (e.g. L_2)
  - if you assume that the set of eigenfunctions span the function space then the rest of the proof is the same as for matrices
  - don't worry if you don't understand this it is enough to remember Mercer's Theorem
- The eigen-expansion holds for any symmetric (square-integrable) kernel, positive semi-definite or not. *Mercer's theorem* is the version for (continuous) positive semi-definite kernels on a bounded domain: then every $\lambda_i \geq 0$ and the sum converges at every point
- If $K(\boldsymbol{x}, \boldsymbol{y})$ is positive semi-definite then there exist real functions $\phi_i(\boldsymbol{x}) = \sqrt{\lambda_i}\psi_i(\boldsymbol{x})$ such that

$$K(\boldsymbol{x}, \boldsymbol{y}) = \sum_i \lambda_i\, \psi_i(\boldsymbol{x})\, \psi_i(\boldsymbol{y}) = \sum_i \phi_i(\boldsymbol{x})\, \phi_i(\boldsymbol{y})$$

  - if $K(\boldsymbol{x}, \boldsymbol{y})$ was not positive semi-definite then some of the eigenvalues would be negative and the functions $\phi_i(\boldsymbol{x}) = \sqrt{\lambda_i}\, \psi_i(\boldsymbol{x})$ would not be real-valued

2.3 Why SVM kernels need to be positive semi-definite

- Recall that we required $K(\boldsymbol{x}, \boldsymbol{y}) = \langle \boldsymbol{\phi}(\boldsymbol{x}), \boldsymbol{\phi}(\boldsymbol{y}) \rangle$ where the components of $\boldsymbol{\phi}(\boldsymbol{x})$ are real, i.e. $\phi_k(x)$ is a real number.
- We used the inner product to compute the separation between the data points $\boldsymbol{\phi}(\boldsymbol{x}_i)$ and the separating plane given by $\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x}) = b$ in the extended feature space.
- But this only makes sense if $\langle \boldsymbol{\phi}(\boldsymbol{x}), \boldsymbol{\phi}(\boldsymbol{y}) \rangle$ defines a proper inner product. In particular $\langle \boldsymbol{\phi}(\boldsymbol{x}), \boldsymbol{\phi}(\boldsymbol{x}) \rangle = \|\boldsymbol{\phi}(\boldsymbol{x})\|^2 \geq 0$ (with equality only if $\boldsymbol{\phi}(\boldsymbol{x}) = \mathbf{0}$).
- Now by Mercer's theorem $\phi_i(\boldsymbol{x}) = \sqrt{\lambda_i}\, \psi_i(\boldsymbol{x})$ where $\psi_i(\boldsymbol{x})$ is the ith eigenfunction of $K(\boldsymbol{x}, \boldsymbol{y})$ with eigenvalue λ_i. But if $\lambda_i < 0$ then $\sqrt{\lambda_i}$ is imaginary. Note that

$$\|\boldsymbol{\phi}(\boldsymbol{x})\|^2 = \langle \boldsymbol{\phi}(\boldsymbol{x}), \boldsymbol{\phi}(\boldsymbol{x}) \rangle = \sum_i \lambda_i\, \psi_i^2(\boldsymbol{x})$$

  but this is not guaranteed to be non-negative for every value of $\boldsymbol{x}$ if some eigenvalues are negative.
- You can also show that the Hessian of the dual objective function is equal to $-y_i\, y_j\, K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ which is negative semi-definite provided $K(\boldsymbol{x}, \boldsymbol{y}) \succeq 0$. If this is not the case the dual objective can be unbounded, with some combination of the Lagrange multipliers going to infinity.

2.4 Positive Semi-Definite Kernels

- SVM Kernels are positive semi-definite symmetric functions
  - There are four equivalent conditions, each of which says a kernel is positive semi-definite
    1. All their eigenvalues are non-negative (i.e. either zero or positive)
    2. They can be decomposed as
    $$K(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{\phi}(\boldsymbol{x})^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{y}) = \sum_{i=1}^{p'} \phi_i(\boldsymbol{x})\, \phi_i(\boldsymbol{y})$$
    where $\phi_i(\boldsymbol{x})$ are real-valued functions
    3. Their quadratic form with any function $f(\boldsymbol{x})$ is non-negative
    4. For any set of points $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n\}$ the matrix
    $$\mathbf{K} = \begin{pmatrix} K(\boldsymbol{x}_1, \boldsymbol{x}_1) & K(\boldsymbol{x}_1, \boldsymbol{x}_2) & \cdots & K(\boldsymbol{x}_1, \boldsymbol{x}_n) \\ K(\boldsymbol{x}_2, \boldsymbol{x}_1) & K(\boldsymbol{x}_2, \boldsymbol{x}_2) & \cdots & K(\boldsymbol{x}_2, \boldsymbol{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ K(\boldsymbol{x}_n, \boldsymbol{x}_1) & K(\boldsymbol{x}_n, \boldsymbol{x}_2) & \cdots & K(\boldsymbol{x}_n, \boldsymbol{x}_n) \end{pmatrix}$$
    is a positive semi-definite matrix
       - such matrices are known as **Gram matrices**
       - this is the most practical test: we can check it numerically for any data set (see the experiment below)
       - the proof that this is a necessary condition follows rather simply from the fact that if we define a matrix $\boldsymbol{\Phi}$ with elements $\Phi_{ik} = \phi_i(\boldsymbol{x}_k)$ then $\mathbf{K} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ and we have seen many times any such matrix is positive semi-definite

- Recall from the previous lecture that any kernel function that allows a decomposition in terms of real functions can be used in an SVM where we can use the kernel trick
  - If we don't use positive semi-definite kernels then our "distances" (used in computing margins) are no-longer proper distances and can be negative (invalidating everything)

2.5 Constructing Kernels

- Most functions of two variables won't be positive semi-definite
- Given a function of two variables it is hard to determine if it is positive-semi definite (none of the definitions are particularly easy to use)
- However we can use simple rules to build positive-semi definite (PSD) kernels from other positive semi-definite kernels
  1. Our starting point is to note the inner product $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \boldsymbol{x}^{\mathsf{T}}\boldsymbol{y}$ is positive semi-definite
     - as an aside we don't necessarily need to use normal vectors as our features so long as we have objects with an inner-product
  2. Adding PSD kernels
     if $K_1(\boldsymbol{x}, \boldsymbol{y})$ and $K_2(\boldsymbol{x}, \boldsymbol{y})$ are PSD kernels then so is $K_3(\boldsymbol{x}, \boldsymbol{y}) = K_1(\boldsymbol{x}, \boldsymbol{y}) + K_2(\boldsymbol{x}, \boldsymbol{y})$
     - To prove this we can use the property that PSD have non-negative quadratic form
     $$\begin{aligned} Q &= \int f(\boldsymbol{x})\, K_3(\boldsymbol{x}, \boldsymbol{y})\, f(\boldsymbol{y})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}\boldsymbol{y} \\ &= \int f(\boldsymbol{x})\, \big(K_1(\boldsymbol{x}, \boldsymbol{y}) + K_2(\boldsymbol{x}, \boldsymbol{y})\big)\, f(\boldsymbol{y})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}\boldsymbol{y} \\ &= \int f(\boldsymbol{x})\, K_1(\boldsymbol{x}, \boldsymbol{y})\, f(\boldsymbol{y})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}\boldsymbol{y} + \int f(\boldsymbol{x})\, K_2(\boldsymbol{x}, \boldsymbol{y})\, f(\boldsymbol{y})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}\boldsymbol{y} \geq 0 \end{aligned}$$

3. Multiplication by a positive scalar
   if $K_1(\boldsymbol{x}, \boldsymbol{y})$ is a PSD kernels and $c > 0$ then so is $K_3(\boldsymbol{x}, \boldsymbol{y}) = c\, K_1(\boldsymbol{x}, \boldsymbol{y})$
   - We can prove this in a similar way to the last proof
4. Multiply PSD kernels
   if $K_1(\boldsymbol{x}, \boldsymbol{y})$ and $K_2(\boldsymbol{x}, \boldsymbol{y})$ are PSD kernels then so is $K_3(\boldsymbol{x}, \boldsymbol{y}) = K_1(\boldsymbol{x}, \boldsymbol{y})\, K_2(\boldsymbol{x}, \boldsymbol{y})$
   - This is easy to prove using the decomposition of PSD to inner products

$$K_3(\boldsymbol{x}, \boldsymbol{y}) = \sum_{i,j} \phi_i^1(\boldsymbol{x})\, \phi_i^1(\boldsymbol{y})\, \phi_j^2(\boldsymbol{x})\, \phi_j^2(\boldsymbol{y}) = \sum_{i,j} \phi_{ij}^3(\boldsymbol{x})\, \phi_{ij}^3(\boldsymbol{y})$$

   where $\phi_{ij}^3(\boldsymbol{x}) = \phi_i^1(\boldsymbol{x})\, \phi_j^2(\boldsymbol{x})$
   - this (double) sum we can treat as an inner-product
   - it is easy to show that the quadratic form with any function $f(\boldsymbol{x})$ is non-negative
5. Powers of PSD kernels
   if $K_1(\boldsymbol{x}, \boldsymbol{y})$ is a PSD kernels then so is $K_1^n(\boldsymbol{x}, \boldsymbol{y})$ for any natural number n
   - Since the product of any two PSD kernels are PSD then the square of a PSD kernel is PSD
   - But by an inductive argument this holds for any integer power
6. Exponential of PSD kernels
   The exponential of a PSD kernel is also a PSD kernel
   - convergent Taylor expansions allow us to approximate a function to any degree of accuracy
   - often Taylor expansions aren't everywhere convergent (so we have to be careful)
   - but Taylor expansions of exponentials are everywhere convergent
   - further Taylor expanding an exponential of a PSD kernel involves a sum of PSD kernels

$$\mathrm{e}^{K(\boldsymbol{x},\boldsymbol{y})} = \sum_{i=0}^{\infty} \frac{1}{i!} K^i(\boldsymbol{x}, \boldsymbol{y}) = 1 + K(\boldsymbol{x}, \boldsymbol{y}) + \frac{1}{2} K^2(\boldsymbol{x}, \boldsymbol{y}) + \cdots$$

   - each term is a PSD kernel (the $i = 0$ term, 1, has $\phi(\boldsymbol{x}) = 1$)
7. If $K(\boldsymbol{x}, \boldsymbol{y}) \succeq 0$ then $K_1(\boldsymbol{x}, \boldsymbol{y}) = g(\boldsymbol{x})\, K(\boldsymbol{x}, \boldsymbol{y})\, g(\boldsymbol{y}) \succeq 0$
   - Consider the quadratic form for any function $f(\boldsymbol{x})$

$$\begin{aligned} Q &= \int f(\boldsymbol{x})\, K_1(\boldsymbol{x}, \boldsymbol{y})\, f(\boldsymbol{y})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}\boldsymbol{y} = \int f(\boldsymbol{x})\, g(\boldsymbol{x})\, K(\boldsymbol{x}, \boldsymbol{y})\, g(\boldsymbol{y})\, f(\boldsymbol{y})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}\boldsymbol{y} \\ &= \int h(\boldsymbol{x})\, K(\boldsymbol{x}, \boldsymbol{y})\, h(\boldsymbol{y})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}\boldsymbol{y} \geq 0 \end{aligned}$$

     where $h(\boldsymbol{x}) = f(\boldsymbol{x})\, g(\boldsymbol{x})$ and we used the fact that $K(\boldsymbol{x}, \boldsymbol{y}) \succeq 0$ so that its quadratic form with any function (in this case $h(\boldsymbol{x})$) is non-negative.
   - As $Q \geq 0$ then $K_1(\boldsymbol{x}, \boldsymbol{y}) \succeq 0$

- **Polynomial kernels**
  - A constant $c \geq 0$ is a PSD kernel ($\phi(\boldsymbol{x}) = \sqrt{c}$), so $c + \boldsymbol{x}^\mathsf{T}\boldsymbol{y}$ is a sum of PSD kernels
  - Hence $(c + \boldsymbol{x}^\mathsf{T}\boldsymbol{y})^d$ is a PSD kernel for any natural number d
  - Expanding it, its features are all the monomials of the x_i of degree up to d, with suitable weights (see the exercise)

- Using these properties we see that $K(\boldsymbol{x}, \boldsymbol{y}) = \mathrm{e}^{-\gamma \|\boldsymbol{x}-\boldsymbol{y}\|^2}$ is a PSD kernel if $\gamma > 0$
  - Now $\boldsymbol{x}^\mathsf{T}\boldsymbol{y}$ is an inner product so a PSD kernel

- Since $2\,\gamma > 0$ then $2\,\gamma\,\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y}$ is a PSD kernel
- But then so is $\mathrm{e}^{2\,\gamma\,\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y}}$ as it is the exponential of a PSD kernel
- Now
$$K_1(\boldsymbol{x},\boldsymbol{y}) = \mathrm{e}^{-\gamma\,\|\boldsymbol{x}-\boldsymbol{y}\|^2} = \mathrm{e}^{-\gamma\,\|\boldsymbol{x}\|^2}\,\mathrm{e}^{2\,\gamma\,\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y}}\,\mathrm{e}^{-\gamma\,\|\boldsymbol{y}\|^2}$$
is of the form $g(\boldsymbol{x})\,K(\boldsymbol{x},\boldsymbol{y})\,g(\boldsymbol{y})$ where $K(\boldsymbol{x},\boldsymbol{y}) = \mathrm{e}^{2\,\gamma\,\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y}} \succeq 0$ and $g(\boldsymbol{x}) = \mathrm{e}^{-\gamma\,\|\boldsymbol{x}\|^2}$ so $K_1(\boldsymbol{x},\boldsymbol{y}) \succeq 0$
- This kernel is known as the *radial basis function* or *RBF* or *Gaussian kernel*
- It has a hyper-parameter, γ, that sets the length scale: $\gamma = 1/(2\ell^2)$ for a length scale ℓ, so larger γ means a shorter range
- this is a very important kernel as it often (but certainly not always) gives good performance (if γ is appropriately chosen)

2.6 Kernels for Structured Data

- **Non-numerical Kernels**
  - When SVMs were fashionable there was a whole industry of researchers finding clever kernels
  - When working with language or trees or graphs it paid to create bespoke kernels for these structures
  - Typically these would all be built up from inner-products
  - e.g. the *ℓ-spectrum kernel* counts every substring of length ℓ in each document and takes the inner product of the two histograms; "statistics" and "computation" share "tat" and "ati", so with $\ell = 3$ the kernel is 2
  - Using clever algorithms you can build very clever kernels functions
  - One down side of SVM kernels is they don't naturally capture prior knowledge about the problem being tackled
    - a clever work around is to build kernels based on other learning machines that are trained on the problem
    - an example of this is the use of *Fisher kernels*, whose features are the gradients of the log-likelihood of a generative model (e.g. a hidden Markov model), weighted by the Fisher information

2.7 Beyond Classification

- There are a lot of other kernel based learning machines
- Many of these use constraints
- They often involve linear operations between vectors where the optimum depends on the inner-product of vectors
  - thus we can use the kernel trick
- *SVR* are support vector machines for regression
  - here we try to find a regression function (a hyperplane in the extended feature space) so that all points lie within a margin ϵ of it (the exact opposite of what we had)
  - We can introduce slack variables to allow some points to lie outside the margin
    - the slack variables must be non-negative
    - we can use a linear punishment s_i or quadratic punishment s_i^2

- We can also do *kernel ridge regression*

$$\min_{\boldsymbol{w}} \eta \|\boldsymbol{w}\|^2 + \sum_i \left(y_i - \boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}(\boldsymbol{x}_i)\right)^2$$

  - $\|\boldsymbol{w}\|^2$ is a regularisation term
  - The optimal weights lie in the space spanned by the set of extended feature vectors $\{\boldsymbol{\phi}(\boldsymbol{x}_k)|k = 1, 2, \ldots, m\}$: any component of $\boldsymbol{w}$ orthogonal to them leaves the predictions unchanged and only increases $\|\boldsymbol{w}\|^2$
  - Thus we can write

$$\boldsymbol{w} = \sum_i \alpha_i \, \boldsymbol{\phi}(\boldsymbol{x}_i)$$

    - Note that here α_i are just parameters; they are not Lagrange multipliers and they can be negative
  - Substituting this into the objective function for ridge regression we get a quadratic optimisation problem in $\boldsymbol{\alpha}$ that just depends on the inner products $\boldsymbol{\phi}^\mathsf{T}(\boldsymbol{x}_i)\,\boldsymbol{\phi}(\boldsymbol{x}_j)$
  - We can use the kernel trick. The solution is $\boldsymbol{\alpha} = (\mathbf{K} + \eta\,\mathbf{I})^{-1}\boldsymbol{y}$, where $\mathbf{K}$ is the Gram matrix, and we predict $f(\boldsymbol{x}) = \sum_i \alpha_i \, K(\boldsymbol{x}_i, \boldsymbol{x})$ (Exercise)

- *Kernel PCA*
  - For kernel PCA we map features into an extended feature space
  - We then use the dual form of PCA (which we've done in an earlier lecture)
  - This allows us to find non-linear manifolds where the data varies

- *Kernel Canonical Correlation Analysis*
  - Canonical correlation analysis finds correlations between datasets
  - The linear form is a bit naff
  - But the kernel form can give nice results

- *Gaussian Processes*
  - Gaussian Processes also use kernels
  - They are a bit different to other kernel methods
    - we don't think of them as working in an extended feature space
    - but their kernels (covariance functions) must be PSD
  - They are one of the most successful methods for doing regression
  - We will look at them later

3 Exercises

3.1 Quadratic Kernels

- Show that the kernel function $K(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{\phi}^\mathsf{T}(\boldsymbol{x})\,\boldsymbol{\phi}(\boldsymbol{y})$, where

$$\boldsymbol{\phi}(\boldsymbol{x}) = (x_1^2, x_2^2, x_3^2, \sqrt{2}\,x_1x_2, \sqrt{2}\,x_1x_3, \sqrt{2}\,x_2x_3)$$

  can be written as $(\boldsymbol{x}^\mathsf{T}\boldsymbol{y})^2$ if $\boldsymbol{x}$ and $\boldsymbol{y}$ are vectors of length 3.

- Answer below

3.2 Kernel Ridge Regression

- Work out the details for kernel ridge regression: substitute

$\boldsymbol{w} = \sum_i \alpha_i \, \boldsymbol{\phi}(\boldsymbol{x}_i)$ and show that $\boldsymbol{\alpha} = (\mathbf{K} + \eta \, \mathbf{I})^{-1} \boldsymbol{y}$ minimises the loss

- Have a go at implementing kernel ridge regression on a real data set
- I'll leave you to work this out

4 Experiments

4.1 Gram Matrix

- Generate ten random vectors $(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_{10})$ where $\boldsymbol{x}_k \in \mathbb{R}^5$
- Compute the Gram matrix $\mathbf{K}$ with components

$$K_{kl} = K(\boldsymbol{x}_k, \boldsymbol{x}_l) = \mathrm{e}^{-\|\boldsymbol{x}_k - \boldsymbol{x}_l\|^2}$$

  - Show that $\mathbf{K}$ is positive definite by computing its

  eigenvalues (for distinct points the RBF Gram matrix is positive definite, not just semi-definite)

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.standard_normal((10, 5))       # ten random vectors in R^5
sq = np.sum((X[:, None, :] - X[None, :, :])**2, axis=-1)
K = np.exp(-sq)                         # Gram matrix of the RBF kernel
print(np.round(K, 3))
print("eigenvalues:", np.round(np.linalg.eigvalsh(K), 4))  # all >= 0
```

5 Answers

5.1 Quadratic Kernel

- This is just straightforward algebra

$$\begin{aligned}\boldsymbol{\phi}^{\mathsf{T}}(\boldsymbol{x})\boldsymbol{\phi}(\boldsymbol{y}) &= x_1^2 y_1^2 + x_2^2 y_2^2 + x_3^2 y_3^2 + 2x_1 x_2 y_1 y_2 + 2\,x_1 x_3 y_1 y_3 + 2x_2 x_3 y_2 y_3 \\ &= (x_1 y_1 + x_2 y_2 + x_3 y_3)^2 = (\boldsymbol{x}^{\mathsf{T}} \boldsymbol{y})^2\end{aligned}$$

- In the lecture notes we did the 2-d case
- Note that the more general polynomial kernel is

$$K_d(\boldsymbol{x}, \boldsymbol{y}) = (1 + \boldsymbol{x}^{\mathsf{T}} \boldsymbol{y})^d$$

  - this is more commonly used as it incorporates the lower degree polynomial kernels

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 16: Probability

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Probability, Random variable, Expectations

2 Main Points

2.1 Basic Language

- Probability is the process of modelling a world with uncertain outcomes
- The model is not perfect, but reflects what we care about
- Mathematicians model the possible outcomes of the world they are considering in terms of a set of *elementary events*
- This set is frequently called Ω (but you are free to give it any name)
- The elementary events are *mutually exclusive*: exactly one of them happens, so for $i \neq j$ the events $\{\omega_i\}$ and $\{\omega_j\}$ do not intersect, $\{\omega_i\} \cap \{\omega_j\} = \emptyset$
- The set is exhaustive so that $\bigcup_i \{\omega_i\} = \Omega$
- This means in our model of the world every possible outcome is included in Ω
- Now we are often not interested in elementary events (they may be too small)
- Instead we consider **events** $\mathcal{E} = \bigcup_{i \in \mathcal{I}} \{\omega_i\}$, sets of outcomes, where $\mathcal{I}$ is some *index set*
- Suppose we are modelling the probability of a darts match
- Ω might represent every location where a dart might land
- $\omega_i \in \Omega$ is a particular position
- I might be interested in the probability of a bull's eye $\mathcal{B} = \bigcup_{i \in \mathcal{I}} \omega_i$
- In this particular case this is a union of an uncountable infinity of points
- In this case I might represent the elementary events by a 2-d vector $\boldsymbol{x}$
- The event of getting a bull's eye corresponds to $\boldsymbol{x}$ being within the region that defines the bull's eye

2.2 Probability and Random Variables

- We can attribute a **probability**, $\mathbb{P}[\mathcal{E}]$, to an event, $\mathcal{E}$
- Probabilities are numbers between 0 and 1: $0 \leq \mathbb{P}[\mathcal{E}] \leq 1$, with $\mathbb{P}[\Omega] = 1$
- 0 means the event never occurs, 1 means the event definitely will occur
- The key rule is *additivity*: if $\mathcal{A}$ and $\mathcal{B}$ cannot both happen ($\mathcal{A} \cap \mathcal{B} = \emptyset$) then $\mathbb{P}[\mathcal{A} \cup \mathcal{B}] = \mathbb{P}[\mathcal{A}] + \mathbb{P}[\mathcal{B}]$
- Hence $\mathbb{P}[\mathcal{E}] + \mathbb{P}[\neg\mathcal{E}] = 1$ where $\neg\mathcal{E} = \Omega \setminus \mathcal{E}$
- That is, the probability of an event occurring and the event not occurring is 1
- In some cases we can interpret $\mathbb{P}[\mathcal{E}]$ as the expected frequency of occurrence of a repetitive trial
- E.g. If the event, $\mathcal{E}$ is rolling an (honest) dice and getting an even number then $\mathbb{P}[\mathcal{E}] = \frac{1}{2}$ can be seen as saying that on average this will happen half the time if you repeat the experiment often enough
- However how do we interpret $\mathbb{P}[\text{Pass AICE3001 exam}]$?
- Hopefully, this is something you only attempt once
- We can think of probability as encoding our informed belief that something will happen
- When our knowledge changes our probability will change
- Random variables, X, are numbers we assign to each elementary event (many events can have the same number assigned)
- That is, a random variable is a function from outcomes to numbers: it partitions Ω and assigns a number to each part
- E.g. for a dice

$$X = \begin{cases} 0 & \text{if } \omega \in \{1, 3, 5\} \\ 1 & \text{if } \omega \in \{2, 4, 6\} \end{cases}$$

- If we let $\mathcal{E}_i$ be the event corresponding to the part with value x_i then $\mathbb{P}[X = x_i] = \mathbb{P}[\mathcal{E}_i]$
- It is common practice (though not universal) to denote random variables with capital letters
- They correspond to events with uncertain outcomes (things that happen in the future)
- We distinguish the scalar that we assign to random variables by writing them in lower case, e.g. x_i
- When we write $\mathbb{P}[X]$ we can view this as short-hand for

$$(\mathbb{P}[X = x] \mid x \in \mathcal{X}) = (\mathbb{P}[X = x_1], \mathbb{P}[X = x_2], \ldots \mathbb{P}[X = x_n])$$

  where $\mathcal{X}$ is the set of possible values that X can take
- That is, $\mathbb{P}[X]$, is the probability mass function which tells us the probability of the random variable X for all possible values it can take
- We differentiate between random variables (capitals) and scalars (lower case) because they behave very differently when we take expectations

- A function, $Y = g(X)$, of a random variable X is a random variable
- **Continuous random variables**
  - When we consider random variables with continuous outcomes (e.g. the time I go to bed, T) then there is an uncountable infinity of possible outcomes and $\mathbb{P}[T = t] = 0$
  - Here we can consider the probability of a random variable, $\boldsymbol{X}$ occurring in a ball, $\mathcal{B}(\boldsymbol{x}, \epsilon)$, of radius ϵ around the point $\boldsymbol{x}$
  - Taking the ratio of that probability to the volume of the ball in the limit $\epsilon \to 0$ defines a **probability density**

$$f_{\boldsymbol{X}}(\boldsymbol{x}) = \lim_{\epsilon \to 0} \frac{\mathbb{P}[\boldsymbol{X} \in \mathcal{B}(\boldsymbol{x}, \epsilon)]}{|\mathcal{B}(\boldsymbol{x}, \epsilon)|}$$

  - Probability densities are not probabilities (they are non-negative, but can be greater than 1)
  - Because they are densities, if I change the way I measure volumes then the numeric values of the density will change (probability density measured in units of probability per cm is different to that measured in units of probability per inch)
  - A consequence is that if we do a change of variables then often the probability density will change
  - Consider a region $\mathcal{R}$—we can describe this using different coordinate systems x or $y = g(x)$

$$\mathbb{P}[X \in \mathcal{R}] = \int_{\mathcal{R}} f_X(x)\,\mathrm{d}x = \mathbb{P}[Y \in \mathcal{R}] = \int_{\mathcal{R}} f_Y(y)\,\mathrm{d}y$$

    (that is the probability of the event occurring in some region should not depend on coordinate region we use)
  - Thus $f_X(x)\,|\mathrm{d}x| = f_Y(y)\,|\mathrm{d}y|$ or

$$f_X(x) = f_Y(y) \left|\frac{\mathrm{d}y}{\mathrm{d}x}\right| = f_Y(g(x))\,|g'(x)|$$

  - In high dimensions if we make a change of variables $\boldsymbol{x} \to \boldsymbol{y}(\boldsymbol{x})$ (which can be seen as a change of random variables $\boldsymbol{X} \to \boldsymbol{Y}(\boldsymbol{X})$) then

$$f_{\boldsymbol{X}}(\boldsymbol{x}) = f_{\boldsymbol{Y}}(\boldsymbol{y})\,|\det(\mathbf{J})|$$

    where $\mathbf{J}$ is the Jacobian matrix

$$\mathbf{J} = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \frac{\partial y_n}{\partial x_2} & \cdots & \frac{\partial y_n}{\partial x_n} \end{pmatrix}$$

  - An alternative definition of probability densities in 1-d is

$$f_X(x) = \lim_{\delta x \to 0} \frac{\mathbb{P}[x \le X < x + \delta x]}{\delta x}$$

  - Thus

$$f_X(x)\,\delta x \approx \mathbb{P}[x \le X < x + \delta x]$$

    so that $f_X(x)\,\delta x \le 1$ (since this is a probability)

- **Cumulative Distribution Functions (CDF)**

$$F_X(x) = \mathbb{P}[X \leq x] = \begin{cases} \sum_{i:x_i \leq x} \mathbb{P}[X = x_i] \\ \int_{-\infty}^{x} f_X(y)\,\mathrm{d}y \end{cases}$$

  - This is a function that goes from 0 to 1 as x goes from $-\infty$ to ∞
  - For continuous random variables

$$f_X(x) = \frac{\mathrm{d}F_X(x)}{\mathrm{d}x}$$

  - Sometimes it is easier to work with the CDF than the PDF

2.3 Probability Is Not Counting

- Probability is often first taught as counting: if all outcomes are equally likely then $\mathbb{P}[\mathcal{E}] = |\mathcal{E}|/|\Omega|$
- But "equally likely" is an assumption about the world (a fair dice, a well-shuffled pack), not part of probability theory
- Most problems of interest have no equally likely outcomes: tomorrow's weather, whether a network misclassifies an image
- Probability is about the *rules* for combining and updating beliefs: additivity, conditioning and marginalisation. Combinatorics is a tool for counting outcomes when symmetry tells us they are equally likely

2.4 The Language of Probability

- Mathematical texts use a standard vocabulary
  - *sample space* Ω, *outcome* ω, *event* $\mathcal{E} \subseteq \Omega$
  - *random variable*: a function from outcomes to numbers
  - *distribution*: described by a *probability mass function* $\mathbb{P}[X = x]$ (discrete), a *density* $f_X(x)$ (continuous) or the CDF $F_X(x)$
  - $X \sim \mathcal{N}(\mu, \sigma^2)$ reads "X is distributed as" a normal with mean μ and variance σ^2
  - *support*: the values with non-zero probability (or density)
  - *i.i.d.*: independent and identically distributed, as for the results of repeated, well conducted experiments
  - *almost surely*: with probability one

2.5 Expectations

- Expectations compute the average value of a quantity (it is a number)
- Mathematicians write expectations as $\mathbb{E}_{\boldsymbol{X}}[\cdots]$ (note that physicist often use $\langle \cdots \rangle$)
- We can define the expectation of $\boldsymbol{Y} = g(\boldsymbol{X})$ as

$$\mathbb{E}_{\boldsymbol{X}}[g(\boldsymbol{X})] = \begin{cases} \sum_{\boldsymbol{x} \in \mathcal{X}} g(\boldsymbol{x})\,\mathbb{P}[\boldsymbol{X} = \boldsymbol{x}] \\ \int g(\boldsymbol{x})\, f_{\boldsymbol{X}}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} \end{cases}$$

- The expectation of a constant c is c

$$\mathbb{E}_{\boldsymbol{X}}[c] = \begin{cases} \sum_{\boldsymbol{x}\in\mathcal{X}} c\,\mathbb{P}[\boldsymbol{X}=\boldsymbol{x}] = c\sum_{\boldsymbol{x}\in\mathcal{X}} \mathbb{P}[\boldsymbol{X}=\boldsymbol{x}] = c \\ \int c\, f_{\boldsymbol{X}}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = c\int f_{\boldsymbol{X}}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = c \end{cases}$$

- A consequence of this is that $\mathbb{E}_X[\mathbb{E}_X[g(X)]] = \mathbb{E}_X[g(X)]$
- Summations and integrals are linear operators

$$\sum_i (a\,x_i + b\,y_i) = a\left(\sum_i x_i\right) + b\left(\sum_i y_i\right)$$
$$\int (a\,f(\boldsymbol{x}) + b\,g(\boldsymbol{x}))\,\mathrm{d}\boldsymbol{x} = a\left(\int f(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}\right) + b\left(\int g(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}\right)$$

  from which it follows that expectations are linear

$$\mathbb{E}[a\,X + b\,Y] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y]$$

- Beware: in general $\mathbb{E}[X\,Y] \neq \mathbb{E}[X]\,\mathbb{E}[Y]$ (equality holds if X and Y are independent, or more generally uncorrelated)
- **Indicator Functions**
  - Indicator functions are functions that take on the value of 0 or 1
  - They are written in different ways. One nice form is the Iverson notation

$$[\![\textit{predicate}]\!] = \begin{cases} 1 & \text{if } \textit{predicate} \text{ is True} \\ 0 & \text{if } \textit{predicate} \text{ is False} \end{cases}$$

  - Sometimes it is written $\boldsymbol{I}_A(x)$ where $A(x)$ is the predicate
  - Donald Knuth is a champion of the Iverson notation (it is easy to manipulate)
  - For probabilities

$$\mathbb{P}[\textit{predicate}] = \mathbb{E}[[\![\textit{predicate}]\!]]$$

    it is not too difficult to convince yourself that this is correct as you are summing the probabilities where the predicate is true
  - As an example the CDF is given by

$$F_X(x) = \mathbb{P}[X \leq x] = \mathbb{E}[[\![X \leq x]\!]]$$

2.6 Calculus of Probabilities

- When we model the world it is common to consider different random variables
- In this case we can consider the **joint probability**

$$p_{X,Y}(x,y) = \mathbb{P}[X=x, Y=y]$$

  i.e. the probability of the event where both $X=x$ and $Y=y$
- $\mathbb{P}[X=x, Y=y] = \mathbb{P}[Y=y, X=x]$ the order in which you write the random variables in a joint distribution function doesn't matter

- There are really only a couple of ways you can manipulate probabilities (this defines the calculus of probability)
- The first is known as *marginalisation*. Given $\mathbb{P}[X, Y]$

$$\mathbb{P}[X = x] = \sum_{y \in \mathcal{Y}} \mathbb{P}[X = x, Y = y]$$

  where $\mathcal{Y}$ is the set of values that the random variable Y takes
- Often we will just write

$$\mathbb{P}[X] = \sum_{Y} \mathbb{P}[X, Y]$$

- **Conditional Probability**
  - The other rule of probability involves introducing **conditional probabilities**
  - We can also define the probability of an event X given that $Y = y$ has occurred

$$\mathbb{P}[X \mid Y = y] = \frac{\mathbb{P}[X, Y = y]}{\mathbb{P}[Y = y]}$$

  - This is a probability over X. If we marginalise over X then

$$\sum_{x \in \mathcal{X}} \mathbb{P}[X = x \mid Y = y] = \frac{\sum_{x \in \mathcal{X}} \mathbb{P}[X = x, Y = y]}{\mathbb{P}[Y = y]} = \frac{\mathbb{P}[Y = y]}{\mathbb{P}[Y = y]} = 1$$

  - In constructing a model it is often much easier to specify conditional probabilities (because you know something) rather than joint probabilities
  - When manipulating probabilities it is often easier to work with joint probabilities because we can simplify them by marginalising out random variables we are not interested in
  - To obtain the joint probability from a conditional probability we use

$$\mathbb{P}[X, Y] = \mathbb{P}[X|Y]\ \mathbb{P}[Y] = \mathbb{P}[Y|X]\ \mathbb{P}[X]$$

  - This generalises to more random variables

$$\mathbb{P}[X, Y, Z] = \mathbb{P}[X, Y|Z]\ \mathbb{P}[Z] = \mathbb{P}[X|Y, Z]\ \mathbb{P}[Y|Z]\ \mathbb{P}[Z]$$

  - This way of breaking down probabilities in not unique. For example, we could have written

$$\mathbb{P}[X, Y, Z] = \mathbb{P}[Y, Z|X]\ \mathbb{P}[X] = \mathbb{P}[Z|Y, X]\ \mathbb{P}[Y|X]\ \mathbb{P}[X]$$

  - Note that $\mathbb{P}[A, B \mid X, Y]$ means the probability of random variables A and B given that X and Y take particular values
  - **Beware**: conditional probabilities, $\mathbb{P}[X \mid Y]$ are probabilities for X, but not Y

$$\sum_{x \in \mathcal{X}} \mathbb{P}[X = x \mid Y] = 1$$
$$\sum_{y \in \mathcal{Y}} \mathbb{P}[X \mid Y = y] \neq 1$$

    (in general)

- Note that

$$\begin{aligned}\mathbb{E}_Y[\mathbb{P}[X \mid Y]] &= \sum_{y\in\mathcal{Y}} \mathbb{P}[Y=y]\ \mathbb{P}[X|Y=y] \\ &= \sum_{y\in\mathcal{Y}} \mathbb{P}[X, Y=y] = \mathbb{P}[X]\end{aligned}$$

  that is taking the expectation of $\mathbb{P}[X \mid Y]$ with respect to Y is the same as marginalising the joint probability $\mathbb{P}[X, Y]$ with respect to Y
- Joint probabilities do not imply causality
- We might believe there is a causal relationship between getting very cold and catching a cold $\mathbb{P}[\text{Catch cold} \mid \text{freezing}] > \mathbb{P}[\text{Catch cold}]$
- However we can use Bayes' rule (see next lecture) to compute $\mathbb{P}[\text{freezing} \mid \text{Catch cold}]$
- That is, given all I know is that Bob has a cold I can deduce the probability that he was freezing shortly before catching the cold. This is a perfectly sensible thing to consider, but clearly it does not imply that freezing yesterday was caused by him catching a cold today

- **Independence**
  - Random variables X and Y are said to be *independent* if

$$\mathbb{P}[X, Y] = \mathbb{P}[X]\ \mathbb{P}[Y]$$

  - Because $\mathbb{P}[X, Y] = \mathbb{P}[X|Y]\ \mathbb{P}[Y]$ and $\mathbb{P}[X, Y] = \mathbb{P}[Y|X]\ \mathbb{P}[X]$ independence implies

$$\mathbb{P}[X|Y] = \mathbb{P}[X] \qquad\qquad \mathbb{P}[Y|X] = \mathbb{P}[Y]$$

  - Probabilistic independence is a mathematical property, not necessarily causal independence. There are examples of events that are causally dependent but nevertheless statistical independent (although such cases are rare)
  - Conversely, two variables with no causal link between them can still be dependent if they share a cause: ice-cream sales and drownings both rise in hot weather. Only variables with no causal connection at all, direct or through a common cause, must be independent
  - If $X \in \{0, 1\}$ represents the outcome of tossing a coin and $Y \in \{1, 2, 3, 4, 5, 6\}$ the outcome of rolling a dice then X and Y are independent
  - In well conducted experiments we expect the results we obtain are independent (that is the outcome of one experiment should not affect the outcome of another experiment)
  - Let $\mathcal{D} = (X_1, X_2, \ldots, X_m)$ represents possible outcomes from a set of m well conducted experiments then

$$\mathbb{P}[\mathcal{D}] = \prod_{i=1}^{m} \mathbb{P}[X_i]$$

  - Denoting a possible sentence I might say by $\mathcal{S} = (W_1, W_2, \ldots, W_m)$ then

$$\mathbb{P}[\mathcal{S}] \neq \prod_{i=1}^{m} \mathbb{P}[W_i]$$

  - If this were not the case then you would expect my sentences to be equally likely irrespective of the order of my words (I certainly hope this is not the case)
  - Independence is a very useful condition that substantially simplifies probabilistic calculations

- Independence is a strong condition that does not happen in many models. However, a far more likely condition is **conditional independence**. For example, X and Y are conditionally independent given Z if

$$\mathbb{P}[X, Y|Z] = \mathbb{P}[X|Z]\ \mathbb{P}[Y|Z]$$

- Me working today, W, and me getting stuck in a traffic jam, J, are not statistically independent events, but they are statistically independent (at least in a simple of model of the world) given the day of the week, D

$$\mathbb{P}[W, J|D] = \mathbb{P}[W|D]\ \mathbb{P}[J|D]$$

3 Exercises

1. Suppose I make an investment of capital C_0 and each day I get a random return R_t such that $C_t = (1 + R_t)\, C_{t-1}$. Taking logs

$$\log(C_t) = \log(1 + R_t) + \log C_{t-1} \approx R_t + \log(C_{t-1})$$

where we assume $|R_t| \ll 1$. Rearranging $\log(C_t) - \log(C_{t-1}) \approx R_t$ and summing

$$\sum_{t=1}^{T} (\log(C_t) - \log(C_{t-1})) = \log(C_T) - \log(C_0) = \sum_{t=1}^{T} R_t$$

Now $\log(C_T) - \log(C_0) = \log(C_T/C_0) = \log(G_T)$ where $G_T = C_T/C_0$ is the multiplicative gain (or loss) in the investment. Let $L_T = \log(G_T)$. Assuming we live in a random world so that $R_t \sim \mathcal{N}(0, \epsilon)$. That is, the return each day is normally distributed with mean 0 and variance ϵ. Furthermore, let us assume that R_t is independent of $R_{t'}$ then L_T is the sum of T normal independent variables so that $L_T \sim \mathcal{N}(0, T\,\epsilon)$ (variances add). That is, $f_L(\ell) = \mathcal{N}(\ell \mid 0, T\,\epsilon)$. Compute the PDF of G_T using

$$f_G(g) = f_L(\ell) \left|\frac{\mathrm{d}\ell}{\mathrm{d}g}\right|$$

where $\ell = \log(g)$

4 Answers

1. This may seem a crazy model, but it is very commonly used in finance and models stock prices extremely well. I'm not going to give the answer, but the distribution is a log-normal distribution which you can look up on Wikipedia. It occurs frequently when a random variable is equal to a product of random variables (taking the logarithm and using the central limit theorem—i.e. the sum of many random variables is approximately normally distributed—gives rise to a log-normal distribution). For the record, with $\mathrm{d}\ell/\mathrm{d}g = 1/g$,

$$f_G(g) = \frac{1}{g\sqrt{2\pi T\epsilon}}\, \mathrm{e}^{-(\log g)^2/(2T\epsilon)}, \qquad g > 0$$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 17: On Becoming a Scientist

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Reproducibility, standard error, error bars, Bernoulli trials, null hypothesis, t-tests, statistical significance

2 Main Points

2.1 Reproducibility and Reporting

- Science requires that experiments can be reproduced, and machine learning is no exception
- A report should tell a story: why the work is interesting, and what specifically is being tested
- It must also give enough detail for someone else to reproduce the work. For a deep learning experiment that means
  - the architecture
  - the dataset, and how it was split into training, validation and test sets
  - any preprocessing
  - the optimiser, mini-batch size and learning rate
  - the number of epochs
  - any hyper-parameter tuning
  - how long training took, on what hardware
- Publishing code is good practice but does not replace describing the method in the report
- Presenting results is not enough; you must interpret them. Every result is uncertain—a different test set would give a different number—so *results are meaningless unless their uncertainty is quantified*

2.2 Experimental Error

- If algorithm A scores 95% and algorithm B scores 97%, could the difference have happened by chance?
- To answer this we assume the test examples are typical and independent (not, for example, the same example repeated). These assumptions are not always true and should be checked

2.3 Estimating the Mean and its Error

- From measurements $X_1, \ldots, X_n$ we estimate the mean and variance by

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} X_i \qquad\qquad \hat{\sigma}_n^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \hat{\mu}_n)^2$$

  These are *unbiased*: $\mathbb{E}[\hat{\mu}_n] = \mu$ and $\mathbb{E}\left[\hat{\sigma}_n^2\right] = \sigma^2$

- The mean squared error of $\hat{\mu}_n$ is

$$\begin{aligned}\mathbb{E}\left[(\hat{\mu}_n - \mu)^2\right] &= \frac{1}{n^2}\mathbb{E}\left[\left(\sum_{i=1}^{n}(X_i - \mu)\right)\left(\sum_{j=1}^{n}(X_j - \mu)\right)\right] \\ &= \frac{1}{n^2}\sum_{i=1}^{n}\left(\mathbb{E}\left[(X_i - \mu)^2\right] + \sum_{j\neq i}\mathbb{E}[X_i - \mu]\,\mathbb{E}[X_j - \mu]\right) = \frac{\sigma^2}{n}\end{aligned}$$

  because independence lets the cross terms factorise, and each factor $\mathbb{E}[X_i - \mu]$ is zero

- The root mean squared error is $\sigma/\sqrt{n}$. The *standard error* (or estimated error in the mean) replaces σ by its estimate:

$$\Delta = \frac{\hat{\sigma}_n}{\sqrt{n}}$$

- Report a result as $\hat{\mu}_n \pm \Delta$, quoting only the significant figures that Δ justifies: not 1.6582356 ± 0.122543 but 1.7 ± 0.1

2.4 Repeated Bernoulli Trials

- A success/failure outcome (e.g. a test example classified correctly or not) with independent trials is a *Bernoulli trial* with unknown success probability p

- The number of successes K in n trials is binomial,

$$\mathbb{P}[K = k] = \binom{n}{k} p^k (1-p)^{n-k}$$

  with $\mathbb{E}[K] = n\,p$ and $\mathbb{E}\left[(K - n\,p)^2\right] = n\,p\,(1-p)$

- The estimate $\hat{p} = K/n$ has mean squared error

$$\mathbb{E}\left[(\hat{p} - p)^2\right] = \frac{1}{n^2}\mathbb{E}\left[(K - n\,p)^2\right] = \frac{p\,(1-p)}{n}$$

- We don't know p, so the standard error is estimated as

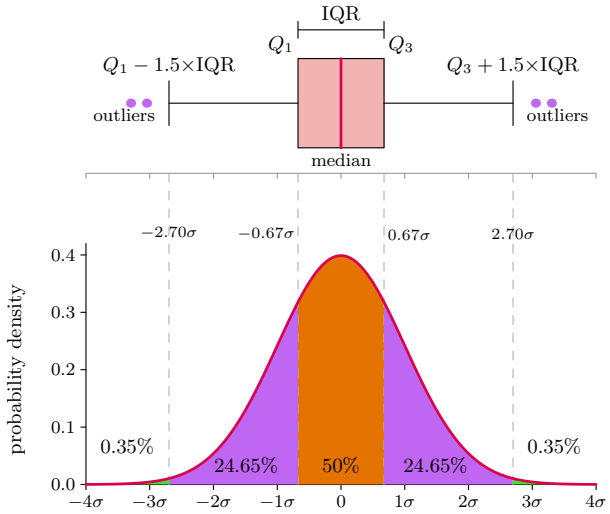
$$\Delta \approx \sqrt{\frac{\hat{p}\,(1-\hat{p})}{n}}$$

- $p(1-p)$ is largest at $p = 1/2$, which gives the worst case bound $\Delta \leq 1/(2\sqrt{n})$. This is pessimistic when p is small, while the estimate above is over-optimistic when $\hat{p} = 0$

- A quick rule of thumb: $n = 10\,000$ test examples give a standard error of at most 0.5%

2.5 Error Bars

- Conventions differ, so always say what your error bars show
  - **one standard error** ($\pm\Delta$): the traditional choice. For roughly normal data the true mean lies outside the bar about 32% of the time
  - **95% confidence interval**: about $\pm 2\Delta$ (more precisely $\pm 1.96\,\Delta$ for large n)
  - **standard deviation**: describes the spread of the data, not the confidence in the mean, so it should not be used as an error bar on a mean
- Box plots summarise a distribution rather than the uncertainty in its median, so you cannot judge statistical significance from them. For a normal distribution $Q_{1,3} = \mp 0.6745\sigma$, the whiskers reach $Q_1 - 1.5\,\mathrm{IQR}$ and $Q_3 + 1.5\,\mathrm{IQR}$, i.e. $\pm 2.698\sigma$, and 0.35% of the data lies beyond each

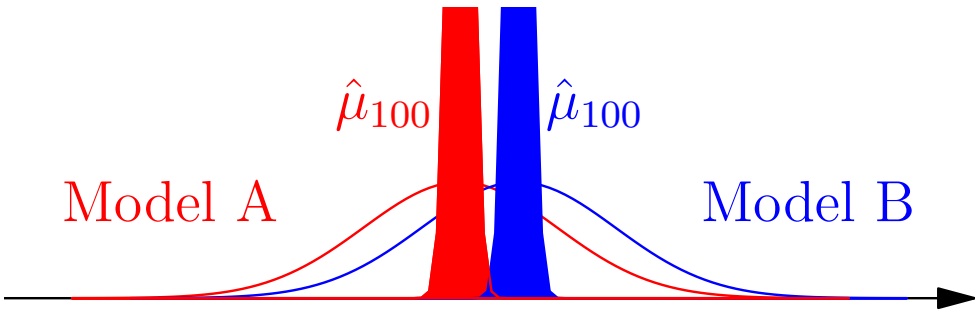


Box plots summarise the *spread* of the data.

They show no error bars on the median, so you cannot tell from them whether two results are statistically significantly different.

2.6 Comparing Models

- How do we decide whether model B really is better than model A? The lecture animation repeatedly simulates two models whose true performances differ, and shows that with few test examples the worse model often appears better; with more data such mistakes become rare



- To quantify this we use a *null hypothesis*: *models A and B have the same performance*. We then ask how likely a gap at least as large as the one observed would be if the null hypothesis were true. This probability is the *p-value*
- The answer depends on the size of the gap, the number of observations and the variance

2.7 T-tests

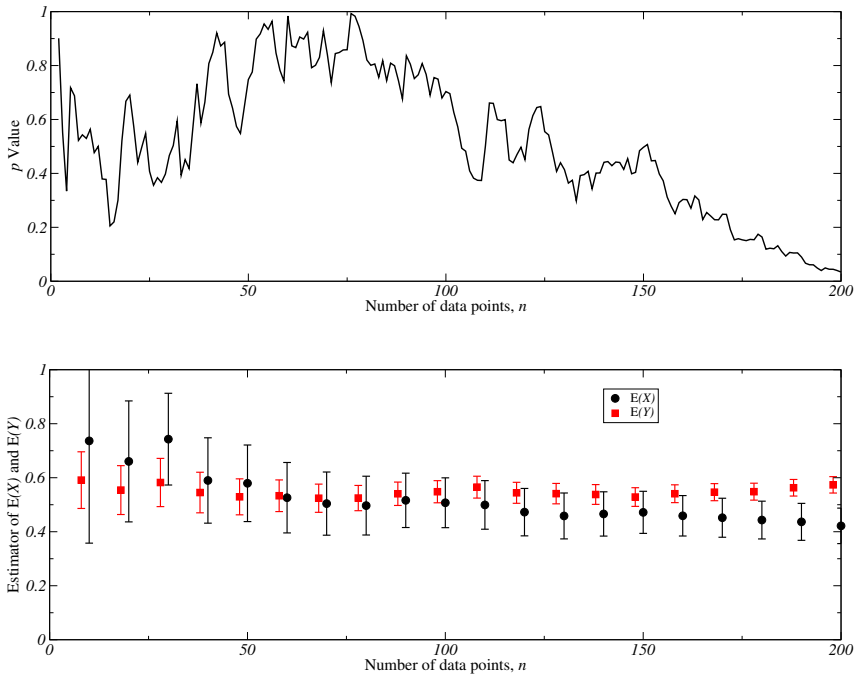
- William Gosset invented the t-test while working for the Guinness brewery, publishing under the pseudonym *Student*. Student's test assumes both sets of results are normal with the same mean and the same variance under the null hypothesis
- *Welch's t-test,* more common now, allows the variances to differ. With n results for A and m for B,

$$t = \frac{\hat{\mu}_A - \hat{\mu}_B}{\hat{\sigma}_{A,B}} \qquad\qquad \hat{\sigma}_{A,B} = \sqrt{\frac{\hat{\sigma}_A^2}{n} + \frac{\hat{\sigma}_B^2}{m}}$$

- The p-value follows from t and the effective number of degrees of freedom (the Welch–Satterthwaite formula)

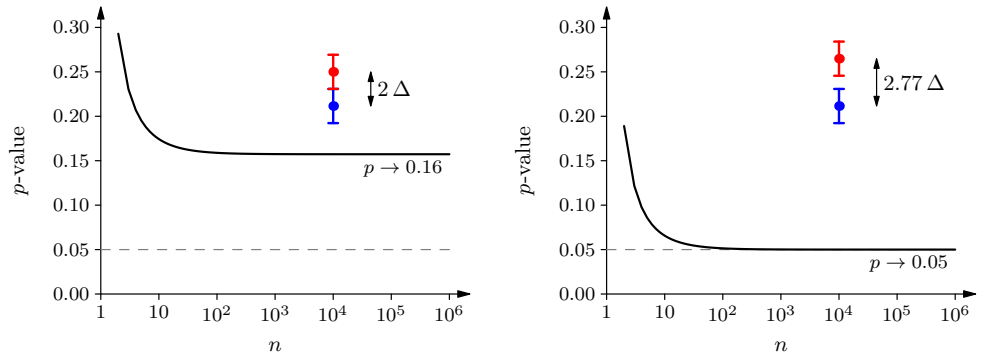
$$\nu = \frac{\hat{\sigma}_{A,B}^4}{\left(\hat{\sigma}_A^2/n\right)^2/(n-1) + \left(\hat{\sigma}_B^2/m\right)^2/(m-1)}$$

  Note the *fourth* power in the numerator: ν is dimensionless
- A small p-value means the observed gap would be unlikely if the null hypothesis were true. By convention A is called *statistically significantly* different from B when $p < 0.05$
- Statistical significance does not mean the difference matters; that is a separate judgement
- The test assumes normally distributed results. Few people check, and it is usually a reasonable approximation, but there are exceptions
- Example with $X \sim \mathcal{N}(0.4, 1)$ and $Y \sim \mathcal{N}(0.6, 0.25)$:



2.8 Error Bars and Significance

- It is tempting to judge significance by whether error bars overlap, but this needs care



- For two independent means with equal standard errors Δ the standard error of the difference is $\sqrt{2}\,\Delta$. If one-standard-error bars just touch, the gap is 2Δ and $t = 2\Delta/(\sqrt{2}\,\Delta) = \sqrt{2} \approx 1.41$, giving $p \approx 0.16$ for large samples—*not* significant at the 5% level
- For significance at 5% the gap must be about $1.96 \times \sqrt{2}\,\Delta \approx 2.8\,\Delta$, i.e. the bars must be separated by a further $0.8\,\Delta$ or so
- Conversely, non-overlapping 95% confidence intervals imply a difference well beyond the 5% level. When in doubt, do the test

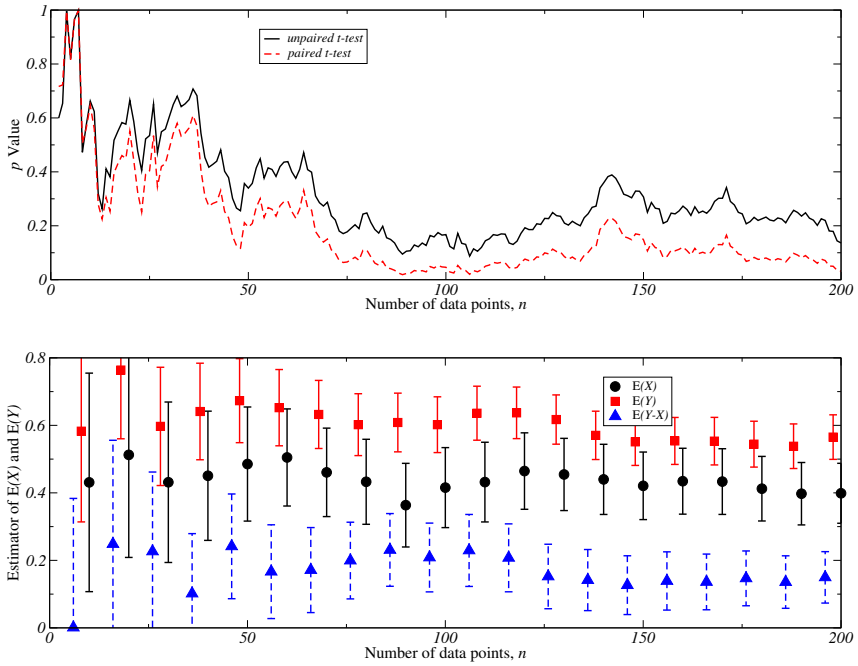
2.9 Paired T-tests

- Sometimes much of the variation comes from the test examples themselves: model A may be consistently better than B, but both do well on easy examples and badly on hard ones
- An ordinary t-test can then miss a real difference
- A *paired* t-test works with the per-example differences $D_i = X_i - Y_i$, whose variance is

$$\hat{\sigma}_D^2 = \hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\,\hat{c}_{X,Y} \qquad\qquad \hat{c}_{X,Y} = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \hat{\mu}_X)(Y_i - \hat{\mu}_Y)$$

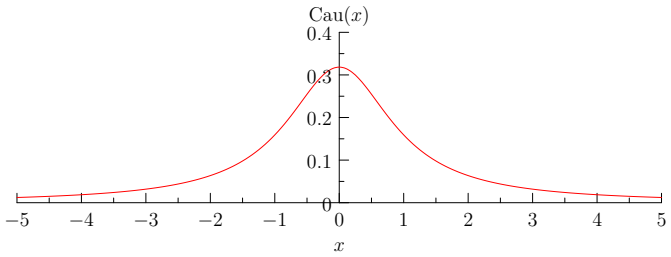
  and then $t = \hat{\mu}_D/(\hat{\sigma}_D/\sqrt{n})$ with $n-1$ degrees of freedom. A strong positive correlation $\hat{c}_{X,Y}$ removes the shared variation

- Example with $X = Z + X_1$, $Y = Z + Y_1$, a shared $Z \sim \mathcal{N}(0, 0.8)$, $X_1 \sim \mathcal{N}(0.4, 0.2)$ and $Y_1 \sim \mathcal{N}(0.6, 0.2)$:



2.10 Pitfalls

- **Run the test once.** At $p < 0.05$ a true null hypothesis produces a "significant" result one time in twenty. Run the test twice and the chance of at least one false positive is $1 - 0.95^2 \approx 0.1$; running tests until one succeeds is cheating
- To compare many models use a method designed for it, such as ANOVA, or correct the threshold for the number of tests
- **Simulating the null hypothesis.** The t-test distribution comes from a difficult calculation, but you can also simulate the null hypothesis many times and count how often the simulated statistic exceeds the observed one. This works whenever the null hypothesis is easy to simulate, even when normality fails
- **Heavy tails.** For distributions with thick tails, such as the Cauchy distribution, averaging many samples need not give a reliable estimate of the mean. When nothing makes sense, plot a histogram



- *The person you cheat when you don't do this is yourself*

3 Exercises

3.1 Reporting a classification accuracy

- A classifier gets 912 of 1000 test examples right. Estimate its accuracy and standard error, and write the result down sensibly

3.2 How big a test set?

- How many test examples do you need to guarantee a standard error of at most 0.5% in an accuracy, whatever the true accuracy?

3.3 Unbiased variance

- Show that $\mathbb{E}\left[\hat{\sigma}_n^2\right] = \sigma^2$ when $\hat{\sigma}_n^2 = \frac{1}{n-1}\sum_i (X_i - \hat{\mu}_n)^2$ and the X_i are independent

3.4 Touching error bars

- Two models are each evaluated on many independent test examples and have equal standard errors Δ. Their one-standard-error bars just touch. Is the difference significant at the 5% level?

3.5 Multiple comparisons

- You compare 10 models against a baseline using separate t-tests at the 5% level. None of them is actually any different from the baseline. What is the probability that at least one test reports a significant difference?

4 Answers

4.1 Reporting a classification accuracy

- $\hat{p} = 0.912$ and $\Delta = \sqrt{0.912 \times 0.088 / 1000} \approx 0.0090$
- So the accuracy is 0.912 ± 0.009, or $(91.2 \pm 0.9)\%$

4.2 How big a test set?

- The worst case is $\Delta \leq 1/(2\sqrt{n})$. Requiring $1/(2\sqrt{n}) \leq 0.005$ gives $\sqrt{n} \geq 100$, so $n \geq 10\,000$

4.3 Unbiased variance

- Write $X_i - \hat{\mu}_n = (X_i - \mu) - (\hat{\mu}_n - \mu)$. Then

$$\begin{aligned}\sum_i (X_i - \hat{\mu}_n)^2 &= \sum_i (X_i - \mu)^2 - 2\,(\hat{\mu}_n - \mu) \sum_i (X_i - \mu) + n\,(\hat{\mu}_n - \mu)^2 \\ &= \sum_i (X_i - \mu)^2 - n\,(\hat{\mu}_n - \mu)^2\end{aligned}$$

  since $\sum_i (X_i - \mu) = n\,(\hat{\mu}_n - \mu)$

- Taking expectations, and using $\mathbb{E}\left[(\hat{\mu}_n - \mu)^2\right] = \sigma^2 / n$,

$$\mathbb{E}\left[\sum_i (X_i - \hat{\mu}_n)^2\right] = n\,\sigma^2 - \sigma^2 = (n-1)\,\sigma^2$$

  so dividing by $n - 1$ gives an unbiased estimate

4.4 Touching error bars

- The gap is 2Δ and the standard error of the difference is $\sqrt{\Delta^2 + \Delta^2} = \sqrt{2}\,\Delta$, so $t = \sqrt{2} \approx 1.41$
- With many examples t is approximately normal, and $\mathbb{P}[|Z| > 1.41] \approx 0.16$. The difference is *not* significant at the 5% level

4.5 Multiple comparisons

- Each test independently gives a false positive with probability 0.05, so the probability of none is 0.95^{10} and of at least one is $1 - 0.95^{10} \approx 0.40$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 18: Bayesian Inference

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Bayes, Conjugate Priors, Uninformative Priors

2 Main Points

** Bayesian Statistics

- Background
  - Bayesian statistics is an approach to making statistical inference
  - It was first proposed by the Rev. Thomas Bayes, who died in 1761; his essay was published in 1763
  - It was championed by the great French polymath Pierre-Simon Laplace
  - It fell out of favour at the beginning of the 20th Century due to philosophical prejudice
    - It was deemed to be unscientific because it requires specifying a prior probability which seemed subjective
  - It was championed from the 1950's by Ed Jaynes who eventually convinced most people that Bayesian inference is consistent
  - Perhaps more importantly with the rise of computers it was possible to demonstrate that Bayesian methods work
- Bayes' Rule
  - Bayes rule follows from a simple identity

$$\mathbb{P}[h_i|\mathcal{D}] = \frac{\mathbb{P}[\mathcal{D}|h_i]\ \mathbb{P}[h_i]}{\mathbb{P}[\mathcal{D}]}$$

    - $\mathbb{P}[h_i|\mathcal{D}]$ is the *posterior* probability of hypothesis h_i given data $\mathcal{D}$
    - $\mathbb{P}[\mathcal{D}|h_i]$ is the *likelihood* of the data given the hypothesis h_i
    - $\mathbb{P}[h_i]$ is our *prior* probability (our belief in hypothesis h_i before seeing the data)
    - $\mathbb{P}[\mathcal{D}]$ is a normalisation term I will call the *evidence* and is just equal to

$$\mathbb{P}[\mathcal{D}] = \sum_{i=1}^{n} \mathbb{P}[h_i, \mathcal{D}] = \sum_{i=1}^{n} \mathbb{P}[\mathcal{D}|h_i]\ \mathbb{P}[h_i]$$

- Although this is just a mathematical identity, to use it in inference requires specifying a prior belief
  - I argued from the first lecture that for machine learning to work we need to make assumptions about the data

- In Bayesian approach these assumptions are built into our likelihood and prior

- A nice property of Bayesian inference is that it turns the problem from an inverse problem (computing the posterior $\mathbb{P}[h_i|\mathcal{D}]$) into a forward problem of computing the likelihood $\mathbb{P}[\mathcal{D}|h_i]$
  - This solves problems of missing data (we only need to calculate the probability of the data we see)
  - It is also much more straightforward to model the forward process (from physics) rather than the inverse process
  - That said, it can still be technically very challenging

- Bayes' rule has the same form whether we are working with discrete variables (where we have probability masses) or continuous variables (where we have probability densities)

2.1 Conjugate Priors

- Technically performing Bayesian inference can be difficult because the posterior can be very ugly to work with

- But for a few of the classic likelihood functions there exist *conjugate prior distributions* such that the posterior is in the same class of distributions as the prior
  - Suppose that the likelihood of observing data $\boldsymbol{X}$ given some parameters $\boldsymbol{\theta}$ is $p(\boldsymbol{X}|\boldsymbol{\theta})$ (this may be a probability mass or density depending on whether $\boldsymbol{X}$ is continuous or not)
  - We want to infer $\boldsymbol{\theta}$ which we do through the posterior $p(\boldsymbol{\theta}|\boldsymbol{X})$
  - If we are lucky there is a family of distributions, $\mathrm{Conj}(\boldsymbol{\theta}|\boldsymbol{\phi})$ parameterised by $\boldsymbol{\phi}$, that describe the parameters of the likelihood $\boldsymbol{\theta}$ and satisfy

    $$\mathrm{Conj}(\boldsymbol{\theta}|\boldsymbol{\phi}') \propto p(\boldsymbol{X}|\boldsymbol{\theta})\,\mathrm{Conj}(\boldsymbol{\theta}|\boldsymbol{\phi})$$

    for some new parameters $\boldsymbol{\phi}'$ that depend on $\boldsymbol{\phi}$ and $\boldsymbol{X}$
  - Thus if we start with a prior $\mathrm{Conj}(\boldsymbol{\theta}|\boldsymbol{\phi}_0)$ then after observing data $\boldsymbol{X}_1$ we would end up with a posterior $\mathrm{Conj}(\boldsymbol{\theta}|\boldsymbol{\phi}_1)$ where $\boldsymbol{\phi}_1$ depends on observation $\boldsymbol{X}_1$ and $\boldsymbol{\phi}_0$
  - We can repeat this
    - When we make a second observation $\boldsymbol{X}_2$ our posterior now becomes our prior (we have updated our beliefs) so now we obtain a new posterior

      $$\mathrm{Conj}(\boldsymbol{\theta}|\boldsymbol{\phi}_2) \propto p(\boldsymbol{X}_2|\boldsymbol{\theta})\,\mathrm{Conj}(\boldsymbol{\theta}|\boldsymbol{\phi}_1)$$

- As an example let us consider a number of Bernoulli trials (we assume observations $X_i \in \{0, 1\}$, independent given p, with probability p of a 1)
  - But we don't know p
  - We assume our prior distribution is a beta distribution $\mathrm{Beta}(p|a_0, b_0) \propto p^{a_0-1}\,(1-p)^{b_0-1}$
  - Our likelihood is $\mathbb{P}[X|p] = p^X\,(1-p)^{1-X}$
  - Using Bayes' rule our posterior is

    $$f(p|X) \propto p^X\,(1-p)^{1-X}\,p^{a_0-1}\,(1-p)^{b_0-1} = p^{a_0+X-1}\,(1-p)^{b_0+1-X-1}$$

  - But this has the same functional form as a beta-distribution
  - It will be a beta-distribution as it has to be normalised (posteriors are always normalised)
  - Thus $f(p|X) = \mathrm{Beta}(p|a_1, b_1)$ where $a_1 = a_0 + X$ and $b_1 = b_0 + 1 - X$

- The Poisson distribution is defined as

$$\mathrm{Pois}[N|\mu] = \frac{\mu^N \, \mathrm{e}^{-\mu}}{N!}$$

  - It describes the probability of observing N events where each event is assumed independent and the expected number of events is μ
  - Suppose we have made observations N_1, N_2, etc. and we are trying to infer μ
  - The Poisson distribution has a conjugate distribution which is the gamma distribution

$$\mathrm{Gamma}(\mu|a,b) = \frac{b^a \, \mu^{a-1} \, \mathrm{e}^{-b\,\mu}}{\Gamma(a)}$$

  - Assuming a prior $\mathrm{Gamma}(\mu|a_0, b_0)$ and assume we make an observation N_1 then the posterior is given by

$$f(\mu|N_1) \propto \mu^{N_1} \, \mathrm{e}^{-\mu} \, \mu^{a_0-1} \, \mathrm{e}^{-b_0\,\mu} = \mu^{a_0+N_1-1} \, \mathrm{e}^{-(b_0+1)\,\mu}$$

  - Thus $f(\mu|N_1) = \mathrm{Gamma}(\mu|a_1, b_1)$ where
    - $a_1 = a_0 + N_1$
    - $b_1 = b_0 + 1$

- Conjugate priors are the exception rather than the rule but they do occur for some classic distributions. E.g.

Likelihood	**Prior**
Binomial/Bernoulli	Beta
Poisson	Gamma
Multinomial	Dirichlet
Normal (unknown mean and variance)	Normal-gamma
Multivariate normal (unknown mean and covariance)	Normal-Wishart

2.2 Uninformative Priors

- What should we do if we have no prior information
- This disturbed statisticians who felt that there might not be an objective answer to this
- However, Ed Jaynes argued that there is a unique answer that we get by requiring invariance
- Scale parameters
  - Often we are trying to infer scale parameters
  - These would include the rate in the Poisson distribution or the standard deviation in a normal distribution
  - They are non-negative numbers that determine the scale (i.e. what units we should measure in)
  - For most problems we have some idea about this (otherwise it would be difficult to do the measurement)
  - But we may still not know the order of magnitude of what we are measuring
  - What should we use as a prior?
  - The answer to this was given by Harold Jeffreys: for a scale parameter it is $p(x) \propto 1/x$, often called *Jeffreys' prior* (his general rule, based on the Fisher information, gives $\mu^{-1/2}$ for a Poisson rate; the $1/\mu$ used in the lecture is the scale-invariant choice)

- Let's see the argument
- If we have no idea about what scale an observable x takes then we should expect there to be equal probability to be in the intervals $[A, B]$ or the interval $[A/c, B/c]$
- This may seem strange as these two intervals have different lengths, but if this wasn't the case we would know something about which scale to use
- If our prior for x is $p(x)$ then we require

$$\int_A^B p(x)\,\mathrm{d}x = \int_{A/c}^{B/c} p(x)\,\mathrm{d}x$$

- Now we can make a change of variables
- $y = c\,x$ so this last integral becomes

$$\int_{A/c}^{B/c} p(x)\,\mathrm{d}x = \int_A^B \frac{1}{c}\,p\Big(\frac{y}{c}\Big)\,\mathrm{d}y$$

- But for this to equal the first integral for any interval $[A, B]$ we require

$$p(x) \propto \frac{1}{x}$$

- We note that if $p(x) = 1/x$ then

$$p\Big(\frac{x}{c}\Big) = \frac{c}{x} = c\,p(x)$$

  as required
- The strange thing about this distribution is that it is improper:

$$\int_0^\infty \frac{1}{x}\,\mathrm{d}x = \infty$$

- Strangely a lot of uninformative priors turn out to be improper
- However, this doesn't seem to matter in Bayesian inference: after making some observations we end up with a proper posterior

- **Benford's Law**
  - There is a strange consequence of Jeffreys prior which was first speculated upon in 1881 when Simon Newcomb noticed that logarithm tables were much more used for numbers beginning with 1 rather 2 and 2 rather than 3, etc. Frank Benford in 1938 looked at the occurrence of numbers in data set and found that the digit of first significant figure was more likely to be 1 than 2 and more likely to be 2 than 3.
  - If we assume these numbers are measured in an arbitrary scale and the occurrence of these numbers followed Jeffreys prior then the probability that real number between 1 and 10 actually took values between n and $n + 1$ is given by

$$\frac{\int_n^{n+1} \frac{1}{x}\mathrm{d}x}{\int_1^{10} \frac{1}{x}\mathrm{d}x} = \frac{\log(n+1) - \log(n)}{\log(10)} = \log_{10}\left(\frac{n+1}{n}\right)$$

  - Note that we would get the same result if the number was between 10 and 100 and we asked the probability that it was in the range $10\,n$ to $10\,(n + 1)$
  - But this means the digit of the most significant figure will be n
  - So numbers beginning with 1 occur more often than numbers beginning with 2, and numbers beginning with 2 occur more often than those beginning with 3, etc.
  - This is weird but true (for most naturally occurring non-negative numbers)

3 Exercises

3.1 Throwing Dice

- There are three dice on the table. Two of them are normal dice, while the other dice has 6 on two faces and 1, 2, 3 and 5 on the other face. A dice is chosen at random and is thrown 10 times. On three occasions the top face is a 6. What is the probability that the dice chosen is the dishonest dice?
- See answers below

4 Experiments

4.1 Benford's Law

- Benford's law is so wacky that to convince yourself it is true you really need to test it out
- Find a set of data with features that are clearly scale parameters and have a go (you need enough data points to be convincing)
    - if you numbers are positive and could take any value then they are likely to be scale parameter

5 Answers

5.1 Throwing Dice

- We use Bayes' rule

$$P(\text{dishonest}|\text{data}) = \frac{P(\text{data}|\text{dishonest})\,P(\text{dishonest})}{P(\text{data}|\text{dishonest})\,P(\text{dishonest}) + P(\text{data}|\text{honest})\,P(\text{honest})}$$

- Now $P(\text{dishonest}) = 1/3$ and $P(\text{honest}) = 2/3$
- The probability of k success out of n trials with a success probability p is given by the binomial distribution

$$\text{Binom}(k|n,p) = \binom{n}{k} p^k\,(1-p)^{n-k}$$

- The probability of 3 out of 10 rolls being a 6 is given by

$$P(\text{data}|\text{dishonest}) = \text{Binom}(3|10,\tfrac{1}{3}) = \binom{10}{3}\left(\frac{1}{3}\right)^3\left(\frac{2}{3}\right)^7$$

$$P(\text{data}|\text{honest}) = \text{Binom}(3|10,\tfrac{1}{6}) = \binom{10}{3}\left(\frac{1}{6}\right)^3\left(\frac{5}{6}\right)^7$$

- Thus

$$P(\text{dishonest}|\text{data}) = \frac{1}{1 + \frac{P(\text{data}|\text{honest})\,P(\text{honest})}{P(\text{data}|\text{dishonest})\,P(\text{dishonest})}}$$

$$= \frac{1}{1 + 2\left(\frac{1}{2}\right)^3\left(\frac{5}{4}\right)^7} \approx 0.456$$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 19: Integral Calculus

Adam Prugel-Bennett

November 2, 2026

1 Keywords

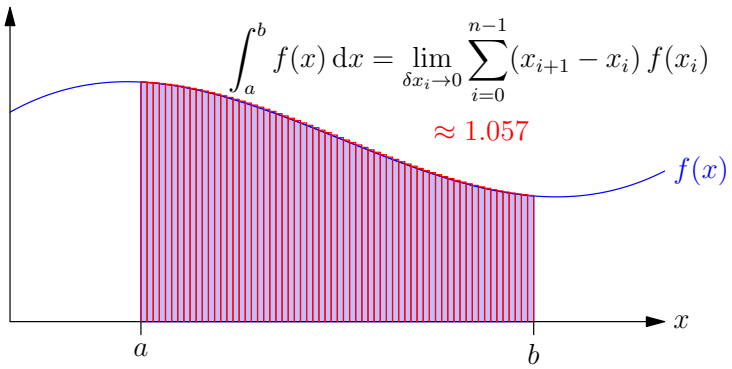
- Riemann integral, fundamental theorem of calculus, integration by parts, substitution, Jacobians, generating functions, gamma function

2 Main Points

2.1 The Riemann Integral

- An integral is the area under a curve. Divide $[a, b]$ into strips at points $a = x_0 < x_1 < \cdots < x_n = b$ and add up the areas of the rectangles:

$$\int_a^b f(x)\,\mathrm{d}x = \lim_{\delta x_i \to 0} \sum_{i=0}^{n-1} (x_{i+1} - x_i)\, f(x_i)$$



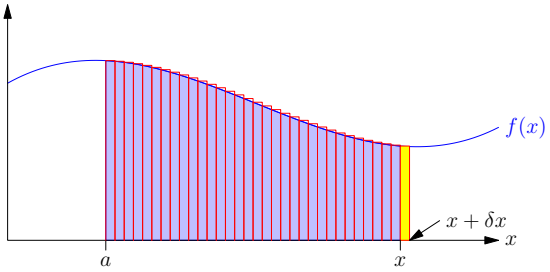
- Integration is linear: splitting the sum,

$$\int_a^b (r\, f(x) + s\, g(x))\,\mathrm{d}x = r \int_a^b f(x)\,\mathrm{d}x + s \int_a^b g(x)\,\mathrm{d}x$$

2.2 The Fundamental Theorem of Calculus

- Let $I(a, x) = \int_a^x f(z)\,\mathrm{d}z$. Extending the upper limit by δx adds a thin strip of area $\delta x\, f(x)$, so

$$\frac{\mathrm{d}I(a,x)}{\mathrm{d}x} = \lim_{\delta x \to 0} \frac{I(a, x+\delta x) - I(a,x)}{\delta x} = f(x)$$



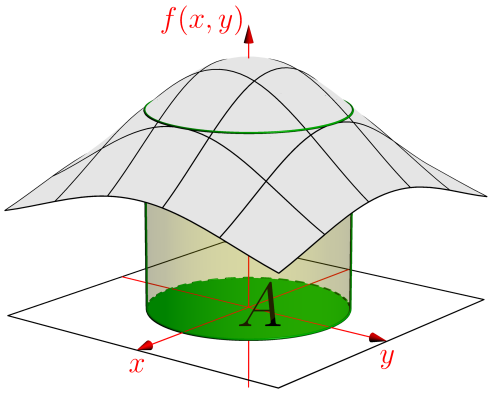
- Conversely, integrating a derivative, the Riemann sum telescopes:

$$\int_a^b \frac{\mathrm{d}f(x)}{\mathrm{d}x}\,\mathrm{d}x = \lim \sum_{i=0}^{n-1} (x_{i+1} - x_i)\,\frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} = \lim \sum_{i=0}^{n-1} \left(f(x_{i+1}) - f(x_i)\right)$$
$$= f(x_n) - f(x_0) = f(b) - f(a)$$

- So integration is an *anti-derivative*: it undoes differentiation

2.3 Indefinite and Multiple Integrals

- An *indefinite integral* is a function $F(x) = \int f(x)\,\mathrm{d}x$ with $F'(x) = f(x)$. Since $F(x)$, $F(x)+1$, $F(x)+\pi, \ldots$ all have the same derivative, F is only defined up to an additive constant
- Definite integrals follow from it: $\int_a^b f(x)\,\mathrm{d}x = F(b) - F(a)$
- For functions of several variables we integrate over regions, e.g. $\iint_A f(x,y)\,\mathrm{d}x\,\mathrm{d}y$. We usually write a single integral sign, $\int f(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}$, for $\int \cdots \int f(x_1, \ldots, x_n)\,\mathrm{d}x_1 \cdots \mathrm{d}x_n$



2.4 Performing Integrals

- The basic method is to recognise an anti-derivative. Since $\mathrm{d}x^n/\mathrm{d}x = n\,x^{n-1}$,

$$\int x^{n-1}\,\mathrm{d}x = \frac{x^n}{n} + c \qquad (n \neq 0), \qquad \int_a^b x^{n-1}\,\mathrm{d}x = \frac{b^n - a^n}{n}$$

- The product and chain rules let us differentiate almost anything, but there are no general rules for $\int f(x)\,g(x)\,\mathrm{d}x$ or $\int f(g(x))\,\mathrm{d}x$. Unlike differentiation there is no small parameter to expand in. *In general integration is hard*

2.5 Integration by Parts

- Integrating the product rule $\mathrm{d}(f\,g)/\mathrm{d}x = f'\,g + f\,g'$ from a to b and rearranging,

$$\int_a^b f(x)\,\frac{\mathrm{d}g(x)}{\mathrm{d}x}\,\mathrm{d}x = \Big[f(x)\,g(x)\Big]_a^b - \int_a^b \frac{\mathrm{d}f(x)}{\mathrm{d}x}\,g(x)\,\mathrm{d}x$$

  Whether this helps depends on whether the new integral is easier

- **Example: the factorial function.** Let $\Pi(z) = \int_0^\infty x^z\,\mathrm{e}^{-x}\,\mathrm{d}x$. Taking $g(x) = -\mathrm{e}^{-x}$,

$$\Pi(z) = \Big[-x^z\,\mathrm{e}^{-x}\Big]_0^\infty + \int_0^\infty z\,x^{z-1}\,\mathrm{e}^{-x}\,\mathrm{d}x = z\,\Pi(z-1)$$

  since the boundary term vanishes for $z > 0$

- $\Pi(0) = \int_0^\infty \mathrm{e}^{-x}\,\mathrm{d}x = \Big[-\mathrm{e}^{-x}\Big]_0^\infty = 1$, so for integers $\Pi(n) = n\,(n-1)\cdots 1 = n!$

2.6 Differentiating Through the Integral

- Sometimes we can do an integral by differentiating a related, easier one. The moments of a random variable are $M_n = \mathbb{E}[X^n] = \int x^n \, f_X(x) \, \mathrm{d}x$
- Define the *moment generating function*

$$Z(\ell) = \mathbb{E}\left[\mathrm{e}^{\ell X}\right] = \int_{-\infty}^{\infty} \mathrm{e}^{\ell x} \, f_X(x) \, \mathrm{d}x$$

- Differentiating n times under the integral and setting $\ell = 0$,

$$\left.\frac{\mathrm{d}^n Z(\ell)}{\mathrm{d}\ell^n}\right|_{\ell=0} = \int_{-\infty}^{\infty} x^n \, f_X(x) \, \mathrm{d}x = M_n$$

2.7 The Cumulant Generating Function

- Expanding $\mathrm{e}^{\ell x} = 1 + \ell \, x + \frac{1}{2}\ell^2 x^2 + \cdots$ inside the integral,

$$Z(\ell) = 1 + \ell \, M_1 + \frac{1}{2}\ell^2 M_2 + \frac{1}{3!}\ell^3 M_3 + \cdots$$

- Taking the logarithm, using $\log(1+\epsilon) = \epsilon - \frac{1}{2}\epsilon^2 + \frac{1}{3}\epsilon^3 - \cdots$, gives the *cumulant generating function*

$$G(\ell) = \log(Z(\ell)) = \ell \, M_1 + \frac{1}{2}\ell^2 \left(M_2 - M_1^2\right) + \frac{1}{3!}\ell^3 \left(M_3 - 3\, M_2 \, M_1 + 2\, M_1^3\right) + \cdots$$

- The *cumulants* are $\kappa_n = G^{(n)}(0)$: κ_1 is the mean, κ_2 the variance and κ_3 is related to the skewness

2.8 Special Functions

- Some integrals have no closed form in terms of elementary functions. $\Pi(z)$ is one: $\Pi(n) = n!$ for integers, but for general z it is a new function in its own right
- History has left us with the *gamma function* instead,

$$\Gamma(z) = \int_0^{\infty} x^{z-1} \, \mathrm{e}^{-x} \, \mathrm{d}x = \Pi(z-1)$$

- Other special functions defined by integrals include the Bessel, Airy, hypergeometric, elliptic and error functions
- Integration is sometimes easier in the complex plane (largely due to Cauchy), and there is an algorithm (the Risch algorithm) that decides whether many functions have elementary integrals. It is complicated enough that it is only available inside symbolic packages such as Mathematica. Both are beyond this course

2.9 Substitution

- Changing variable from x to u, with $x = x(u)$, the Riemann sum becomes

$$\int_a^b f(x) \, \mathrm{d}x = \int_{u(a)}^{u(b)} f(x(u)) \, \frac{\mathrm{d}x(u)}{\mathrm{d}u} \, \mathrm{d}u$$

  where $u(x)$ is the inverse of $x(u)$

- **Example.** For $I(n) = \int_0^\infty x^n \, \mathrm{e}^{-x^2/2} \, \mathrm{d}x$, put $u = x^2/2$, so $x = \sqrt{2u}$, $\mathrm{d}x/\mathrm{d}u = 1/\sqrt{2u}$, and the limits stay 0 and ∞:

$$I(n) = \int_0^\infty (2u)^{n/2} \, \mathrm{e}^{-u} \, \frac{\mathrm{d}u}{\sqrt{2u}} = 2^{\frac{n-1}{2}} \int_0^\infty u^{\frac{n-1}{2}} \, \mathrm{e}^{-u} \, \mathrm{d}u = 2^{\frac{n-1}{2}} \, \Pi\left(\frac{n-1}{2}\right)$$

- So $I(1) = 1$, $I(3) = 2 \times 1! = 2$ and $I(5) = 2^2 \times 2! = 8$. Even n need Π at half integers, e.g. $I(0) = \Pi(-1/2)/\sqrt{2}$, and $I(2) = \sqrt{2}\,\Pi(1/2) = \Pi(-1/2)/\sqrt{2}$ using $\Pi(1/2) = \frac{1}{2}\Pi(-1/2)$

- **A recipe.** (1) Choose $u(x)$, often an "inner" function whose derivative also appears in the integrand; (2) write $\mathrm{d}u = u'(x)\,\mathrm{d}x$ and replace every x; (3) change the limits; (4) integrate in u

- **Example.** With $u = 1 + x^2$, $\mathrm{d}u = 2x\,\mathrm{d}x$ and limits $u : 1 \to 2$,

$$\int_0^1 2x \, (1 + x^2)^3 \, \mathrm{d}x = \int_1^2 u^3 \, \mathrm{d}u = \frac{16 - 1}{4} = \frac{15}{4}$$

- **The limits can swap over.** With $u = \cos(x)$, $\mathrm{d}u = -\sin(x)\,\mathrm{d}x$ and $u : 1 \to 0$,

$$\int_0^{\pi/2} \sin(x) \cos^2(x) \, \mathrm{d}x = \int_1^0 -u^2 \, \mathrm{d}u = \int_0^1 u^2 \, \mathrm{d}u = \frac{1}{3}$$

- **A linear change of variable stretches** $\mathrm{d}x$**.** With $z = (x - \mu)/\sigma$, $\mathrm{d}x = \sigma \, \mathrm{d}z$; the factor σ is the one-dimensional Jacobian determinant

2.10 Changing Variables in Many Dimensions

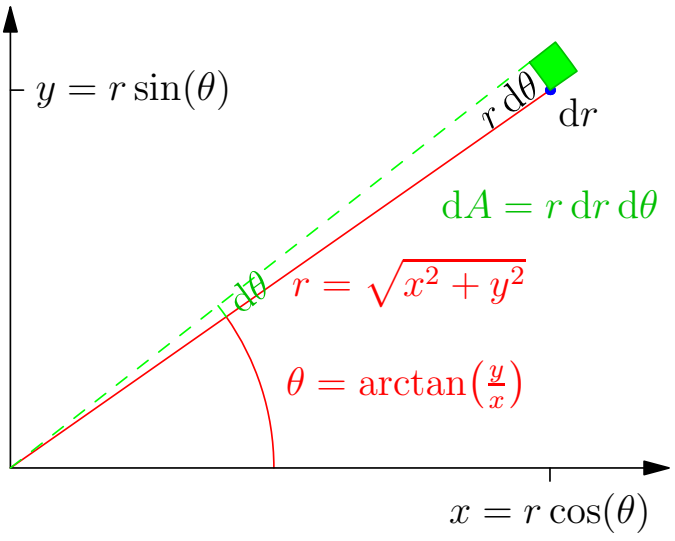
- Changing variables $\boldsymbol{x} \to \boldsymbol{u}$, volume elements are rescaled by the determinant of the Jacobian:

$$\int f(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} = \int f(\boldsymbol{x}(\boldsymbol{u})) \, |\det(\mathbf{J})| \, \mathrm{d}\boldsymbol{u}, \qquad J_{ij} = \frac{\partial x_i}{\partial u_j}$$

- **Polar coordinates.** With $x = r\cos(\theta)$ and $y = r\sin(\theta)$,

$$|\det(\mathbf{J})| = \left| \det \begin{pmatrix} \cos(\theta) & -r\sin(\theta) \\ \sin(\theta) & r\cos(\theta) \end{pmatrix} \right| = r\left(\cos^2(\theta) + \sin^2(\theta)\right) = r$$

  so $\mathrm{d}x\,\mathrm{d}y = r\,\mathrm{d}r\,\mathrm{d}\theta$: a small patch at radius r has sides $\mathrm{d}r$ and $r\,\mathrm{d}\theta$



- Gaussian integrals, which use these changes of variables, are the subject of the next lecture (see the Gaussian Integrals notes)

3 Exercises

3.1 Integration by parts

- Evaluate $\int_0^1 x\,\mathrm{e}^x\,\mathrm{d}x$

3.2 Substitution

- Evaluate $\int_0^\infty x\,\mathrm{e}^{-x^2}\,\mathrm{d}x$

3.3 Mean absolute value of a normal

- For a standard normal Z, use $I(1)$ to find $\mathbb{E}[|Z|]$

4 Answers

4.1 Integration by parts

- Take $f(x) = x$ and $g(x) = \mathrm{e}^x$:

$$\int_0^1 x\,\mathrm{e}^x\,\mathrm{d}x = \Big[x\,\mathrm{e}^x\Big]_0^1 - \int_0^1 \mathrm{e}^x\,\mathrm{d}x = \mathrm{e} - (\mathrm{e}-1) = 1$$

4.2 Substitution

- With $u = x^2$, $\mathrm{d}u = 2\,x\,\mathrm{d}x$:

$$\int_0^\infty x\,\mathrm{e}^{-x^2}\,\mathrm{d}x = \frac{1}{2}\int_0^\infty \mathrm{e}^{-u}\,\mathrm{d}u = \frac{1}{2}$$

4.3 Mean absolute value of a normal

- By symmetry

$$\mathbb{E}[|Z|] = \frac{2}{\sqrt{2\pi}}\int_0^\infty z\,\mathrm{e}^{-z^2/2}\,\mathrm{d}z = \frac{2\,I(1)}{\sqrt{2\pi}} = \sqrt{\frac{2}{\pi}} \approx 0.798$$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 20: Gaussian Integrals

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Gaussian integrals, completing the square, moment generating functions, marginals, conditionals, Kalman filter

2 Main Points

2.1 Why Gaussians?

- Gaussians are used throughout machine learning (Gaussian processes, Kalman filters, linear regression, the noise in diffusion models) because all their integrals can be done exactly
- Marginalising, conditioning, adding or linearly transforming Gaussians gives another Gaussian, so inference reduces to linear algebra on means and covariances
- The algebra is long-winded but uses only three tricks: the basic Gaussian integral, completing the square, and diagonalising a matrix. Learn the results; read the derivations once so you trust them

2.2 One Dimension

- The basic integral, by squaring it and changing to polar coordinates,

$$\int_{-\infty}^{\infty} \mathrm{e}^{-x^2/2}\,\mathrm{d}x = \sqrt{2\pi}$$

  and, changing variables to $z = (x-\mu)/\sigma$, the normal density

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma}\,\mathrm{e}^{-(x-\mu)^2/(2\sigma^2)}$$

  integrates to one

- **Completing the square** (for $a > 0$): $-\frac{a}{2}x^2 + bx = -\frac{a}{2}(x - b/a)^2 + b^2/(2a)$, so

$$\int_{-\infty}^{\infty} \mathrm{e}^{-\frac{a}{2}x^2 + b\,x}\,\mathrm{d}x = \sqrt{\frac{2\pi}{a}}\,\mathrm{e}^{\frac{b^2}{2a}}$$

- **Moment generating function** (MGF) $M(t) = \mathbb{E}\left[\mathrm{e}^{tX}\right]$. For $X \sim \mathcal{N}(\mu, \sigma^2)$

$$M(t) = \mathrm{e}^{\mu t + \frac{1}{2}\sigma^2 t^2}$$

  - $M^{(n)}(0) = \mathbb{E}[X^n]$, so $M'(0) = \mu$ and $M''(0) = \mu^2 + \sigma^2$
  - the cumulant generating function $\log M(t)$ stops at t^2: a Gaussian has no cumulants beyond the mean and variance

- An MGF determines its distribution, and for independent X, Y the MGF of $X + Y$ is the product of their MGFs. So the sum of independent Gaussians is Gaussian, with the means added and the variances added

2.3 Many Dimensions

- For a positive definite covariance $\mathbf{\Sigma} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\mathsf{T}$, rotating to $\boldsymbol{y} = \mathbf{V}^\mathsf{T}(\boldsymbol{x} - \boldsymbol{\mu})$ (Jacobian $\mathbf{V}$, with $|\det \mathbf{V}| = 1$) turns the integral into a product of one-dimensional ones:

$$\int \mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\mathsf{T}\mathbf{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}\,\mathrm{d}\boldsymbol{x} = \prod_i \sqrt{2\pi\lambda_i} = \sqrt{\det(2\pi\mathbf{\Sigma})}$$

  so

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \mathbf{\Sigma}) = \frac{1}{\sqrt{\det(2\pi\mathbf{\Sigma})}}\,\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\mathsf{T}\mathbf{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}$$

- **The central result.** Completing the square, $-\frac{1}{2}\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} + \boldsymbol{b}^\mathsf{T}\boldsymbol{x} = -\frac{1}{2}(\boldsymbol{x} - \mathbf{A}^{-1}\boldsymbol{b})^\mathsf{T}\mathbf{A}(\boldsymbol{x} - \mathbf{A}^{-1}\boldsymbol{b}) + \frac{1}{2}\boldsymbol{b}^\mathsf{T}\mathbf{A}^{-1}\boldsymbol{b}$, gives

$$\int \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} + \boldsymbol{b}^\mathsf{T}\boldsymbol{x}}\,\mathrm{d}\boldsymbol{x} = \sqrt{\frac{(2\pi)^n}{\det \mathbf{A}}}\,\mathrm{e}^{\frac{1}{2}\boldsymbol{b}^\mathsf{T}\mathbf{A}^{-1}\boldsymbol{b}}$$

  (sometimes called the Hubbard–Stratonovich transformation)

- The MGF of $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{\Sigma})$ is

$$\mathbb{E}\left[\mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{x}}\right] = \mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{\mu} + \frac{1}{2}\boldsymbol{t}^\mathsf{T}\mathbf{\Sigma}\,\boldsymbol{t}}$$

  from which $\mathbb{E}[\boldsymbol{x}] = \boldsymbol{\mu}$ and the covariance is $\mathbf{\Sigma}$

- **Linear transformations.** If $\boldsymbol{z} = \mathbf{W}\boldsymbol{x} + \boldsymbol{c}$ then $\mathbb{E}\left[\mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{z}}\right] = \mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{c}}\,M(\mathbf{W}^\mathsf{T}\boldsymbol{t})$, so

$$\boldsymbol{z} \sim \mathcal{N}\big(\mathbf{W}\boldsymbol{\mu} + \boldsymbol{c},\ \mathbf{W}\mathbf{\Sigma}\mathbf{W}^\mathsf{T}\big)$$

  ($\mathbf{W}$ need not be square)

2.4 Marginals and Conditionals

- Partition a Gaussian vector into $\boldsymbol{z}_1$ and $\boldsymbol{z}_2$, with means $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$ and covariance blocks $\mathbf{\Sigma}_{11}$, $\mathbf{\Sigma}_{12}$, $\mathbf{\Sigma}_{21} = \mathbf{\Sigma}_{12}^\mathsf{T}$, $\mathbf{\Sigma}_{22}$
- **Marginal.** Setting $\boldsymbol{t} = (\boldsymbol{t}_1, \mathbf{0})$ in the MGF gives the MGF of $\mathcal{N}(\boldsymbol{\mu}_1, \mathbf{\Sigma}_{11})$: to marginalise, strike out the rows and columns of the variables integrated away
- **Conditional.** Let $\boldsymbol{w} = \boldsymbol{z}_1 - \mathbf{\Sigma}_{12}\mathbf{\Sigma}_{22}^{-1}\boldsymbol{z}_2$. Then $\mathrm{Cov}(\boldsymbol{w}, \boldsymbol{z}_2) = \mathbf{\Sigma}_{12} - \mathbf{\Sigma}_{12}\mathbf{\Sigma}_{22}^{-1}\mathbf{\Sigma}_{22} = \mathbf{0}$. Jointly Gaussian variables that are uncorrelated are independent (their joint MGF factorises), so $\boldsymbol{w}$ is independent of $\boldsymbol{z}_2$, and

$$\boldsymbol{z}_1 \mid \boldsymbol{z}_2 \sim \mathcal{N}\big(\boldsymbol{\mu}_1 + \mathbf{\Sigma}_{12}\mathbf{\Sigma}_{22}^{-1}(\boldsymbol{z}_2 - \boldsymbol{\mu}_2),\ \mathbf{\Sigma}_{11} - \mathbf{\Sigma}_{12}\mathbf{\Sigma}_{22}^{-1}\mathbf{\Sigma}_{21}\big)$$

  - the mean moves linearly with the observation; the covariance shrinks by an amount that does not depend on the value observed
  - $\mathbf{\Sigma}_{11} - \mathbf{\Sigma}_{12}\mathbf{\Sigma}_{22}^{-1}\mathbf{\Sigma}_{21}$ is called the *Schur complement* of $\mathbf{\Sigma}_{22}$
- **The long way round.** The same results can be obtained by doing the integrals directly; see the derivation below

2.5 Inference with Gaussians

- **A noisy measurement.** Prior $x \sim \mathcal{N}(\mu_0, s_0^2)$, measurement $y = x + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Then (x, y) is jointly Gaussian with covariance $\begin{pmatrix} s_0^2 & s_0^2 \\ s_0^2 & s_0^2 + \sigma^2 \end{pmatrix}$ and

$$\mathbb{E}[x \mid y] = \mu_0 + \frac{s_0^2}{s_0^2 + \sigma^2}(y - \mu_0), \qquad \frac{1}{\mathrm{Var}(x \mid y)} = \frac{1}{s_0^2} + \frac{1}{\sigma^2}$$

  precisions (inverse variances) add

- **Combining sensors.** Two measurements with noise variances σ_1^2, σ_2^2 and a flat prior give the precision-weighted average $(y_1/\sigma_1^2 + y_2/\sigma_2^2)/(1/\sigma_1^2 + 1/\sigma_2^2)$, with precision $1/\sigma_1^2 + 1/\sigma_2^2$

- **Predicting one variable from another.** For two variables with correlation ρ, $\mathbb{E}[x_2 \mid x_1] = \mu_2 + \rho\,(\sigma_2/\sigma_1)(x_1 - \mu_1)$ and $\mathrm{Var}(x_2 \mid x_1) = \sigma_2^2(1-\rho^2)$: linear regression, and regression to the mean

- **Kalman filter.** For a drifting position $x_{t+1} = x_t + \eta_t$, $\eta_t \sim \mathcal{N}(0, q)$, observed as $y_t = x_t + \epsilon_t$, the belief $\mathcal{N}(m_t, v_t)$ is updated by a *predict* step (a sum of Gaussians: the variance becomes $v_t + q$) and an *update* step (a noisy measurement):

$$k = \frac{v_t + q}{v_t + q + \sigma^2}, \qquad m_{t+1} = m_t + k\,(y_{t+1} - m_t), \qquad v_{t+1} = (1-k)(v_t + q)$$

- **Gaussian processes.** With the unknown value f_* as $\boldsymbol{z}_1$ and noisy observations $\boldsymbol{y}$ as $\boldsymbol{z}_2$, the conditional formula gives the GP prediction

$$f_* \mid \boldsymbol{y} \sim \mathcal{N}\big(\boldsymbol{k}_*^\mathsf{T}(\mathbf{K} + \sigma^2\mathbf{I})^{-1}\boldsymbol{y},\ k_* - \boldsymbol{k}_*^\mathsf{T}(\mathbf{K} + \sigma^2\mathbf{I})^{-1}\boldsymbol{k}_*\big)$$

2.6 Derivation: Marginalising and Conditioning by Integration

- This is the long way round. Take zero means (shift the variables otherwise) and write the covariance in blocks

$$\boldsymbol{\Sigma} = \begin{pmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^\mathsf{T} & \mathbf{C} \end{pmatrix}$$

  for the variables $\boldsymbol{x}$ (first block) and $\boldsymbol{y}$ (second)

- **Block factorisation.** With $\mathbf{S} = \mathbf{A} - \mathbf{B}\mathbf{C}^{-1}\mathbf{B}^\mathsf{T}$ (multiply out to check),

$$\boldsymbol{\Sigma} = \mathbf{U}\mathbf{D}\mathbf{U}^\mathsf{T}, \qquad \mathbf{U} = \begin{pmatrix} \mathbf{I} & \mathbf{B}\mathbf{C}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}, \qquad \mathbf{D} = \begin{pmatrix} \mathbf{S} & \mathbf{0} \\ \mathbf{0} & \mathbf{C} \end{pmatrix}$$

  The inverse of $\mathbf{U}$ just flips the sign of the off-diagonal block, so

$$\boldsymbol{\Sigma}^{-1} = (\mathbf{U}^\mathsf{T})^{-1}\mathbf{D}^{-1}\mathbf{U}^{-1} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\mathbf{C}^{-1}\mathbf{B}^\mathsf{T} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{S}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{C}^{-1} \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{B}\mathbf{C}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}$$

  and, as $\det \mathbf{U} = 1$, $\det \boldsymbol{\Sigma} = \det \mathbf{S}\ \det \mathbf{C}$

- **The exponent splits.** With $\boldsymbol{u} = \boldsymbol{x} - \mathbf{B}\mathbf{C}^{-1}\boldsymbol{y}$,

$$(\boldsymbol{x}^\mathsf{T}, \boldsymbol{y}^\mathsf{T})\,\boldsymbol{\Sigma}^{-1} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{pmatrix} = \boldsymbol{u}^\mathsf{T}\mathbf{S}^{-1}\boldsymbol{u} + \boldsymbol{y}^\mathsf{T}\mathbf{C}^{-1}\boldsymbol{y}$$

- **Marginal.** Integrating over $\boldsymbol{x}$ is the same as integrating over $\boldsymbol{u}$ (a shift), which gives $\sqrt{\det(2\pi\mathbf{S})}$, so

$$p(\boldsymbol{y}) = \frac{\sqrt{\det(2\pi\mathbf{S})}}{\sqrt{\det(2\pi\boldsymbol{\Sigma})}}\,\mathrm{e}^{-\frac{1}{2}\boldsymbol{y}^\mathsf{T}\mathbf{C}^{-1}\boldsymbol{y}} = \frac{1}{\sqrt{\det(2\pi\mathbf{C})}}\,\mathrm{e}^{-\frac{1}{2}\boldsymbol{y}^\mathsf{T}\mathbf{C}^{-1}\boldsymbol{y}} = \mathcal{N}(\boldsymbol{y} \mid \mathbf{0}, \mathbf{C})$$

  using $\det(2\pi\boldsymbol{\Sigma}) = \det(2\pi\mathbf{S})\ \det(2\pi\mathbf{C})$

- **Conditional.** Dividing the joint density by $p(\boldsymbol{y})$ leaves

$$p(\boldsymbol{x} \mid \boldsymbol{y}) = \frac{1}{\sqrt{\det(2\pi\mathbf{S})}}\,\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x} - \mathbf{B}\mathbf{C}^{-1}\boldsymbol{y})^\mathsf{T}\mathbf{S}^{-1}(\boldsymbol{x} - \mathbf{B}\mathbf{C}^{-1}\boldsymbol{y})} = \mathcal{N}\big(\boldsymbol{x} \mid \mathbf{B}\mathbf{C}^{-1}\boldsymbol{y},\ \mathbf{A} - \mathbf{B}\mathbf{C}^{-1}\mathbf{B}^\mathsf{T}\big)$$

  the same result as the light derivation in the lecture

3 Exercises

3.1 Fourth moment

- Use the MGF of $\mathcal{N}(0, \sigma^2)$ to show $\mathbb{E}\left[X^4\right] = 3\sigma^4$. (Expand $\mathrm{e}^{\sigma^2 t^2/2}$ as a power series and compare with $\sum_n \mathbb{E}[X^n]\, t^n/n!$.)

3.2 Combining sensors

- Two sensors read $y_1 = 10.0$ (noise standard deviation 1) and $y_2 = 12.0$ (standard deviation 2). With a flat prior, what are the mean and variance of the combined estimate?

3.3 Prediction

- Two standardised variables ($\mu_1 = \mu_2 = 0$, $\sigma_1 = \sigma_2 = 1$) have correlation $\rho = 0.8$. Given $x_1 = 2$, find the mean and variance of x_2.

3.4 Conditional covariance

- Using the linear-transformation rule, show that $\boldsymbol{w} = \boldsymbol{z}_1 - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{z}_2$ has covariance $\boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}$.

3.5 Steady-state Kalman filter

- After many steps the Kalman variance settles to a fixed value v. Write $p = v + q$ and show that $p^2 - q\,p - q\,\sigma^2 = 0$. Find v for $q = 0.1$ and $\sigma^2 = 1$.

3.6 The normal moment generating function

- Show that for $X \sim \mathcal{N}(\mu, \sigma^2)$ the moment generating function is $Z(\ell) = \mathrm{e}^{\mu\ell + \sigma^2\ell^2/2}$, by completing the square in the exponent. What does this say about the cumulants of a normal distribution?

4 Experiments

4.1 Checking the formulae

- The first part checks the conditional formula by keeping samples whose z_2 is close to an observed value; the second runs a one-dimensional Kalman filter

```
import numpy as np

rng = np.random.default_rng(0)

# 1. Check the conditional formula by sampling: keep samples whose z2 is
#    close to an observed value and compare with the formula
mu = np.array([1.0, -1.0, 0.5])
L = rng.standard_normal((3, 3))
Sigma = L @ L.T + 0.5 * np.eye(3)                 # a random covariance
z = rng.multivariate_normal(mu, Sigma, size=2_000_000)
z2_obs = np.array([0.0, 1.0])                     # observe the last two
near = np.all(np.abs(z[:, 1:] - z2_obs) < 0.05, axis=1)
S11, S12, S22 = Sigma[:1, :1], Sigma[:1, 1:], Sigma[1:, 1:]
mean = mu[:1] + S12 @ np.linalg.solve(S22, z2_obs - mu[1:])
var = S11 - S12 @ np.linalg.solve(S22, S12.T)
```

```
print(f"{near.sum()} samples near z2 = {z2_obs}")
print(f"conditional mean: formula {mean[0]:.3f}, samples {z[near, 0].mean():.3f}")
print(f"conditional var:  formula {var[0, 0]:.3f}, samples {z[near, 0].var():.3f}")

# 2. A one-dimensional Kalman filter tracking a drifting position
T, q, sigma2 = 50, 0.1, 1.0
x = np.cumsum(rng.normal(0, np.sqrt(q), T))       # true positions
y = x + rng.normal(0, np.sqrt(sigma2), T)         # noisy observations
m, v = 0.0, 10.0                                  # vague initial belief
est = []
for t in range(T):
    v = v + q                                     # predict
    k = v / (v + sigma2)                          # update
    m, v = m + k * (y[t] - m), (1 - k) * v
    est.append(m)
est = np.array(est)
print(f"RMS error: raw observations {np.sqrt(np.mean((y - x)**2)):.3f}, "
      f"Kalman estimate {np.sqrt(np.mean((est - x)**2)):.3f}, final variance {v:.3f}")
```

- Running it: the sampled conditional mean and variance (1.30, 0.85 from about 900 samples) agree with the formula (1.27, 0.82) to within sampling error; the Kalman filter reduces the RMS error from 0.96 (raw observations) to 0.57, and its variance settles at 0.270, the value found in the exercise

5 Answers

5.1 Fourth moment

- $\mathrm{e}^{\sigma^2 t^2/2} = 1 + \frac{\sigma^2 t^2}{2} + \frac{\sigma^4 t^4}{8} + \cdots$. The coefficient of t^4 is $\mathbb{E}\left[X^4\right]/4!$, so $\mathbb{E}\left[X^4\right] = 24\,\sigma^4/8 = 3\sigma^4$

5.2 Combining sensors

- Precisions 1 and $1/4$, so the mean is $(10 + 12/4)/(1 + 1/4) = 13/1.25 = 10.4$ and the variance is $1/1.25 = 0.8$: closer to, and better than, the better sensor

5.3 Prediction

- $\mathbb{E}[x_2 \mid x_1] = 0.8 \times 2 = 1.6$ and $\mathrm{Var}(x_2 \mid x_1) = 1 - 0.64 = 0.36$

5.4 Conditional covariance

- $\boldsymbol{w} = \mathbf{W}\boldsymbol{z}$ with $\mathbf{W} = (\mathbf{I},\ -\boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1})$, so its covariance is

$$\begin{aligned}\mathbf{W}\boldsymbol{\Sigma}\mathbf{W}^{\mathsf{T}} &= \boldsymbol{\Sigma}_{11} - 2\,\boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21} + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{22}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21} \\ &= \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}\end{aligned}$$

5.5 Steady-state Kalman filter

- At the fixed point $v = (1-k)p = p\,\sigma^2/(p+\sigma^2)$ with $v = p - q$. So $(p-q)(p+\sigma^2) = p\,\sigma^2$, i.e. $p^2 - q\,p - q\,\sigma^2 = 0$
- With $q = 0.1$, $\sigma^2 = 1$: $p = (0.1 + \sqrt{0.01 + 0.4})/2 = 0.370$, so $v = 0.270$ (as in the experiment)

5.6 The normal moment generating function

- $$Z(\ell) = \frac{1}{\sqrt{2\pi}\,\sigma} \int \mathrm{e}^{\ell x - (x-\mu)^2/(2\sigma^2)}\,\mathrm{d}x$$
- Completing the square,

$$\ell\, x - \frac{(x-\mu)^2}{2\sigma^2} = -\frac{\big(x - \mu - \sigma^2\ell\big)^2}{2\sigma^2} + \mu\,\ell + \frac{\sigma^2\ell^2}{2}$$

- The remaining integral is a normal density with mean $\mu + \sigma^2\ell$, which integrates to 1, so $Z(\ell) = \mathrm{e}^{\mu\ell + \sigma^2\ell^2/2}$
- Hence $G(\ell) = \log Z(\ell) = \mu\,\ell + \frac{1}{2}\sigma^2\ell^2$ exactly: $\kappa_1 = \mu$, $\kappa_2 = \sigma^2$, and every higher cumulant is zero. The normal is the distribution whose cumulants stop after the second

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 21: Gaussian Processes

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Gaussian Processes, regression

2 Main Points

2.1 The Big Picture

- Gaussian processes are a machine learning method for regression
  - You can use them for classification but it is regression where they excel
- They put a prior on functions (preferring smooth functions)
- Conceptually they are easy
- They are easy to use
- But they are mathematically fiddly to understand
- Gaussian processes were first used for spatial modelling where the technique was know as *krigging*
- They are one of the most powerful methods we have of doing regression
- They are quite general purpose as the prior we use is reasonable for a large number of applications

2.2 Gaussian Processes

- Gaussian Processes are also mathematical objects
- A Gaussian Process assigns a probability to functions depending on their spatial smoothness
  - We assume that the probability of $f(\boldsymbol{x})$ taking a particular value is normally/Gaussian distributed in a way that depends on the value of value of the function at other points
- This will be governed by a *covariance* or *kernel* function $k(\boldsymbol{x},\boldsymbol{y})$ which tells us the covariance of the prior
$$\mathbb{E}_f\left[(f(\boldsymbol{x})-m(\boldsymbol{x}))\,(f(\boldsymbol{y})-m(\boldsymbol{y}))\right]=k(\boldsymbol{x},\boldsymbol{y})$$
  - This is an expectation over all functions from our probability distribution
  - $m(\boldsymbol{x})$ encodes a prior belief in the value of $f(\boldsymbol{x})$—normally we choose this to be 0 as most often we don't have any prior knowledge of the function
  - Because the covariance is quadratic it will be positive semi-definite

- The kernel function is critical to determining the type of function we will learn
- Choosing this kernel function correctly is essential to make GP work

- The values at each point $\boldsymbol{x}$ is normally or Gaussianly distributed

$$p(f|m,k) \propto \mathrm{e}^{-\frac{1}{2}\int (f(\boldsymbol{x})-m(\boldsymbol{x}))\,k^{-1}(\boldsymbol{x},\boldsymbol{y})\,(f(\boldsymbol{y})-m(\boldsymbol{y}))\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}\boldsymbol{y}}$$

  - It is a little unclear exactly what this means for functions: working with function spaces one has to be very careful (e.g. to normalise this we have to sum over all functions)
  - Nevertheless it is clear that we can use this for comparing the likelihood of different functions
  - If you discretise the domain of $\boldsymbol{x}$ then

$$p(f|m,k) \propto \mathrm{e}^{-\frac{1}{2}\sum_{i,j}(f(\boldsymbol{x}_i)-m(\boldsymbol{x}_i))\,k^{-1}(\boldsymbol{x}_i,\boldsymbol{x}_j)\,(f(\boldsymbol{x}_j)-m(\boldsymbol{x}_j))}$$

    - the probability function is now a well-define multi-dimensional normal distribution
    - here it is clear that the value at each point is normally distributed
    - to get back to the integral form we have to take a limit where our discrete points become every closer together (this stresses out mathematicians who point out that nasty things can happen, but for the rest of us we just go ahead)
  - We can sample Gaussian processes from this distribution
    - in low dimensions this allows us to visualise the type of functions we are likely to learn
    - you can try this out in the experiment section

2.3 Bayesian Inference

- We can use Gaussian Processes as a prior for performing regression
- We assume we have a set of data points $\mathcal{D} = \big((\boldsymbol{x}_i, y_i)\big|i = 1,\ldots,m\big)$
- The likelihood for data is taken to be a normal distribution

$$p(\mathcal{D}|f) = \prod_{i=1}^{m} \mathcal{N}\left(y_i\middle|f(\boldsymbol{x}_i),\sigma^2\right)$$

  - We assume that $f(\boldsymbol{x}_i)$ is the true value of the function at $\boldsymbol{x}_i$, but, because of measurement error, y_i will be distributed according to a normal distribution
  - σ is the accuracy of our measured data (it is another hyper-parameter we must choose)
  - Note we are assuming the data points are independent of each other (which is usually a good approximation)

- We can now compute a posterior

$$p(f|\mathcal{D},m,k,\sigma) = \frac{p(\mathcal{D}|f,\sigma)\,p(f|m,k)}{p(\mathcal{D}|m,k,\sigma)}$$

  - I have explicitly written in the dependencies on they hyper-parameters m, k and σ
  - The Gaussian Process prior is a conjugate distribution for the normal likelihood
  - Computing probabilities over functions is slightly wild

- It is more useful to compute the probability at a particular point $\boldsymbol{x}^*$

- To compute this we marginalise over all other points
  - This is non-trivial but doable
  - If you have a multivariate normal distribution

$$\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu},\mathbf{C}) = \frac{1}{\sqrt{|2\,\pi\,\mathbf{C}|}}\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\mathbf{C}^{-1}\,(\boldsymbol{x}-\boldsymbol{\mu})}$$

    and you integrate over x_i (marginalise it out) then

$$\int_{-\infty}^{\infty}\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu},\mathbf{C})\,\mathrm{d}x_i = \mathcal{N}(\hat{\boldsymbol{x}}|\hat{\boldsymbol{\mu}},\hat{\mathbf{C}})$$

    where $\hat{\boldsymbol{x}}$ and $\hat{\boldsymbol{\mu}}$ are identical to $\boldsymbol{x}$ and $\boldsymbol{\mu}$ except with their i^{th} component removed and $\hat{\mathbf{C}}$ is identical to $\mathbf{C}$ except with the ith row and column removed
  - Intuitively this seems very natural (if x_i can take any value it doesn't change the mean or covariance between other random variables)
  - You can prove this algebraically but it is not a trivial calculation
  - Because of this for a Gaussian Process we only care about those set of points where we have data values
- For all point $\boldsymbol{x}$ (except the point $\boldsymbol{x}^*$) we integrate over the possible values that $f(\boldsymbol{x})$ can take
- Where we have no prior this integral is just a constant—and will cancel with the denominator $p(\mathcal{D}|m,k,\sigma)$
- We are left with

$$p(f(\boldsymbol{x}^*)|\mathcal{D}) \propto \left(\prod_i \int \mathrm{d}f(\boldsymbol{x}_i)\right) \mathrm{e}^{-\frac{1}{2\sigma^2}\sum_{i=1}^{m}(y_i-f(\boldsymbol{x}_i))^2}\;\mathrm{e}^{-\frac{1}{2}\sum_{i,j=1}^{m}(f(\boldsymbol{x}_i)-m(\boldsymbol{x}_i))\,k^{-1}(\boldsymbol{x}_i,\boldsymbol{x}_j)\,(f(\boldsymbol{x}_j)-m(\boldsymbol{x}_j))}$$
$$\mathrm{e}^{-\sum_{j=1}^{m}(f(\boldsymbol{x}^*)-m(\boldsymbol{x}^*))\,k^{-1}(\boldsymbol{x}^*,\boldsymbol{x}_j)\,(f(\boldsymbol{x}_j)-m(\boldsymbol{x}_j))}$$
$$\mathrm{e}^{-\frac{1}{2}(f(\boldsymbol{x}^*)-m(\boldsymbol{x}^*))\,k^{-1}(\boldsymbol{x}^*,\boldsymbol{x}^*)(f(\boldsymbol{x}^*)-m(\boldsymbol{x}^*))}$$

  - Writing $f^* = f(\boldsymbol{x}^*)$, $f_i = f(\boldsymbol{x}_i)$, $m_i = m(\boldsymbol{x}_i)$, $k^{-1}_{ij} = k^{-1}(\boldsymbol{x}_i,\boldsymbol{x}_j)$, $k^{*-1}_j = k^{-1}(\boldsymbol{x}^*,\boldsymbol{x}_j)$ and $k^{*-1} = k^{-1}(\boldsymbol{x}^*,\boldsymbol{x}^*)$ then

$$p(f^*|\mathcal{D}) \propto \mathrm{e}^{-\frac{1}{2}k^{*-1}(f^*-m^*)^2}\left(\prod_i \int \mathrm{d}f_i\right)\mathrm{e}^{-\frac{1}{2\sigma^2}\sum_{i=1}^{m}(y_i-f_i)^2-\frac{1}{2}\sum_{i,j=1}^{m}(f_i-m_i)\,k^{-1}_{ij}\,(f_j-m_j)}$$
$$\mathrm{e}^{-\sum_{j=1}^{m}(f^*-m^*)\,k^{*-1}_j\,(f_j-m_j)}$$

  - Doing Gaussian integrals is a pain—*I don't expect you to do this*
  - But if you are brave enough to try let $g_i = f_i - m_i$, $g^* = f^* - m^*$ and $\hat{y}_i = y_i - m_i$ then we can rewrite this as

$$p(f^*|\mathcal{D}) \propto \mathrm{e}^{-\frac{1}{2}k^{*-1}g^{*2}}\left(\prod_i \int \mathrm{d}g_i\right)\mathrm{e}^{-\frac{1}{2\sigma^2}\sum_{i=1}^{m}(\hat{y}_i-g_i)^2-\frac{1}{2}\sum_{i,j=1}^{m}g_i\,k^{-1}_{ij}\,g_j-g^*\sum_{j=1}^{m}k^{*-1}_i\,g_j}$$

    * where we made a change of variable in the integral
  - In matrix form we can write this as

$$p(f^*|\mathcal{D}) \propto \mathrm{e}^{-\frac{1}{2}k^{*-1}g^{*2}}\left(\prod_i \int \mathrm{d}g_i\right)\mathrm{e}^{(\sigma^{-2}\,\hat{\boldsymbol{y}}-g^*\boldsymbol{k}^{*-1})^{\mathsf{T}}\,\boldsymbol{g}-\frac{1}{2}\boldsymbol{g}^{\mathsf{T}}\,(\mathbf{k}^{-1}-\sigma^{-2}\mathbf{I})\,\boldsymbol{g}}$$

* I've dropped a term $\mathrm{e}^{-\frac{\|\hat{\boldsymbol{y}}\|^2}{2\,\sigma^2}}$ since this is a constant that can be absorbed into the constant of proportionality

- To perform the integrals over g_i we *complete the square*. To make the algebra easier to follow we define $\mathbf{M} = \mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}$ and $\boldsymbol{b} = \sigma^{-2}\,\hat{\boldsymbol{y}} - g^*\boldsymbol{k}^{*-1}$. We now rewrite the exponent, E, of the integral as

$$\begin{aligned} E &= (\sigma^{-2}\,\hat{\boldsymbol{y}} - g^*\boldsymbol{k}^{*-1})^{\mathsf{T}}\,\boldsymbol{g} - \frac{1}{2}\boldsymbol{g}^{\mathsf{T}}(\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I})\,\boldsymbol{g} = \boldsymbol{b}^{\mathsf{T}}\,\boldsymbol{g} - \frac{1}{2}\boldsymbol{g}^{\mathsf{T}}\mathbf{M}\boldsymbol{g} \\ &= -\frac{1}{2}(\boldsymbol{g} - \mathbf{M}^{-1}\boldsymbol{b})^{\mathsf{T}}\mathbf{M}\,(\boldsymbol{g} - \mathbf{M}^{-1}\boldsymbol{b}) + \frac{1}{2}\boldsymbol{b}^{\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{b} \end{aligned}$$

  * You simply have to expand out all the terms to see these are the same
- Defining $\boldsymbol{h} = \boldsymbol{g} - \mathbf{M}^{-1}\boldsymbol{b}$ and substituting this back into $p(f^*|\mathcal{D})$ we get

$$p(f^*|\mathcal{D}) \propto \mathrm{e}^{-\frac{1}{2}k^{*-1}g^{*2} + \frac{1}{2}\boldsymbol{b}^{\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{b}} \left(\prod_i \int \mathrm{d}h_i\right) \mathrm{e}^{-\frac{1}{2}\boldsymbol{h}^{\mathsf{T}}\mathbf{M}\,\boldsymbol{h}}$$

- The last integral is a Gaussian integral that integrates to a constant (and can be absorbed into the constant of proportionality)
- We are left with

$$\begin{aligned} p(f^*|\mathcal{D}) &\propto \mathrm{e}^{-\frac{1}{2}k^{*-1}g^{*2} + \frac{1}{2}\boldsymbol{b}^{\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{b}} \\ &= \mathrm{e}^{-\frac{1}{2}k^{*-1}g^{*2} + \frac{1}{2}(\sigma^{-2}\,\hat{\boldsymbol{y}} - g^*\boldsymbol{k}^{*-1})^{\mathsf{T}}\mathbf{M}^{-1}(\sigma^{-2}\,\hat{\boldsymbol{y}} - g^*\boldsymbol{k}^{*-1})} \\ &\propto \mathrm{e}^{-\frac{1}{2}(k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1})g^{*2} - \frac{g^*}{\sigma^2}\hat{\boldsymbol{y}}^{\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1}} \end{aligned}$$

  * where we have used $\boldsymbol{b} = \sigma^{-2}\,\hat{\boldsymbol{y}} - g^*\boldsymbol{k}^{*-1}$
  * we have also thrown away terms not involving g^* (as they are constants that can be absorbed into the constant of proportionality)
- We can again *complete the square* this time for g^*

$$p(f^*|\mathcal{D}) \propto \mathrm{e}^{-\frac{1}{2}(k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1})(g^* - \frac{\hat{\boldsymbol{y}}^{\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1}}{\sigma^2(k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1})})}$$

  * We've again dropped terms that don't contain g^*
- But $g^* = f^* - m^*$ so we can write this as

$$\begin{aligned} p(f^*|\mathcal{D}) &\propto \mathrm{e}^{-\frac{1}{2}(k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1})(f^* - m^* - \frac{\hat{\boldsymbol{y}}^{\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1}}{\sigma^2(k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1})})} \\ &\propto \mathcal{N}\left(f^*\middle| m^* + \frac{\hat{\boldsymbol{y}}^{\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1}}{\sigma^2(k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1})}, k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{k}^{*-1}\right) \end{aligned}$$

  * where we use the fact that we end up with a term that has the form of a normal distribution
  * because the posterior term is normalise, in fact it has to be exactly equal to this normal distribution
- Using $\mathbf{M} = \mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}$

$$p(f^*|\mathcal{D}) = \mathcal{N}\left(f^*\middle| m^* + \frac{\hat{\boldsymbol{y}}^{\mathsf{T}}\left(\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}\right)^{-1}\boldsymbol{k}^{*-1}}{\sigma^2(k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\left(\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}\right)^{-1}\boldsymbol{k}^{*-1})}, \frac{1}{k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\left(\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}\right)^{-1}\boldsymbol{k}^{*-1}}\right)$$

- In other words the expected mean value of $f^* = f(\boldsymbol{x}^*)$ is

$$\mathbb{E}[f(\boldsymbol{x}^*)] = m(\boldsymbol{x}^*) + \frac{\hat{\boldsymbol{y}}^{\mathsf{T}}\left(\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}\right)^{-1}\boldsymbol{k}^{*-1}}{\sigma^2(k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\left(\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}\right)^{-1}\boldsymbol{k}^{*-1})}$$

  (note that usually $m(\boldsymbol{x}^*) = 0$) and the expected variance in the value is

$$\mathbb{V}\mathrm{ar}[f(\boldsymbol{x}^*)] = \frac{1}{k^{*-1} - \boldsymbol{k}^{*-1\mathsf{T}}\left(\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}\right)^{-1}\boldsymbol{k}^{*-1}}$$

- The result is pretty horrible because it involves inverting a matrix K with components $k(\boldsymbol{x}, \boldsymbol{y})$ evaluated at the set of points $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m, \boldsymbol{x}^*\}$
- This is usually feasible but there is a simpler form that occurs because of identities involving inverse matrices
- This is best obtained using a second way of deriving the results
- I went to the pain of deriving the result this way because it is just a consequence of Bayes' rule

- In the second derivation we consider joint probability of the observations $\{y_i | i = 1, 2, \ldots, m\}$ and $f^* = f(\boldsymbol{x}^*)$
  - Even this derivation is a pain and you are **not** expected to know it
  - Now we assume $y_i = f(\boldsymbol{x}_i) + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ (that is our observations have a normally distributed error) then

$$\mathbb{E}[y_i\, y_j] = \mathbb{E}[(f(\boldsymbol{x}_i) + \epsilon_i)(f(\boldsymbol{x}_j) + \epsilon_j)] = k(\boldsymbol{x}_i, \boldsymbol{x}_j) + \sigma^2\, \delta_{ij}$$

    - since the observation error, ϵ_i, is assumed independent of the function value, $f(\boldsymbol{x}_i)$
    - here we assume $m(\boldsymbol{x}) = 0$ just to make life simple
  - Similarly

$$\mathbb{E}[y_i\, f^*] = \mathbb{E}[(f(\boldsymbol{x}_i) + \epsilon_i) f(\boldsymbol{x}^*)] = k(\boldsymbol{x}_i, \boldsymbol{x}^*)$$

    and

$$\mathbb{E}\left[(f^*)^2\right] = k(\boldsymbol{x}^*, \boldsymbol{x}^*)$$

  - Thus

$$p(f^*, \boldsymbol{y}) = \mathcal{N}\left(\begin{pmatrix}\boldsymbol{y}\\ f_*\end{pmatrix} \middle| \boldsymbol{0}, \begin{pmatrix}\mathsf{K} + \sigma^2\,\mathsf{I} & \boldsymbol{k}^*\\ \boldsymbol{k}^{*\mathsf{T}} & k^*\end{pmatrix}\right)$$

    - where K is a matrix with components $k(\boldsymbol{x}_i, \boldsymbol{x}_j)$, $\boldsymbol{k}^*$ is a vector with components $k(\boldsymbol{x}_i, \boldsymbol{x}^*)$ and $k^* = k(\boldsymbol{x}^*, \boldsymbol{x}^*)$
  - We now use an identity involving matrix inverses

$$\begin{pmatrix}\mathsf{K} + \sigma^2\,\mathsf{I} & \boldsymbol{k}^*\\ \boldsymbol{k}^{*\mathsf{T}} & k^*\end{pmatrix}^{-1} = \begin{pmatrix}\left(\mathsf{K} + \sigma^2\,\mathsf{I} - \frac{\boldsymbol{k}^{*\mathsf{T}}\boldsymbol{k}^*}{k^*}\right)^{-1} & -\frac{(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{k}^*}{k^* - \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{k}^*}\\ -\frac{\boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}}{k^* - \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{k}^*} & \frac{1}{k^* - \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{k}^*}\end{pmatrix}$$

    - these identities involving inverses of matrices seem to come from nowhere—they make working with normal distributions a real pain
    - you can prove the identity by multiply the right-hand side by $\begin{pmatrix}\mathsf{K} + \sigma^2\,\mathsf{I} & \boldsymbol{k}^*\\ \boldsymbol{k}^{*\mathsf{T}} & k^*\end{pmatrix}$ and showing you get the identity
    - if you are tempted to do this then set $\sigma^2 = 0$ and rename $\boldsymbol{k}^*$ and k^* so its easier to do the algebra—it is a tedious calculation (see exercises)
  - Now we use $p(f^*|\mathcal{D}) = p(f^*|\boldsymbol{y}) = p(f^*, \boldsymbol{y})/p(\boldsymbol{y})$
  - All you need to do is collect the terms in $p(f^*, \boldsymbol{y})$ involving f^*

$$p(f^*|\mathcal{D}) \propto \mathrm{e}^{-\frac{f^{*2}}{2(k^* - \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{k}^*)} + \frac{f^*\,\boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{y}}{k^* + \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{k}^*}}$$

  - *Completing the square*

$$p(f^*|\mathcal{D}) \propto \mathrm{e}^{-\frac{1}{2(k^* - \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{k}^*)}\left(f^* - \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K}+\sigma^2\,\mathsf{I})^{-1}\boldsymbol{y}\right)^2}$$

  - But as $p(f^*|\mathcal{D})$ must be normalised

$$p(f(\boldsymbol{x}^*)|\mathcal{D}) = \mathcal{N}\left(f(\boldsymbol{x}^*)\middle| \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K} + \sigma^2\,\mathsf{I})^{-1}\boldsymbol{y},\; k^* - \boldsymbol{k}^{*\mathsf{T}}(\mathsf{K} + \sigma^2\,\mathsf{I})^{-1}\boldsymbol{k}^*\right)$$

- That is the mean value of f at a point $\boldsymbol{x}^*$ is

$$\mathbb{E}[f(\boldsymbol{x}^*)] = m(\boldsymbol{x}^*) + \boldsymbol{k}^{*\mathsf{T}}(\mathbf{K} + \sigma^2\,\mathbf{I})^{-1}\boldsymbol{y}$$

  - I've re-instated $m(\boldsymbol{x}^*)$ (even though it is usually taken to be zero)
- The variance is given by

$$\mathbb{V}\mathrm{ar}[f(\boldsymbol{x}^*)] = k^* - \boldsymbol{k}^{*\mathsf{T}}(\mathbf{K} + \sigma^2\,\mathbf{I})^{-1}\boldsymbol{k}^*$$

- This may look completely different from the previous expression but surprisingly it is identical (you can demonstrate this numerically)
- The advantage over the previous methods is that we only need to do one matrix inversion (and we get a slightly easier expression)
- Although the mathematics to get here was horrible the expression is extremely easy to compute
- One of the advantages of Bayesian inference is it provides an estimate of its own uncertainty (in this case $\mathbb{V}\mathrm{ar}[f(\boldsymbol{x}^*)]$)

2.4 Learning Hyperparameters

- **Hyperparameters in the Bayesian Framework**
  - As I have said many times to get machine learning to work it is vital to choose the hyperparameter properly (we will see this is true for GPs)
  - In the Bayesian framework if $\boldsymbol{\phi}$ are hyperparameters then Bayes' rules says

$$p(\boldsymbol{x}|\mathcal{D}, \boldsymbol{\phi}) = \frac{p(\mathcal{D}|\boldsymbol{x}, \boldsymbol{\phi})\,p(\boldsymbol{x}|\boldsymbol{\phi})}{p(\mathcal{D}|\boldsymbol{\phi})}$$

    - this is just Bayes' rule with everything conditioned on the hyperparameters $\boldsymbol{\phi}$
  - We can select hyperparameters by considering the **evidence**, $p(\mathcal{D}|\boldsymbol{\phi})$
    - when the hyperparameters are viewed as different models this is known as **model selection**
    - this is also called the evidence framework
    - We can view the relatively likelihood of one model, $\boldsymbol{\phi}_1$ compare to a second, $\boldsymbol{\phi}_2$ by examining

$$\frac{p(\mathcal{D}|\boldsymbol{\phi}_1)}{p(\mathcal{D}|\boldsymbol{\phi}_2)}$$

  - If we want to be hyper-Bayesian then we can put a prior, $p(\boldsymbol{\phi})$, over our hyperparameters and the calculate the joint posterior for the parameters of the likelihood $\boldsymbol{\theta}$ and the hyper-parameters $\boldsymbol{\phi}$

$$p(\boldsymbol{\theta}, \boldsymbol{\phi}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\theta}, \boldsymbol{\phi})\,p(\boldsymbol{\theta}|\boldsymbol{\phi})\,p(\boldsymbol{\phi})}{p(\mathcal{D})}$$

    - to compute the posterior we are interested in we marginalise out the hyperparameters

$$p(\boldsymbol{\theta}|\mathcal{D}) = \int p(\boldsymbol{\theta}, \boldsymbol{\phi}|\mathcal{D})\,\mathrm{d}\boldsymbol{\phi}$$

    - Often this integral is not computable in closed form and we are forced to estimate it using Monte-Carlo techniques
- **Hyperparameter for Gaussian Processes**
  - For Gaussian Processes this means choosing the right kernel function

- Here we are in a better positions than in SVMs in that the kernel represents the covariance function
- Given data we could estimate the kernel function using

$$\mathbb{E}[y_i, y_j] = k(\boldsymbol{x}_i, \boldsymbol{x}_j) + \sigma^2 \delta_{ij}$$

- It would be an inverse problem to estimate $k(\boldsymbol{x}_i, \boldsymbol{x}_j)$
- Often we start with a guess of the form of the kernel
- A very common kernel function is the Gaussian kernel

$$k(\boldsymbol{x}, \boldsymbol{y}) = \mathrm{e}^{-\frac{\|\boldsymbol{x}-\boldsymbol{y}\|^2}{2\ell^2}}$$

- Now ℓ is a *scale parameter* that has to be determined
- However, a great advantage of Gaussian Processes is that we can compute the evidence in closed form

$$\log(p(\mathcal{D}|\boldsymbol{\theta})) = -\frac{1}{2}\boldsymbol{y}^\mathsf{T}(\mathbf{K} + \sigma^2\mathbf{I})^{-1}\boldsymbol{y} - \frac{1}{2}\log\big(|\mathbf{K} + \sigma^2\mathbf{I}|\big) - \frac{m}{2}\log(2\,\pi)$$

  - This allows us to rapidly compare different hyperparameters (kernels and noise level)
  - We are not doing this in a fully Bayesian way (we are maximising our evidence rather than updating a distribution for the hyperparameters) therefore we could overfit
  - However, usually we have relatively few hyperparameters so overfitting is less severe

3 Exercises

3.1 Working with Gaussians

This is optional. Learning to work with Gaussians allows you to understand a lot of machine learning techniques, but it is not going to be examined.

1. Integrals involving Gaussians can be done in closed form. This means that they are heavily used in machine learning. It does, however, take a lot of practice to learn how to do these integrals. The starting point is the integral

$$I_1 = \int_{-\infty}^{\infty} \mathrm{e}^{-x^2/2}\,\mathrm{d}x = \sqrt{2\,\pi}.$$

   This integral isn't easy. The trick is to turn this into a two dimensional integral and perform the integral in polar coordinates. Have a go.

2. Show by a change of variables

$$I_2 = \int_{-\infty}^{\infty} \mathrm{e}^{-x^2/(2\,\sigma^2)}\,\mathrm{d}x = \sqrt{2\,\pi}\,\sigma.$$

3. Show that

$$I_3 = \int_{-\infty}^{\infty} \mathrm{e}^{-x^2/2 + a\,x}\,\frac{\mathrm{d}x}{\sqrt{2\,\pi}} = \mathrm{e}^{a^2/2}$$

   This involves completing the square and using a change of variables. It is probably the most frequently used trick working with Gaussian integrals.

4. Show that

$$I_4 = \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\|\boldsymbol{x}\|^2} \mathrm{d}\boldsymbol{x} = (2\,\pi)^{\frac{n}{2}}$$

   Often we don't bother writing all the integral signs (one will do)

5. By writing $\mathbf{M} = \mathbf{V}\,\mathbf{\Lambda}\,\mathbf{V}^{\mathsf{T}}$ show that

$$I_5 = \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\mathbf{M}\boldsymbol{x}} \mathrm{d}\boldsymbol{x} = (2\,\pi)^{\frac{n}{2}}\,\frac{1}{\sqrt{|\mathbf{M}|}}$$

   where $|\mathbf{M}|$ is the determinant of $\mathbf{M}$. Note that $|\mathbf{M}| = |\mathbf{\Lambda}| = \prod_i \lambda_i$.

6. Show that

$$I_6 = \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\mathbf{M}^{-1}\boldsymbol{x}} \mathrm{d}\boldsymbol{x} = (2\,\pi)^{\frac{n}{2}}\,\sqrt{|\mathbf{M}|}$$

3.2 Working with Matrix Inverses

- Modelling using normal (aka Gaussian) distributions is very powerful because we can usually computer everything in closed form—integrals involving Gaussians are always doable
- However, there is a twist which makes it extremely painful
  - The normal distribution uses the inverse of the covariance matrix
  - The inverse of matrices are a pain to work with
- To get nice results we often consider partition matrices into four blocks—the inverse of the full matrix can be written in terms of the inverse of each block
- Let's look at a very simple example
  - Show that

$$\begin{pmatrix} \mathbf{K} & \boldsymbol{\ell} \\ \boldsymbol{\ell}^{\mathsf{T}} & m \end{pmatrix}^{-1} = \begin{pmatrix} \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^{\mathsf{T}}}{m}\right)^{-1} & -\frac{\mathbf{K}^{-1}\boldsymbol{\ell}}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}} \\ -\frac{\boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}} & \frac{1}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}} \end{pmatrix}$$

    - $\mathbf{K}$ is a symmetric matrix, ℓ is a vector and m a scalar
    - Similar identities exist when ℓ and m are matrices, but let's keep thing simple
  - Now we can show this by multiply both sides by $\begin{pmatrix} \mathbf{K} & \boldsymbol{\ell} \\ \boldsymbol{\ell}^{\mathsf{T}} & m \end{pmatrix}$ (the algebra is easier if we multiply on the right)
  - This is still very fiddly to show, so break it down
    - Show that

$$\begin{pmatrix} \mathbf{A} & \boldsymbol{b} \\ \boldsymbol{b}^{\mathsf{T}} & c \end{pmatrix} \begin{pmatrix} \mathbf{K} & \boldsymbol{\ell} \\ \boldsymbol{\ell}^{\mathsf{T}} & m \end{pmatrix} = \begin{pmatrix} \mathbf{A}\,\mathbf{K} + \boldsymbol{b}\,\boldsymbol{\ell}^{\mathsf{T}} & \mathbf{A}\,\boldsymbol{\ell} + m\,\boldsymbol{b} \\ \boldsymbol{b}^{\mathsf{T}}\mathbf{K} + c\,\boldsymbol{\ell}^{\mathsf{T}} & \boldsymbol{b}^{\mathsf{T}}\boldsymbol{\ell} + c\,m \end{pmatrix}$$

    - Now you have to show that when $\mathbf{A} = \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^{\mathsf{T}}}{m}\right)^{-1}$, $\boldsymbol{b} = -\frac{\mathbf{K}^{-1}\boldsymbol{\ell}}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}}$ and $c = \frac{1}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}}$ that
      1. $\mathbf{A}\,\mathbf{K} + \boldsymbol{b}\,\boldsymbol{\ell}^{\mathsf{T}} = \mathbf{I}$
      2. $\mathbf{A}\,\boldsymbol{\ell} + m\,\boldsymbol{b} = \mathbf{0}$
      3. $\boldsymbol{b}^{\mathsf{T}}\mathbf{K} + c\,\boldsymbol{\ell}^{\mathsf{T}} = \mathbf{0}^{\mathsf{T}}$
      4. $\boldsymbol{b}^{\mathsf{T}}\boldsymbol{\ell} + c\,m = 1$
    - To show 1. and 2. take out $\mathbf{A}$ as a common factor using $\mathbf{I} = \mathbf{A}\,\mathbf{A}^{-1}$ (answer below)

4 Experiments

4.1 Generate Samples from a Gaussian Processes

- To generate a Gaussian Process, $\mathcal{GP}(0, k)$, with $m(\boldsymbol{x}) = 0$ and covariance

$$k(\boldsymbol{x}, \boldsymbol{y}) = \mathrm{e}^{-\frac{\|\boldsymbol{x}-\boldsymbol{y}\|^2}{2\,\ell^2}}$$

  a points $\{\boldsymbol{x}_i | i = 1, 2, \ldots, n\}$

  1. We use first compute the covariance matrix $\mathbf{K}$ with elements $K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$
  2. Compute the *Cholesky decompostion* $\mathbf{L}$ such that $\mathbf{K} = \mathbf{L}\,\mathbf{L}^\mathsf{T}$
     - For any positive definite matrices we can always compute a Cholesky decomposition (although beware, due to numerical rounding some matrices that should be positive definite, appear not to have Cholesky decompositions—you can $\epsilon\,\mathbf{I}$ to your matrix to help)
     - The Cholesky decomposition is a lower diagonal matrix
     - It is used to efficiently solve linear problems involving positive definite matrices (it is more efficient and more stable than Gaussian elimination)
  3. Now let $\boldsymbol{f} = \mathbf{L}\,\boldsymbol{\eta}$ where $\boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
     - We note that

$$\mathbb{E}\left[\boldsymbol{f}\,\boldsymbol{f}^\mathsf{T}\right] = \mathbb{E}\left[\mathbf{L}\,\boldsymbol{\eta}\,\boldsymbol{\eta}^\mathsf{T}\mathbf{L}^\mathsf{T}\right] = \mathbf{L}\,\mathbb{E}\left[\boldsymbol{\eta}\,\boldsymbol{\eta}^\mathsf{T}\right]\mathbf{L}^\mathsf{T} = \mathbf{L}\,\mathbf{I}\,\mathbf{L}^\mathsf{T} = \mathbf{L}\,\mathbf{L}^\mathsf{T} = \mathbf{K}$$

     - Thus $\boldsymbol{f}$ can be viewed as samples from a Gaussian Process
     - Draw many different samples and plot them
  4. Experiment with different values of ℓ

1-d Gaussian Processes

```
x = [-10:0.2:10];    % define some points
n = length(x)
l = 1.0                      % define length scale
K = zeros(n,n);         % define holder
for i = 1:n
  K(i,i) = 1;
  for j = 1:i
    K(j,i) = K(i,j) = exp(-(x(i)-x(j))^2/(2*l)); % define covariance matrix
  endfor
endfor
L = chol(K+0.0001*eye(n));  % cheat to persuade octave K is positive definite
f1 = L*randn(n,1);
f2 = L*randn(n,1);
f3 = L*randn(n,1);
plot(x,f1,x,f2,x,f3)
```

2-d Gaussian Process

```
range = [-4:0.2:4];    % define some points
n = length(range)
X = zeros(n*n,2);
for i = 1:n
  for j = 1:n
     X((i-1)*n+j,1) = range(i);
```

```
    X((i-1)*n+j,2) = range(j);
  endfor
endfor
l = 1.0                       % define length scale
N =length(X)
K = zeros(N,N);
for i = 1:N
  K(i,i) = 1;
  for j = 1:i
    K(j,i) = K(i,j) = exp(-norm(X(i,:)-X(j,:))^2/(2*l)); % define covariance matrix
  endfor
endfor

L = chol(K+0.0001*eye(N));  % cheat to persuade octave K is positive definite
f = L*randn(N,1);
fm = reshape(f,n,n);
[xx, yy] = meshgrid(range, range);
mesh(xx, yy, fm);
xlabel ("x");
ylabel ("y");
zlabel ("f(x,y)");
title ("2-d Gaussian Process");
```

5 Answers

5.1 Working with Gaussians

1. The Gaussian Integral.

   - We consider the two dimensional integral

$$I_1^2 = \left(\int_{-\infty}^{\infty} \mathrm{e}^{-x^2/2}\,\mathrm{d}x\right)^2 \overset{\text{(a)}}{=} \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} \mathrm{e}^{-x^2/2-y^2/2}\,\mathrm{d}x\,\mathrm{d}y$$
$$\overset{\text{(b)}}{=} \int_0^{\infty}\int_0^{2\pi} r\,\mathrm{e}^{-r^2}\mathrm{d}r\,\mathrm{d}\theta \overset{\text{(c)}}{=} \int_0^{\infty} r\,\mathrm{e}^{-r^2}\mathrm{d}r \int_0^{2\pi}\,\mathrm{d}\theta = 2\,\pi$$

     (a) Changing the name of the dummy index from x to y
     (b) Using the change of variable $x = r\,\cos(\theta)$ and $y = r\,\sin(\theta)$ so that $\mathrm{d}x\mathrm{d}y = r\mathrm{d}r\,\mathrm{d}\theta$
     (c) Reordering the integrals and using $\int_0^{\infty} r\,\mathrm{e}^{-r^2}\mathrm{d}r = 1$ (which you can prove by a change of variables $u = r^2/2$ so that $r\,\mathrm{d}r = \mathrm{d}u$)

     Therefore $I_1 = \sqrt{2\,\pi}$.

2.
$$I_2 = \int_{-\infty}^{\infty} \mathrm{e}^{-x^2/(2\,\sigma^2)}\,\mathrm{d}x \overset{\text{(a)}}{=} \sigma \int_{-\infty}^{\infty} \mathrm{e}^{-u^2/2}\,\mathrm{d}u \overset{\text{(b)}}{=} \sqrt{2\,\pi}\,\sigma.$$

     (a) Using $u = x/\sigma$ so that $\mathrm{d}x = \sigma\,\mathrm{d}u$
     (b) Using I_1

3.
$$I_2 = \int_{-\infty}^{\infty} \mathrm{e}^{-x^2/2+a\,x}\,\frac{\mathrm{d}x}{\sqrt{2\,\pi}} \overset{\text{(a)}}{=} \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}(x-a)^2+\frac{a^2}{2}}\,\frac{\mathrm{d}x}{\sqrt{2\,\pi}}$$
$$\overset{\text{(b)}}{=} \mathrm{e}^{a^2/2}\int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}(x-a)^2}\,\frac{\mathrm{d}x}{\sqrt{2\,\pi}} \overset{\text{(c)}}{=} \mathrm{e}^{a^2/2}$$

(a) Using $-x^2/2 + a\,x = -\frac{1}{2}(x-a)^2 + \frac{a^2}{2}$

(b) Factoring out $\mathrm{e}^{a^2/2}$

(c) Making a change of variables $u = x - a$ and using integral I_1

4.

$$I_4 = \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\|\boldsymbol{x}\|^2} \mathrm{d}\boldsymbol{x} \overset{\text{(a)}}{=} \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\sum_{i=1}^{n} x_i^2} \prod_{i=1}^{n} \mathrm{d}x_i$$
$$\overset{\text{(b)}}{=} \prod_{i=1}^{n} \left(\int_{-\infty}^{\infty} \mathrm{e}^{-\frac{x_i^2}{2}} \, \mathrm{d}x_i \right) \overset{\text{(c)}}{=} (2\,\pi)^{\frac{n}{2}}$$

(a) Using $\|\boldsymbol{x}\|^2 = \sum_{i=1}^{n} x_i^2$

(b) Using $\exp(\sum_i a_i) = \prod_i \mathrm{e}^{a_i}$ and factoring out the different terms

(c) Using integral I_1

5.

$$I_5 = \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^\mathsf{T}\mathbf{M}\boldsymbol{x}} \mathrm{d}\boldsymbol{x} \overset{\text{(a)}}{=} \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^\mathsf{T}\mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}\boldsymbol{x}} \mathrm{d}\boldsymbol{x}$$
$$\overset{\text{(b)}}{=} \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\boldsymbol{u}^\mathsf{T}\boldsymbol{\Lambda}\boldsymbol{u}} \, \mathrm{d}\boldsymbol{u} \overset{\text{(c)}}{=} \int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\sum_i \lambda_i u_i^2} \, \mathrm{d}\boldsymbol{u}$$
$$\overset{\text{(d)}}{=} \prod_i \left(\int_{-\infty}^{\infty} \mathrm{e}^{-\frac{1}{2}\lambda_i u_i^2} \, \mathrm{d}u_i \right) \overset{\text{(e)}}{=} \prod_i \left(\sqrt{\frac{2\pi}{\lambda_i}} \right) \overset{\text{(f)}}{=} (2\,\pi)^{n/2} \frac{1}{\sqrt{\prod_i \lambda_i}} \overset{\text{(g)}}{=} \frac{(2\,\pi)^{\frac{n}{2}}}{\sqrt{|\mathbf{M}|}}$$

(a) Using $\mathbf{M} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}$ where

(b) Writing $\boldsymbol{u} = \mathbf{V}^\mathsf{T}\boldsymbol{x}$. Note that $\mathrm{d}\boldsymbol{u} = |\mathbf{V}^\mathsf{T}| \, \mathrm{d}\boldsymbol{x}$, but $\mathbf{V}$ is an orthogonal matrix so $|\mathbf{V}^\mathsf{T}| = 1$. So $\mathrm{d}\boldsymbol{u} = \mathrm{d}\boldsymbol{x}$.

(c) $\boldsymbol{\Lambda}$ is a diagonal matrix with elements $\Lambda_{ii} = \lambda_i$

(d) Rearranging

(e) Using integral I_2 with $\sigma = 1/\sqrt{\lambda_i}$

(f) Rearranging

(g) Using $\prod_i \lambda_i = |\mathbf{M}|$.

6. This follows using I_5 because $|\mathbf{M}^{-1}| = \frac{1}{|\mathbf{M}|}$.

5.2 Working with Matrix Inverses

- Using $\mathbf{A} = \left(\mathbf{K} - \frac{\boldsymbol{\ell}\boldsymbol{\ell}^\mathsf{T}}{m}\right)^{-1}$, $\boldsymbol{b} = -\frac{\mathbf{K}^{-1}\boldsymbol{\ell}}{m - \boldsymbol{\ell}^\mathsf{T}\mathbf{K}^{-1}\boldsymbol{\ell}}$ and $c = \frac{1}{m - \boldsymbol{\ell}^\mathsf{T}\mathbf{K}^{-1}\boldsymbol{\ell}}$, we show the four identities

  1.

$$\begin{aligned}
\mathbf{A}\,\mathbf{K} + \boldsymbol{b}\,\boldsymbol{\ell}^\mathsf{T} &= \mathbf{A}\,\left(\mathbf{K} + \mathbf{A}^{-1}\boldsymbol{b}\,\boldsymbol{\ell}^\mathsf{T}\right) \\
&= \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}}{m}\right)^{-1} \left(\mathbf{K} + \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}}{m}\right) \frac{(-\mathbf{K}^{-1}\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T})}{m - \boldsymbol{\ell}^\mathsf{T}\mathbf{K}^{-1}\boldsymbol{\ell}}\right) \\
&= \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}}{m}\right)^{-1} \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}}{m - \boldsymbol{\ell}^\mathsf{T}\mathbf{K}^{-1}\boldsymbol{\ell}} + \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}\mathbf{K}^{-1}\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}/m}{m - \boldsymbol{\ell}^\mathsf{T}\mathbf{K}^{-1}\boldsymbol{\ell}}\right) \\
&= \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}}{m}\right)^{-1} \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}}{m} \frac{m - \boldsymbol{\ell}^\mathsf{T}\mathbf{K}^{-1}\boldsymbol{\ell}}{m - \boldsymbol{\ell}^\mathsf{T}\mathbf{K}^{-1}\boldsymbol{\ell}}\right) \\
&= \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}}{m}\right)^{-1} \left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^\mathsf{T}}{m}\right) = \mathbf{I}
\end{aligned}$$

2.

$$\begin{aligned}\mathbf{A}\,\boldsymbol{\ell} + m\,\boldsymbol{b} &= \mathbf{A}\left(\boldsymbol{\ell} + m\,\mathbf{A}^{-1}\boldsymbol{b}\right)\\ &= \mathbf{A}\left(\boldsymbol{\ell} - m\left(\mathbf{K} - \frac{\boldsymbol{\ell}\,\boldsymbol{\ell}^{\mathsf{T}}}{m}\right)\frac{\mathbf{K}^{-1}\boldsymbol{\ell}}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}}\right)\\ &= \mathbf{A}\left(\boldsymbol{\ell} - \frac{m\,\boldsymbol{\ell}}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}}(1 - \frac{\boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}}{m})\right)\\ &= \mathbf{A}\,(\boldsymbol{\ell} - \boldsymbol{\ell}) = \mathbf{0}\end{aligned}$$

3.

$$\boldsymbol{b}^{\mathsf{T}}\mathbf{K} + c\,\boldsymbol{\ell}^{\mathsf{T}} = -\frac{\boldsymbol{\ell}^{\mathsf{T}}}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}} + \frac{\boldsymbol{\ell}^{\mathsf{T}}}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}} = \mathbf{0}^{\mathsf{T}}$$

4.

$$\boldsymbol{b}^{\mathsf{T}}\boldsymbol{\ell} + c\,m = -\frac{\boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}} + \frac{m}{m - \boldsymbol{\ell}^{\mathsf{T}}\mathbf{K}^{-1}\boldsymbol{\ell}} = 1$$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 22: Probabilistic Inference

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Generative models, Hierarchical Models, MAP, Empirical Bayes, Mixture of Gaussians, Expectation Maximisation

2 Main Points

2.1 Beyond Conjugate Priors

- In the lecture on Bayesian inference we chose priors so that the posterior had the same form as the prior (conjugate priors)
- Real problems involve many interacting random variables, some of which we never observe, and the posterior then has no simple closed form
- This lecture is about how to set up such models and about simple strategies for using them: MAP, empirical Bayes and the EM algorithm. The lecture on MCMC shows how to compute the full posterior
- Everything rests on the two rules from the lecture on probability
  - the sum rule (marginalisation): $\mathbb{P}[X] = \sum_y \mathbb{P}[X, Y = y]$, or $f(x) = \int f(x, y)\,\mathrm{d}y$
  - the product rule (the definition of conditional probability): $\mathbb{P}[X, Y] = \mathbb{P}[X|Y]\ \mathbb{P}[Y] = \mathbb{P}[Y|X]\ \mathbb{P}[X]$, of which Bayes' rule is a rearrangement
  - conditioning does not imply causation

2.2 Building Probabilistic Models

- **Discriminative Models**
  - Often in machine learning our goal is to learn the probability distribution $\mathbb{P}[Y|\boldsymbol{X}]$ where Y is our target and $\boldsymbol{X}$ is a data point
  - We may parameterise this distribution with some parameters $\boldsymbol{\Theta}$ and our task would be to learn these parameters based on training data
- **Generative Models**
  - Surprisingly it is often easier to model the joint probability $\mathbb{P}[Y, \boldsymbol{X}]$
  - This means that we model the process of both generating the targets and the feature vectors together
  - These are known as *generative models* as they allow us to generate random examples

- We don't necessarily want to use them to generate random samples; it just makes the modelling process easier (although you need to get used to this as it feels counter-intuitive)
- We can use generative models to do discrimination since $\mathbb{P}[Y|\boldsymbol{X}] = \mathbb{P}[Y, \boldsymbol{X}]/\mathbb{P}[\boldsymbol{X}]$ where $\mathbb{P}[\boldsymbol{X}] = \sum_Y \mathbb{P}[Y, \boldsymbol{X}]$ (the sum is over the few values of the target, not over all possible feature vectors)

- **Latent Variables**
  - To model complicated processes we introduce intermediate random variables that we never observe
  - These are known as **latent variables**; they are not our target, but they make the model easy to write down
  - To predict we marginalise them out: $\mathbb{P}[\boldsymbol{X}, Y] = \sum_Z \mathbb{P}[\boldsymbol{X}, Y, Z]$

- **Generative Stories**
  - A good way to build a model is to write down how the data could have been generated, as if writing a simulator. Each line draws one variable given variables drawn earlier. For the ratings model below

```
theta = rng.beta(a, b, size=J)    # a rate for each product
k = rng.binomial(n, theta)        # observed positive reviews
```

- By the product rule the joint distribution is the product of the distributions in each line

$$f(\boldsymbol{\theta}, \boldsymbol{k}) = \prod_{j=1}^{J} \mathrm{Beta}(\theta_j|a, b)\,\mathrm{Bin}(k_j|n_j, \theta_j)$$

- *Probabilistic programming languages* such as Stan and PyMC let you write a model in this form and then do the inference (usually by MCMC) for you

- **Graphical Models**
  - The same information can be drawn as a diagram (Figure 1): each random variable is a node, and arrows point to a variable from the variables it is drawn from
  - We use dark nodes for fixed numbers (parameters or known quantities), shaded nodes for observed variables and white nodes for latent variables
  - A box (a *plate*) means "repeat for each": here for each product j
  - The joint distribution is the product, over nodes, of the distribution of each node given its parents
  - Graphical models are also used to read off which variables are independent, but we only use them to design and explain models

2.3 Hierarchical Models

- A shop lists products with the fraction of reviews that are positive. Product A has 3 positive reviews out of 3 (100%), product B 1840 out of 2000 (92%). Which is better?

- Ranking by the fraction puts every product with a handful of good reviews at the top. Intuitively we trust A's 100% less, and our experience that most products are rated about 80% should pull it down

- **Model.** Product j has an unknown probability θ_j that a review is positive, and k_j of its n_j reviews are positive

$$k_j \sim \mathrm{Bin}(n_j, \theta_j), \qquad \theta_j \sim \mathrm{Beta}(a, b)$$

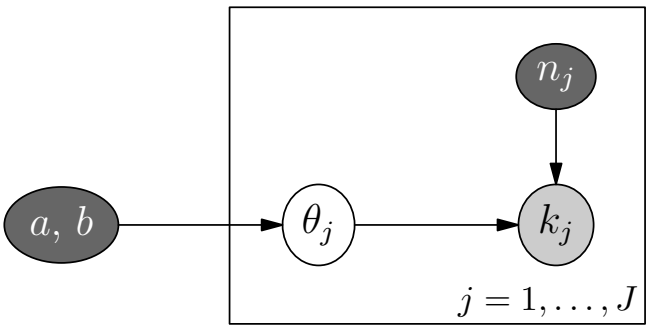


Figure 1: Graphical model for the hierarchical ratings model

- the products are different, but similar: their θ_j come from a common population
- a and b are *hyperparameters*: parameters of the distribution of the parameters. $a/(a+b)$ is the typical rate and $a+b$ measures how alike the products are (the variance of $\text{Beta}(a,b)$ is $ab/((a+b)^2(a+b+1))$, which is small when $a+b$ is large)
- the model has levels (population, product, reviews), so it is called a *hierarchical model*

- **One product at a time.** If we knew a and b, each product would be the beta-binomial problem of the lecture on Bayesian inference, with posterior

$$f(\theta_j|k_j) = \text{Beta}(\theta_j|a+k_j,\, b+n_j-k_j)$$

The posterior mean is a weighted average of the typical rate and the observed fraction

$$\mathbb{E}[\theta_j|k_j] = \frac{a+k_j}{a+b+n_j} = \frac{a+b}{a+b+n_j}\,\frac{a}{a+b} + \frac{n_j}{a+b+n_j}\,\frac{k_j}{n_j}$$

so the prior counts as $a+b$ extra reviews. With $a=20$ and $b=5$ (typical rate 0.8, worth 25 reviews) product A gets $23/28 \approx 0.82$ and product B gets $1860/2025 \approx 0.92$: B is better

- **Shrinkage.** Products with few reviews are pulled (*shrunk*) towards the typical rate, while those with many reviews barely move (Figure 2). Each product *borrows strength* from the others

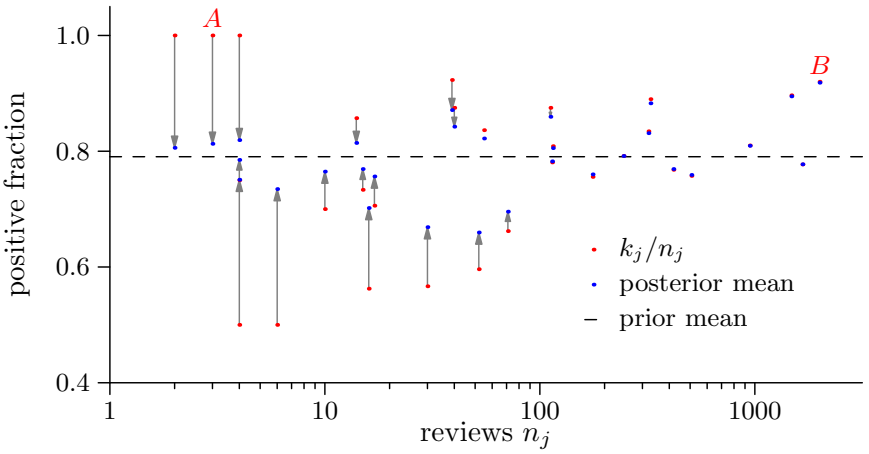


Figure 2: Fraction of positive reviews (red) and posterior means (blue) for 30 products; A and B are the examples above. The hyperparameters were fitted by empirical Bayes

- **Partial pooling.** There are two extremes: estimate each product on its own (k_j/n_j: no pooling), which is noisy when n_j is small, or treat all products as the same (complete pooling), which ignores real differences. A hierarchical model sits in between, and the data

decide where, through $a + b$. The same idea is used for schools, hospitals, adverts, sports players and the users of a recommender system

- But we don't know a and b. If we give them a prior too (a *hyperprior*), the posterior $f(a, b, \boldsymbol{\theta}|\boldsymbol{k})$ is no longer conjugate and has no closed form: a and b appear inside the beta function that normalises $\mathrm{Beta}(\theta_j|a, b)$

2.4 Solution Strategies

- **Full Bayes.** Bayes' rule gives the posterior over all the parameters

$$\mathbb{P}[\boldsymbol{\Theta}|\mathcal{D}] = \frac{\mathbb{P}[\mathcal{D}|\boldsymbol{\Theta}]\ \mathbb{P}[\boldsymbol{\Theta}]}{\mathbb{P}[\mathcal{D}]}$$

  - it describes our uncertainty and, because it averages over parameters rather than choosing the best, it is much less prone to over-fitting
  - but it is difficult: the posterior is rarely a standard distribution, and the *evidence* or *marginal likelihood*

$$\mathbb{P}[\mathcal{D}] = \sum_{\boldsymbol{\Theta}} \mathbb{P}[\mathcal{D}|\boldsymbol{\Theta}]\ \mathbb{P}[\boldsymbol{\Theta}] \qquad \text{or} \qquad f(\mathcal{D}) = \int f(\mathcal{D}|\boldsymbol{\theta})\, f(\boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\theta}$$

    is a sum or integral over far too many values when $\boldsymbol{\Theta}$ is high dimensional
  - Monte Carlo methods (the lecture on MCMC) can sample from the posterior without computing the evidence, but they can be slow

- **Maximum a posteriori (MAP).** Take the mode of the posterior

$$\boldsymbol{\theta}_{\mathrm{MAP}} = \operatorname*{argmax}_{\boldsymbol{\theta}} f(\mathcal{D}|\boldsymbol{\theta})\, f(\boldsymbol{\theta}) = \operatorname*{argmax}_{\boldsymbol{\theta}} \log(f(\mathcal{D}|\boldsymbol{\theta})) + \log(f(\boldsymbol{\theta}))$$

  - we don't need the evidence, which does not depend on $\boldsymbol{\theta}$
  - $-\log(f(\boldsymbol{\theta}))$ acts as a regulariser. For example a Gaussian prior $w_i \sim \mathcal{N}\big(w_i\big|0, s^2\big)$ on weights adds $\|\boldsymbol{w}\|^2/(2s^2)$ to the loss: ridge regression, or weight decay in neural networks. We use this in the lecture on matrix factorisation
  - but it isn't really Bayesian (although some users will claim it is). It keeps a single point and throws away our uncertainty about $\boldsymbol{\theta}$, so we lose our error bars and can over-fit
  - the peak of a probability density is not invariant: change variables (e.g. from σ to $\log \sigma$) and the peak moves, although the probabilities do not
  - in high dimensions most of the posterior probability can lie far from the peak, so the MAP point can be untypical

- **Maximum likelihood.** With a uniform prior MAP is the *maximum likelihood estimate*

  - this was developed and popularised by Ronald Fisher (1912–1922) at a time when Bayesian inference was considered taboo, and was accepted by the statistical community despite its strong connection to Bayesian inference
  - in high-dimensional problems a non-uniform prior can make a big difference

- **Empirical Bayes.** For a hierarchical model a half-way house is to integrate out the many parameters θ_j, choose the few hyperparameters to maximise the evidence (as we did for the hyperparameters of a Gaussian process), and then use the conjugate posterior for each product

- for the ratings model the integral is in closed form, the *beta-binomial* distribution

$$\mathbb{P}[k_j|a,b] = \int_0^1 \binom{n_j}{k_j}\theta^{k_j}(1-\theta)^{n_j-k_j}\frac{\theta^{a-1}(1-\theta)^{b-1}}{B(a,b)}\,\mathrm{d}\theta = \binom{n_j}{k_j}\frac{B(a+k_j,\, b+n_j-k_j)}{B(a,b)}$$

  where $B(a,b) = \int_0^1 \theta^{a-1}(1-\theta)^{b-1}\mathrm{d}\theta = \Gamma(a)\Gamma(b)/\Gamma(a+b)$ is the beta function
- we maximise $\sum_j \log \mathbb{P}[k_j|a,b]$ numerically; for the 30 products of Figure 2 this gives $a = 19.9$ and $b = 5.3$
- it is cheap and often works well, but it ignores our uncertainty in a and b

- **A ladder of strategies**

Strategy	What you get	Cost
Maximum likelihood	a point estimate	an optimiser
MAP	a regularised point estimate	an optimiser
Empirical Bayes	posteriors given fitted hyperparameters	an integral
EM	ML or MAP with latent variables	iteration
Full Bayes	all the uncertainty	MCMC

  Choose the cheapest strategy that answers your question

2.5 You Already Know EM

- k-means clustering repeats two steps: assign each point to its nearest centre, then move each centre to the mean of its points
- The first step guesses the hidden cluster labels; the second fits the parameters as if those guesses were right
- The EM algorithm does the same, but with *probabilities* of belonging to each cluster, and it comes from a probabilistic model, a mixture

2.6 Mixtures of Gaussians

- To illustrate latent variables and a simple hierarchical model we consider a classic probabilistic model known as a *mixture of Gaussians*
- We consider a concrete scenario
- We suppose we are detecting two types (A and B) of short-lived particles [1]
- We measure the mass, X_i, of each particle with a noisy detector, but we don't know the type of particle
- Let $Z_i \in \{0,1\}$ equal 1 if particle i is of type A and 0 if it is of type B, with $\mathbb{P}[Z_i = 1] = p$
- The probability density of the measurement is therefore

$$f(X_i|Z_i,\boldsymbol{\Theta}) = Z_i\,\mathcal{N}\big(X_i\big|\mu_A,\sigma_A^2\big) + (1-Z_i)\,\mathcal{N}\big(X_i\big|\mu_B,\sigma_B^2\big)$$

  - where μ_A and μ_B are the masses of particles of type A and B respectively
  - σ_A and σ_B are the standard deviations in the measurements
  - this says that if the ith particle is of type A then the density of X_i is $\mathcal{N}\big(X_i\big|\mu_A,\sigma_A^2\big)$, while if it is of type B, then X_i is distributed according to $\mathcal{N}\big(X_i\big|\mu_B,\sigma_B^2\big)$
- We show some typical data from $m = 1\,000$ observations in Figure 3

[1] If you prefer, you can think of an autonomous vehicle using lidar where it detects reflections from two different, but close by, objects, A and B. We make multiple noisy measurements of the distance from the two objects.

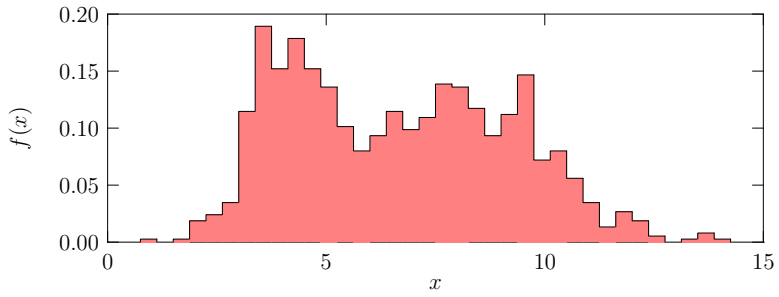


Figure 3: Example of data measuring the masses of two types of particles

- Our job is to infer the parameters $\boldsymbol{\Theta} = (\mu_A, \sigma_A^2, \mu_B, \sigma_B^2, p)$
- As a graphical model (Figure 4): for each data point draw a type Z_i, then a mass X_i from that type's normal distribution. Z_i is latent; the parameters are shared by all the data points, so they sit outside the plate

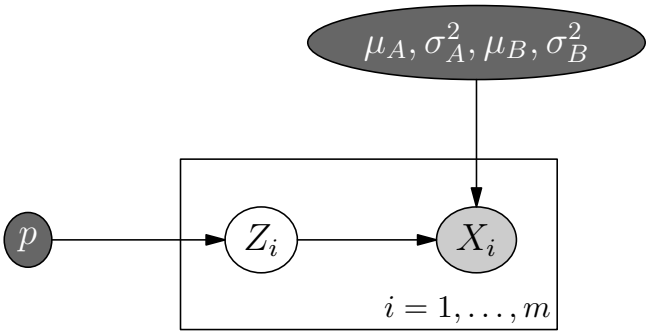


Figure 4: Graphical model for a mixture of two Gaussians

- We could do a full Bayesian calculation, but let us just use maximum likelihood
- Assuming the data points are independent, the likelihood of the data $\mathcal{D} = \{X_i | i = 1, 2, \ldots, m\}$ is

$$
\begin{aligned}
f(\mathcal{D}|\boldsymbol{\Theta}) &\overset{(1)}{=} \sum_{\boldsymbol{Z} \in \{0,1\}^m} f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}) \\
&\overset{(2)}{=} \prod_{i=1}^{m} \sum_{Z_i \in \{0,1\}} f(X_i, Z_i|\boldsymbol{\Theta}) \overset{(3)}{=} \prod_{i=1}^{m} \sum_{Z_i \in \{0,1\}} f(X_i|Z_i, \boldsymbol{\Theta})\, \mathbb{P}[Z_i|\boldsymbol{\Theta}]
\end{aligned}
$$

  1. where we marginalise out the latent variables $\boldsymbol{Z} = (Z_1, Z_2, \ldots Z_m)$
  2. we assume the data points are independent, so the sum over $\boldsymbol{Z}$ factorises into a product of sums over each Z_i
  3. we use the identity $f(X_i, Z_i|\boldsymbol{\Theta}) = f(X_i|Z_i, \boldsymbol{\Theta})\, \mathbb{P}[Z_i|\boldsymbol{\Theta}]$

- It is usually easier working with the log-likelihood

$$
\begin{aligned}
\log(f(\mathcal{D}|\boldsymbol{\Theta})) &= \sum_{i=1}^{m} \log\big(f(X_i|Z_i = 1)\, \mathbb{P}[Z_i = 1] + f(X_i|Z_i = 0)\, \mathbb{P}[Z_i = 0]\big) \\
&= \sum_{i=1}^{m} \log\big(p\, \mathcal{N}\big(X_i \big| \mu_A, \sigma_A^2\big) + (1 - p)\, \mathcal{N}\big(X_i \big| \mu_B, \sigma_B^2\big)\big)
\end{aligned}
$$

- We could do gradient ascent on this, but it is an ugly expression to work with: the log of a sum does not simplify

- Strictly, the likelihood has no maximum: put $\mu_A = X_1$ and let $\sigma_A \to 0$. Then $f(X_1|\boldsymbol{\Theta}) \geq p\mathcal{N}\big(X_1\big|X_1, \sigma_A^2\big) = p/(\sqrt{2\pi}\,\sigma_A) \to \infty$, while every other data point still has a density of at least $(1-p)\mathcal{N}\big(X_i\big|\mu_B, \sigma_B^2\big) > 0$. What we want is a good *local* maximum

2.7 Expectation Maximisation

- Rather than maximise the likelihood directly we can start from some initial guess $\boldsymbol{\Theta}^{(0)}$ and iteratively improve it

$$\boldsymbol{\Theta}^{(t+1)} = \operatorname*{argmax}_{\boldsymbol{\Theta}} Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}), \qquad Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) = \sum_{\boldsymbol{Z}\in\{0,1\}^m} \mathbb{P}\Big[\boldsymbol{Z}\Big|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\Big]\, \log(f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})) \tag{1}$$

  - $\log(f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}))$ is the log-likelihood we would have if we knew the latent variables (the *complete-data* log-likelihood); it is usually simple
  - we don't know $\boldsymbol{Z}$, so we average over it using its distribution given the data and our current parameters

- This is a general optimisation strategy that is regularly used when we have latent variables
- It is known as **expectation maximisation** or the **EM-algorithm**
  - E-step: compute $\mathbb{P}\big[\boldsymbol{Z}\big|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\big]$
  - M-step: maximise $Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})$
- This looks very different to maximising the log-likelihood: it takes some effort to understand why this works
- By the definition of conditional probability

$$f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}) = \mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]\; f(\mathcal{D}|\boldsymbol{\Theta})$$

  from which (taking logs and rearranging), for every $\boldsymbol{Z}$,

$$\log(f(\mathcal{D}|\boldsymbol{\Theta})) = \log(f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})) - \log(\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}])$$

- Multiply both sides by $\mathbb{P}\big[\boldsymbol{Z}\big|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\big]$ and sum over $\boldsymbol{Z}$. The left-hand side does not involve $\boldsymbol{Z}$, and the probabilities sum to 1, so

$$\log(f(\mathcal{D}|\boldsymbol{\Theta})) = Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) + S(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})$$

  where Q is the function we maximise in equation (1) and

$$S(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) = -\sum_{\boldsymbol{Z}\in\{0,1\}^m} \mathbb{P}\Big[\boldsymbol{Z}\Big|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\Big]\, \log(\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}])$$

- The change in log-likelihood when we update our parameters is therefore

$$\begin{aligned}\Delta L &= \log\Big(f(\mathcal{D}|\boldsymbol{\Theta}^{(t+1)})\Big) - \log\Big(f(\mathcal{D}|\boldsymbol{\Theta}^{(t)})\Big) \\ &= Q(\boldsymbol{\Theta}^{(t+1)}|\boldsymbol{\Theta}^{(t)}) - Q(\boldsymbol{\Theta}^{(t)}|\boldsymbol{\Theta}^{(t)}) + S(\boldsymbol{\Theta}^{(t+1)}|\boldsymbol{\Theta}^{(t)}) - S(\boldsymbol{\Theta}^{(t)}|\boldsymbol{\Theta}^{(t)})\end{aligned}$$

- The difference of the S terms is

$$\sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \log\left(\frac{P_t(\boldsymbol{Z})}{P_{t+1}(\boldsymbol{Z})}\right) = \mathrm{KL}(P_t \| P_{t+1})$$

  where $P_t(\boldsymbol{Z}) = \mathbb{P}\big[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\big]$; this is the *KL divergence,* which we meet again in the lecture on entropy

- KL divergences are never negative. Using $\log(x) \le x - 1$,

$$-\mathrm{KL}(P_t \| P_{t+1}) = \sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \log\left(\frac{P_{t+1}(\boldsymbol{Z})}{P_t(\boldsymbol{Z})}\right) \le \sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \left(\frac{P_{t+1}(\boldsymbol{Z})}{P_t(\boldsymbol{Z})} - 1\right) = 1 - 1 = 0$$

- In expectation maximisation we choose $\boldsymbol{\Theta}^{(t+1)}$ to maximise $Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})$, so $Q(\boldsymbol{\Theta}^{(t+1)}|\boldsymbol{\Theta}^{(t)}) \ge Q(\boldsymbol{\Theta}^{(t)}|\boldsymbol{\Theta}^{(t)})$
- Thus $\Delta L \ge 0$: no step decreases the log-likelihood. (We only need Q to increase, so a partial M-step also works.)
- To find a MAP solution instead, add $\log(f(\boldsymbol{\Theta}))$ to Q
- Let us return to our two types of particles A and B
- The complete-data likelihood is $f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}) = \prod_{i=1}^{m} f(X_i, Z_i|\boldsymbol{\Theta})$ where

$$f(X_i, Z_i|\boldsymbol{\Theta}) = p\, Z_i\, \mathcal{N}\big(X_i \big| \mu_A, \sigma_A^2\big) + (1-p)\,(1-Z_i)\,\mathcal{N}\big(X_i \big| \mu_B, \sigma_B^2\big)$$

- E-step. Given the data, the Z_i are independent and by Bayes' rule

$$\gamma_i^{(t)} = \mathbb{P}\left[Z_i = 1 \middle| X_i, \boldsymbol{\Theta}^{(t)}\right] = \frac{p^{(t)}\,\mathcal{N}\left(X_i \middle| \mu_A^{(t)}, \sigma_A^{2(t)}\right)}{p^{(t)}\,\mathcal{N}\left(X_i \middle| \mu_A^{(t)}, \sigma_A^{2(t)}\right) + (1-p^{(t)})\,\mathcal{N}\left(X_i \middle| \mu_B^{(t)}, \sigma_B^{2(t)}\right)}$$

  and $\mathbb{P}\left[Z_i = 0 \middle| X_i, \boldsymbol{\Theta}^{(t)}\right] = 1 - \gamma_i^{(t)}$. $\gamma_i^{(t)}$ is called the *responsibility* of type A for data point i
- M-step. Because $\log(f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}))$ is a sum over data points, the sum over $\boldsymbol{Z}$ in Q reduces to a sum over each Z_i

$$\begin{aligned} Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) &= \sum_{i=1}^{m} \gamma_i^{(t)} \log\big(p\,\mathcal{N}\big(X_i \big| \mu_A, \sigma_A^2\big)\big) + (1-\gamma_i^{(t)}) \log\big((1-p)\,\mathcal{N}\big(X_i \big| \mu_B, \sigma_B^2\big)\big) \\ &= \sum_{i=1}^{m} \gamma_i^{(t)} \left(\log(p) - \frac{(X_i - \mu_A)^2}{2\sigma_A^2} - \frac{1}{2}\log\big(2\,\pi\,\sigma_A^2\big)\right) \\ &\qquad + (1-\gamma_i^{(t)}) \left(\log(1-p) - \frac{(X_i - \mu_B)^2}{2\sigma_B^2} - \frac{1}{2}\log\big(2\,\pi\,\sigma_B^2\big)\right) \end{aligned}$$

- To maximise this we just set the derivatives to 0
  - With respect to p

$$\frac{\partial Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})}{\partial p} = \frac{1}{p}\sum_{i=1}^{m} \gamma_i^{(t)} - \frac{1}{1-p}\sum_{i=1}^{m} (1-\gamma_i^{(t)}) = 0 \quad \Rightarrow \quad p^{(t+1)} = \frac{1}{m}\sum_{i=1}^{m} \gamma_i^{(t)}$$

  - With respect to μ_A

$$\frac{\partial Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})}{\partial \mu_A} = \sum_{i=1}^{m} \gamma_i^{(t)} \frac{X_i - \mu_A}{\sigma_A^2} = 0$$

    so (and similarly for μ_B)

$$\mu_A^{(t+1)} = \frac{\sum_{i=1}^{m} \gamma_i^{(t)} X_i}{\sum_{i=1}^{m} \gamma_i^{(t)}}, \qquad \mu_B^{(t+1)} = \frac{\sum_{i=1}^{m} (1-\gamma_i^{(t)}) X_i}{\sum_{i=1}^{m} (1-\gamma_i^{(t)})}$$

- With respect to σ_A^2

$$\frac{\partial Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})}{\partial \sigma_A^2} = \frac{1}{2\,\sigma_A^4}\sum_{i=1}^{m}\gamma_i^{(t)}(X_i-\mu_A)^2 - \frac{1}{2\,\sigma_A^2}\sum_{i=1}^{m}\gamma_i^{(t)} = 0$$

so, putting in the optimal $\mu_A^{(t+1)}$ (and similarly for σ_B^2)

$$\sigma_A^{2(t+1)} = \frac{\sum_{i=1}^{m}\gamma_i^{(t)}(X_i-\mu_A^{(t+1)})^2}{\sum_{i=1}^{m}\gamma_i^{(t)}}, \qquad \sigma_B^{2(t+1)} = \frac{\sum_{i=1}^{m}(1-\gamma_i^{(t)})(X_i-\mu_B^{(t+1)})^2}{\sum_{i=1}^{m}(1-\gamma_i^{(t)})}$$

- These are very natural update equations
  - we estimate the probability, $\gamma_i^{(t)}$, that observation X_i is a particle of type A based on our current parameters
  - we then compute the fraction, means and variances of each type, weighting each data point by these probabilities
  - if the $\gamma_i^{(t)}$ were all 0 or 1 these would be the ordinary fraction, means and variances of each group
- We are guaranteed that our EM-algorithm never decreases the likelihood (although it could reach a local rather than global optimum)
- For the data set we showed earlier (which was a random sample of size 1000 generated using $p = 0.3$, $\mu_A = 4$, $\sigma_A = 0.8$, $\mu_B = 8$ and $\sigma_B = 2$) we get the results shown in Figure 5

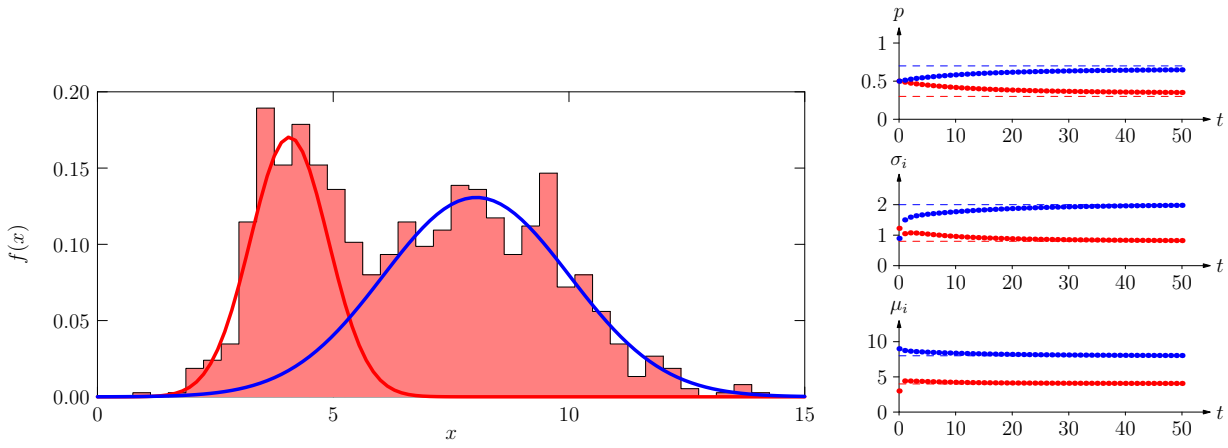


Figure 5: Example of EM algorithm to compute the statistics for the masses of our two particles

- The EM algorithm often leads to very natural update equations, but its convergence is often rather slow

2.8 EM and k-Means

- Give both clusters the same fixed variance σ^2. Then

$$\gamma_i = \frac{1}{1+\frac{1-p}{p}\,\mathrm{e}^{((X_i-\mu_A)^2-(X_i-\mu_B)^2)/(2\sigma^2)}}$$

and as $\sigma \to 0$ (with $0 < p < 1$ fixed) $\gamma_i \to 1$ if X_i is nearer μ_A and $\gamma_i \to 0$ if it is nearer μ_B

- The E-step becomes "assign each point to its nearest mean" and the M-step "move each mean to the average of its points": k-means is EM with hard assignments
- EM's soft assignments let a point in the overlap count for both clusters, and EM also learns the size (p) and width (σ) of each cluster

2.9 EM in Practice

- EM finds a local maximum that depends on the starting point, so run it from several starting points and keep the best
- A component can collapse onto a single point ($\sigma \to 0$), where the likelihood is infinite. A prior on σ (i.e. MAP, which just adds $\log(f(\boldsymbol{\Theta}))$ to Q) prevents this
- Convergence can be slow when the components overlap, because the latent variables are then very uncertain
- The update equations are simple and need no step size
- The same idea applies to any model with latent variables: missing data, hidden Markov models, and many more

2.10 Lower Bounds and VAEs

- (This section goes beyond the lecture: it is a pointer to where these ideas lead.)
- The proof that EM works can be written for *any* distribution $q(\boldsymbol{Z})$ over the latent variables. Since $f(\mathcal{D}|\boldsymbol{\Theta}) = f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})/\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]$ for every $\boldsymbol{Z}$,

$$\begin{aligned}\log(f(\mathcal{D}|\boldsymbol{\Theta})) &= \sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \log\left(\frac{f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})}{\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]}\right) \\ &= \sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \log\left(\frac{f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})}{q(\boldsymbol{Z})}\right) + \sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \log\left(\frac{q(\boldsymbol{Z})}{\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]}\right)\end{aligned}$$

  The second term is $\mathrm{KL}\big(q \,\|\, \mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]\big) \geq 0$, so the first term is a lower bound on the log-likelihood, called the *evidence lower bound* (ELBO)
- EM is coordinate ascent on the ELBO: the E-step sets q to the posterior $\mathbb{P}\big[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\big]$, which makes the KL term zero so that the bound touches the log-likelihood; the M-step maximises the bound over $\boldsymbol{\Theta}$ (the $-\sum q \log q$ part does not depend on $\boldsymbol{\Theta}$, so this is maximising Q)
- A *variational autoencoder* (VAE) is a latent-variable model in which a neural network (the decoder) generates data from a latent vector. Its posterior over the latent vector is intractable, so a second network (the encoder) learns an approximation q, and both are trained by gradient ascent on the same bound
- We meet the KL divergence again in the lecture on entropy

3 Exercises

3.1 Posterior means for ratings

- With $a = 20$ and $b = 5$, product C has 9 positive reviews out of 10 and product D 85 out of 100. Rank them by the fraction of positive reviews and by the posterior mean.
- Show that the posterior mean $(a + k)/(a + b + n)$ is the weighted average of $a/(a + b)$ and k/n given in the notes.

3.2 The beta-binomial

- Derive the marginal likelihood $\mathbb{P}[k|a, b]$ of the ratings model, using $\int_0^1 \theta^{\alpha-1}(1-\theta)^{\beta-1}\mathrm{d}\theta = B(\alpha, \beta)$. Why can the factor $\binom{n}{k}$ be ignored when choosing a and b?

3.3 Generative to discriminative

- In a mixture of two Gaussians with $p = 0.3$, $\mu_A = 4$, $\sigma_A = 0.8$, $\mu_B = 8$ and $\sigma_B = 2$, a particle of mass $X = 5.5$ is measured. What is the probability that it is of type A?

3.4 One EM step by hand

- Data $X = (0, 1, 5, 6)$; current parameters $p = \frac{1}{2}$, $\mu_A = 1$, $\mu_B = 5$, $\sigma_A = \sigma_B = 1$ (keep the variances fixed). Compute the responsibilities and the new p, μ_A and μ_B.

3.5 Mysterious Disease

- We assume that we are tracking some disease
- Let $Z(t)$ be the number of people that catch the disease on day t, but this is unknown (a latent variable)
- We assume the rate of growth of the disease is

$$\mathbb{P}[Z(t+1)] = \text{Poi}\Big(Z(t+1)\Big|\frac{r_0}{3}\left(Z(t) + Z(t-1) + Z(t-2)\right)\Big)$$

  - We are assuming that someone is infectious for the first three days after catching the disease
  - We assume $Z(1) = 1$ and $Z(0) = Z(-1) = 0$
  - In expectation everyone with the disease will infect r_0 new people
- We observe $X(t)$, the number of new cases that are reported, where

$$\mathbb{P}[X(t) = k] = \text{Binom}(k|Z(t), p) = \binom{Z(t)}{k} p^k \left(1-p\right)^{Z(t)-k}$$

  - We assume that each patient will be tested with a probability p
  - We are given a time series $(X(1), X(2), \ldots, X(T))$
- Build a probabilistic model to estimate p and r_0
- See answers

4 Experiments

4.1 Empirical Bayes for ratings

- Simulate 200 products from the ratings model, fit a and b by maximising the evidence, and compare the posterior means with the raw fractions as estimates of the true θ_j

```
import numpy as np
from scipy.optimize import minimize
from scipy.special import betaln

rng = np.random.default_rng(3)
J = 200
n = np.round(np.exp(rng.uniform(np.log(2), np.log(2000), J))).astype(int)
theta = rng.beta(16, 4, J)                    # true a = 16, b = 4
k = rng.binomial(n, theta)
```

```
def neg_log_evidence(log_ab):                # binomial coefficients omitted:
    a, b = np.exp(log_ab)                    # they do not depend on a, b
    return -np.sum(betaln(a + k, b + n - k) - betaln(a, b))


a, b = np.exp(minimize(neg_log_evidence, np.log([1.0, 1.0]),
                       method="Nelder-Mead").x)
post = (a + k) / (a + b + n)
print("a, b:", a, b)
small = n < 10
for name, est in [("k/n", k / n), ("posterior mean", post)]:
    print(name, "RMS error, all:", np.sqrt(np.mean((est - theta) ** 2)),
          " n<10:", np.sqrt(np.mean((est - theta)[small] ** 2)))
```

- Which estimate is better, and for which products does it matter most? (See the answers.)

4.2 EM for a mixture of Gaussians

- Generate the data of the lecture and run EM, printing the log-likelihood as we go

```
import numpy as np

rng = np.random.default_rng(0)
m = 1000
p, muA, sA, muB, sB = 0.3, 4.0, 0.8, 8.0, 2.0      # true parameters
Z = rng.random(m) < p                                # latent types
X = np.where(Z, rng.normal(muA, sA, m), rng.normal(muB, sB, m))


def normal(x, mu, var):
    return np.exp(-(x - mu) ** 2 / (2 * var)) / np.sqrt(2 * np.pi * var)


th = dict(p=0.5, muA=2.0, vA=1.0, muB=10.0, vB=1.0)   # initial guess
for t in range(51):
    a = th["p"] * normal(X, th["muA"], th["vA"])
    b = (1 - th["p"]) * normal(X, th["muB"], th["vB"])
    loglik = np.sum(np.log(a + b))
    if t in (0, 1, 2, 5, 10, 20, 50):
        print(f"t={t:2d}  log-lik {loglik:9.2f}  p={th['p']:.3f}  "
              f"muA={th['muA']:.2f}  sA={np.sqrt(th['vA']):.2f}  "
              f"muB={th['muB']:.2f}  sB={np.sqrt(th['vB']):.2f}")
    g = a / (a + b)                                  # E-step: responsibilities
    th["p"] = g.mean()                               # M-step
    th["muA"] = np.sum(g * X) / np.sum(g)
    th["muB"] = np.sum((1 - g) * X) / np.sum(1 - g)
    th["vA"] = np.sum(g * (X - th["muA"]) ** 2) / np.sum(g)
    th["vB"] = np.sum((1 - g) * (X - th["muB"]) ** 2) / np.sum(1 - g)
```

- Does the log-likelihood ever decrease? How quickly do the parameters settle down? Try other starting points. (See the answers.)

4.3 Mysterious Disease

- Build a simulator of your model (assume you know p and r_0)

- If you are feeling very adventurous you could try to solve your model to predict p and r_0, but be warned this is hard (you probably need to use MCMC, which is the subject of a later lecture)

```
import numpy as np

rng = np.random.default_rng(1)
r0, p, T = 2.0, 0.2, 30
Z = np.zeros(T, dtype=int)          # Z[t]: new infections on day t+1
Z[0] = 1
for t in range(1, T):
    Z[t] = rng.poisson(r0 / 3 * Z[max(t - 3, 0):t].sum())
X = rng.binomial(Z, p)              # each new case is reported with probability p
print("Z:", Z)
print("X:", X)
```

5 Answers

5.1 Posterior means for ratings

- By fraction C (0.90) beats D (0.85). By posterior mean C gets $29/35 \approx 0.83$ and D gets $105/125 = 0.84$, so D ranks higher: C's ten reviews are not enough to be sure it is better than a typical product
- $\frac{a+b}{a+b+n} \cdot \frac{a}{a+b} + \frac{n}{a+b+n} \cdot \frac{k}{n} = \frac{a}{a+b+n} + \frac{k}{a+b+n} = \frac{a+k}{a+b+n}$

5.2 The beta-binomial

- Collecting the powers of θ,

$$\begin{aligned}\mathbb{P}[k|a,b] &= \int_0^1 \binom{n}{k}\theta^k(1-\theta)^{n-k}\,\frac{\theta^{a-1}(1-\theta)^{b-1}}{B(a,b)}\,\mathrm{d}\theta = \frac{\binom{n}{k}}{B(a,b)}\int_0^1 \theta^{a+k-1}(1-\theta)^{b+n-k-1}\mathrm{d}\theta \\ &= \binom{n}{k}\frac{B(a+k,\,b+n-k)}{B(a,b)}\end{aligned}$$

- $\binom{n}{k}$ does not depend on a or b, so it only adds a constant to the log-evidence and does not change where its maximum is

5.3 Empirical Bayes for ratings

- The fit gives $a = 19.6$ and $b = 4.8$ (the true values are 16 and 4; the typical rates, 0.80 and 0.80, agree well)
- The root-mean-square error of the estimates of θ_j falls from 0.116 for the raw fractions to 0.050 for the posterior means. For the products with fewer than 10 reviews it falls from 0.236 to 0.073: shrinkage helps most where the data are few

5.4 Generative to discriminative

- By Bayes' rule

$$\mathbb{P}[Z=1|X] = \frac{0.3\,\mathcal{N}\big(5.5\big|4, 0.64\big)}{0.3\,\mathcal{N}\big(5.5\big|4, 0.64\big) + 0.7\,\mathcal{N}\big(5.5\big|8, 4\big)} = \frac{0.3 \times 0.0860}{0.3 \times 0.0860 + 0.7 \times 0.0913} \approx 0.29$$

- Although 5.5 is closer to μ_A than to μ_B, type B is both more common and much more spread out, so it is the more likely explanation

5.5 One EM step by hand

- $\mathcal{N}\big(x\big|\mu,1\big) \propto \mathrm{e}^{-(x-\mu)^2/2}$ and $p=\frac{1}{2}$, so $\gamma_i = 1/(1+\mathrm{e}^{-((X_i-\mu_B)^2-(X_i-\mu_A)^2)/2})$. For $X=0,1,5,6$ the exponents $(X_i-\mu_B)^2-(X_i-\mu_A)^2$ are $24, 16, -16, -24$, so $\gamma \approx (1.000, 1.000, 0.000, 0.000)$ (more precisely $1-6\times10^{-6}$, $1-3.4\times10^{-4}$, 3.4×10^{-4}, 6×10^{-6})
- $p=\frac{1}{4}\sum\gamma_i = 0.5$, $\mu_A \approx (0+1)/2 = 0.50$ and $\mu_B \approx (5+6)/2 = 5.50$: the clusters are so well separated that EM reduces to splitting the data into two groups and averaging each

5.6 EM for a mixture of Gaussians

- Running the code gives

t	log-likelihood	p	μ_A	σ_A	μ_B	σ_B
0	-4316.14	0.500	2.00	1.00	10.00	1.00
1	-2264.00	0.396	4.33	0.93	8.44	1.59
5	-2250.37	0.350	4.29	0.97	8.18	1.81
20	-2244.50	0.282	4.08	0.82	7.89	1.96
50	-2243.99	0.257	4.02	0.77	7.78	2.03

- The log-likelihood never decreases. The first step does almost all the work, but the parameters then creep for tens of iterations: by 100 they have settled at $p=0.255$, $\mu_A=4.01$, $\sigma_A=0.76$, $\mu_B=7.77$ and $\sigma_B=2.03$
- These are the maximum-likelihood estimates for this sample, not the true values; with 1000 points p is uncertain by a few hundredths, because the two groups overlap

5.7 Mysterious Disease

- We want to compute $f(p, r_0|\mathcal{D})$ where $\mathcal{D} = (X(1), X(2), \ldots, X(T))$
- We have a likelihood of

$$\mathbb{P}[X(t)|Z(t), p] = \mathrm{Binom}(X(t)|Z(t), p)$$

  where

$$\mathbb{P}[Z(t+1)|r_0, Z(t), Z(t-1), Z(t-2)] = \mathrm{Poi}\Big(Z(t+1)\Big|\frac{r_0}{3}\left(Z(t)+Z(t-1)+Z(t-2)\right)\Big)$$

- To perform a Bayesian calculation we would have to put priors, $f(p)$ and $f(r_0)$, on p and r_0
  - A reasonable prior to use for p is $f(p) = \mathrm{Beta}(p|a,b)$—you could use $a=b=1$ as an uninformative (uniform) prior, but you might have some prior knowledge, e.g. $a=b=2$ which says $f(p) = 6\,p\,(1-p)$
  - A reasonable prior for r_0 would be $\mathrm{Gamma}(r_0|a,b)$—you could use $a=b=0$ as an uninformative (improper) prior but again you might have some prior belief, e.g. $a=2$, $b=1$
- Bayes' rule tells us

$$f(p, r_0, \boldsymbol{Z}|\boldsymbol{X}) = \frac{\prod_{t=1}^{T} \mathbb{P}[X(t)|Z(t),p]\ \mathbb{P}[Z(t)|Z(t-1),Z(t-2),Z(t-3),r_0]\ f(r_0)\,f(p)}{\mathbb{P}[\boldsymbol{X}]}$$

- To get estimates of p and r_0 we have to marginalise out $\boldsymbol{Z}$
- Now the problem is this is rather horrible to compute (your priors are not conjugate priors in this problem and the posterior is very complicated)

- Probably the best way to do this is to use Markov Chain Monte Carlo (MCMC), which is the subject of a later lecture
- The simulator above gives, for $r_0 = 2$ and $p = 0.2$, case numbers that grow by roughly a factor of 1.45 a day; the reported numbers $X(t)$ are about a fifth of $Z(t)$, but noisy while they are small

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 23: Matrix Factorisation

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Matrix completion, low-rank approximation, recommender systems, alternating least squares, non-negative matrix factorisation

2 Main Points

2.1 Missing Data and Recommenders

- We are often given a matrix of data with missing entries (e.g. forms that people have only partly filled in)
  - most learning machines need complete data, so we either throw away rows or columns with missing values or try to infer them
- We can only infer missing values if the matrix has structure. The structure we use is that *structured data typically lies close to a low-dimensional subspace* (the idea behind PCA and the SVD)
- The best-known application is *collaborative filtering*: predicting the ratings users would give to items they haven't rated, from the ratings of all users
  - the classic method finds similar users and averages their ratings, weighted by similarity
    $$\hat{R}_{ui} = \frac{\sum_{u'} \mathrm{sim}(u,u')\, R_{u'i}}{\sum_{u'} \mathrm{sim}(u,u')}$$
  - the simplest baseline uses means: $R_{ui} \approx \bar{R}_u + \bar{R}_i - \bar{R}$ (user mean plus item mean minus overall mean)

2.2 Low-Rank Approximation

- The SVD of a complete $m \times n$ matrix is
  $$\mathbf{X} = \mathbf{U}\,\mathbf{S}\,\mathbf{V}^\mathsf{T} = \sum_k s_k\, \boldsymbol{u}_k\, \boldsymbol{v}_k^\mathsf{T}, \qquad s_1 \geq s_2 \geq \cdots \geq 0$$
- Keeping the K largest singular values gives the rank-K matrix $\mathbf{X}_K = \sum_{k=1}^{K} s_k\, \boldsymbol{u}_k\, \boldsymbol{v}_k^\mathsf{T}$
- *Eckart–Young theorem*: $\mathbf{X}_K$ is the best rank-K approximation to $\mathbf{X}$
  - in the Frobenius norm, with $\|\mathbf{X} - \mathbf{X}_K\|_F^2 = \sum_{k>K} s_k^2$
  - and in the spectral norm, with $\|\mathbf{X} - \mathbf{X}_K\|_2 = s_{K+1}$
  - if the data lies close to a K-dimensional subspace, the discarded singular values are small

- **Completing a matrix with the SVD.** The SVD needs a complete matrix, so we iterate

  1. fill the missing entries (e.g. with user means)
  2. compute the best rank-K approximation $\mathbf{L}(t)$ of the filled-in matrix $\mathbf{X}(t)$
  3. copy its values into the missing entries only

  $$\mathbf{X}(t+1) = \mathbf{W} \odot \mathbf{X} + \overline{\mathbf{W}} \odot \mathbf{L}(t)$$

  where $W_{ij} = 1$ if X_{ij} is observed and 0 otherwise, $\overline{\mathbf{W}} = 1 - \mathbf{W}$ and $\odot$ is the element-by-element product
  4. repeat from 2
  5. in the lecture's example (a rank-2 matrix with a quarter of its entries hidden) the RMS error on the hidden entries falls from 0.95 (user means) to 0.22 after four iterations

- More directly, write $\mathbf{X} \approx \mathbf{A}\,\mathbf{B}$ with $\mathbf{A}$ an $m \times K$ and $\mathbf{B}$ a $K \times n$ matrix
  - $\mathbf{AB}$ has rank at most K, and has $(m+n)K$ parameters rather than mn
  - K controls the complexity; choose it by cross-validation, hiding some of the observed entries
  - fit by least squares on the observed entries $\mathcal{D}$

  $$\min_{\mathbf{A},\mathbf{B}} \sum_{(i,j)\in\mathcal{D}} \Big(X_{ij} - \sum_{k=1}^{K} A_{ik}\, B_{kj}\Big)^2$$

  - **LoRA.** The same parameter count makes fine-tuning large neural networks cheap. Low-rank adaptation (LoRA; Hu et al.,

    1. freezes a pretrained weight matrix $\mathbf{W}_0$ ($m \times n$)

    and learns only a low-rank change $\Delta\mathbf{W} = \mathbf{AB}$ with $K \ll m, n$, one factor starting at zero so that training starts from $\mathbf{W}_0$. For $m = n = 4096$ and $K = 8$ that is about 6.6×10^4 trainable parameters instead of 1.7×10^7. The assumption is that the change a new task needs is close to low rank. Each task then stores only its own small $\mathbf{A}$ and $\mathbf{B}$, and after training $\mathbf{AB}$ can be added into $\mathbf{W}_0$, so the adapted network is no slower to run
  - $\mathbf{A}$ and $\mathbf{B}$ are never unique: $\mathbf{AB} = (\mathbf{AG})(\mathbf{G}^{-1}\mathbf{B})$ for any invertible $K \times K$ matrix $\mathbf{G}$

2.3 Probabilistic Approach

- We use the *maximum a posteriori* (MAP) solution from the lecture on probabilistic inference: the most probable parameters, $\boldsymbol{\theta}_{\mathrm{MAP}} = \mathrm{argmax}_{\boldsymbol{\theta}} \big(\log \mathbb{P}[\mathcal{D}|\boldsymbol{\theta}] + \log \mathbb{P}[\boldsymbol{\theta}]\big)$, which does not need the evidence. Its limitations (no error bars, dependence on how we parametrise $\boldsymbol{\theta}$) are discussed there

- Treat the ratings as noisy: $X_{ij} \sim \mathcal{N}\big(\sum_k A_{ik}B_{kj},\, \sigma^2\big)$, with Gaussian priors $A_{ik} \sim \mathcal{N}(0, \sigma_A^2)$ and $B_{kj} \sim \mathcal{N}(0, \sigma_B^2)$

- The full posterior is intractable, but the *maximum a posteriori* (MAP) solution maximises

$$\mathcal{E} = -\frac{1}{2\sigma^2} \sum_{(i,j)\in\mathcal{D}} \Big(X_{ij} - \sum_{k=1}^{K} A_{ik}\, B_{kj}\Big)^2 - \frac{1}{2\sigma_A^2} \sum_{i,k} A_{ik}^2 - \frac{1}{2\sigma_B^2} \sum_{k,j} B_{kj}^2 + \text{const}$$

- **Each step is ridge regression.** Fix $\mathbf{B}$ and write $\boldsymbol{a}_i$ for row i of $\mathbf{A}$ and $\boldsymbol{b}_j$ for column j of $\mathbf{B}$. Then $\boldsymbol{a}_i$ minimises

$$\sum_{j|(i,j)\in\mathcal{D}} \big(X_{ij} - \boldsymbol{a}_i^{\mathsf{T}}\boldsymbol{b}_j\big)^2 + \eta\, \|\boldsymbol{a}_i\|^2, \qquad \eta = \frac{\sigma^2}{\sigma_A^2}$$

which is ridge regression with features $\boldsymbol{b}_j$ and targets X_{ij}. Its solution is

$$\boldsymbol{a}_i = \left(\eta\,\mathbf{I} + \sum_j \boldsymbol{b}_j \boldsymbol{b}_j^{\mathsf{T}}\right)^{-1} \sum_j \boldsymbol{b}_j\, X_{ij}$$

(sums over the j with $(i, j) \in \mathcal{D}$): one $K \times K$ linear solve per row

- Solving for $\mathbf{B}$ with $\mathbf{A}$ fixed is the same with the roles swapped. Alternating the two is *alternating least squares* (ALS)
  - each step is a convex problem and cannot decrease $\mathcal{E}$, so ALS converges, usually quickly
  - but the whole problem is not convex, so it converges to a local optimum that can depend on the starting point
- **Embeddings.** Row i of $\mathbf{A}$ is a K-dimensional vector for user i and column j of $\mathbf{B}$ a vector for item j; the predicted rating is their inner product. Modern recommenders learn such embeddings with neural networks, mostly from *implicit feedback* (clicks, views, purchases), where a missing entry is weak evidence of not liking rather than simply unknown
- Matrix factorisation was central to the winning entries of the Netflix Prize (2006–09) and remains a strong baseline

2.4 Non-Negative Matrix Factorisation

- Factorisation is also a way of modelling data. For a words $\times$ books count matrix, the rank-3 SVD gives factors with mixed signs, which are hard to interpret
- *Non-negative matrix factorisation* (NMF) minimises $\|\mathbf{X} - \mathbf{AB}\|_F^2$ with every element of $\mathbf{A}$ and $\mathbf{B}$ non-negative
  - projected gradient descent: take a gradient step, then set negative elements to zero
  - or Lee and Seung's multiplicative updates ($\oslash$ is element-by-element division)

    $$\mathbf{B} \leftarrow \mathbf{B} \odot \left(\mathbf{A}^{\mathsf{T}}\mathbf{X}\right) \oslash \left(\mathbf{A}^{\mathsf{T}}\mathbf{AB}\right), \qquad \mathbf{A} \leftarrow \mathbf{A} \odot \left(\mathbf{XB}^{\mathsf{T}}\right) \oslash \left(\mathbf{ABB}^{\mathsf{T}}\right)$$

    which never make an element negative
  - in the lecture's example the three factors pick out mathematics, physics and biology
- Caveats
  - NMF cannot fit better than the truncated SVD, which is the best rank-K fit (Eckart–Young). In the example the relative errors are 0.164 (NMF) and 0.158 (SVD)
  - the problem is not convex, so the result depends on the starting point and on K
  - the factors are not unique, and a "parts-based" interpretation of them is not guaranteed
  - so compare against simple baselines such as the SVD or PCA

3 Exercises

3.1 Counting parameters

- A recommender has $m = 10^6$ users and $n = 10^5$ items. How many numbers does a rank $K = 50$ factorisation store, compared with the full matrix?

3.2 Eckart–Young

- A matrix has singular values 5, 3 and 1. What are the Frobenius-norm and spectral-norm errors of its best rank-1 and rank-2 approximations?

3.3 The prior fixes the scale

- Show that replacing $\mathbf{A} \to c\,\mathbf{A}$ and $\mathbf{B} \to \mathbf{B}/c$ (for $c > 0$) leaves the likelihood unchanged.
- Show that the prior terms in $\mathcal{E}$ are then $-\alpha\, c^2 - \beta/c^2$ with $\alpha = \sum_{i,k} A_{ik}^2/(2\sigma_A^2)$ and $\beta = \sum_{k,j} B_{kj}^2/(2\sigma_B^2)$, and find the c that maximises them. What does this say about the sizes of $\mathbf{A}$ and $\mathbf{B}$ at the MAP solution?

3.4 Ridge regression for one row

- Differentiate $\sum_j (X_{ij} - \boldsymbol{a}_i^{\mathsf{T}} \boldsymbol{b}_j)^2 + \eta\, \|\boldsymbol{a}_i\|^2$ with respect to $\boldsymbol{a}_i$ and derive the update for $\boldsymbol{a}_i$ given above.

3.5 Multiplicative updates and KKT

- Show that if every element of $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{X}$ is positive, the multiplicative update for $\mathbf{B}$ keeps $\mathbf{B}$ positive.
- At a fixed point of the update show that for every element either $B_{kj} = 0$ or $\big(\mathbf{A}^{\mathsf{T}}\mathbf{A}\mathbf{B} - \mathbf{A}^{\mathsf{T}}\mathbf{X}\big)_{kj} = 0$. Relate this to the complementary slackness condition in the KKT conditions for minimising E subject to $B_{kj} \geq 0$ (the gradient of E with respect to $\mathbf{B}$ is $2(\mathbf{A}^{\mathsf{T}}\mathbf{A}\mathbf{B} - \mathbf{A}^{\mathsf{T}}\mathbf{X})$).

4 Experiments

4.1 Alternating least squares

- Make a noisy rank-3 matrix, observe 30% of its entries, hold some of those back, and fit by ALS for several K and η

```
import numpy as np

rng = np.random.default_rng(1)
m, n, K_true = 60, 40, 3
X_true = rng.standard_normal((m, K_true)) @ rng.standard_normal((K_true, n))
X = X_true + 0.3 * rng.standard_normal((m, n))     # noisy ratings
observed = rng.random((m, n)) < 0.3                  # see 30% of entries
train = observed & (rng.random((m, n)) < 0.8)        # hold some back
test = observed & ~train


def als(X, W, K, eta, iters=50):
    """Alternating least squares: each step is ridge regression."""
    A = 0.1 * rng.standard_normal((X.shape[0], K))
    B = 0.1 * rng.standard_normal((K, X.shape[1]))
    for _ in range(iters):
        for i in range(X.shape[0]):           # row i of A, B fixed
            Bi = B[:, W[i]]
            A[i] = np.linalg.solve(eta * np.eye(K) + Bi @ Bi.T, Bi @ X[i, W[i]])
        for j in range(X.shape[1]):           # column j of B, A fixed
            Aj = A[W[:, j]]
```

```
            B[:, j] = np.linalg.solve(eta * np.eye(K) + Aj.T @ Aj, Aj.T @ X[W[:, j], j])
    return A, B


def rmse(P, mask):
    return np.sqrt(np.mean((P - X)[mask] ** 2))


mean = X[train].mean()
print(f"predict the mean:  test RMSE {rmse(np.full_like(X, mean), test):.3f}")
for K in [1, 2, 3, 5, 10]:
    for eta in [0.01, 1.0]:
        A, B = als(X, train, K, eta)
        print(f"K={K:2d} eta={eta:4}:  train {rmse(A @ B, train):.3f}  test {rmse(A @ B, te
```

- What happens to the training and test errors as K grows? What does the regulariser η do? (See the answers.)

5 Answers

5.1 Counting parameters

- $(m+n)K = 1.1 \times 10^6 \times 50 = 5.5 \times 10^7$ numbers, against $mn = 10^{11}$ for the full matrix: about 1800 times fewer

5.2 Eckart–Young

- Rank 1: Frobenius error $\sqrt{3^2+1^2} = \sqrt{10} \approx 3.16$, spectral error 3
- Rank 2: Frobenius error 1, spectral error 1

5.3 The prior fixes the scale

- $(c\mathbf{A})(\mathbf{B}/c) = \mathbf{AB}$, so the likelihood term is unchanged
- The prior terms become $-\alpha c^2 - \beta/c^2$. Setting the derivative $-2\alpha c + 2\beta/c^3$ to zero gives $c^4 = \beta/\alpha$
- At the MAP solution $c = 1$, so $\alpha = \beta$: the prior balances the sizes, $\sum A_{ik}^2/\sigma_A^2 = \sum B_{kj}^2/\sigma_B^2$. The prior removes the scaling freedom, but not rotations: $\mathbf{A} \to \mathbf{AQ}$, $\mathbf{B} \to \mathbf{Q}^\mathsf{T}\mathbf{B}$ with $\mathbf{Q}$ orthogonal changes neither $\mathbf{AB}$ nor the sums of squares

5.4 Ridge regression for one row

- The gradient is $-2\sum_j \boldsymbol{b}_j(X_{ij} - \boldsymbol{b}_j^\mathsf{T}\boldsymbol{a}_i) + 2\eta\,\boldsymbol{a}_i$. Setting it to zero, $\big(\eta\,\mathbf{I} + \sum_j \boldsymbol{b}_j\boldsymbol{b}_j^\mathsf{T}\big)\boldsymbol{a}_i = \sum_j \boldsymbol{b}_j X_{ij}$, which is $\mathbf{M}^i\boldsymbol{a}^i = \boldsymbol{c}^i$ in the lecture

5.5 Multiplicative updates and KKT

- Every element of $\mathbf{A}^\mathsf{T}\mathbf{X}$ and $\mathbf{A}^\mathsf{T}\mathbf{AB}$ is a sum of products of positive numbers, so positive. The update multiplies each B_{kj} by the positive ratio of the two, so B_{kj} stays positive
- At a fixed point $B_{kj} = B_{kj}\,(\mathbf{A}^\mathsf{T}\mathbf{X})_{kj}/(\mathbf{A}^\mathsf{T}\mathbf{AB})_{kj}$, so $B_{kj}\big((\mathbf{A}^\mathsf{T}\mathbf{AB})_{kj} - (\mathbf{A}^\mathsf{T}\mathbf{X})_{kj}\big) = 0$
- This is $B_{kj}\,\partial E/\partial B_{kj} = 0$: either the constraint is active ($B_{kj} = 0$) or the gradient component vanishes, as complementary slackness requires. (A full KKT point also needs $\partial E/\partial B_{kj} \geq 0$ where $B_{kj} = 0$, which the fixed-point condition alone does not guarantee)

5.6 Alternating least squares

- Running the code gives (test RMSE; predicting the mean gives 1.67)

K	$\eta = 0.01$	$\eta = 1$
1	1.163	1.185
2	1.116	1.075
3	4.324	0.695
5	2.286	0.898
10	1.884	0.947

- The training error falls steadily with K (to 0.006 for $K = 10$, $\eta = 0.01$), but the test error is best at the true rank, $K = 3$, with regularisation
- Without regularisation the larger models over-fit badly: there are only about 10 training entries per row and 14 per column to fit K parameters each
- The test error cannot reach the noise level (0.3) because so few entries are observed

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 24: MCMC

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Monte Carlo methods, MCMC, Metropolis-Hastings, Gibbs sampling, Hamiltonian Monte Carlo, Convergence diagnostics, Variational Methods

2 Main Points

2.1 Monte Carlo

- Except in a few cases Bayesian inference is hard
  - It is easy when we have a conjugate prior giving us a simple posterior
  - But, when we want to model complicated situations our likelihood function usually doesn't have a conjugate prior
  - This means the posterior is not expressible as a simple probability distribution
  - Furthermore it is usually impossible to compute the marginal likelihood or evidence

$$\mathbb{P}[\mathcal{D}] = \sum_{\boldsymbol{\Theta}} \mathbb{P}[\mathcal{D}|\boldsymbol{\Theta}]\ \mathbb{P}[\boldsymbol{\Theta}]$$

  - The sum or integral over variables $\boldsymbol{\Theta}$ is often too large to make this feasible
  - You could try to obtain a histogram for the posterior, but if $\boldsymbol{\Theta}$ has many components the curse of dimensionality makes this difficult
- There is a simple answer which is to obtain samples from the posterior
  - We can use this to estimate expectations of any function, $g(\boldsymbol{\Theta})$, of $\boldsymbol{\Theta}$

$$\mathbb{E}_{\boldsymbol{\Theta}|\mathcal{D}}[g(\boldsymbol{\Theta})] \approx \frac{1}{n}\sum_{i=1}^{n} g(\boldsymbol{\Theta}_i)$$

    - $\boldsymbol{\Theta}_i$ are random deviates drawn from the posterior $\mathbb{P}[\boldsymbol{\Theta}_i|\mathcal{D}]$
    - (random deviates or random variates are values drawn from a probability distribution)
    - When the posterior distribution is well behaved (doesn't have a thick tail, so that $g(\boldsymbol{\Theta})$ has a finite variance) then the error in the approximation falls off as $1/\sqrt{n}$
    - For independent samples $\mathrm{Var}\big(\frac{1}{n}\sum_i g(\boldsymbol{\Theta}_i)\big) = \mathrm{Var}(g)/n$, whatever the dimension of $\boldsymbol{\Theta}$: the dimension only enters through $\mathrm{Var}(g)$
- The problem is you have to be able to sample random deviates from the posterior, which is proportional to $f(\mathcal{D}|\boldsymbol{\Theta})\, f(\boldsymbol{\Theta})$
- For simple distributions there are two classic solutions to generating random deviates

1. *Transformation Methods*
   - If we can compute the cumulative probability function
   $$F_X(x) = \int_{-\infty}^{x} f(t)\,\mathrm{d}t$$
   and invert $F_X(x)$, then $X = F_X^{-1}(U)$ will be a random deviate with density f, where U is a uniform deviate between 0 and 1
   - This works because $\mathbb{P}[X \le x] = \mathbb{P}\left[F_X^{-1}(U) \le x\right] = \mathbb{P}[U \le F_X(x)] = F_X(x)$
   - E.g. for the exponential distribution $F_X(x) = 1 - \mathrm{e}^{-\lambda x}$, so $X = -\log(1-U)/\lambda$
   - This only works for a few classic distributions
2. *Rejection Method*
   - In the rejection methods we find another density $g_Y(y)$ that we can sample from that satisfies
   $$c\,g_Y(x) \ge f_X(x) \quad \text{for all } x$$
   - Then we generate a deviate $Y \sim g_Y$ and accept it with a probability $f_X(Y)/(c\,g_Y(Y))$ (draw U uniformly from $(0,1)$ and accept if $U\,c\,g_Y(Y) \le f_X(Y)$)
   - The probability of accepting is $\int g_Y(y)\,f_X(y)/(c\,g_Y(y))\,\mathrm{d}y = 1/c$, so on average we need c tries, or $c-1$ rejections, for each deviate
   - The accepted deviates have density f_X: the density of proposing y and accepting it is $g_Y(y)\,f_X(y)/(c\,g_Y(y)) = f_X(y)/c$, which is proportional to $f_X(y)$
   - This is a more general method, but is efficient only if we have a good approximating function $g_Y(y)$ (good in the sense that c is not too large)
   - Particularly for multivariate (i.e. high dimensional) random variables it is often very hard to find a good approximating function

2.2 Markov-Chain Monte Carlo (MCMC)

- Fortunately there is another method for generating random deviates which is very general
- This uses a Markov-Chain where we update an initial state of our system
- If we choose the dynamics correctly eventually we will reach a state that is correctly distributed
- Suppose we have a set of states, $\mathcal{S}$, and we want to choose a random sample from a distribution $\boldsymbol{\pi} = (\pi_i | i \in \mathcal{S})$
  - π_i is the probability of being in state i—we get to choose this (for Bayesian inference we will want to choose the posterior probability)
- Let M_{ij} be the transition probability from state j to state i
  - $\sum_i M_{ij} = 1$ (if we start in any state, j say, we have to end up in some state, possibly j itself)
  - $M_{ij} \ge 0$ (it is a probability)
  - Matrices of non-negative elements whose columns sum to one (with this convention) are known as **stochastic matrices**
- If we choose M_{ij} so that
  $$M_{ij}\,\pi_j = M_{ji}\,\pi_i$$
  - then after some time the probability of being in state i will be π_i (there are some mild conditions on M for this to be true)

- This condition is known as **detailed balance**
- Note that if we sum the detailed balance equation then

$$\sum_{j\in\mathcal{S}} M_{ij}\,\pi_j = \sum_{j\in\mathcal{S}} M_{ji}\,\pi_i = \pi_i$$

  - since $\sum_j M_{ji} = 1$
- Writing this in matrix form

$$\mathbf{M}\boldsymbol{\pi} = \boldsymbol{\pi}$$

  - That is, $\boldsymbol{\pi}$ is an eigenvector of $\mathbf{M}$ with eigenvalue 1
- Because $\mathbf{M}$ is a stochastic matrix all eigenvalues have $|\lambda| \leq 1$
  - To prove this let $\boldsymbol{v}$ be an eigenvector of $\mathbf{M}$ with eigenvalue λ

$$\sum_{j\in\mathcal{S}} M_{ij}\,v_j = \lambda\,v_i$$

    taking the absolute value of both sides

$$\left|\sum_{j\in\mathcal{S}} M_{ij}\,v_j\right| = |\lambda\,v_i| = |\lambda|\,|v_i|$$

  - But

$$\left|\sum_{j\in\mathcal{S}} M_{ij}\,v_j\right| \leq \sum_{j\in\mathcal{S}} |M_{ij}\,v_j| = \sum_{j\in\mathcal{S}} M_{ij}\,|v_j|$$

    where we have used $M_{ij} \geq 0$ (and essentially $|a+b| \leq |a| + |b|$)
  - Thus

$$\sum_{j\in\mathcal{S}} M_{ij}\,|v_j| \geq |\lambda|\,|v_i|$$

  - Summing both sides with respect to i

$$\begin{aligned}
\sum_{i\in\mathcal{S}}\sum_{j\in\mathcal{S}} M_{ij}\,|v_j| &\geq |\lambda| \sum_{i\in\mathcal{S}} |v_i| \\
\sum_{j\in\mathcal{S}}\sum_{i\in\mathcal{S}} M_{ij}\,|v_j| &\geq |\lambda| \sum_{i\in\mathcal{S}} |v_i| \\
\sum_{j\in\mathcal{S}} |v_j| &\geq |\lambda| \sum_{i\in\mathcal{S}} |v_i|
\end{aligned}$$

    where we used $\sum_{i\in\mathcal{S}} M_{ij} = 1$
  - It follows that $|\lambda| \leq 1$
  - Let $\boldsymbol{p}(t)$ be a vector whose elements $p_i(t)$ equal the probability of being in state i at time t
  - If the set of eigenvectors spans the whole space, then any starting vector $\boldsymbol{p}(0)$ can be written in terms of the eigenvectors

$$\boldsymbol{p}(0) = \sum_i c_i\,\boldsymbol{v}_i$$

    (this is just a change of basis)
    - detailed balance guarantees this: with $\mathbf{D} = \mathrm{diag}(\boldsymbol{\pi})$ (all $\pi_i > 0$), the matrix $\mathbf{D}^{-1/2}\mathbf{M}\mathbf{D}^{1/2}$ has elements $M_{ij}\sqrt{\pi_j/\pi_i}$, which is symmetric by detailed balance, so $\mathbf{M}$ has real eigenvalues and a full set of eigenvectors

- The probability vector after one step is given by

$$\boldsymbol{p}(1) = \mathbf{M}\,\boldsymbol{p}(0) = \mathbf{M}\sum_i c_i\,\boldsymbol{v}_i = \sum_i c_i\,\mathbf{M}\,\boldsymbol{v}_i = \sum_i c_i\,\lambda_i\,\boldsymbol{v}_i$$

- After t steps

$$\boldsymbol{p}(t) = \mathbf{M}^t\,\boldsymbol{p}(0) = \sum_i c_i\,\lambda_i^t\,\boldsymbol{v}_i$$

- If $|\lambda_i| < 1$ then λ_i^t will shrink exponentially fast so that

$$\lim_{t\to\infty} \boldsymbol{p}(t) = \sum_{i:\lambda_i=1} c_i \boldsymbol{v}_i$$

- That is we converge onto the set of eigenvectors with eigenvalue 1
- But if detailed balance is conserved then $\boldsymbol{\pi}$ satisfies this
- There are some conditions to ensure that there is only one such eigenvector with eigenvalue 1 (and no eigenvalue -1)
  * We have to ensure that we can get from any state to any other through some series of transitions
  * We have to prevent periodic behaviour (e.g. if our set of states where from 1 to n and we could only move from state i to state $i-1$ or $i+1$ then if we started on an odd state we would always be on an odd state after an even number of moves)
  * It is very easy to ensure both conditions

- Now the problem is to choose the transition probabilities to satisfy detailed balance
- This is easy and can be done in different ways
- **Metropolis Algorithm**
  - A very easy way to achieve detailed balance is that starting from state j we propose a neighbouring state i
    - We have to make sure the probability, q, of proposing state i when in state j is the same as the probability of proposing state j when in state i (e.g. every state has the same number of neighbours, each chosen with equal probability)
  - We now move to state i if
    - $\pi_i \geq \pi_j$
    - or otherwise we move anyway with a probability π_i/π_j
  - Detailed balance: if $\pi_i \geq \pi_j$ then $M_{ij} = q$ and $M_{ji} = q\,\pi_j/\pi_i$, so $M_{ij}\,\pi_j = q\,\pi_j = M_{ji}\,\pi_i$ (and the case $\pi_i < \pi_j$ is the same with i and j swapped)
- **Metropolis-Hastings Algorithm**
  - Sometimes it is difficult to ensure that choosing state i from state j is the same as choosing state j from state i
    - As an example considering our states were non-negative integers, we move from state i to state $i-1$ and $i+1$ with probability $\frac{1}{2}$, but we have a problem at state 0 where we can't move to state $i-1$
  - We can modify the Metropolis algorithm. Suppose $p(i|j)$ is the probability of choosing state i starting in state j then let $r = p(j|i)\,\pi_i/(p(i|j)\,\pi_j)$
    - we accept the move if $r > 1$
    - or with a probability r

  - in practice we can just draw a random number U uniformly between 0 and 1 and accept the move if $U < r$
  - this is the same as the Metropolis algorithm if $p(j|i) = p(i|j)$

- We can equally well apply this to (multi-dimensional) continuous variables, $\boldsymbol{\theta}$
  - If $\boldsymbol{\theta}$ is our current set of parameters, we choose a new set of parameter $\boldsymbol{\theta}'$ with probability $p(\boldsymbol{\theta}'|\boldsymbol{\theta})$ and then accept this proposal if $r = (p(\boldsymbol{\theta}|\boldsymbol{\theta}')\,\pi(\boldsymbol{\theta}'))/(p(\boldsymbol{\theta}'|\boldsymbol{\theta})\,\pi(\boldsymbol{\theta})) \geq 1$ or otherwise with a probability r
  - We now choose $p(\boldsymbol{\theta}'|\boldsymbol{\theta})$ so that with high probability $\boldsymbol{\theta}'$ is close to $\boldsymbol{\theta}$, this ensures that there is a reasonably high acceptance rate
  - But the step size is a trade-off: tiny steps are almost always accepted but the chain explores very slowly, while huge steps are almost always rejected and the chain sits still

- For Bayes calculations then $\pi(\boldsymbol{\theta})$ would be our posterior $f(\boldsymbol{\theta}|\mathcal{D})$
  - MCMC has the nice property that the update depends only on the ratio

$$\frac{\pi(\boldsymbol{\theta}')}{\pi(\boldsymbol{\theta})} = \frac{f(\boldsymbol{\theta}'|\mathcal{D})}{f(\boldsymbol{\theta}|\mathcal{D})} = \frac{f(\mathcal{D}|\boldsymbol{\theta}')\,f(\boldsymbol{\theta}')}{f(\mathcal{D}|\boldsymbol{\theta})\,f(\boldsymbol{\theta})}$$

  - That is, the normalising factor $f(\mathcal{D})$ that appears in Bayes' rule cancels
  - This is important because this is usually incomputable (although we can use MCMC to estimate this)
  - There is also often another advantage of this ratio: if we only change one variable at a time then frequently we can compute the ratio much more efficiently than the full likelihood and priors
  - When this happens then it pays to choose a single variable, θ_i, at a time, choose a neighbour θ_i' from a distribution $p(\theta_i'|\theta_i)$ and decide whether to accept this update

- **Gibbs Sampling**
  - A special case of updating one variable at a time is to draw the new value θ_k' from its conditional distribution $\pi(\theta_k'|\boldsymbol{\theta}_{-k})$, where $\boldsymbol{\theta}_{-k}$ are all the other components, which are kept fixed
  - This is *Gibbs sampling*
  - It is Metropolis-Hastings with proposal $p(\boldsymbol{\theta}'|\boldsymbol{\theta}) = \pi(\theta_k'|\boldsymbol{\theta}_{-k})$. Writing $\pi(\boldsymbol{\theta}) = \pi(\theta_k|\boldsymbol{\theta}_{-k})\,\pi(\boldsymbol{\theta}_{-k})$,

$$r = \frac{\pi(\theta_k|\boldsymbol{\theta}_{-k})\,\pi(\theta_k'|\boldsymbol{\theta}_{-k})\,\pi(\boldsymbol{\theta}_{-k})}{\pi(\theta_k'|\boldsymbol{\theta}_{-k})\,\pi(\theta_k|\boldsymbol{\theta}_{-k})\,\pi(\boldsymbol{\theta}_{-k})} = 1$$

    so every move is accepted
  - It is useful when the conditional distributions are standard distributions that we can sample from (often because of conjugate priors), even though the joint posterior is not
  - When the parameters are strongly correlated, moving one at a time only allows small steps, and the chain mixes slowly

- **Convergence**
  - A problem with MCMC is that your initial parameter, $\boldsymbol{p}(0)$, is different from $\boldsymbol{\pi}$ so you have to wait some considerable time before your samples are from posterior distribution
  - As we saw earlier the convergence rate is determined by the second largest eigenvalue of the transition matrix $\mathbf{M}$ (we never explicit calculate $\mathbf{M}$ so we don't know what this eigenvalue is)

    - If it is very close to 1 then your convergence can be slow
  - Furthermore if your posterior distribution is broad you need lots of independent samples to accurately compute expectations
  - To obtain an independent sample we have to run our MCMC a long time
  - In practice you have to throw away some samples in a *burn-in* period
  - You can then average over all the remaining samples you have (including repetitions where you don't accept a move)
  - If you do this long enough then you should get samples that cover values of $\boldsymbol{\theta}$ with a high posterior probability
  - Successive samples are correlated, so n samples are worth fewer than n independent ones (the *effective sample size*)
  - MCMC is slow because of this
  - There are a lot of clever tricks to speed up convergence and decorrelation times
- Often MCMC seems daunting because of the apparent difficulties
  - But don't be put off
  - You only need to use the tricks for extremely complicated problems
  - Usually a simple implementation works fine
  - Modern computers are so fast that, for problems with a modest number of parameters, a simple random-walk Metropolis-Hastings sampler gives accurate answers in seconds
    - Convergence can be a problem in pathological cases—these occur when using MCMC to model some physical systems—but for many Bayesian problems the Markov Chain is *rapidly mixing* meaning the convergence time is not excessively long
    - Random walks do get slow when there are many parameters or the parameters are strongly correlated; that is what Hamiltonian Monte Carlo (below) is for

2.3 Checking Convergence

- We never know for certain that a chain has converged, but there are standard checks, and MCMC libraries carry them out for you
- Run several chains from widely different starting points and discard the burn-in of each
- Plot the traces (the value of each parameter against time): after burn-in the chains should overlap and look alike. A chain stuck in one region, or drifting, shows that something is wrong
- $\hat{R}$ (the Gelman–Rubin statistic) compares the variance between the chains with the variance within them. If the chains are all sampling the same distribution these agree and $\hat{R} \approx 1$; a common rule is to require $\hat{R} < 1.01$
- Successive samples are correlated. If the correlation between samples k steps apart is ρ_k, the mean of n samples has the variance of the mean of about

$$n_{\text{eff}} = \frac{n}{1 + 2\sum_{k=1}^{\infty} \rho_k}$$

  independent samples, the *effective sample size*. Report it with your estimates: the error in a posterior mean is roughly the posterior standard deviation divided by $\sqrt{n_{\text{eff}}}$

2.4 Hamiltonian Monte Carlo

- A random walk explores by diffusion: to cross a distance D with steps of size s takes about $(D/s)^2$ steps. When the posterior is a long narrow ridge (strongly correlated parameters) the step must be as small as the ridge is narrow, and progress along the ridge is very slow. The same happens in high dimensions
- Hamiltonian (or hybrid) Monte Carlo (HMC) uses the gradient of the log posterior to make long moves that are still accepted
- Treat $U(\boldsymbol{\theta}) = -\log \pi(\boldsymbol{\theta})$ as a landscape. As before this only needs likelihood times prior, since the evidence only adds a constant to U
- Introduce a fictitious momentum $\boldsymbol{p}$ of the same dimension and the energy

  $$H(\boldsymbol{\theta}, \boldsymbol{p}) = U(\boldsymbol{\theta}) + \tfrac{1}{2}\|\boldsymbol{p}\|^2$$

  The joint density $\propto \mathrm{e}^{-H} = \pi(\boldsymbol{\theta})\,\mathrm{e}^{-\|\boldsymbol{p}\|^2/2}$, so $\boldsymbol{\theta}$ and $\boldsymbol{p}$ are independent, $\boldsymbol{\theta}$ has the distribution we want and $\boldsymbol{p}$ is standard normal
- Each HMC step
  1. draws a fresh momentum $\boldsymbol{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  2. follows the frictionless motion of a ball in the landscape, $\dot{\boldsymbol{\theta}} = \boldsymbol{p}$, $\dot{\boldsymbol{p}} = -\boldsymbol{\nabla} U(\boldsymbol{\theta})$, for L small time steps of size ϵ using the *leapfrog* method (a half step in $\boldsymbol{p}$, alternate full steps in $\boldsymbol{\theta}$ and $\boldsymbol{p}$, and a final half step in $\boldsymbol{p}$)
  3. accepts the end point with probability $\min\bigl(1, \mathrm{e}^{H_{\text{start}} - H_{\text{end}}}\bigr)$
- The exact motion conserves H, so the acceptance probability would be 1; the leapfrog method makes small errors, and step 3 is a Metropolis–Hastings step that corrects for them, so the samples come exactly from π. (This relies on the leapfrog method being reversible and preserving volume, which it does)
- Because H is almost conserved, moves a long way across the distribution are almost always accepted
- In practice
  - HMC needs $\boldsymbol{\nabla} \log \pi(\boldsymbol{\theta})$; automatic differentiation (as used to train neural networks in PyTorch or JAX) provides it
  - it needs a step size ϵ and a number of steps L: too short a path and HMC is a random walk, too long and the path wastes time coming back. The *No-U-Turn Sampler* (NUTS; Hoffman and Gelman, 2014) chooses these automatically, stopping the path when it starts to turn back on itself
  - NUTS is the default sampler in probabilistic programming languages such as Stan, PyMC and NumPyro: you write down the generative story and get posterior samples
- In the lecture's example (a Gaussian with correlation 0.95, starting in the tail) 300 random-walk steps crawl along the ridge, while 15 HMC trajectories of 20 steps each, the same number of evaluations, cross it every time (Figure 1)

2.5 The Wider Landscape

- *Variational inference* fits an approximate posterior by optimisation (maximising the ELBO, see the notes on probabilistic inference and below). It is fast but only approximate

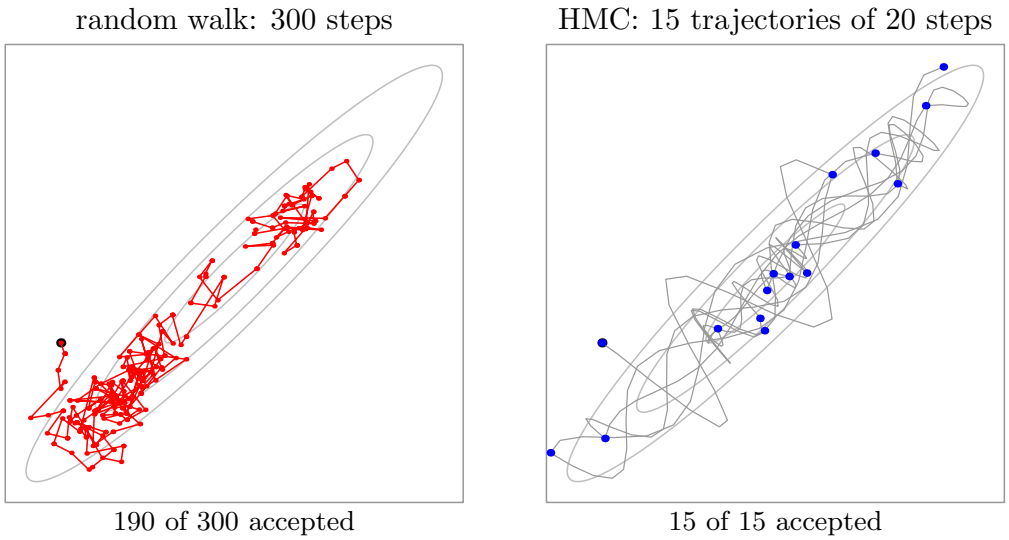


Figure 1: Random-walk Metropolis (left) and HMC (right) started at the same point on a Gaussian with correlation 0.95, with the same number of evaluations. Grey curves: HMC trajectories; blue: HMC samples

- *Stochastic-gradient MCMC* (e.g. stochastic-gradient Langevin dynamics) takes noisy gradient steps on minibatches of data, which makes MCMC possible for very large data sets and for Bayesian neural networks
- *Parallel tempering* and *sequential Monte Carlo* help when the posterior has several well-separated peaks, which a single chain rarely crosses between
- Diffusion models, which generate images in modern generative AI, draw their samples using Langevin-type dynamics, a close relative of MCMC

2.6 Variational Techniques

- This is a completely different approach to MCMC, but it tries to solve the same problem, but uses a different strategy
- This is again technically challenging and I don't expect you to memorise the derivation
- We saw in earlier lectures that we can cheat by seeking the parameters that maximise the posterior (the, so called, MAP solution)
- But this sacrifices all the probabilistic information
- In variational techniques we try to approximate the posterior with a simpler distribution
- That is, we approximate the posterior $f(\boldsymbol{\theta}|\mathcal{D})$ with a simpler distribution $q(\boldsymbol{\theta}|\boldsymbol{w})$ where we get choose $\boldsymbol{w}$ to make q close to f
- Usually we choose q to be a factorisable distribution

$$q(\boldsymbol{\theta}|\boldsymbol{w}) = \prod_i q(\theta_i|w_i)$$

- These techniques were first developed in the physics community and so come with a strange language

- In the *variational approximations* we consider the **variational free energy**

$$\Phi(\boldsymbol{w}) = \int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log\left(\frac{q(\boldsymbol{\theta}|\boldsymbol{w})}{f(\boldsymbol{\theta}, \mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta}$$

  - writing $f(\boldsymbol{\theta}, \mathcal{D}) = f(\boldsymbol{\theta}|\mathcal{D}) \, f(\mathcal{D})$ we can rearrange the variational free energy as

$$\begin{aligned}\Phi(\boldsymbol{w}) &= \int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \left(\log\left(\frac{q(\boldsymbol{\theta}|\boldsymbol{w})}{f(\boldsymbol{\theta}|\mathcal{D})}\right) - \log(f(\mathcal{D}))\right) \mathrm{d}\boldsymbol{\theta} \\ &= \mathrm{KL}\big(q(\boldsymbol{\theta}|\boldsymbol{w}) \big\| f(\boldsymbol{\theta}|\mathcal{D})\big) - \log(f(\mathcal{D}))\end{aligned}$$

  - where $\mathrm{KL}\big(q(\boldsymbol{\theta}|\boldsymbol{w}) \big\| f(\boldsymbol{\theta}|\mathcal{D})\big)$ is the KL divergence that measures the "distance" between $q(\boldsymbol{\theta}|\boldsymbol{w})$ and $f(\boldsymbol{\theta}|\mathcal{D})$
  - the KL-divergence is non-negative and equal to zero only if the two distributions are the same (we showed this in the notes on probabilistic inference, and meet it again in the lecture on entropy)
  - The term $\log(f(\mathcal{D}))$ is the *marginal likelihood* or *evidence* and does not depend on the parameters $\boldsymbol{w}$
  - If we choose $\boldsymbol{w}$ to minimise $\Phi(\boldsymbol{w})$ then we minimise the KL-divergence and force $q(\boldsymbol{\theta}|\boldsymbol{w})$ to approximate the posterior $f(\boldsymbol{\theta}|\mathcal{D})$
  - The variational free energy can also be written as

$$\begin{aligned}\Phi(\boldsymbol{w}) &= -\int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log(f(\boldsymbol{\theta}, \mathcal{D})) \, \mathrm{d}\boldsymbol{\theta} + \int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log(q(\boldsymbol{\theta}|\boldsymbol{w})) \, \mathrm{d}\boldsymbol{\theta} \\ &= -\big(L_q(\boldsymbol{w}) + H_q(\boldsymbol{w})\big)\end{aligned}$$

  - Minimising the variational free energy is equivalent to maximising $L_q(\boldsymbol{w}) + H_q(\boldsymbol{w})$
  - $L_q(\boldsymbol{w}) = \int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log(f(\boldsymbol{\theta}, \mathcal{D})) \, \mathrm{d}\boldsymbol{\theta}$ is the expected log of the joint probability of data and parameters (likelihood times prior), averaged over our approximate posterior $q(\boldsymbol{\theta}|\boldsymbol{w})$
  - $H_q(\boldsymbol{w}) = -\int q(\boldsymbol{\theta}|\boldsymbol{w}) \, \log(q(\boldsymbol{\theta}|\boldsymbol{w})) \, \mathrm{d}\boldsymbol{\theta}$ is the entropy of our approximate posterior (this measures the uncertainty in $\boldsymbol{\theta}$)
  - We thus balance maximising the likelihood of the data with maximising the uncertainty in our approximate posterior
  - This means we fit the data, but guard against over fitting by allowing there to be a non-zero probability wherever it does not strongly contradict the data
  - In practice both $L_q(\boldsymbol{w})$ and $H_q(\boldsymbol{w})$ tend to be quite easy to compute
  - We are left with an optimisation problem for the parameters, $\boldsymbol{w}$, but this is usually quite quick to compute

3 Exercises

3.1 Transformation method

- Find a transformation of a uniform deviate U on $(0, 1)$ that gives a deviate with density $f(x) = 2x$ for $0 \leq x \leq 1$.

3.2 Rejection method

- We draw normal deviates $\mathcal{N}\big(x\big|0, 1\big)$ using Cauchy deviates with density $g(x) = 1/(\pi(1 + x^2))$ as proposals (as in the lecture). Find the smallest c such that $c\, g(x) \geq \mathcal{N}\big(x\big|0, 1\big)$ for all x, and the fraction of proposals that are accepted.

3.3 A two-state chain

- A chain has two states with $\boldsymbol{\pi} = (\frac{1}{4}, \frac{3}{4})$. Each state's only neighbour is the other state. Using the Metropolis algorithm, write down the transition matrix $\mathbf{M}$, check that $\mathbf{M}\boldsymbol{\pi} = \boldsymbol{\pi}$, and find the other eigenvalue. How does $\boldsymbol{p}(t)$ approach $\boldsymbol{\pi}$?

3.4 Mysterious Disease

- Recall in the lecture on probabilistic inference, we defined $Z(t)$ to be the number of people that catch a disease on day t
- We assumed the growth rate is given by

$$\mathbb{P}[Z(t+1)] = \mathrm{Poi}\Big(Z(t+1)\Big|\frac{r_0}{3}\left(Z(t) + Z(t-1) + Z(t-2)\right)\Big)$$

  - that is, people are infectious for three days

- The observed number of new people with the disease, $X(t)$, on day t is

$$\mathbb{P}[X(t) = k] = \mathrm{Binom}(k|Z(t), p) = \binom{Z(t)}{k} p^k \left(1-p\right)^{Z(t)-k}$$

- Here is $X(t)$ for the first 30 days
  0,0,0,0,1,1,2,1,0,0,3,10,19,34,44,93,117,221,376,633,
  1021,1643,2701,4503,7414,12091,19999,33286,54612,90283

- Estimate p and r_0 from the data
- (If you prefer simulate your own data and estimate your parameters)

4 Experiments

4.1 Random walk against HMC

- Sample a Gaussian with correlation 0.95 by random-walk Metropolis and by HMC with the same number of evaluations, and compare their effective sample sizes

```
import numpy as np

rng = np.random.default_rng(0)
rho, d = 0.95, 2
P = np.linalg.inv(np.array([[1, rho], [rho, 1]]))   # precision matrix
U = lambda x: 0.5 * x @ P @ x                       # -log pi + const
gradU = lambda x: P @ x


def ess(x):
    """Effective sample size from the autocorrelations (summed to the first
    negative one)."""
    x = x - x.mean()
    acf = np.correlate(x, x, "full")[len(x) - 1:] / (x @ x)
    k = np.argmax(acf < 0)
    return len(x) / (1 + 2 * acf[1:k].sum())
```

```
budget = 30000                                  # evaluations for each
x, rw = np.zeros(d), []
for t in range(budget):                         # random-walk Metropolis
    y = x + 0.3 * rng.standard_normal(d)
    if np.log(rng.random()) < U(x) - U(y):
        x = y
    rw.append(x[0])

L, eps = 20, 0.15
x, hmc = np.zeros(d), []
for t in range(budget // L):                    # HMC, L gradients each
    p = rng.standard_normal(d)
    q, H0 = x.copy(), U(x) + 0.5 * p @ p
    p = p - 0.5 * eps * gradU(q)                # leapfrog
    for s in range(L):
        q = q + eps * p
        if s < L - 1:
            p = p - eps * gradU(q)
    p = p - 0.5 * eps * gradU(q)
    if np.log(rng.random()) < H0 - (U(q) + 0.5 * p @ p):
        x = q
    hmc.append(x[0])

for name, xs in [("random walk", np.array(rw)), ("HMC", np.array(hmc))]:
    print(name, len(xs), "samples, mean", round(xs.mean(), 3),
          "ESS", round(ess(xs)))
```

- (See the answers.)

4.2 Metropolis for the traffic rate

- The lecture's traffic example with a random-walk proposal $\mu' = \mu + s\,\epsilon$, $\epsilon \sim \mathcal{N}\big(\epsilon\big|0,1\big)$, compared with the exact posterior $\mathrm{Gamma}(\mu|\sum_i N_i, n)$

```
import numpy as np

rng = np.random.default_rng(0)
N = rng.poisson(5.0, size=20)           # cars on 20 days, true rate 5
n, S = len(N), N.sum()


def log_post(mu):                       # log(likelihood x prior), prior 1/mu
    return -n * mu + S * np.log(mu) - np.log(mu) if mu > 0 else -np.inf


for step in [0.05, 0.5, 5.0]:
    mu, samples, accepted = 1.0, [], 0
    for t in range(60000):
        mu_new = mu + step * rng.standard_normal()   # symmetric proposal
        if np.log(rng.random()) < log_post(mu_new) - log_post(mu):
            mu, accepted = mu_new, accepted + 1
        if t >= 10000:                  # discard burn-in
            samples.append(mu)          # count repeats when we don't move
    samples = np.array(samples)
```

```
    print(f"step {step:4}: accept {accepted/60000:.2f}  "
          f"mean {samples.mean():.3f}  sd {samples.std():.3f}")
print(f"exact Gamma({S},{n}): mean {S/n:.3f}  sd {np.sqrt(S)/n:.3f}")
```

- Note that we work with the log of the posterior (products of many probabilities underflow) and that a proposal $\mu' \le 0$ is simply rejected (its posterior is 0). Try changing the step size and the length of the run. (See the answers.)

5 Answers

5.1 Random walk against HMC

- Over three random seeds the 30000 random-walk samples have an effective sample size of 150–190, while the 1500 HMC samples have about 1500: essentially independent (the estimate is capped at n because successive HMC samples are slightly anti-correlated). For the same work HMC gives about ten times as many effective samples here, and the advantage grows with the dimension

5.2 Transformation method

- $F(x) = \int_0^x 2t\,\mathrm{d}t = x^2$, so $X = F^{-1}(U) = \sqrt{U}$

5.3 Rejection method

- We need $c \ge \mathcal{N}\big(x\big|0,1\big)\,/g(x) = \sqrt{\pi/2}\,(1+x^2)\,\mathrm{e}^{-x^2/2}$ for all x. Differentiating, $2x - x(1+x^2) = x(1-x^2) = 0$, so the maximum is at $x = \pm 1$, giving $c = \sqrt{2\pi}\,\mathrm{e}^{-1/2} \approx 1.52$ (the lecture uses 1.53)

- The acceptance probability is $1/c \approx 0.66$: about a third of the proposals are wasted

5.4 A two-state chain

- From state 1 we always propose state 2, and $\pi_2 > \pi_1$ so we always accept: $M_{21} = 1$, $M_{11} = 0$. From state 2 we propose state 1 and accept with probability $\pi_1/\pi_2 = \frac{1}{3}$: $M_{12} = \frac{1}{3}$, $M_{22} = \frac{2}{3}$

$$\mathsf{M} = \begin{pmatrix} 0 & \frac{1}{3} \\ 1 & \frac{2}{3} \end{pmatrix}, \qquad \mathsf{M}\boldsymbol{\pi} = \begin{pmatrix} \frac{1}{3}\cdot\frac{3}{4} \\ \frac{1}{4} + \frac{2}{3}\cdot\frac{3}{4} \end{pmatrix} = \begin{pmatrix} \frac{1}{4} \\ \frac{3}{4} \end{pmatrix}$$

- The eigenvalues sum to the trace, $\frac{2}{3}$, so the other is $-\frac{1}{3}$. $\boldsymbol{p}(t) - \boldsymbol{\pi}$ shrinks by a factor of 3 each step and changes sign each step: the chain overshoots and oscillates as it converges

5.5 Metropolis for the traffic rate

- Running the code gives (the exact posterior has mean 5.300 and standard deviation 0.515)

step s	acceptance	mean	sd
0.05	0.97	5.278	0.496
0.5	0.71	5.300	0.510
5	0.13	5.296	0.506

- All three agree with the exact answer to within a few hundredths, but for different reasons: with $s = 0.05$ nearly every move is accepted but each moves only a little, so successive samples are strongly correlated; with $s = 5$ most proposals are rejected. A step comparable to the width of the posterior works best. With shorter runs the poor choices of s give visibly worse estimates

5.6 Mysterious Disease

- We need to decide on a prior
  - A reasonable prior for p is a Beta distribution perhaps with $a = b = 1$ (this is a uniform prior)
  - For r_0 we could use a Gamma prior with $a = 2$ and $b = 1$, which has a mean of $r_0 = 2$ (it seems to be considerably greater than 1)—we need to include $r_0 \, \mathrm{e}^{-r_0}$ as our prior (the normalisation is irrelevant)
- Now we have to choose proposal distributions for the new values of p and r_0
  - You could use $p' = p + U(-0.01, 0.01)$ and $r_0' = r_0 + U(-0.1, 0.1)$ where $U(a, b)$ is a uniform random deviate between a and b
  - These proposals are symmetric; a proposal outside the allowed range ($0 \le p \le 1$ and $r_0 \ge 0$) has zero posterior probability and is simply rejected
  - We could instead choose $p' \sim \mathrm{Beta}(10\,p, 10{-}10\,p)$ and $r_0' \sim r_0 \, \mathrm{Gamma}(5, 5) = \mathrm{Gamma}(5, 5/r_0)$, which never leave the allowed range and ensure that in expectation p' equals p and r_0' equals r_0 with small variations
    - Beware that there are two common definitions of Gamma distributions

$$\mathrm{Gamma}(z|a, b) = \frac{b^a}{\Gamma(a)} \, z^{a-1} \, \mathrm{e}^{-b\,z}$$

$$\mathrm{Gamma}^*(z|\alpha, \beta) = \frac{1}{\beta^\alpha \, \Gamma(\alpha)} z^{\alpha-1} \, \mathrm{e}^{-z/\beta}$$

    - You have to check which of these your libraries use (numpy's `rng.gamma(shape, scale)` uses the second)
    - These proposals are not symmetric, so we would need Metropolis-Hastings to ensure we get unbiased samples
- The variables $Z(t)$ are latent variables: we also have to sample them (e.g. by proposing changes to one $Z(t)$ at a time), and their samples estimate the actual number of cases of people with the virus
  - My data was generated with $p = 0.1$ and $r_0 = 2.5$
  - A check: with people infectious for three days the cases grow by a factor λ per day with $\lambda^3 = \frac{r_0}{3}(\lambda^2 + \lambda + 1)$. The data grow by a factor of about 1.65 a day, and $\lambda = 1.65$ gives $r_0 \approx 2.5$. The growth rate fixes r_0, but p is only determined by the early days, when the numbers are small

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 25: Entropy

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Shannon entropy, bits and nats, code length, Huffman coding, Stirling's approximation, maximum entropy

2 Main Points

2.1 Measuring Uncertainty

- Which is more uncertain: three coin tosses or one throw of a die? It depends on whether we care about the order of the tosses—but how would we answer at all?
- For a discrete random variable X, Shannon's entropy gives a precise answer,

$$H_X = -\sum_{x\in\mathcal{X}} \mathbb{P}[X=x]\,\log_2(\mathbb{P}[X=x])$$

- **A fair die**, $\mathbb{P}[D=i]=1/6$:

$$H_D = -\sum_{i=1}^{6}\frac{1}{6}\log_2\left(\frac{1}{6}\right) = \log_2(6) \approx 2.585 \text{ bits}$$

- **Three ordered coin tosses**, eight equally likely outcomes: $H_C = \log_2(8) = 3$ bits. More equally likely outcomes means more uncertainty, as we'd expect
- **Three unordered coin tosses** (we only record the number of heads): the outcomes 0 and 3 heads have probability $1/8$ each, and 1 and 2 heads have probability $3/8$ each, so

$$H_U = -\frac{1}{4}\log_2\left(\frac{1}{8}\right) - \frac{3}{4}\log_2\left(\frac{3}{8}\right) \approx 1.811 \text{ bits}$$

- This seems reasonable, but it is not obvious how we would have arrived at a number without entropy. Why this particular formula?

2.2 Additivity

- For two independent random variables, $\mathbb{P}[X,Y]=\mathbb{P}[X]\,\mathbb{P}[Y]$, so

$$\begin{aligned}
H_{(X,Y)} &= -\sum_{x,y}\mathbb{P}[X=x]\,\mathbb{P}[Y=y]\,\log_2(\mathbb{P}[X=x]\,\mathbb{P}[Y=y]) \\
&= -\sum_{x,y}\mathbb{P}[X=x]\,\mathbb{P}[Y=y]\,(\log_2(\mathbb{P}[X=x])+\log_2(\mathbb{P}[Y=y])) \\
&= -\sum_{x}\mathbb{P}[X=x]\,\log_2(\mathbb{P}[X=x]) - \sum_{y}\mathbb{P}[Y=y]\,\log_2(\mathbb{P}[Y=y]) = H_X + H_Y
\end{aligned}$$

  using $\sum_y \mathbb{P}[Y=y] = \sum_x \mathbb{P}[X=x] = 1$

- The uncertainty of two independent events adds. Shannon's entropy is one of very few functions with this property

2.3 Entropy as Code Length

- To communicate one of 2^n equally likely outcomes (say n coin tosses) we need a binary string of exactly n bits
- With 5 equally likely outcomes, 3 bits would do but would waste $3/8$ of the possible codes. Batching 3 independent messages gives $5^3 = 125$ outcomes, which fit in 7 bits ($2^7 = 128$), wasting only $3/128$: that is $7/3 \approx 2.33$ bits per message
- Batching more and more messages, the cost per message approaches $\log_2(5) \approx 2.32$ bits. In general N equally likely outcomes need $\log_2(N) = -\log_2(1/N)$ bits each
- When outcomes have different probabilities we can still find codes that use very close to $-\log_2(\mathbb{P}[X = x])$ bits for outcome x, by transmitting many messages. This code length measures the *surprise* of the outcome: rare outcomes cost more
- Shannon's entropy is the *expected code length*,

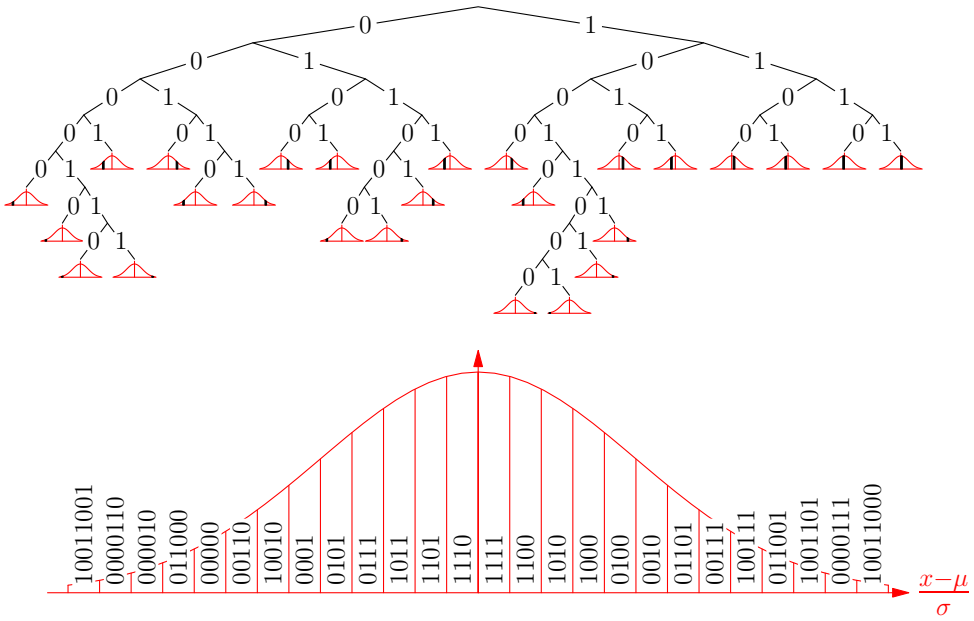
$$H_X = \mathbb{E}_X[-\log_2(\mathbb{P}[X])]$$

  which is the average amount of information needed to convey the outcome
- The lecture animation illustrates this with a loaded die: each outcome's bar is the length $-\log_2(\mathbb{P}[X = x])$ of its code word, and the entropy is the probability-weighted average bar length

X_i:	1	2	3	4	5	6
$p(X_i)$:	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{4}$	$\frac{1}{4}$
Code:	000	001	010	011	10	11
$L = -\log_2\big(p(X_i)\big)$:	3	3	3	3	2	2

2.4 Real Codes

- Code words of different lengths cause a practical problem: the receiver needs to know where each one ends
- *Huffman coding* solves this by placing the code words at the leaves of a binary tree, so no code word is a prefix of another and any message can be decoded by walking the tree
- A greedy algorithm (repeatedly merge the two least probable symbols) builds the optimal tree. Its expected code length L satisfies $H_X \leq L < H_X + 1$



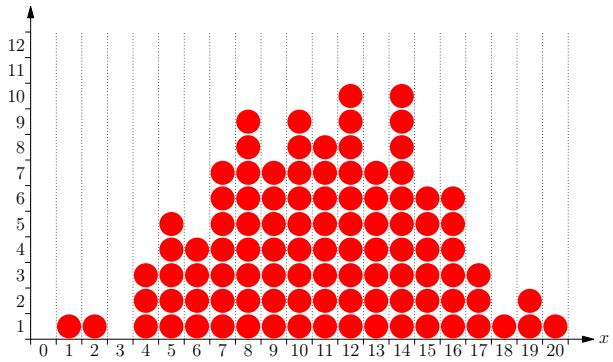
- The lecture illustrates coding a normal distribution discretised to accuracy Δx: finer accuracy means more possible values and a longer code

2.5 Bits and Nats

- With $\log_2$, entropy is measured in *bits*
- With natural logarithms, $H_X = -\sum_x \mathbb{P}[X = x] \ln(\mathbb{P}[X = x])$, entropy is measured in *nats*. One nat is $\log_2(\mathrm{e}) \approx 1.443$ bits
- Nats are more convenient whenever we do calculus on entropy, which is why the rest of this note uses natural logarithms

2.6 Counting States

- Put N balls into K boxes so that n_i land in box i

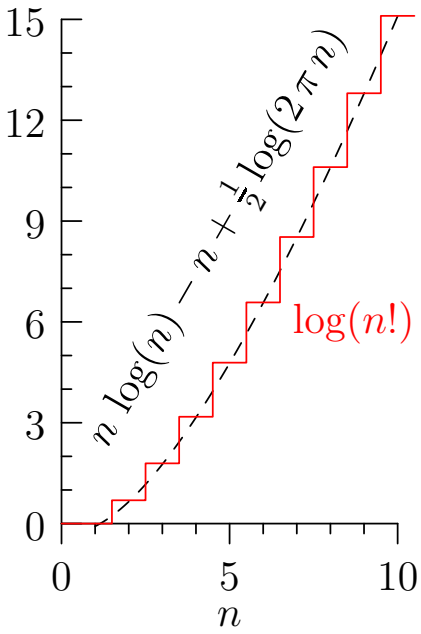


- The number of ways of doing this is the multinomial coefficient

$$W(\boldsymbol{n}) = \frac{N!}{n_1!\, n_2! \cdots n_K!}$$

- For large n, *Stirling's approximation* gives

$$n! \approx \sqrt{2\pi n}\; n^n\, \mathrm{e}^{-n} \qquad\qquad \log(n!) \approx n\log(n) - n + \frac{1}{2}\log(2\pi n)$$



- Substituting, the $-n$ terms cancel because $\sum_i n_i = N$, and to leading order

$$W(\boldsymbol{n}) = \mathrm{e}^{\log(N!) - \sum_i \log(n_i!)} \approx C\,\mathrm{e}^{N\log(N) - \sum_i n_i \log(n_i)} = C\,\mathrm{e}^{-N\sum_i \frac{n_i}{N}\log\left(\frac{n_i}{N}\right)}$$

- Writing $p_i = n_i/N$ for the fraction of balls in box i,

$$W(\boldsymbol{n}) \approx C\,\mathrm{e}^{N\,H} \qquad \text{where} \qquad H = -\sum_i p_i \log(p_i)$$

- So entropy measures the logarithm of the number of microscopic arrangements consistent with the proportions p_i. Because of the factor N in the exponent, as $N \to \infty$ the arrangements consistent with any constraints are overwhelmingly concentrated on the proportions that maximise the entropy

2.7 Entropy Is Concave

- Each term $h(p) = -p\ln(p)$ has $h''(p) = -1/p < 0$, so the entropy $H = \sum_i h(p_i)$ is a concave function of the probabilities, and $-H$ is convex
- For K outcomes, Jensen's inequality for the concave $\log$ gives

$$H_X = \mathbb{E}\left[\log_2\left(\frac{1}{\mathbb{P}[X]}\right)\right] \le \log_2\left(\mathbb{E}\left[\frac{1}{\mathbb{P}[X]}\right]\right) = \log_2(K)$$

  since $\mathbb{E}[1/\mathbb{P}[X]] = \sum_x \mathbb{P}[X = x]\,/\mathbb{P}[X = x] = K$. The uniform distribution has the most uncertainty
- Maximising a concave function subject to linear constraints (such as fixed averages) is a convex problem with a single maximum, so the maximum entropy method gives one answer
- Because $-H$ is convex and favours spread-out distributions, it is a convenient regulariser; we use it for optimal transport

2.8 The Maximum Entropy Method

- To infer a distribution from limited information, maximise its entropy subject to constraints expressing what we know. The entropy acts as a prior

- Two justifications:
  - it picks by far the most likely set of configurations consistent with the observations
  - it maximises our uncertainty given what we know, making the estimate as unbiased as possible
- It relies on the underlying arrangements being equally likely a priori. If there are other constraints we have not included, it can give poor answers
- **Known mean and variance.** For a continuous X with $\mathbb{E}[X] = \mu$ and $\mathbb{E}\left[X^2\right] = \mu_2 = \mu^2 + \sigma^2$, form the Lagrangian

$$\mathcal{L}(f) = -\int f_X(x) \log(f_X(x))\, \mathrm{d}x + \lambda_0 \left(\int f_X(x)\, \mathrm{d}x - 1 \right) + \lambda_1 \left(\int f_X(x)\, x\, \mathrm{d}x - \mu \right) + \lambda_2 \left(\int f_X(x)\, x^2\, \mathrm{d}x - \mu_2 \right)$$

- Setting the functional derivative to zero,

$$\frac{\delta \mathcal{L}}{\delta f_X(x)} = -\log(f_X(x)) - 1 + \lambda_0 + \lambda_1\, x + \lambda_2\, x^2 = 0$$

  so $f_X(x) = \mathrm{e}^{-1+\lambda_0+\lambda_1 x+\lambda_2 x^2}$
- Choosing the multipliers to satisfy the three constraints gives

$$f_X(x) = \frac{1}{\sqrt{2\pi}\,\sigma}\, \mathrm{e}^{-(x-\mu)^2/(2\sigma^2)}$$

  The *normal distribution is the maximum entropy distribution with a given mean and variance*
- Maximum entropy works superbly in statistical physics, which infers distributions over microscopic states from observations such as volume and pressure. Temperature appears, rather strangely, as a Lagrange multiplier

2.9 Softmax Is Maximum Entropy

- A model gives scores s_i to K classes. Among the probability vectors with a given average score, $\sum_i p_i s_i = \bar{s}$, which has maximum entropy? As for the normal distribution, form

$$\mathcal{L} = -\sum_i p_i \ln(p_i) + \lambda_0 \left(\sum_i p_i - 1 \right) + \beta \left(\sum_i p_i s_i - \bar{s} \right)$$

- Setting $\partial \mathcal{L}/\partial p_i = -\ln(p_i) - 1 + \lambda_0 + \beta s_i = 0$ gives $p_i \propto \mathrm{e}^{\beta s_i}$, and normalising gives the *softmax*

$$p_i = \frac{\mathrm{e}^{s_i/T}}{\sum_j \mathrm{e}^{s_j/T}}, \qquad T = \frac{1}{\beta}$$

- The temperature T is the inverse of a Lagrange multiplier, exactly as in statistical physics. It is the same "temperature" used when sampling from a language model: as $T \to 0$ the most likely class takes all the probability, as $T \to \infty$ the distribution becomes uniform
- With scores $s_i = \boldsymbol{w}_i^{\mathsf{T}} \boldsymbol{x}$ this model is multinomial logistic regression, also known as the *maximum entropy classifier*

2.10 A Little History

- Entropy was introduced in physics by Rudolf Clausius in 1865 (Macquorn Rankine discussed it in 1850)
- Ludwig Boltzmann interpreted it as counting the number of states; Josiah Willard Gibbs and James Clerk Maxwell completed the picture
- Claude Shannon founded information theory on entropy around 1948
- Ed Jaynes realised that statistical physics can be seen as an inference problem—the maximum entropy method

3 Exercises

3.1 The uniform distribution

- Using a Lagrange multiplier for normalisation, show that the maximum entropy distribution over K outcomes with no other constraints is uniform, and find its entropy

3.2 Knowing only the mean

- Find the maximum entropy density on $x \geq 0$ with a given mean $\mathbb{E}[X] = \mu$

3.3 Entropy of a biased coin

- Find the entropy $H(p)$ in bits of a coin that comes up heads with probability p. For which p is it largest? Evaluate $H(0.1)$

3.4 Finishing the normal derivation

- Show that the multipliers in the maximum entropy derivation must be $\lambda_2 = -1/(2\sigma^2)$ and $\lambda_1 = \mu/\sigma^2$

4 Answers

4.1 The uniform distribution

- Maximise $-\sum_i p_i \log(p_i) + \lambda\left(\sum_i p_i - 1\right)$. Differentiating with respect to p_i gives $-\log(p_i) - 1 + \lambda = 0$, so every $p_i = \mathrm{e}^{\lambda - 1}$ takes the same value
- Normalisation forces $p_i = 1/K$, with entropy $\log(K)$ nats or $\log_2(K)$ bits

4.2 Knowing only the mean

- Maximise $-\int_0^\infty f \log f \,\mathrm{d}x + \lambda_0\left(\int_0^\infty f \,\mathrm{d}x - 1\right) + \lambda_1\left(\int_0^\infty f\, x \,\mathrm{d}x - \mu\right)$. The stationarity condition gives $f(x) = \mathrm{e}^{-1+\lambda_0+\lambda_1 x}$
- Normalisability on $[0, \infty)$ needs $\lambda_1 < 0$; write $f(x) = A\,\mathrm{e}^{-ax}$. Normalisation gives $A = a$, and the mean of $a\,\mathrm{e}^{-ax}$ is $1/a$, so $a = 1/\mu$:

  $$f(x) = \frac{1}{\mu}\,\mathrm{e}^{-x/\mu}$$

  the exponential distribution

4.3 Entropy of a biased coin

- $H(p) = -p\log_2(p) - (1-p)\log_2(1-p)$
- $H'(p) = \log_2((1-p)/p)$, which vanishes at $p = 1/2$, where $H = 1$ bit. $H''(p) < 0$, so this is the maximum
- $H(0.1) \approx 0.469$ bits

4.4 Finishing the normal derivation

- The log density is quadratic in x, $\log f_X(x) = -1 + \lambda_0 + \lambda_1 x + \lambda_2 x^2$
- The log of a normal density is $-(x-\mu)^2/(2\sigma^2) + \text{const} = -\frac{x^2}{2\sigma^2} + \frac{\mu\, x}{\sigma^2} + \text{const}$
- Matching coefficients of x^2 and x gives $\lambda_2 = -1/(2\sigma^2)$ and $\lambda_1 = \mu/\sigma^2$. The constant, and hence λ_0, is then fixed by normalisation. This works because a normal with the right mean and variance satisfies all three constraints, and a quadratic exponent determines a normal distribution uniquely

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 26: Information Theory

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Conditional entropy, joint entropy, mutual information, channel capacity, KL-divergence, variational inference, ELBO, cross-entropy, minimum description length, variational auto-encoders

2 Main Points

2.1 Communicating Down a Noisy Channel

- Information theory studies sending a message X, drawn with probability $\mathbb{P}[X]$, down a noisy channel that delivers Y with probability $\mathbb{P}[Y \mid X]$
- Having received $Y = y$, our remaining uncertainty about what was sent is

$$H_{X|Y=y} = -\sum_{x \in \mathcal{X}} \mathbb{P}[X = x \mid Y = y] \log(\mathbb{P}[X = x \mid Y = y])$$

- Averaging over what we might receive gives the *conditional entropy*

$$H_{X|Y} = \sum_{y \in \mathcal{Y}} \mathbb{P}[Y = y]\ H_{X|Y=y} = -\sum_{x,y} \mathbb{P}[X = x, Y = y] \log(\mathbb{P}[X = x \mid Y = y])$$

2.2 Joint Entropy

- The *joint entropy* is the entropy of the pair,

$$H_{X,Y} = -\sum_{x,y} \mathbb{P}[X = x, Y = y] \log(\mathbb{P}[X = x, Y = y])$$

- If X and Y are independent then $H_{X,Y} = H_X + H_Y$ (see the previous lecture); if they are dependent, $H_{X,Y} < H_X + H_Y$
- Taking logs of $\mathbb{P}[X, Y] = \mathbb{P}[Y \mid X]\ \mathbb{P}[X] = \mathbb{P}[X \mid Y]\ \mathbb{P}[Y]$ and averaging gives the *chain rule* for entropy

$$H_{X,Y} = H_X + H_{Y|X} = H_Y + H_{X|Y}$$

and hence $H_X - H_{X|Y} = H_Y - H_{Y|X}$

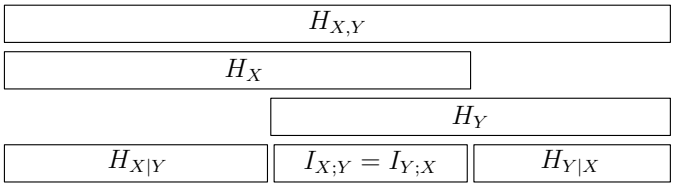
2.3 Mutual Information

- Before receiving anything our uncertainty about X is H_X. Shannon defined the *mutual information* as the expected reduction in that uncertainty when we receive the message:

$$I_{X;Y} = H_X - H_{X|Y}$$

- By the chain rule result above, $I_{X;Y} = I_{Y;X}$: Y tells us as much about X as X tells us about Y

- The relationships between these quantities are summarised in the diagram below



- The *capacity* of a channel is the largest information gain we can achieve by choosing the distribution of messages,

$$C = \max_{\mathbb{P}[X]} I_{X;Y}$$

- Mutual information is widely used in machine learning; for example, two sets of images can be aligned by choosing the non-linear transformation that maximises the (normalised) mutual information between them

2.4 The Binary Symmetric Channel

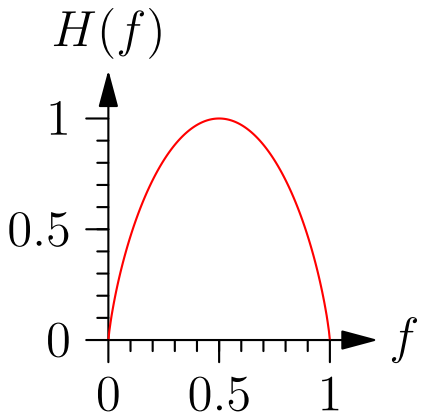
- The simplest noisy channel sends bits, each flipped independently with probability f:

$$\mathbb{P}[Y = 1 \mid X = 0] = \mathbb{P}[Y = 0 \mid X = 1] = f$$

- With equally likely inputs, an elementary calculation gives, per bit,

$$H_{X_i|Y_i} = -(1-f)\log_2(1-f) - f\log_2(f) = H(f)$$

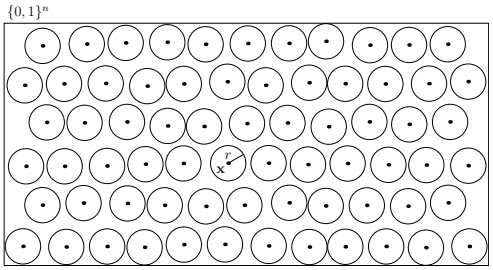
  and for a message of n bits $H_{X|Y} = n\,H(f)$



- Since $H_X = n$ bits for uniform inputs, $I_{X;Y} = n\,(1 - H(f))$. Uniform inputs are optimal here, so the capacity is $1 - H(f)$ bits per bit sent

2.5 Error Correcting Codes and Shannon's Bound

- To communicate reliably we use an error correcting code: choose code words far apart in the space of binary strings, so that a corrupted code word is still closer to the original than to any other



- A *Hamming ball* $B(\boldsymbol{x}, r)$ is the set of strings that differ from $\boldsymbol{x}$ in at most r positions
- A string of length n suffers about $n\,f$ errors, and for large n almost never more than $(f+\epsilon)\,n$. So if each code word owns a ball of radius $r = (f+\epsilon)\,n$, and the balls don't overlap, errors can be corrected
- By Stirling's approximation a ball of radius $f\,n$ contains $\binom{n}{fn} \approx 2^{nH(f)}$ strings, so at most

$$\frac{2^n}{2^{nH(f)}} = 2^{n(1-H(f))}$$

  non-overlapping balls fit. We can therefore send at most $n\,(1-H(f))$ bits of information in n bits: the rate cannot exceed the capacity
- Shannon also showed that choosing $2^{nI_{X;Y}}$ code words at random keeps them far enough apart with high probability, so this rate is *achievable*
- **Example.** For $f = 0.1$, $H(0.1) \approx 0.469$ bits, so the capacity is $C = 1 - 0.469 \approx 0.531$ bits. Reliable communication needs codes about $1/0.531 \approx 1.9$ times as long as the message—just under twice as long—for a channel that corrupts 10% of bits
- Random codes cannot be decoded efficiently, so Shannon's result told us the bound was achievable without telling us how. Practical codes that come close to it (such as turbo and LDPC codes) took decades more to find

2.6 KL-Divergence

- The *Kullback–Leibler divergence* is

$$\mathrm{KL}\,(p \,\|\, q) = \mathbb{E}_{X\sim p}\left[\log\left(\frac{p(X)}{q(X)}\right)\right] = \mathbb{E}_{X\sim p}[-\log(q(X))] - H_p$$

- $-\log(q(x))$ is the code length for x in the optimal code for the distribution q. So

$$\mathbb{E}_{X\sim p}[-\log(q(X))]$$

  is the expected message length when data from p is coded as if it came from q, and $\mathrm{KL}\,(p \,\|\, q)$ is the expected *extra* length paid for using the wrong distribution
- It is also called the *relative entropy*. It is never negative, is zero only when $p = q$, and is not symmetric: $\mathrm{KL}\,(p \,\|\, q) \neq \mathrm{KL}\,(q \,\|\, p)$ in general

2.7 Cross-Entropy Loss

- A classifier gives probabilities $q(y \mid \boldsymbol{x})$ and is usually trained by minimising the *cross-entropy*

$$\mathcal{L} = -\frac{1}{m}\sum_{k=1}^{m} \log(q(y_k \mid \boldsymbol{x}_k))$$

  which is the average code length for the labels using the model's probabilities

- If, at a given $\boldsymbol{x}$, the labels really come from $p(y \mid \boldsymbol{x})$,

$$\mathbb{E}_{Y\sim p}[-\log(q(Y \mid \boldsymbol{x}))] = H_p + \mathrm{KL}\,(p \,\|\, q)$$

  (add and subtract $\log p$). The entropy H_p does not depend on the model, so minimising the cross-entropy minimises the KL-divergence from the data to the model

2.8 Variational Approximation

- In Bayesian inference the posterior $f(\boldsymbol{\theta} \mid \mathcal{D})$ is often too complicated to write down, which is why we used MCMC

- The *variational approximation* instead fits a simpler distribution, typically factorised,

$$f(\boldsymbol{\theta} \mid \mathcal{D}) \approx g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) = \prod_i g(\theta_i \mid \phi_i)$$

  by maximising the *evidence lower bound* (ELBO; it is minus the *variational free energy* of physics)

$$\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \, \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta}, \mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta}$$

2.9 The Evidence Lower Bound (ELBO)

- Writing $f(\boldsymbol{\theta}, \mathcal{D}) = f(\boldsymbol{\theta} \mid \mathcal{D})\, f(\mathcal{D})$,

$$\begin{aligned}\Phi(\boldsymbol{\phi}) &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \left(\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta} \mid \mathcal{D})}\right) - \log(f(\mathcal{D}))\right) \mathrm{d}\boldsymbol{\theta} \\ &= \log(f(\mathcal{D})) - \mathrm{KL}\,(g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \,\|\, f(\boldsymbol{\theta} \mid \mathcal{D}))\end{aligned}$$

- Because the KL-divergence is non-negative, $\Phi(\boldsymbol{\phi}) \le \log(f(\mathcal{D}))$: Φ is a lower bound on the log evidence, hence the name ELBO

- Maximising Φ minimises the KL-divergence, making g close to the true posterior and $\Phi \approx \log(f(\mathcal{D}))$

- Equivalently $\Phi(\boldsymbol{\phi}) = L_g(\boldsymbol{\phi}) + H_g(\boldsymbol{\phi})$ with

$$L_g(\boldsymbol{\phi}) = \int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \, \log(f(\mathcal{D} \mid \boldsymbol{\theta})\, f(\boldsymbol{\theta})) \, \mathrm{d}\boldsymbol{\theta} \qquad H_g(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \, \log(g(\boldsymbol{\theta} \mid \boldsymbol{\phi})) \, \mathrm{d}\boldsymbol{\theta}$$

  The first term rewards putting g where the joint density is large (compare MAP); the entropy term rewards keeping g as spread out as possible

- Variational methods are usually much faster than MCMC and can be very accurate, but the quality is not guaranteed. MCMC is a controlled approximation that improves with more iterations; its slowness is rarely a problem on modern computers

- Minimising the divergence the other way round, $\mathrm{KL}\,(f \,\|\, g)$, leads to a different family of methods (expectation propagation)

2.10 Minimum Description Length

- Outside the Bayesian framework, choosing between models of different complexity is hard. A validation set is the usual answer, but is not always possible
- A principled alternative: prefer the model that *compresses the data most*. If a model compresses the data it has captured real structure
- **Alice and Bob.** Alice has data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) \mid i = 1, \ldots, m\}$; Bob has only the inputs $\boldsymbol{x}_i$. Both have a model $\hat{f}(\boldsymbol{x} \mid \boldsymbol{\theta})$. Rather than sending all the y_i, Alice sends the parameters $\boldsymbol{\theta}$ and the errors $\delta_i = y_i - \hat{f}(\boldsymbol{x}_i \mid \boldsymbol{\theta})$
- If the errors have density p_δ, sending one error to accuracy Δ costs $-\log(p_\delta(\delta_i)\,\Delta)$. The *description length* is

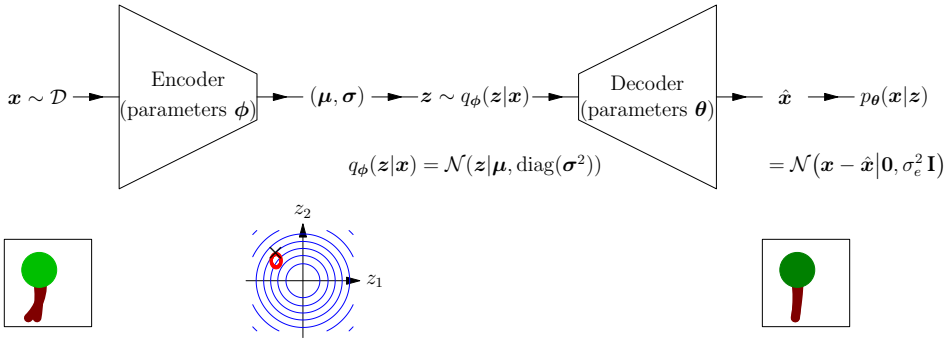
$$L = \sum_{k=1}^{n} \ell(\theta_k) - \sum_{i=1}^{m} \left(\log\Big(p_\delta\Big(y_i - \hat{f}(\boldsymbol{x}_i \mid \boldsymbol{\theta}) \Big) \Big) + \log(\Delta) \right)$$

  where $\ell(\theta_k)$ is the cost of sending parameter θ_k, to whatever precision is worthwhile
- The *minimum description length* (MDL) principle chooses the model with the smallest L. The $-m \log(\Delta)$ term is the same for every model and does not affect the choice
- For Gaussian errors $-\log(p_\delta(\delta))$ is the squared error plus a constant, so MDL is least squares plus a penalty for model complexity
- MDL trades accuracy against complexity and is often the only principled method available. It can be fiddly because we must decide how precisely to store the parameters, though good models often survive surprisingly low precision

2.11 Variational Auto-Encoders

- A variational auto-encoder (VAE) encodes an input $\boldsymbol{x}$ into a distribution $q_{\boldsymbol{\theta}}(\boldsymbol{z} \mid \boldsymbol{x})$ over a latent code $\boldsymbol{z}$ and decodes $\boldsymbol{z}$ back to a reconstruction. Its loss is

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x} \sim \mathcal{D}}[\mathrm{KL}\left(q_{\boldsymbol{\theta}}(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\right) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{z}(\boldsymbol{x})))]$$



- The original paper derived this as a variational lower bound on the likelihood of the data: the loss is minus an ELBO, with $q_{\boldsymbol{\theta}}(\boldsymbol{z} \mid \boldsymbol{x})$ as the variational distribution. That derivation is hard to follow. The loss has a much more natural reading as a description length:
  - Alice encodes each image into a latent code $q_{\boldsymbol{\theta}}(\boldsymbol{z} \mid \boldsymbol{x})$ and sends it, at a cost of

$$\mathrm{KL}\left(q_{\boldsymbol{\theta}}(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\right)$$

    (strictly, this is the cost with "bits-back" coding, Hinton and van Camp 1993: the randomness in the sampled code can carry other information, which is refunded)

- she also sends the errors needed to repair Bob's reconstruction, at a cost of

$$-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x} \mid \boldsymbol{z}(\boldsymbol{x})))$$

- Minimising the loss minimises the total description length. The clever part is that the network also chooses how precisely to send the code, trading code cost against repair cost

3 Exercises

3.1 The binary symmetric channel

- Verify that with equally likely inputs $H_{X_i|Y_i} = H(f)$, and hence that $I_{X_i;Y_i} = 1 - H(f)$ bits. Evaluate the capacity for $f = 0.1$ and $f = 0.5$, and explain the second answer

3.2 Mutual information from entropies

- Use the chain rule for entropy to show that $I_{X;Y} = H_X + H_Y - H_{X,Y}$

3.3 The KL-divergence is not symmetric

- For two coins with heads probabilities $p = 0.5$ and $q = 0.9$, compute $\mathrm{KL}\,(p \,\|\, q)$ and $\mathrm{KL}\,(q \,\|\, p)$ in bits

3.4 KL-divergence between normals (harder)

- Show that

$$\mathrm{KL}\left(\mathcal{N}(\mu_1, \sigma_1^2) \,\|\, \mathcal{N}(\mu_2, \sigma_2^2)\right) = \log\left(\frac{\sigma_2}{\sigma_1}\right) + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\,\sigma_2^2} - \frac{1}{2}$$

  in nats

4 Answers

4.1 The binary symmetric channel

- With uniform inputs and symmetric noise, Bayes' rule gives $\mathbb{P}[X = y \mid Y = y] = 1 - f$ and $\mathbb{P}[X \neq y \mid Y = y] = f$ whichever y is received, so $H_{X_i|Y=y} = H(f)$ for both y, and the average is $H(f)$
- $H_{X_i} = 1$ bit, so $I_{X_i;Y_i} = 1 - H(f)$
- $f = 0.1$: $C \approx 1 - 0.469 = 0.531$ bits. $f = 0.5$: $H(0.5) = 1$, so $C = 0$. When each bit is flipped with probability one half the output is independent of the input, and nothing gets through

4.2 Mutual information from entropies

- From $H_{X,Y} = H_Y + H_{X|Y}$ we have $H_{X|Y} = H_{X,Y} - H_Y$, so

$$I_{X;Y} = H_X - H_{X|Y} = H_X + H_Y - H_{X,Y}$$

4.3 The KL-divergence is not symmetric

- $\mathrm{KL}\,(p \,\|\, q) = 0.5 \log_2(0.5/0.9) + 0.5 \log_2(0.5/0.1) \approx 0.737$ bits
- $\mathrm{KL}\,(q \,\|\, p) = 0.9 \log_2(0.9/0.5) + 0.1 \log_2(0.1/0.5) \approx 0.531$ bits

4.4 KL-divergence between normals (harder)

- With $p = \mathcal{N}(\mu_1, \sigma_1^2)$ and $q = \mathcal{N}(\mu_2, \sigma_2^2)$,

$$\log\left(\frac{p(x)}{q(x)}\right) = \log\left(\frac{\sigma_2}{\sigma_1}\right) - \frac{(x-\mu_1)^2}{2\,\sigma_1^2} + \frac{(x-\mu_2)^2}{2\,\sigma_2^2}$$

- Take the expectation over $X \sim p$. $\mathbb{E}\left[(X-\mu_1)^2\right] = \sigma_1^2$, and writing $X - \mu_2 = (X-\mu_1) + (\mu_1 - \mu_2)$, $\mathbb{E}\left[(X-\mu_2)^2\right] = \sigma_1^2 + (\mu_1-\mu_2)^2$ because the cross term has zero mean

- Hence

$$\mathrm{KL}\left(p \,\|\, q\right) = \log\left(\frac{\sigma_2}{\sigma_1}\right) - \frac{1}{2} + \frac{\sigma_1^2 + (\mu_1-\mu_2)^2}{2\,\sigma_2^2}$$

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 27: Optimal Transport and the Wasserstein Distance

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Wasserstein distance, optimal transport, duality, Lipschitz functions, WGANs, Sinkhorn, FID

2 Main Points

2.1 Comparing Distributions

- Many machine learning tasks need a measure of how different two probability distributions are, e.g. to make the samples of a generative model look like the data
- The most common is the KL-divergence

$$\mathrm{KL}(p\|q) = \int p(\boldsymbol{x})\, \log\left(\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\right)\, \mathrm{d}\boldsymbol{x}$$

  - it is non-negative and zero only when $p = q$
  - it is not a distance (it is not symmetric and does not satisfy the triangular inequality), but that is not the main problem
  - if $q(\boldsymbol{x}) = 0$ where $p(\boldsymbol{x}) \neq 0$ it is infinite, so two distributions that look very similar can have a huge (or infinite) KL-divergence
- Real data such as images lie close to a thin, low-dimensional manifold in a very high-dimensional space. Two such distributions can be close without overlapping at all, and then the KL-divergence tells us nothing useful

2.2 Optimal Transport

- The *Wasserstein* or *Earth Mover's* distance is the least total work needed to move the probability mass of p so that it becomes q, where work is mass times the distance it is moved
- A *transportation policy* $\gamma(\boldsymbol{x}, \boldsymbol{y}) \geq 0$ says how much mass moves from $\boldsymbol{x}$ to $\boldsymbol{y}$. Everything that leaves $\boldsymbol{x}$ must add up to $p(\boldsymbol{x})$, and everything that arrives at $\boldsymbol{y}$ to $q(\boldsymbol{y})$:

$$\int \gamma(\boldsymbol{x}, \boldsymbol{y})\, \mathrm{d}\boldsymbol{y} = p(\boldsymbol{x}), \qquad \int \gamma(\boldsymbol{x}, \boldsymbol{y})\, \mathrm{d}\boldsymbol{x} = q(\boldsymbol{y})$$

  so γ is a joint distribution with marginals p and q; we call the set of these $\Lambda(p, q)$
- With a distance $d(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|$ the cost of a policy is $C(\gamma) = \mathbb{E}_\gamma[d(\boldsymbol{x}, \boldsymbol{y})]$ and

$$W(p, q) = \min_{\gamma \in \Lambda(p,q)} \mathbb{E}_\gamma[d(\boldsymbol{x}, \boldsymbol{y})]$$

- More generally, with cost d^k, $W_k(p,q) = \big(\min_\gamma \mathbb{E}_\gamma\big[d(\boldsymbol{x},\boldsymbol{y})^k\big]\big)^{1/k}$. The lecture's W is W_1; FID (below) uses W_2
- **One dimension.** Here there is a closed form in terms of the cumulative distribution functions,
$$W_1(p,q) = \int_{-\infty}^{\infty} \big|F_p(x) - F_q(x)\big|\,\mathrm{d}x$$
the area between the two curves. For two samples of the same size, sort both and match them in order: $W_1 = \frac{1}{n}\sum_i |x_{(i)} - y_{(i)}|$. In more than one dimension there is no such shortcut

2.3 Duality

- For discrete distributions the unknowns are the numbers $\gamma(\boldsymbol{x}_i,\boldsymbol{y}_j) \geq 0$, the objective $\sum_{i,j} d(\boldsymbol{x}_i,\boldsymbol{y}_j)\,\gamma(\boldsymbol{x}_i,\boldsymbol{y}$ is linear and so are the constraints: it is a linear programme, like those in the constrained-optimisation lecture
- With Lagrange multipliers $\alpha(\boldsymbol{x}_i)$ and $\beta(\boldsymbol{y}_j)$ for the two sets of constraints the Lagrangian rearranges to
$$\mathcal{L} = \sum_i \alpha(\boldsymbol{x}_i)\,p(\boldsymbol{x}_i) + \sum_j \beta(\boldsymbol{y}_j)\,q(\boldsymbol{y}_j) - \sum_{i,j} \gamma(\boldsymbol{x}_i,\boldsymbol{y}_j)\big(\alpha(\boldsymbol{x}_i) + \beta(\boldsymbol{y}_j) - d(\boldsymbol{x}_i,\boldsymbol{y}_j)\big)$$
Because $\gamma \geq 0$, the dual problem is to maximise $\sum_i \alpha(\boldsymbol{x}_i)\,p(\boldsymbol{x}_i) + \sum_j \beta(\boldsymbol{y}_j)\,q(\boldsymbol{y}_j)$ subject to $\alpha(\boldsymbol{x}_i) + \beta(\boldsymbol{y}_j) \leq d(\boldsymbol{x}_i,\boldsymbol{y}_j)$ for all i, j. For continuous distributions the sums become integrals and α, β become functions
- **Eliminating** β. We need $\alpha(\boldsymbol{x}) + \beta(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y})$ for all $\boldsymbol{x}$, $\boldsymbol{y}$, and want β as large as possible. Taking $\boldsymbol{x} = \boldsymbol{y}$, where $d(\boldsymbol{y},\boldsymbol{y}) = 0$, gives $\beta(\boldsymbol{y}) \leq -\alpha(\boldsymbol{y})$: so $-\alpha$ is the largest β could possibly be
- Try that largest value, $\beta = -\alpha$. The constraint becomes
$$\alpha(\boldsymbol{x}) - \alpha(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y}) \quad \text{for all } \boldsymbol{x},\boldsymbol{y}$$
which says α is *1-Lipschitz*. For any such α, $\beta = -\alpha$ satisfies every constraint, and no other β can do better because $\beta \leq -\alpha$ always
- We lose nothing by restricting to 1-Lipschitz α (*Kantorovich–Rubinstein*). Take any feasible pair (α,β) and let $f(\boldsymbol{x}) = \min_{\boldsymbol{y}}\big(d(\boldsymbol{x},\boldsymbol{y}) - \beta(\boldsymbol{y})\big)$. The constraint gives $f \geq \alpha$, so (f,β) is still feasible and the objective is no smaller. f is a minimum of functions that are each 1-Lipschitz in $\boldsymbol{x}$ (by the triangular inequality), so f is 1-Lipschitz, and by the previous point we can then replace β by $-f$ without making the objective smaller
- This gives the *Kantorovich–Rubinstein* duality
$$W(p,q) = \max_{\alpha\ \text{1-Lipschitz}} \Big(\mathbb{E}_p[\alpha(\boldsymbol{X})] - \mathbb{E}_q[\alpha(\boldsymbol{X})]\Big)$$
Instead of the huge object $\gamma(\boldsymbol{x},\boldsymbol{y})$ we need one function $\alpha(\boldsymbol{x})$, and the expectations can be estimated from samples
- For differentiable α, being 1-Lipschitz is the same as $\|\boldsymbol{\nabla}\alpha(\boldsymbol{x})\| \leq 1$ everywhere: the function can change no faster than its input

2.4 Applications

- **Wasserstein GANs.** A GAN has a generator $G(\boldsymbol{z}, \boldsymbol{w}_G)$ that turns noise $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ into images, and a discriminator trained to tell them from real images. GANs are hard to train: a discriminator that is too good gives the generator no useful gradient. A Wasserstein GAN trains the generator to minimise W between the data distribution and the generator's distribution,

  $$\min_{\boldsymbol{w}_G} \max_{\boldsymbol{w}_\alpha} \Big(\frac{1}{|\mathcal{B}|} \sum_{\boldsymbol{x} \in \mathcal{B}} \alpha(\boldsymbol{x}, \boldsymbol{w}_\alpha) - \frac{1}{n} \sum_{i=1}^{n} \alpha(G(\boldsymbol{z}_i, \boldsymbol{w}_G), \boldsymbol{w}_\alpha) \Big)$$

  with expectations estimated from a minibatch $\mathcal{B}$ of real images and n generated ones

  - the *critic* α replaces the discriminator; it still gives a gradient when the distributions do not overlap
  - it should be 1-Lipschitz, which in practice is only roughly enforced: the original WGAN clipped the weights, WGAN-GP penalises $\|\nabla \alpha\|$ away from 1, and spectral normalisation scales each weight matrix to spectral norm 1 (with 1-Lipschitz activation functions the network is then at most 1-Lipschitz)

- **Sinkhorn.** Solving the linear programme for n points costs roughly $O(n^3)$. Cuturi (2013) added an entropy term,

  $$\min_{\gamma} \sum_{i,j} d_{ij}\, \gamma_{ij} + \epsilon \sum_{i,j} \gamma_{ij} \log \gamma_{ij}$$

  with the same constraints. The solution has the form $\gamma_{ij} = u_i\, K_{ij}\, v_j$ with $K_{ij} = \mathrm{e}^{-d_{ij}/\epsilon}$ (Exercise), and $\boldsymbol{u}$, $\boldsymbol{v}$ are found by alternately rescaling rows and columns until the marginals are right,

  $$\boldsymbol{v} \leftarrow \boldsymbol{q} \oslash (\mathbf{K}^{\mathsf{T}} \boldsymbol{u}), \qquad \boldsymbol{u} \leftarrow \boldsymbol{p} \oslash (\mathbf{K} \boldsymbol{v})$$

  (Sinkhorn's matrix scaling algorithm; $\oslash$ is element-by-element division)

  - each step is two matrix–vector products, so it is fast on GPUs and differentiable
  - as $\epsilon \to 0$ the cost approaches the exact transport cost; larger ϵ converges faster but blurs the plan (see the experiment)

- **FID.** For Gaussians, W_2 has a closed form

  $$W_2^2 = \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2 + \operatorname{Tr}\Big(\boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2 - 2\big(\boldsymbol{\Sigma}_1 \boldsymbol{\Sigma}_2\big)^{1/2} \Big)$$

  The *Fr\'echet Inception Distance* (Heusel et al., 2017) fits Gaussians to a pretrained network's features of real and of generated images and reports this quantity. It is the standard score for image generators, GANs and diffusion models alike (lower is better), though it only compares the means and covariances of the features

3 Exercises

3.1 Disjoint supports

- p puts probability $\frac{1}{2}$ on each of $x = 0$ and $x = 2$; q puts probability 1 on $x = 1$. Find $W_1(p, q)$, $\mathrm{KL}(p \| q)$ and $\mathrm{KL}(q \| p)$.

3.2 Shifting a distribution

- Let p be uniform on $[0, 1]$ and q uniform on $[a, 1 + a]$ with $0 < a < 1$. Use the one-dimensional formula to find $W_1(p, q)$. What is $\mathrm{KL}(p \| q)$?

3.3 FID in one dimension

- Show that for one-dimensional Gaussians the W_2^2 formula reduces to $(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2$.

3.4 The Sinkhorn solution

- Write the Lagrangian of the entropic problem with multipliers α_i and β_j for the two sets of constraints. Set its derivative with respect to γ_{ij} to zero and show $\gamma_{ij} = u_i \, \mathrm{e}^{-d_{ij}/\epsilon} \, v_j$. What are u_i and v_j?

4 Experiments

4.1 Exact transport, sorting and Sinkhorn

- For two samples on a line, compare the linear programme, the sort-and-match formula and Sinkhorn with several ϵ

```
import numpy as np
from scipy.optimize import linprog

rng = np.random.default_rng(0)
n = 8
x = np.sort(rng.normal(0, 1, n)); y = np.sort(rng.normal(1, 0.5, n))
p = np.full(n, 1/n); q = np.full(n, 1/n)
D = np.abs(x[:, None] - y[None, :])                  # cost |x - y|

# exact optimal transport as a linear programme
A = np.vstack([np.kron(np.eye(n), np.ones(n)), np.kron(np.ones(n), np.eye(n))])
res = linprog(D.ravel(), A_eq=A, b_eq=np.r_[p, q], bounds=(0, None))
print(f"LP:               W1 = {res.fun:.4f}")

# one dimension, equal weights: sort both samples and match in order
print(f"sort and match:   W1 = {np.mean(np.abs(x - y)):.4f}")

# Sinkhorn: entropic OT, gamma = diag(u) K diag(v)
for eps in [0.5, 0.1, 0.02]:
    K = np.exp(-D / eps); u = np.ones(n)
    for _ in range(5000):
        v = q / (K.T @ u)
        u = p / (K @ v)
    G = u[:, None] * K * v[None, :]
    print(f"Sinkhorn eps={eps:4}: cost = {np.sum(G * D):.4f}  (marginal error {np.abs(G.sum
```

- The linear programme and sorting both give $W_1 = 0.2909$. Sinkhorn gives 0.436, 0.312 and 0.293 for $\epsilon = 0.5$, 0.1 and 0.02: it approaches the exact cost as ϵ shrinks, with the marginals matched exactly

5 Answers

5.1 Disjoint supports

- Each half unit of mass moves a distance 1, so $W_1 = 1$
- $\mathrm{KL}(p\|q) = \infty$, because $q(0) = q(2) = 0$ where $p > 0$; $\mathrm{KL}(q\|p) = \infty$ too, because $p(1) = 0$. The distributions are close but have no overlap, which is exactly where the Wasserstein distance is useful

5.2 Shifting a distribution

- $F_p(x) - F_q(x) = x$ on $[0, a]$, a on $[a, 1]$ and $1 + a - x$ on $[1, 1 + a]$, so $W_1 = \frac{a^2}{2} + a(1 - a) + \frac{a^2}{2} = a$: the distance the mass was shifted
- $\mathrm{KL}(p\|q) = \infty$, since $q = 0$ on $[0, a)$ where $p > 0$

5.3 FID in one dimension

- $\mathrm{Tr}(\sigma_1^2 + \sigma_2^2 - 2(\sigma_1^2\sigma_2^2)^{1/2}) = \sigma_1^2 + \sigma_2^2 - 2\sigma_1\sigma_2 = (\sigma_1 - \sigma_2)^2$

5.4 The Sinkhorn solution

- The Lagrangian is

$$\mathcal{L} = \sum_{ij} d_{ij}\gamma_{ij} + \epsilon \sum_{ij} \gamma_{ij} \log \gamma_{ij} - \sum_i \alpha_i \Big(\sum_j \gamma_{ij} - p_i \Big) - \sum_j \beta_j \Big(\sum_i \gamma_{ij} - q_j \Big)$$

- $\partial\mathcal{L}/\partial\gamma_{ij} = d_{ij} + \epsilon(\log \gamma_{ij} + 1) - \alpha_i - \beta_j = 0$, so $\gamma_{ij} = \mathrm{e}^{(\alpha_i - \epsilon/2)/\epsilon}\, \mathrm{e}^{-d_{ij}/\epsilon}\, \mathrm{e}^{(\beta_j - \epsilon/2)/\epsilon}$: $u_i = \mathrm{e}^{\alpha_i/\epsilon - 1/2}$ and $v_j = \mathrm{e}^{\beta_j/\epsilon - 1/2}$ (any split of the constant e^{-1} between them will do). The entropy term keeps every $\gamma_{ij} > 0$, so the constraint $\gamma \geq 0$ never binds

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 28: When Machine Learning Works

Adam Prugel-Bennett

November 2, 2026

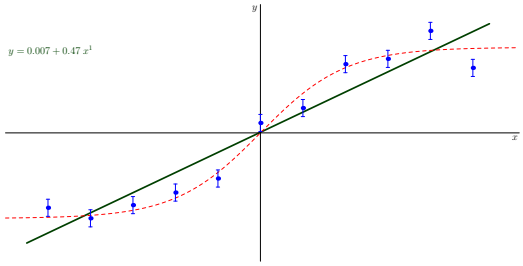
1 Keywords

- When ML Works, Bias Variance, Shrinkage, Ridge Regression, Ensembles

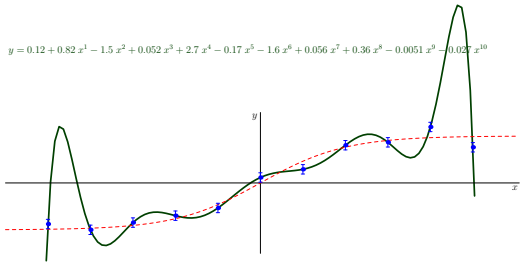
2 Main Points

2.1 Generalisation

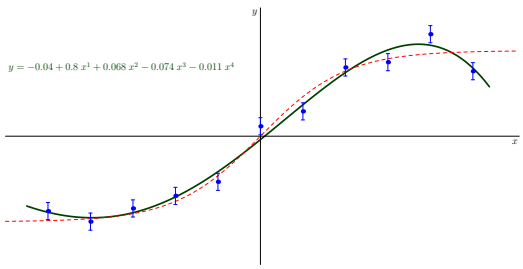
- We train our learning machines on a finite data set
- But we use our learning machines on unseen data
- If we have a too simple machine we might not be able to fit the training data and are unlikely to do well on unseen data



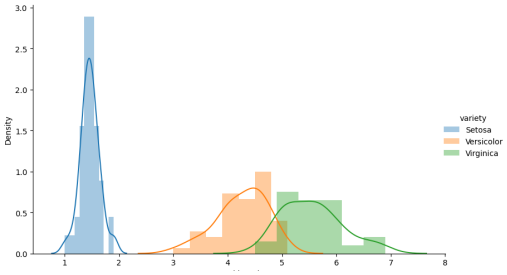
- If we have a too complicated machine we might be able to fit the training data almost perfectly, but we might have learnt a too complex rule that doesn't fit the test set



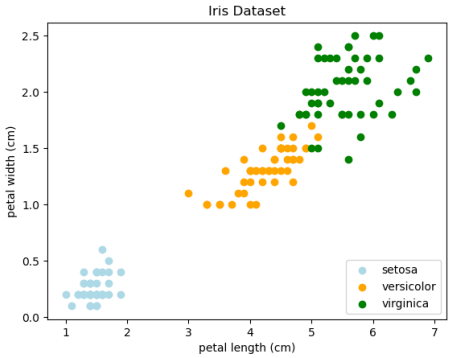
- Often there is a good compromise so that the learning machine learns a simple rule that fits the training data quite well but isn't too complicated



- We can think of the features and labels coming from an underlying distribution $\mu(\boldsymbol{x}, y)$. For example, in the famous iris dataset we have four features (petal/sepal length/width). We imagine there is some probability density function for each feature. The petal length might be distributed as shown



- The features are actually jointly distributed (in 4 "dimensions"). E.g. the petal length and petal width are quite correlated



- An assumption we make (which has to be approximately correct for machine learning to have a chance of working) is that the testing data (where we are going to use the machine) is similar to the training data. Mathematically we can model this by assuming they both come from the same underlying distribution, $\mu(\boldsymbol{x}, y)$

2.2 Bias-Variance Dilemma

- We draw training sets $\mathcal{D}$ of m independent examples from $\mu(\boldsymbol{x}, y)$ and train a machine $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}})$ on each
- Its generalisation loss is

$$L_G(\boldsymbol{\theta}_{\mathcal{D}}) = \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x}, y) \left(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}}) - y\right)^2$$

  For any particular data set we might do better or worse than the expected generalisation loss $\bar{L}_G = \mathbb{E}_{\mathcal{D}}[L_G(\boldsymbol{\theta}_{\mathcal{D}})]$

- The *mean machine* averages the predictions of machines trained on all possible data sets (a thought experiment, not something practical)

$$\hat{f}_m(\boldsymbol{x}) = \mathbb{E}_{\mathcal{D}}\left[\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}})\right]$$

- Adding and subtracting $\hat{f}_m(\boldsymbol{x})$,

$$\begin{aligned}\bar{L}_G &= \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x}, y)\, \mathbb{E}_{\mathcal{D}}\left[\left(\left(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}}) - \hat{f}_m(\boldsymbol{x})\right) + \left(\hat{f}_m(\boldsymbol{x}) - y\right)\right)^2\right] \\ &= \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x}, y)\, \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}}) - \hat{f}_m(\boldsymbol{x})\right)^2\right] + \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x}, y)\left(\hat{f}_m(\boldsymbol{x}) - y\right)^2 = V + B\end{aligned}$$

- The cross term vanishes because $\mathbb{E}_{\mathcal{D}}\left[\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}}) - \hat{f}_m(\boldsymbol{x})\right] = 0$, while $\hat{f}_m(\boldsymbol{x}) - y$ does not depend on the data set
- We used $\mathbb{E}[A+B] = \mathbb{E}[A] + \mathbb{E}[B]$ and $\mathbb{E}[c\,A] = c\,\mathbb{E}[A]$ for c not depending on the random variable

- The **bias** B is the generalisation loss of the mean machine; the **variance** V measures how much a single machine varies with the data set
- **Where is the noise?** If $y = f^*(\boldsymbol{x}) + \epsilon$ with $\mathbb{E}[\epsilon] = 0$ and $\mathrm{Var}(\epsilon) = \sigma^2$, the same trick (the cross term again vanishes) gives

$$B = \sigma^2 + \sum_{\boldsymbol{x}} \mu(\boldsymbol{x}) \big(\hat{f}_m(\boldsymbol{x}) - f^*(\boldsymbol{x})\big)^2$$

  so $\bar{L}_G = \text{noise} + \text{bias}^2 + \text{variance}$, the form often seen elsewhere. The noise is the same for every machine; here we keep it inside B
- The **Bias-Variance Dilemma** is that
  - simple machines are likely to have high bias: they cannot represent the data well, so neither can their mean
  - complex machines are likely to have high variance: they are sensitive to the particular training set
  - this is a tendency, not a law: the average of many decision trees can represent a far more complex boundary than a single tree

2.3 Shrinkage

- Estimate the mean θ of a distribution with variance σ^2 from m samples. The sample mean $\bar{x}$ is unbiased with variance σ^2/m
- The deliberately biased estimate $\hat{\theta} = c\,\bar{x}$, $0 \le c \le 1$, has $\mathbb{E}\left[\hat{\theta}\right] = c\,\theta$ and $\mathrm{Var}(\hat{\theta}) = c^2\sigma^2/m$, so

$$\mathbb{E}\left[\big(\hat{\theta} - \theta\big)^2\right] = (1-c)^2\,\theta^2 + c^2\,\frac{\sigma^2}{m}$$

- Its derivative with respect to c at $c = 1$ is $2\sigma^2/m > 0$, so shrinking a little always reduces the error: a biased estimate beats the unbiased one. The best c depends on the unknown θ, so in practice it is chosen by validation

2.4 Ridge Regression Through the SVD

- Data $\boldsymbol{y} = \mathbf{X}\boldsymbol{w}^* + \boldsymbol{\epsilon}$ with independent noise of variance σ^2 on each target. Ridge regression minimises $\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2 + \eta\|\boldsymbol{w}\|^2$, giving $\hat{\boldsymbol{w}} = (\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$
- With the SVD $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$ (columns $\boldsymbol{u}_k$, $\boldsymbol{v}_k$, singular values s_k), $\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I} = \mathbf{V}(\mathbf{S}^2 + \eta\mathbf{I})\mathbf{V}^\mathsf{T}$, so in the basis of the $\boldsymbol{v}_k$ each component separates. Writing $w_k^* = \boldsymbol{v}_k^\mathsf{T}\boldsymbol{w}^*$ for the component of $\boldsymbol{w}^*$ along $\boldsymbol{v}_k$ (not its \$k\$-th coordinate), and using $\boldsymbol{u}_k^\mathsf{T}\mathbf{X}\boldsymbol{w}^* = \sum_j s_j(\boldsymbol{u}_k^\mathsf{T}\boldsymbol{u}_j)(\boldsymbol{v}_j^\mathsf{T}\boldsymbol{w}^*) = s_k w_k^*$:

$$\hat{w}_k = \frac{s_k\,\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}}{s_k^2 + \eta}, \qquad \boldsymbol{u}_k^\mathsf{T}\boldsymbol{y} = s_k\,w_k^* + \boldsymbol{u}_k^\mathsf{T}\boldsymbol{\epsilon}$$

  and $\boldsymbol{u}_k^\mathsf{T}\boldsymbol{\epsilon}$ has mean 0 and variance σ^2

- Hence

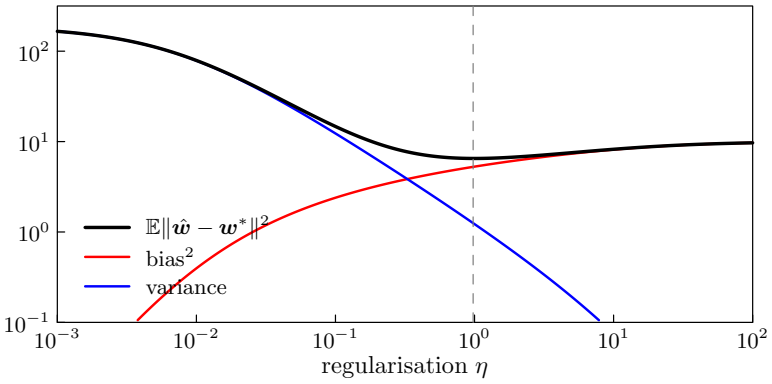
$$\mathbb{E}[\hat{w}_k] = \frac{s_k^2}{s_k^2+\eta}\, w_k^*, \qquad \mathrm{Var}(\hat{w}_k) = \frac{\sigma^2 s_k^2}{(s_k^2+\eta)^2}$$

and

$$\mathbb{E}\left[\|\hat{\boldsymbol{w}} - \boldsymbol{w}^*\|^2\right] = \sum_k \left(\frac{\eta}{s_k^2+\eta}\right)^2 w_k^{*2} + \frac{\sigma^2 s_k^2}{(s_k^2+\eta)^2}$$

For test inputs with identity covariance this is the expected test error minus the noise

- With $\eta = 0$ (least squares) the bias is zero but the variance is $\sum_k \sigma^2/s_k^2$: small singular values, i.e. an ill-conditioned $\mathbf{X}$, make it huge. A small η shrinks exactly those directions. In the lecture's example (ten singular values from 3 to 0.1, $\sigma = 1$) least squares has error 188 and the best $\eta \approx 1$ gives 6.5



2.5 Ensembles

- An ensemble averages M predictors trained with different randomness, $\bar{f}_M(\boldsymbol{x}) = \frac{1}{M}\sum_{i=1}^M f_i(\boldsymbol{x})$. The mean, and so the bias, is unchanged
- If each $f_i(\boldsymbol{x})$ has variance s^2 and each pair has correlation ρ, then

$$\mathrm{Var}\big(\bar{f}_M(\boldsymbol{x})\big) = \frac{1}{M^2}\Big(Ms^2 + M(M-1)\rho\, s^2\Big) = \rho\, s^2 + \frac{1-\rho}{M}\, s^2$$

More members remove the second term, but correlation leaves a floor $\rho\, s^2$

- *Bagging* trains each member on a bootstrap resample of the data; *random forests* also choose each split of each tree from a random subset of the features, making each tree worse but the trees less correlated; *deep ensembles* train several networks from different random initialisations
- The spread of the members' predictions is a useful measure of uncertainty

3 Exercises

3.1 Limits of ridge regression

- Using the formula for $\mathbb{E}\left[\|\hat{\boldsymbol{w}} - \boldsymbol{w}^*\|^2\right]$, what happens to the bias and the variance as $\eta \to 0$ and as $\eta \to \infty$?

3.2 How big an ensemble?

- Members of an ensemble have pairwise correlation $\rho = 0.2$. How many members are needed for the variance of the average to be within 10% of its floor $\rho\, s^2$?

3.3 Where is the noise?

- Derive $B = \sigma^2 + \sum_{\boldsymbol{x}} \mu(\boldsymbol{x}) \big(\hat{f}_m(\boldsymbol{x}) - f^*(\boldsymbol{x})\big)^2$ when $y = f^*(\boldsymbol{x}) + \epsilon$.

4 Experiments

4.1 Bias and variance of ridge regression

- Train ridge regression on many independent training sets, estimate the mean machine and the variance, and compare with the formulas

```
import numpy as np

rng = np.random.default_rng(0)
m, p, sigma = 30, 10, 1.0
s = np.geomspace(3, 0.1, p)                          # singular values
U, _ = np.linalg.qr(rng.standard_normal((m, p)))
V, _ = np.linalg.qr(rng.standard_normal((p, p)))
X = U @ np.diag(s) @ V.T
w_star = V @ np.ones(p)

for eta in [0.0, 0.1, 1.0, 10.0]:
    W = []
    for _ in range(5000):                            # many training sets
        y = X @ w_star + sigma * rng.standard_normal(m)
        W.append(np.linalg.solve(X.T @ X + eta * np.eye(p), X.T @ y))
    W = np.array(W)
    w_mean = W.mean(0)                               # the "mean machine"
    bias2 = ((w_mean - w_star) ** 2).sum()
    var = ((W - w_mean) ** 2).sum(1).mean()
    f = s**2 / (s**2 + eta)                          # theory, SVD basis
    print(f"eta {eta:5}: bias2 {bias2:7.3f} ({((1 - f)**2).sum():7.3f})"
          f"  variance {var:7.2f} ({(sigma**2 * f**2 / s**2).sum():7.2f})")
```

- The numbers in brackets are the formulas. (See the answers.)

5 Answers

5.1 Limits of ridge regression

- As $\eta \to 0$ the bias goes to 0 and the variance to $\sum_k \sigma^2/s_k^2$ (least squares). As $\eta \to \infty$ every component is shrunk to 0: the variance goes to 0 and the bias to $\sum_k w_k^{*2} = \|\boldsymbol{w}^*\|^2$

5.2 How big an ensemble?

- We need $(1-\rho)s^2/M \le 0.1\,\rho\,s^2$, i.e. $M \ge (1-\rho)/(0.1\,\rho) = 0.8/0.02 = 40$

5.3 Where is the noise?

- With $y = f^*(\boldsymbol{x}) + \epsilon$, $\big(\hat{f}_m(\boldsymbol{x}) - y\big)^2 = \big(\hat{f}_m(\boldsymbol{x}) - f^*(\boldsymbol{x})\big)^2 - 2\big(\hat{f}_m(\boldsymbol{x}) - f^*(\boldsymbol{x})\big)\epsilon + \epsilon^2$. Averaging over ϵ at fixed $\boldsymbol{x}$, the middle term vanishes because $\mathbb{E}[\epsilon] = 0$ and the last gives σ^2

5.4 Bias and variance of ridge regression

- The simulation gives

η	bias^2 (formula)	variance (formula)
0	0.021 (0)	190.6 (188.5)
0.1	2.44 (2.39)	12.32 (12.30)
1	5.31 (5.28)	1.22 (1.22)
10	8.15 (8.15)	0.08 (0.08)

- The simulation agrees with the formulas; the small "bias" at $\eta = 0$ is sampling noise in the estimate of the mean machine (about $190/5000 \approx 0.04$)

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 29: Over-Fitting

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Overfitting, regularisation, feature selection, early stopping, margins, double descent

2 Main Points

- Over-fitting
  - Over-fitting is when we learn **spurious rules** that explain the training set
  - If we use an infinitely flexible machine then we can't generalise at all
- Controlling Complexity
  - More training examples improve generalisation by allowing us to eliminate spurious rules
  - To achieve good generalisation performance machine must match the underlying structure of the problem
  - By preprocessing or careful feature engineering you can sometimes make the learning task easier
  - Deep learning works partly because it uses a lot of training data and partly because architectures such as convolutional networks match the structure of the data (see the next lecture)
  - We usually don't know the structure of the inputs (they are too high dimensional)
    - Try different machines to see what fits
    - Fine-tune the models (adjusting hyper-parameters)
    - Feature engineering very often helps
      - Feature selection
      - Projecting onto a few principal components (PCA)
      - Normalise your input features
    - Need a validation set to do this
    - Beware you are very likely to over-estimate your generalisation error if you use your test set to select your model
    - Can use K-fold cross-validation to get a better estimate of generalisation error
- Regularisation
  - Adding regularisation terms can help choose a simpler model
  - L_2 regularisers punish large weights making the fitting function smoother; L_1 regularisers (the lasso) can do automatic feature selection

- **Regularisers are priors.** The MAP solution maximises $\log f(\mathcal{D}|\boldsymbol{w}) + \log f(\boldsymbol{w})$, so $-\log f(\boldsymbol{w})$ acts as a regulariser. Gaussian noise of variance σ^2 with a Gaussian prior $w_i \sim \mathcal{N}(0, s^2)$ gives ridge regression with $\eta = \sigma^2/s^2$; a Laplace prior $f(w_i) \propto \mathrm{e}^{-|w_i|/b}$ gives the lasso penalty $\eta \sum_i |w_i|$
- **Early stopping is implicit ridge.** Gradient descent on $\frac{1}{2}\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$ with learning rate r is $\boldsymbol{w} \leftarrow \boldsymbol{w} - r\,\mathbf{X}^\mathsf{T}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})$. With the SVD $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$, component k of $\boldsymbol{w}$ along $\boldsymbol{v}_k$ obeys $w_k \leftarrow (1 - r s_k^2)\, w_k + r\, s_k\, \boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}$. Starting from $\boldsymbol{w} = \mathbf{0}$, after t steps

$$w_k(t) = \left(1 - (1 - r\, s_k^2)^t\right) \frac{\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}}{s_k}$$

  where $\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}/s_k$ is the least squares component (see the exercises). Ridge regression multiplies the same component by $s_k^2/(s_k^2 + \eta)$. Both factors are near 1 for large s_k and near 0 for small s_k, where the variance lives, so stopping after t steps acts roughly like ridge with $\eta \approx 1/(r\,t)$
- **Why SVMs generalise.** For data in a ball of radius R separated with margin γ, the capacity of the separator depends on $(R/\gamma)^2$ rather than on the dimension of the feature space (Vapnik), which is why the kernel trick, with enormous or infinite feature spaces, need not over-fit. Also, removing a training point that is not a support vector leaves the solution unchanged, so the leave-one-out error is at most (number of support vectors)$/m$

2.1 Double Descent

- Least squares on $m = 40$ points using the first p of 100 features: the test error shows the classical U-curve, a spike at $p = m$, and then a second descent (Figure 1)

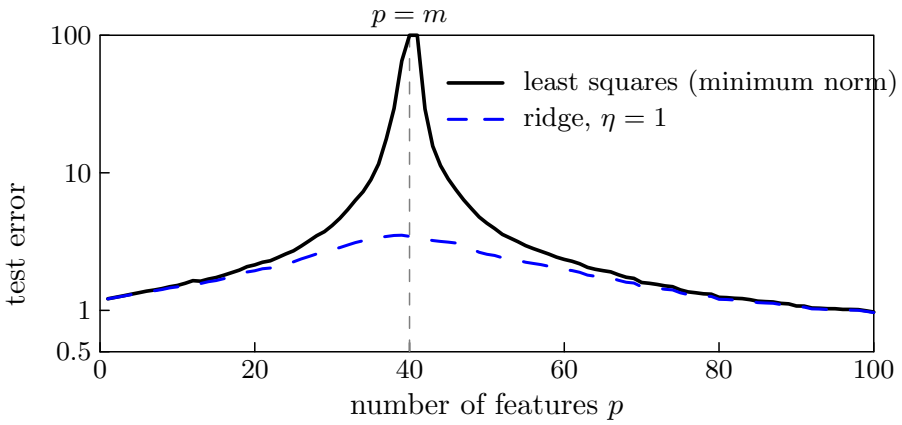


Figure 1: Test error of least squares (minimum norm for $p > m$) and of ridge regression against the number of features

- At $p = m$ there is exactly one fit through every noisy point. The smallest singular value of $\mathbf{X}$ is then close to 0, and the variance $\sum_k \sigma^2/s_k^2$ (lecture on bias and variance) explodes
- For $p > m$ there are infinitely many exact fits. We take the one with the smallest norm, $\hat{\boldsymbol{w}} = \mathbf{X}^\mathsf{T}(\mathbf{X}\mathbf{X}^\mathsf{T})^{-1}\boldsymbol{y}$
- Gradient descent from $\boldsymbol{w} = \mathbf{0}$ finds exactly this solution (see the exercises): the optimiser itself regularises
- Modern neural networks have far more parameters than training examples and still often generalise well; counting parameters is not the same as measuring complexity (Belkin et al., 2019)

3 Exercises

3.1 Estimating the error in the mean

- Consider a binary classification problem
- Suppose we measure the accuracy on a validation set of size $n = 10\,000$
- If we correctly predict $8\,000$ of the validation set what is
  1. an estimate of the mean accuracy?
  2. the accuracy of this estimate?
- State you assumptions
- What would the answer be if $n = 100$ and we got 80 correct on the validation set?
- Answer at the end

3.2 Early stopping

- Derive the formula for $w_k(t)$ above from the update $w_k \leftarrow (1 - r s_k^2)\, w_k + r\, s_k\, \boldsymbol{u}_k^\mathsf{T} \boldsymbol{y}$ with $w_k(0) = 0$. What happens as $t \to \infty$ if $0 < r\, s_k^2 < 2$?

3.3 Minimum norm

- Show that every gradient descent step on $\frac{1}{2}\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$ adds a vector of the form $\mathbf{X}^\mathsf{T}\boldsymbol{a}$. Hence, starting from $\boldsymbol{w} = \mathbf{0}$, explain why the limit for $p > m$ is the minimum-norm exact fit.

4 Experiments

4.1 Overfitting Game

- Write your own simulation (see lecture notes)
- Generate n random numbers $X_i \sim U(0, 3)$ (you can try different distributions)
- Add some noise to generate $Y_i = X_i + Z_i$ where $Z_i \sim \mathcal{N}(0, 1)$
- Choose i^* with the largest value of Y_i
- Compute $\Delta = Z_{i^*} = Y_{i^*} - X_{i^*}$
- Repeat many times and plot a histogram

4.2 Double descent

- Fit least squares (minimum norm when $p > m$) with $m = 40$ points and p of 100 features, and print the median test error over many training sets. (The mean is useless at $p = m$: the expected error is infinite there.)

```
import numpy as np

rng = np.random.default_rng(0)
D, m, sigma, trials = 100, 40, 0.5, 200
beta = rng.standard_normal(D) / np.sqrt(D)
for p in [10, 30, 38, 40, 42, 60, 100]:
    err = []
```

```
    for _ in range(trials):
        X = rng.standard_normal((m, D))
        y = X @ beta + sigma * rng.standard_normal(m)
        w = np.linalg.lstsq(X[:, :p], y, rcond=None)[0]   # min-norm if p > m
        err.append(((w - beta[:p])**2).sum() + (beta[p:]**2).sum() + sigma**2)
    print(p, round(np.median(err), 2))
```

4.3 Cross-validation

- Try using cross validation

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn import svm

X, y = datasets.load_iris(return_X_y=True)
X.shape, y.shape

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=0)

X_train.shape, y_train.shape

X_test.shape, y_test.shape


clf = svm.SVC(kernel='linear', C=1).fit(X_train, y_train)
clf.score(X_test, y_test)
```

4.4 Practical work

- There are a lot of practical tips that you can use in your project
    - Try them out and see what works and what doesn't
    - Many modifications won't make much difference, but occasionally one will make a lot of difference
    - Be very wary of estimating your generalisation performance
        - You are measuring on a finite validation set so you will have errors
        - Once you've used your validation set for selecting models, you are likely to over-estimate your generalisation performance (under-estimate the generalisation error)

5 Answers

5.1 Estimating the error in the mean

- We assume that the examples in the validation set are independent so the number of successes, k, will be binomially distributed

$$\mathrm{Bin}(k|n,p) = \binom{n}{k} p^k \left(1-p\right)^{n-k}$$

- We don't know the success probability p, but an unbiased estimate of it is the number of observed success divided by n, that is 0.8

- For a binomial the variance in k is given by $n\,p\,(1-p)$ or 1600 if $p = 0.8$ and $n = 10\,000$
- The typical size of the fluctuations in k would be $\sqrt{n\,p\,(1-p)} = 40$
- Our estimate of p would therefore typically have fluctuations of size $40/n = 0.004$
- Thus our estimated success rate is 0.8 ± 0.004 or 80% with an error of about half a percent
- This is small because we used a large validation set. If $n = 100$ and we got 80 successes then we would have an error of 0.8 ± 0.04 so we have an error of 4%.
- Note that by choosing the best of 10 machines we might expect to over-estimate the performance by around 5% in this case

5.2 Early stopping

- Let $c_k = s_k\,\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}$ and $a = 1 - r s_k^2$. Then $w_k(t) = c_k\,r\,(1 + a + \cdots + a^{t-1}) = c_k\,r\,(1-a^t)/(1-a) = (1-a^t)\,c_k/s_k^2 = (1-a^t)\,\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}/s_k$
- If $0 < r s_k^2 < 2$ then $|a| < 1$, so $a^t \to 0$ and w_k converges to the least squares component $\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}/s_k$. Small s_k means a close to 1 and slow convergence: these are the last directions to be fitted

5.3 Minimum norm

- The step is $-r\,\mathbf{X}^\mathsf{T}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})$, of the form $\mathbf{X}^\mathsf{T}\boldsymbol{a}$. Starting from $\mathbf{0}$, $\boldsymbol{w}$ always stays in the space spanned by the rows of $\mathbf{X}$
- For $p > m$ (and $\mathbf{X}\mathbf{X}^\mathsf{T}$ invertible) the limit fits the data exactly, so $\boldsymbol{w} = \mathbf{X}^\mathsf{T}\boldsymbol{a}$ with $\mathbf{X}\mathbf{X}^\mathsf{T}\boldsymbol{a} = \boldsymbol{y}$, giving $\boldsymbol{w} = \mathbf{X}^\mathsf{T}(\mathbf{X}\mathbf{X}^\mathsf{T})^{-1}\boldsymbol{y}$
- Any other exact fit is $\boldsymbol{w} + \boldsymbol{z}$ with $\mathbf{X}\boldsymbol{z} = \mathbf{0}$. Then $\boldsymbol{z}$ is orthogonal to every row of $\mathbf{X}$, hence to $\boldsymbol{w}$, so $\|\boldsymbol{w} + \boldsymbol{z}\|^2 = \|\boldsymbol{w}\|^2 + \|\boldsymbol{z}\|^2 \geq \|\boldsymbol{w}\|^2$: gradient descent finds the minimum-norm fit

5.4 Double descent

- The median test errors are about 1.47 ($p = 10$), 3.89 ($p = 30$), 14.8 ($p = 38$), 86 ($p = 40$), 16.4 ($p = 42$), 2.2 ($p = 60$) and 0.98 ($p = 100$): a peak at $p = m$ and a second descent that ends below anything the small models achieve

AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 30: Symmetry

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Inductive Bias, Symmetry, Invariance, Equivariance, Group theory, Deep Sets, Transformers

2 Main Points

2.1 Inductive Bias

- Learning machines are not universal approximators, but rather have a propensity to learn a specific set of rules
- This is known as their *inductive bias*
- We want to choose the learning machine so the inductive bias reflects the properties of the problem
- In almost all problems we use some smoothness property to select slowly varying functions

2.2 Invariances

- For many problems there are symmetries that you want your learning machine to respect
- For example in image classification we consider the class of an object to be invariant to its position in an image
- Learning machines whose outputs are invariant to translations have a much smaller set of functions to learn and therefore are much less likely to overfit
- CNNs are equivariant in that if we translate the input the output will be translated in the same way
- Building classifiers using layers of CNN builds in a lot of translational invariance
- Ideally we want a classifier system to also be scale and rotation invariant, although we don't know how to do this well
- Sometimes we use pyramid systems where we rescale the image and run the same operations on all scales
- However, in many datasets the foreground object is rescaled to fill most of the image with smaller object treated as the background so scale invariance is not necessarily required
- Often we try to learn invariance by augmenting the training set by artificially transforming the image (e.g. by reflecting, cropping, shearing and rescaling an image)
- Data augmentation usually gives a very significant improvement in performance
- We can sometimes achieve invariance by suitably normalising our data. Again when done right this can give a significant improvement in performance

2.3 Group Theory

- The mathematical language of symmetry is group theory
- You should know group theory exists, but I'm not going to examine you on details of group theory
- This considers a set of objects (we can think of these as transformations)
- Together with a binary operation we can think of as composition
- That is if $a, b \in \mathcal{G}$ are two transformations the $a \cdot b$ can be interpreted as applying transformation b followed by transformation a
- For the set, $\mathcal{G}$, and binary operation, "$\cdot$" to form a group they must satisfy four conditions
  1. Closure: if $a, b \in \mathcal{G}$ the $a \cdot b \in \mathcal{G}$
  2. Associativity: for all $a, b, c \in \mathcal{G}$, we have $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
  3. Existence of an identity element where $e \cdot a = a \cdot e = a$
  4. Every element, $a \in \mathcal{G}$ has an inverse such that $a^{-1} \cdot a = a \cdot a^{-1} = e$
- (Group theory doesn't really care if the elements of the group are transformations, just that the axioms are satisfied)
- An example is the cyclic group C_2
  - We can think of this as tossing a coin with two actions
    - flip the coin: f
    - leave the coin alone: e
  - These form a group as
    1. Closure: $f \cdot e = f$, $f \cdot f = e$, $e \cdot e = e$ and $e \cdot f = f$ (if you don't believe this get out a coin and try. The first identity is don't do anything then flip the coin is the same as just flipping the coin)
    2. Associativity: this follows automatically from the operation being a composition (we will see this in innerProduct lecture where we look at matrix multiplication). You can also check rather boringly, e,g, $(e \cdot f) \cdot f = f \cdot f = e$ while $e \cdot (f \cdot f) = e \cdot e = e$ thus $(e \cdot f) \cdot f = e \cdot (f \cdot f)$. You can try this for all triples
    3. e is clearly the identity
    4. $e^{-1} = e$ and $f^{-1} = f$
  - The cyclic group C_3 corresponds to the rotations of an equilateral triangle maps the triangle to itself. While C_n are the rotations that keep an n sided polygon with identical sides onto itself
  - The *four group* is a second group of size 4 (besides C_4) that can be thought of as describing the flipping around a vertical and horizontal axis a rectangle, or rotating it by 180 degress around its centre of mass. We can describe the group in terms of a *multiplication table* where the first row and first column corresponds to the elements a_i and the ith row and j^{th} column corresponds to the multiplication $a_j \cdot a_i$

	e	v	h	r
e	e	v	h	r
v	v	e	r	h
h	h	r	e	v
r	r	h	v	e

    - This is an "Abelian group" meaning that for any elements a, b we have $a \cdot b = b \cdot a$

- The group of permutations of n objects forms a group of size $n!$ called the symmetric group S_n
  - This is a group as the composition of a permutation is a permutation
  - There is an identity (doing nothing)
  - For every element there is an inverse permutation (which is a permutation)
  - For $n > 2$ (where $S_2 = C_2$) the group is non-Abelian meaning that there are permutations p_i and p_j where $p_i \cdot p_j \neq p_j \cdot p_i$
  - Despite this composing permutations is associative
  - Permutations have a lot of structure and S_n is very well studied
- There exists groups with an infinite number of elements. The set of integers with the operation of addition form a group with identity, 0, and the inverse of n is $-n$
  - You can think of these describing an infinite set of discrete shifts
  - You first started studying this group when you first learn to count
- The set of rational numbers under addition also forms a group
- The reals under addition form a group describing any one dimensional translation
- The set of reals excluding 0 forms a group under multiplication with an identity 1 and where the inverse of x is $1/x$
  - These can be thought of as describing scale transformations
- Groups can be interpreted in different ways with their elements having different names (e.g. one, two, three, $\cdots$ or un, deux, trois, $\cdots$). The permutation of two elements has the same structure (is isomorphic to) flipping a coin. What counts is the structure of the multiplication table (up to re-ordering the elements)

2.3.1 Lie Groups

- When we consider continuous transformation in high dimensional space then we find a set of very interesting (smooth) groups known as Lie groups
- These describe things like general translational invariance, rotational invariance, relativistic invariance
- The set of transformations can be represented as matrices, with the composition operator being matrix multiplication
- Different groups correspond to different sets of matrices.
- A well known group is the set of $n \times n$ orthogonal matrices $O(n)$
- By definition of orthogonal matrices $\mathbf{M}^\mathsf{T}\mathbf{M} = \mathbf{M}\,\mathbf{M}^\mathsf{T} = \mathbf{I}$
- These correspond to rotations and reflections
- Note that, in general, $\det(\mathbf{M}^\mathsf{T}) = \det(\mathbf{M})$ and $\det(\mathbf{A}\,\mathbf{B}) = \det(\mathbf{A}) \times \det \mathbf{B}$
- As a consequence $\det(\mathbf{M}^\mathsf{T}\mathbf{M}) = \det(\mathbf{I}) = 1$ so that $\det(\mathbf{M})^2 = 1$ or $\det(\mathbf{M}) = \pm 1$
- If $\det(\mathbf{M}) = -1$ then the transformation involves a reflection
- The special orthogonal group, $SO(n)$, corresponds to a subgroup of matrices in $O(n)$ with $\det(\mathbf{M}) = 1$. These correspond to rotations only
- The group $SO(2)$ corresponds to 2-d rotations where the elements of the group are matrices

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

2.4 Symmetry in Machine Learning

- **Invariance by averaging.** For any function f and finite group G of transformations, $\bar{f}(\boldsymbol{x}) = \frac{1}{|G|}\sum_{g\in G} f(g\cdot\boldsymbol{x})$ is invariant: for $h \in G$,

$$\bar{f}(h\cdot\boldsymbol{x}) = \frac{1}{|G|}\sum_{g\in G} f(g\cdot h\cdot\boldsymbol{x}) = \frac{1}{|G|}\sum_{g'\in G} f(g'\cdot\boldsymbol{x}) = \bar{f}(\boldsymbol{x})$$

  because $g \mapsto g' = g\cdot h$ is a one-to-one map of G onto itself (by closure, and because $g = g'\cdot h^{-1}$). The group axioms are exactly what is needed. Data augmentation approximates this average during training; *test-time augmentation* computes it at prediction time

- **Equivariance and convolution.** A map is equivariant if $f(g\cdot\boldsymbol{x}) = g\cdot f(\boldsymbol{x})$. Convolution commutes with shifts, and the linear maps that commute with all (circular) shifts are exactly the convolutions. A 3×3 filter has 9 weights against n^4 for a dense layer on an $n\times n$ image: building in the symmetry removes parameters and with them variance

- **Permutation symmetry.** For a set of inputs, any function $f(\{\boldsymbol{x}_1,\dots,\boldsymbol{x}_n\}) = \rho\big(\sum_i \boldsymbol{\phi}(\boldsymbol{x}_i)\big)$ is invariant to their order (*Deep Sets*, Zaheer et al., 2017). Graph neural networks update each node from a sum over its neighbours, so they are equivariant to relabelling the nodes

- **Transformers.** In self-attention each token computes $\boldsymbol{q}_i$, $\boldsymbol{k}_i$, $\boldsymbol{v}_i$ from its own input and outputs

$$\boldsymbol{y}_i = \sum_j \mathrm{softmax}_j\Big(\frac{\boldsymbol{q}_i^{\mathsf{T}}\boldsymbol{k}_j}{\sqrt{d}}\Big)\boldsymbol{v}_j$$

  If the tokens are permuted, each output is computed from the same set of terms, so the outputs are permuted in the same way: self-attention is permutation-equivariant. It therefore cannot see word order, which is why transformers add *positional encodings*

- **Rotations and molecules.** The energy of a molecule is invariant to rotations and translations, and the forces on its atoms are equivariant. Equivariant networks build this in and are now standard for molecules and materials; this area is called *geometric deep learning*

2.5 Looking Back

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange multipliers, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- The same ideas recur: the SVD in PCA, ridge regression and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

- Machine learning works when the model's structure matches the structure of the world. Priors, regularisers and symmetries are three ways of saying what that structure is, and mathematics is the language for saying it precisely

3 Exercises

3.1 Averaging over flips

- Let $G = \{e, r\}$ where r reflects an image left to right. Show that G is a group, and that $\bar{f}(\boldsymbol{x}) = \frac{1}{2}\big(f(\boldsymbol{x}) + f(r\cdot\boldsymbol{x})\big)$ is invariant under r.

3.2 Self-attention is permutation-equivariant

- Write the inputs as the rows of a matrix $\mathbf{X}$ and a permutation as a permutation matrix $\mathbf{P}$. With $\mathbf{Q} = \mathbf{X}\mathbf{W}_q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_k$, $\mathbf{V} = \mathbf{X}\mathbf{W}_v$ and the softmax taken along each row, show that permuting the inputs permutes the outputs.

4 Experiments

4.1 Checking the symmetries

- Check numerically that self-attention is permutation-equivariant and that averaging over the flip group gives invariance

```
import numpy as np

rng = np.random.default_rng(0)
n, d = 5, 4                                  # 5 tokens, dimension 4
X = rng.standard_normal((n, d))
Wq, Wk, Wv = (rng.standard_normal((d, d)) for _ in range(3))


def attention(X):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    A = np.exp(Q @ K.T / np.sqrt(d))
    A /= A.sum(1, keepdims=True)             # softmax over j
    return A @ V


perm = rng.permutation(n)
print(np.allclose(attention(X[perm]), attention(X)[perm]))   # equivariant

# invariance by averaging over the group {identity, left-right flip}
img = rng.standard_normal((6, 6))
f = lambda x: (x * np.arange(36).reshape(6, 6)).sum()      # not invariant
fbar = lambda x: (f(x) + f(x[:, ::-1])) / 2
print(f(img), f(img[:, ::-1]))
print(fbar(img), fbar(img[:, ::-1]))
```

- This prints `True`; f gives different values for the image and its mirror image (-27.1 and -6.5), while $\bar{f}$ gives the same value (-16.8) for both

5 Answers

5.1 Averaging over flips

- $r \cdot r = e$, so the set is closed, e is the identity, each element is its own inverse, and composition is associative: it is the group C_2
- $\bar{f}(r \cdot \boldsymbol{x}) = \frac{1}{2}\big(f(r \cdot \boldsymbol{x}) + f(r \cdot r \cdot \boldsymbol{x})\big) = \frac{1}{2}\big(f(r \cdot \boldsymbol{x}) + f(\boldsymbol{x})\big) = \bar{f}(\boldsymbol{x})$

5.2 Self-attention is permutation-equivariant

- Permuting the inputs gives $\mathbf{PX}$, so $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ become $\mathbf{PQ}$, $\mathbf{PK}$, $\mathbf{PV}$ and the score matrix becomes $\mathbf{P}\mathbf{Q}\mathbf{K}^{\mathsf{T}}\mathbf{P}^{\mathsf{T}}/\sqrt{d}$: its rows and columns are permuted in the same way

- The softmax acts on each row separately and a permutation of a row's entries just permutes its softmax, so the attention matrix becomes $\mathbf{PAP}^\mathsf{T}$
- The output is $\mathbf{PAP}^\mathsf{T}\mathbf{PV} = \mathbf{PAV}$, using $\mathbf{P}^\mathsf{T}\mathbf{P} = \mathbf{I}$: the outputs are permuted